

384.65 / A958

107

T-E2

ESCUELA POLITECNICA NACIONAL

FAACULTAD DE INGENIERIA ELECTRICA

TESIS DE GRADO

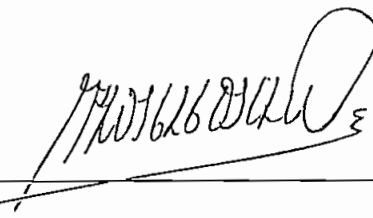
PREVIA A LA OBTENCION DEL TITULO DE
INGENIERO EN ELECTRONICA Y TELECOMUNICACIONES

DISEÑO Y CONSTRUCCION DE UN CODEC DIDACTICO
PARA TRANSMISION DIGITAL EN BANDA BASE

NELSON HERNAN AVILA JIMENEZ

JUNIO DE 1994

CERTIFICO QUE, BAJO MI DIRECCION,
LA PRESENTE TESIS FUE REALIZADA
EN SU TOTALIDAD POR EL SEÑOR
NELSON H. AVILA JIMENEZ

A handwritten signature in dark ink, appearing to read 'Pablo Hidalgo Lascano', is written over a horizontal line. The signature is stylized with a large loop at the end.

ING. PABLO HIDALGO LASCANO
DIRECTOR DE TESIS

INDICE

Pág.

INTRODUCCION	v
CAPITULO I	
ESTUDIO DE LOS CODIGOS DE LINEA	1
1.1. GENERALIDADES	1
1.1.1. Los sistemas de comunicación	1
1.1.2. Transmisión de datos	2
1.1.3. Transmisión de datos en banda base	4
1.1.4. Transmisión de datos usando modulación	6
1.1.5. Medios de transmisión	8
1.2. CLASIFICACION	13
1.2.1. Clasificación según la polaridad	15
1.2.2. Clasificación general	16
1.3. ALGORITMOS DE CODIFICACION	20
1.3.1. Código NRZ-neutral o unipolar	20
1.3.2. Código NRZ-polar	20
1.3.3. Código AMI	21
1.3.4. Códigos diferenciales	23
1.3.5. Códigos bifase	24
1.3.6. Código de modulación por retardo o código Miller	27
1.3.7. Códigos pseudoternarios	28
1.3.8. Códigos entrelazados	51
1.4. COMPARACION Y APLICACIONES	53
1.4.1. Criterios de evaluación	54
1.4.2. Códigos diferenciales NRZ	56

1.4.3.	Codificación con retorno a cero (RZ) . . .	58
1.4.4.	Códificación bifase	59
1.4.5.	Modulación por retardo o código Miller .	60
1.4.6.	Codificación bipolar	62
1.5.	ANALISIS DE LA DENSIDAD ESPECTRAL DE POTENCIA DE, LOS CODIGOS DE LINEA	65
1.5.1.	Tratamiento estadístico de la densidad espectral de potencia	66
1.5.2.	Densidad espectral de potencia de los códigos de línea	72
1.6.	DESCRIPCION DEL SOFTWARE PARA EL ANALISIS	83
1.6.1.	Programa principal	84
1.6.2.	Comparación de la d.e.p. de diferentes códigos	86
1.6.3.	Variación de la d.e.p. en función de la probabilidad p	90
CAPITULO II		
	DISEÑO Y CONSTRUCCION DEL CODEC	93
2.1.	ESPECIFICACIONES Y REQUERIMIENTOS DEL EQUIPO . . .	93
2.2.	DISEÑO DEL CODIFICADOR	97
2.2.1.	Configuración para el microcontrolador .	100
2.2.2.	Mapeo de memoria	105
2.2.3.	Circuito de inicialización	106
2.2.4.	Circuito de configuración	108
2.2.5.	Circuito de ingreso de datos	109
2.2.6.	Reloj maestro	110
2.2.7.	Circuito de selección del ritmo de transmisión	111
2.2.8.	Circuito de sincronización	113

2.2.9.	Circuito de selección del nivel de salida	116
2.3.	DISEÑO DEL DECODIFICADOR	118
2.3.1.	Circuito de ingreso de señal	120
2.3.2.	Circuito de selección del ritmo de transmisión	121
2.3.3.	Circuito de sincronización	123
2.3.4.	Circuito de salida de datos	126
2.4.	CONSTRUCCION DEL EQUIPO	127
2.5.	DESCRIPCION DEL SOFTWARE DE CONTROL DEL EQUIPO . .	133
2.5.1.	Rutina de inicialización	133
2.5.2.	Rutinas para el código NRZ polar	140
2.5.3.	Rutinas para el código AMI	143
2.5.4.	Rutinas para el código RZ polar	145
2.5.5.	Rutinas del código Manchester diferencial	149
2.5.6.	Rutinas para el código bifase-M	152
2.5.7.	Rutinas para el código de modulación por retardo (Miller)	155
2.5.8.	Rutinas para el código 4B-3T	159
2.5.9.	Rutinas para el código MS43	168
2.5.10.	Rutinas para el código B3ZS	173
2.5.11.	Rutinas para el código HDB3	175

CAPITULO III

EVALUACION EXPERIMENTAL DEL EQUIPO	180
--	-----

3.1. PRESENTACION DEL EQUIPO	180
--	-----

3.2. PRUEBAS DE FORMATO	185
-----------------------------------	-----

3.3. VELOCIDAD DE TRANSMISION	212
---	-----

3.4. COMPARACION DE LA ELIMINACION DE LA COMPONENTE CONTINUA	214
---	-----

3.5. LIMITACIONES DEL EQUIPO	221
--	-----

CAPITULO IV

CONCLUSIONES	224
------------------------	-----

BIBLIOGRAFIA	231
------------------------	-----

ANEXOS

ANEXO A: GUIA DEL USUARIO

ANEXO B: HOJAS DE DATOS DE LOS PRINCIPALES ELEMENTOS
UTILIZADOS

ANEXO C: RECOMENDACION G.703 DEL CCITT

ANEXO D: LISTADO DE PROGRAMAS

INTRODUCCION

Varias han sido las formas que se han desarrollado para transmitir la información de un lugar a otro, la más actual y de mayor futuro es la transmisión digital que permite el envío de información inherentemente digital o proveniente de una conversión analógica a digital, entre dos equipos terminales de datos.

Un método para realizar esta transmisión es mediante el empleo del popular **modem** que permite enviar la información utilizando una portadora analógica, el otro consiste en enviar los datos de manera codificada. Si bien este último ha sido tratado desde hace muchos años, su estudio se ha presentado de manera aislada para diferentes códigos que se han propuesto para determinadas aplicaciones.

El objetivo del presente trabajo es el estudio teórico y práctico de los códigos de línea empleados para transmitir datos mediante pulsos eléctricos, cuya conformación determinará la aptitud del código para vencer problemas como la limitación del ancho de banda, la correcta ubicación de la señal dentro del espectro de potencia, la sincronización del reloj de recepción, etc.

Para ello, en el capítulo I se realiza un estudio de los fundamentos de los códigos de línea, sus tipos, los algoritmos de codificación y su utilización. Luego se trata sobre la realización de un programa para computador personal que permita realizar el análisis de la densidad espectral de potencia de diferentes códigos de línea.

El capítulo II abarca el diseño del CODEC a partir de

los requerimientos y especificaciones que se establecen para el equipo tanto en lo referente al *hardware* como al *software*. En el capítulo III se realiza la evaluación experimental del CODEC estableciendo el cumplimiento de los requerimientos y sus limitaciones, para finalmente en el capítulo IV exponer las conclusiones a las que se llegó en la realización del trabajo.

Con el estudio desarrollado y el equipo implementado, a criterio del autor, se ha cumplido con el objetivo de crear un equipo que permita el estudio práctico de la transmisión digital en banda base y el de reforzar la infraestructura del Laboratorio de Comunicación Digital. Las pruebas que se establecen en la evaluación experimental del equipo son una guía de las posibilidades que se ofrecen, pero que pueden ser ampliadas con otros ensayos cuya ejecución estará condicionada a la disponibilidad del equipo de medición necesario.

CAPITULO I

ESTUDIO DE LOS CODIGOS DE LINEA

1.1. GENERALIDADES.

1.1.1. Los sistemas de comunicación.

Antes de hacer un estudio sobre los códigos de línea, conviene situar el tema dentro del campo de las telecomunicaciones.

Un sistema de comunicación es aquel que permite transmitir información de un lugar a otro en el tiempo y el espacio. Esta definición engloba el hecho de que los mensajes a ser comunicados van desde el ~~emisor~~ hasta el ~~destinatario~~, separados en el espacio como en el caso de una conversación telefónica, y en el tiempo como cuando se almacenan datos en ~~el~~ disco fijo de un computador. El ~~conjunto~~ de los agentes que intervienen en la transmisión, incluyendo el medio físico, corresponden al canal de transmisión.

Un mensaje se lo envía como datos que pueden ser de muy diversos tipos, según el sistema de comunicación que se use. ~~En~~ sentido estricto, datos e información no son la misma cosa. Se dice que un dato contiene información, cuando éste proporciona un conocimiento del que antes se ~~carecía~~ o cuando incrementa este conocimiento; en este sentido los datos contienen más información cuanto menos probable es el conocimiento que éstos proveen.

Un sistema de comunicación sin embargo, se encarga de transmitir los datos independientemente de la cantidad de información que aporten; por esta razón, el presente trabajo tratará en general de la transmisión de cualquier

tipo de datos sin considerar la cantidad de información con que éstos contribuyan.

En el caso que nos ocupa, los mensajes se enviarán mediante datos digitales, es decir usando niveles discretos de señales que pueden corresponder ya sea a un dato analógico digitalizado o a uno inherentemente digital.

"Los datos, en su acepción más común se restringe todavía más al caso en que alguno de los interlocutores no es un ser humano sino un ordenador"¹. En general, los datos se referirán a señales digitales que pueden ser generadas por un computador o por cualquier otro tipo de procesamiento digital. En consecuencia no interesará mayormente el origen de los datos sino su transmisión.

1.1.2. Transmisión de datos.

El intercambio de mensajes entre un terminal local y otro remoto, es realizado mediante equipos destinados a este objetivo. La comunicación se origina en una fuente de datos que junto con un controlador de comunicaciones constituyen el **equipo terminal de datos (DTE)**; los datos pasan luego al **equipo terminal del circuito de datos (DCE)** mediante un interfaz. A partir del DCE se realiza la transmisión a través del canal de comunicación. Para la recepción de los datos, se necesita al igual que en el transmisor, de un equipo terminal del circuito de datos, el cual estará conectado de manera similar a través de un interfaz apropiado al equipo terminal de datos; este proceso general se lo ilustra en un sólo sentido en la figura 1.1. Aquí se tiene una transmisión *simplex*. En el caso de que se unan dos sistemas *simplex* se puede tener una transmisión *half-duplex* o *full-duplex*, según el canal de transmisión sea uno sólo o se disponga de dos vías,

¹ VIDALLER L. y OTROS, Transmisión de datos, E.T.S. Ingenieros Telecomunicaciones-Universidad Politécnica de Madrid, Madrid, 1979, p. 3.

respectivamente; en el primer caso se tendrá una comunicación alternada en los dos sentidos, en tanto que en el segundo la comunicación podrá ser simultánea en las dos direcciones.

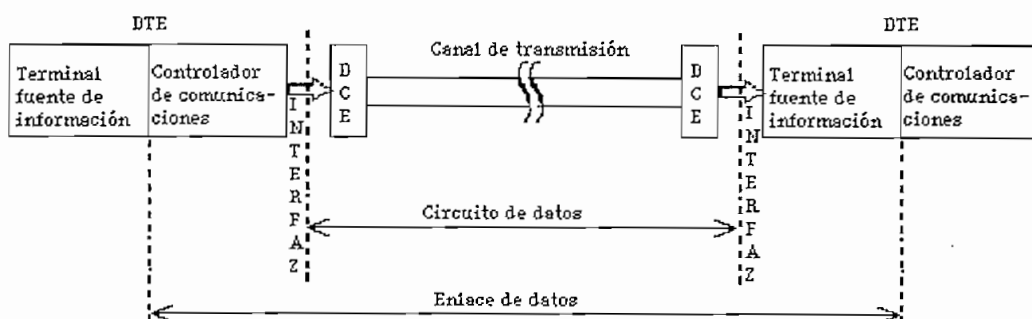


Fig. 1.1. Sistema básico de transmisión de datos

Ahora bien, los datos digitales de los equipos terminales pueden ser generados directamente o mediante un proceso de conversión analógico a digital. El controlador de comunicaciones permitirá un manejo adecuado de los datos, de manera que éstos tengan un formato, un protocolo, etc.

El canal de comunicación lo constituye el medio físico a través del cual se transmiten las señales eléctricas, que llevan los datos que se quiere transportar de un lugar a otro. Una de las características más importantes del medio de transmisión, es su ancho de banda, lo que se interpreta como la capacidad para permitir el paso (de la manera más fiel) de una gama de frecuencias. Una señal a transmitirse está compuesta por una serie de componentes de frecuencia, las que deberán ser aceptadas por el canal. Por lo tanto, mientras mayor sea el rango de frecuencias (ancho de banda) que un canal permita transmitir, mayor será su capacidad para transmitir los datos.

Existen básicamente dos maneras de transmitir los datos, usando los datos digitales como tales (en banda base) o usando una señal analógica (modulación) para representar los mismos.

1.1.3. Transmisión de datos en banda base.

Los equipos generadores de datos digitales, en general, presentan en su salida en una serie de pulsos de corriente continua separados por ausencia de ellos (unos y ceros binarios). Estos se suceden en secuencias que definen unívocamente el carácter o el dato que representan¹.

Para conocer el espectro de frecuencias que una señal como la mencionada genera, se recurre al análisis de Fourier. Es así, que un dato binario digital que puede ser caracterizado por un pulso de duración $2a$, tiene como transformada de Fourier la señal que se muestra en la figura 1.2.

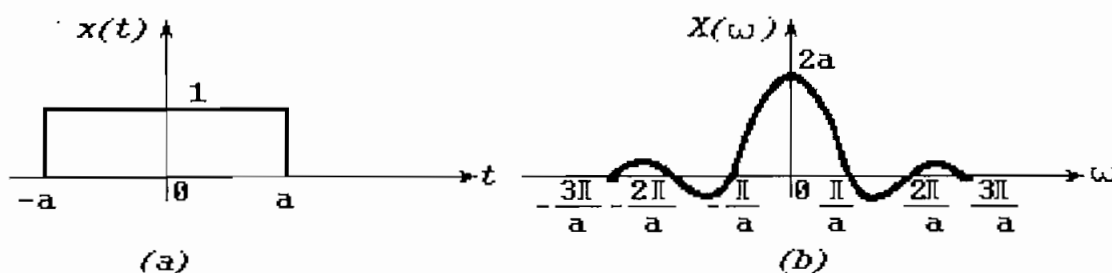


Fig. 1.2. Transformada de Fourier de un pulso

¹ DAVENPORT W., Comunicación moderna de datos, GLEM, Buenos Aires, 1974, p. 115-116.

$$X(\omega) = \int_{-\infty}^{+\infty} x(t) e^{-j\omega t} dt$$

$$x(t) = \Pi\left(\frac{t}{2a}\right)$$

$$X(\omega) = 2 \frac{\sin(a\omega)}{\omega} \quad [1.1]$$

Como se puede observar tanto del gráfico 1.2.b como de la ecuación¹ [1.1], el ancho de banda del pulso es infinito, pues posee componentes de frecuencia que van desde continua (en mayor porcentaje), hasta las más altas frecuencias (en porcentaje despreciable).

De lo anterior se deduce que la transmisión en banda base tiene como principales problemas el gran ancho de banda y la gran componente de corriente continua.

Si se consideran estos inconvenientes, se concluye que la señal tal como está no podría ser enviada por ningún canal real de transmisión, ya que éstos tienen un ancho de banda finito, como es el caso de la línea telefónica que tiene su ancho de banda limitado entre 300 y 3400 Hz.

Como se plantea la cuestión, parecería que no es posible realizar una transmisión de los datos digitales como tales, sin embargo no es así. Observando el espectro de la figura 1.2, se tiene que a medida que aumenta la frecuencia, la contribución al espectro total es muy pobre, de lo que puede deducirse que bastará con enviar una parte de las componentes de frecuencia para obtener una buena aproximación al pulso original.

1

$$\Pi\left(\frac{t}{2a}\right) = \begin{cases} 1 & -a < t < a \\ 0 & \text{para otro } t \end{cases}$$

En este punto surgen los **códigos de banda base**, que permiten básicamente superar los inconvenientes del excesivo ancho de banda y de la gran componente DC, además de aportar con otros elementos para tener una transmisión adecuada, elementos éstos que serán tratados posteriormente.

Existe también otra forma de afrontar los inconvenientes de la transmisión en banda base y consiste en la transmisión de la señal digital mediante modulación de una señal analógica.

1.1.4. Transmisión de datos usando modulación.

La red telefónica conmutada constituye en todo el mundo el sistema más grande de comunicaciones, por lo cual, cuando se vio la necesidad de realizar un intercambio de datos digitales a gran escala, se pensó en la posibilidad de aprovechar este sistema.

Sin embargo, de acuerdo a lo enunciado anteriormente, los datos digitales se generan como pulsos de corriente continua con un ancho de banda exageradamente grande. Como contraparte, el ancho de banda efectivo del canal telefónico está comprendido entre 300 Hz y 3400 Hz, lo cual constituye un espacio insuficiente y un medio inadecuado para transmitir la señal digital en su forma original.

Para no desaprovechar la enorme capacidad instalada de la red telefónica, se ideó la forma de transmitir señales analógicas (dentro del ancho de banda establecido) que lleven la información digital. El método usado para este fin es la modulación de una portadora analógica utilizando la señal digital a transmitir.

Con esta modificación, no sólo que es posible usar

la red telefónica conmutada, sino que se pueden usar otros sistemas de transmisión como radio digital, satélite, etc.

En la modulación lo que se hace es usar una señal analógica de una frecuencia determinada, que constituye la portadora; se modifica entonces su amplitud, frecuencia, fase o una combinación de estos parámetros para transmitir los datos. En la modulación de amplitud, lo más usual es tener dos amplitudes diferentes de la señal portadora para representar el 1_L y el 0_L, respectivamente.

En la modulación de frecuencia, se utilizan dos o más tonos diferentes para representar los dos dígitos binarios o arreglos de estos dígitos. Existe además la modulación de fase, en cuya forma más general, la portadora se desplaza en forma sistemática en el tiempo, en intervalos espaciados de manera uniforme, y por cada uno de estos desplazamientos de fase se transmitirán uno o más bits, dependiendo del número de fases establecidas. Se usa además la combinación de técnicas de modulación para obtener mejores resultados, sobre todo en cuanto a aumentar la velocidad binaria de transmisión, que es uno de los parámetros más importantes en transmisión de datos.

Al dispositivo que acepta como entrada los flujos de bits en serie, y al mismo tiempo produce una portadora modulada como salida (o viceversa), se le conoce comúnmente como **modem** (derivado de los términos modulador-demodulador). El modem constituye un equipo de terminación de circuito de datos y es uno de los más utilizados cuando se trata de realizar una transmisión de datos a larga distancia; sin embargo, dada la facilidad, también se lo usa en cortas distancias cuando la velocidad no es un serio requerimiento.

1.1.5. Medios de transmisión.

En sentido estricto, el objetivo de la transmisión de datos es el transportar un flujo original de bits, de un equipo a otro. Para realizar este transporte existe una variedad de medios físicos, cada uno de los cuales tiene sus características propias que les hacen ser útiles en determinadas condiciones.

a. Par trenzado.

Constituye el medio de transmisión más antiguo y que todavía se lo utiliza ampliamente por su facilidad de implementación. Consiste en dos conductores de cobre aislados, en general de 1 mm de diámetro. Los alambres se entrelazan en forma helicoidal para reducir la interferencia eléctrica con respecto a los pares que se encuentran a su alrededor; dos cables paralelos constituyen una antena simple, en tanto que un par trenzado no lo es¹.

La aplicación más común del par trenzado está en el sistema telefónico, ya que la casi totalidad de aparatos telefónicos están conectados a la central mediante un par trenzado. La distancia que se puede recorrer con estos cables es de varios kilómetros, sin necesidad de amplificar las señales, pero en caso de requerir distancias más grandes, será necesario incluir repetidores.

Los pares trenzados se pueden utilizar ya sea para transmitir en banda base, así como para transmitir usando una portadora modulada; su ancho de banda depende de las dimensiones del conductor y de la distancia que recorre. En muchos casos, se pueden obtener transmisiones de varios Mbit/s en distancias cortas como es el caso de la red de computadoras *Token Ring* de IBM que opera a 4 Mbit/s

¹ TANENBAUM A., Redes de ordenadores, Prentice Hall, México, 1991, p. 66.

utilizando par trenzado¹. Debido a su adecuado comportamiento y bajo costo, los pares trenzados se utilizan ampliamente para transmisión de datos².

b. Cable coaxial.

El tipo de cable coaxial que generalmente se usa en transmisión de datos en banda base es el cable de 50 Ω (el de 75 Ω es más bien usado en transmisión analógica).

El cable coaxial constituye uno de los medios físicos más utilizados cuando se trata de realizar intercambio de datos a nivel de computadores, sobre todo en redes de área local, debido a que el cable coaxial permite evitar la interferencia exterior por tener su cubierta conectada a tierra, y permite transmitir datos a alta velocidad. Por el contrario, con el par trenzado se debe tener más cuidado en su instalación para evitar interferencias.

El ancho de banda que se obtiene con el cable coaxial, en distancias relativamente cortas es lo suficientemente grande como para tener un buen desempeño. Para cables de 1 km por ejemplo, se puede obtener velocidades de hasta 10 Mbit/s, y si se disminuye la distancia, es posible tener un incremento en el ancho de banda.

Por lo general, a velocidades de transmisión bajas o altas, un cable de pares trenzados sufre una pérdida de señal mucho menor que la que experimenta un coaxial. Sin embargo, las líneas coaxiales pueden instalarse prácticamente en cualquier lugar y como el cable tiene conectado su blindaje a tierra, se puede pasar el cable al

¹ BLACK U., Redes de computadoras, Macrobit, México, 1990, p. 143.

² TANENBAUM A., Op. Cit., p. 67.

lado de objetos metálicos sin ningún problema.

Cuando se emplea transmisión usando modulación, se puede usar los canales analógicos (generalmente en base a cable coaxial de 75 Ω), los cuales son diseñados para tener un gran ancho de banda, permitiendo el paso de señales como la de video que requiere de 6 MHz para su transmisión. Un cable típico de 300 MHz, por lo general, puede permitir velocidades de transmisión de datos de hasta 150 Mbit/s.

c. Fibra óptica.

El desarrollo tecnológico ha permitido que el uso de las comunicaciones ópticas se vuelvan cada vez más competitivas con los medios de transmisión tradicionales. Esta técnica de transmisión usa luz infrarroja, cuya frecuencia está en el orden de 10^{14} Hz, por lo que el ancho de banda de un canal óptico representa un potencial muy grande.

Básicamente, un sistema de transmisión óptica consta del medio de transmisión (fibra óptica), la fuente y el detector de luz.

La fuente de luz, que modulada en intensidad, frecuencia o fase permitirá llevar la información, lo constituye un diodo emisor de luz (LED) o un diodo láser (LD), este último necesario en aplicaciones de gran velocidad. El detector lo constituye un fotodiodo PIN (~~positive-intrinsic-negative~~) o un APD (~~avalanche-photo-diode~~), polarizado inversamente de modo que produzca un pulso eléctrico en el momento en que se reciba un rayo de luz. Al colocar un LED o un LD en el extremo de una fibra óptica, y un fotodetector en el otro, se tiene un canal unidireccional que acepta una señal eléctrica, la convierte y la transmite por medio de pulsos de luz, la cual es

recibida y reconvertida en señal eléctrica para su posterior procesamiento.

El principio físico en que se basa la transmisión a través de fibras ópticas se conoce como reflexión interna total, y consiste en que la luz emitida llega al receptor mediante una reflexión continua a lo largo de la fibra, sin permitir su refracción, ya que ésta daría lugar a una pérdida de potencia. Según el número de modos de transmisión (señales de diferente longitud de onda) que se permita transmitir en una fibra óptica, se tendrán las fibras multimodo que se usan en aplicaciones de baja o mediana velocidad, y las monomodo, usadas en aplicaciones de alta velocidad.

Las velocidades de transmisión binaria usadas en los sistemas europeos con fibras ópticas son normalmente 8 Mbit/s, 34 Mbit/s, 140 Mbit/s, 565 Mbit/s. Incluso se ha podido alcanzar transmisiones de datos¹ de 1000 Mbit/s en 1 km. Obviamente se debe tener un compromiso entre la distancia máxima entre regeneradores y el ancho de banda disponible del canal, lo que dará el factor de mérito para la fibra óptica.

En la transmisión de datos, es ilustrativo el realizar una comparación entre el cable coaxial y la fibra óptica. Las fibras ópticas proporcionan un ancho de banda muy grande con pocas pérdidas de potencia, no son afectadas por interferencias electromagnéticas y son fáciles de manejar por su estructura bastante delgada. Sin embargo, la realización de conexiones y empalmes no es fácil y requiere de equipo especial y personal calificado, además de que su resistencia mecánica es bastante pobre.

¹ TANENBAUM A., Op. Cit., p. 73.

d. Canales radioeléctricos.

Cuando se trata de transmitir información en donde las distancias o los lugares determinan que no se pueda o no convenga usar un medio físico, es preciso usar la transmisión radioeléctrica. En este sentido, se utiliza una portadora de radiofrecuencia (RF) la cual será modulada para transportar la información.

El sistema más usado con este propósito es el que corresponde a la microonda, en donde se pueden encontrar desde simples enlaces punto a punto, hasta complicados sistemas satelitales de transmisión de datos usando modulación.

Muchos desarrollos en las comunicaciones han contribuido al crecimiento de aplicaciones en sistemas digitales de microonda. Algunos de los más importantes requerimientos son: el incremento cada vez mayor del tráfico telefónico que puede ser manejado económicamente por métodos completamente digitales, la demanda de nuevos servicios como facsímil, televisión digitalizada y transmisión de datos a alta velocidad.

La velocidad de transmisión binaria que se ha alcanzado con estos sistemas es del orden de 1 Mbit/s para sistemas de baja capacidad y de unos 300 a 400 Mbit/s para sistemas de alta capacidad. Un esquema básico de un sistema radioeléctrico para transmisión de datos se muestra en la figura 1.3; la fuente digital puede incluir cualquier canal de voz digitalizada (PCM), uno o más conversores analógico/digitales para transmisión de señales de televisión de alta calidad, uno o más computadores u otros canales de datos.

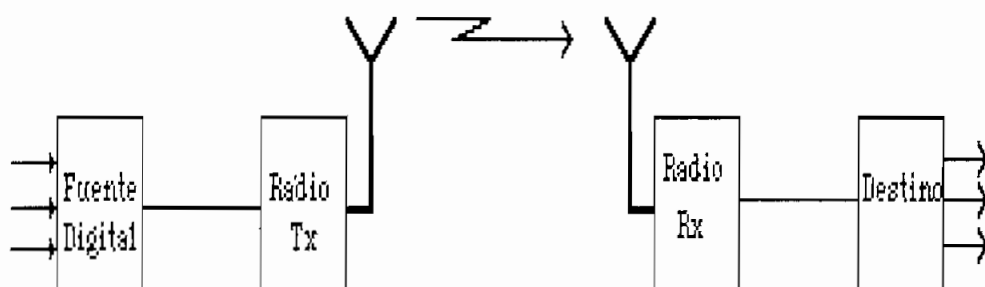


Fig 1.3. Sistema básico de microonda digital

La unidad de transmisión de microonda (radio Tx) acepta la información desde la fuente digital en forma de uno o más trenes de bits, con una velocidad binaria de transmisión especificada, y los convierte en una portadora de radiofrecuencia modulada digitalmente por los datos. El receptor de radio (radio Rx) demodula la señal de radiofrecuencia y provee la información digital al destino¹.

1.2. CLASIFICACION.

Para pensar en las posibles clasificaciones que se podrían hacer de los códigos de línea, es necesario considerar las características que éstos presentan.

Según el CCITT, un código de línea es un "código elegido de modo que convenga a las características de un canal y que define la equivalencia entre un conjunto de dígitos presentados para su transmisión y la correspondiente secuencia de elementos de señal transmitidos por ese canal"².

Como se ha manifestado, los códigos de línea se

¹ FEHER K., Digital communications, Prentice Hall, Englewood Cliffs, 1981, p. 1-6.

² CCITT, Recomendaciones - Libro Rojo, tomo III, fascículo III.3, Rec. G.701, Málaga, 1984,

usan para permitir una mejor transmisión de las señales digitales consideradas como pulsos de corriente continua. Esta mejoría se la entiende de la siguiente manera: las señales digitales que se transmiten por canales de un determinado ancho de banda (limitado), serán atenuadas en sus componentes de alta frecuencia por lo que se pierde la forma característica de pulso con lo cual, en la recepción no se podrá discernir el estado lógico correcto, a menos que el instante de muestreo sea el adecuado.

Y es que las líneas de transmisión tienen una respuesta de frecuencia que permite una comunicación óptima en un rango de frecuencia limitado, por lo cual es conveniente trasladar lo más significativo del espectro de la señal en banda base al rango donde el medio presente su respuesta óptima; esto generalmente implica que se deba disminuir las componentes que se ubican cerca de la frecuencia cero y que el ancho de banda necesario no sea exageradamente grande.

De lo que antecede se deduce que el código de línea debe permitir la **conformación de un adecuado espectro de energía** que garantice la respuesta óptima del medio de transmisión.

Pero, el hecho de incluir una codificación, cualquiera que ésta sea, no debe afectar la realización de la comunicación en sí, y por lo tanto, en el origen como en el destino de los datos la codificación/decodificación deberán ser transparentes y no afectarán, excepto tal vez en un pequeño retardo, la ejecución del sistema. En este sentido, cabe indicar que se debe cuidar que el proceso de codificación/decodificación no incremente los errores de la comunicación o que por lo menos no lo haga en un porcentaje significativo, a cambio de superar condiciones adversas de transmisión.

A parte de cumplir con estos requisitos básicos, se puede sacar ventaja de la transmisión codificada para obtener una utilidad en la recuperación de la señal de reloj en el lado de recepción, a partir de los datos recibidos; esto permitirá obtener una mejor sincronización que asegure una correcta decodificación. En todo esto se debe tener muy en cuenta que la eficiencia de la codificación sea adecuada, pues no será provechoso que por tratar de mejorar la transmisión sólo se la complique.

El establecer una clasificación de los diferentes códigos de línea no es una tarea fácil, puesto que se ha propuesto por parte de la industria diferentes tipos de códigos, muchos de los cuales no se ajustan a un determinado tipo sino que constituyen una individualidad en sí mismos, ya que han sido planteados de manera aislada. De todos modos, se intentará una clasificación que basándose en características generales, permita obtener una visión de la universalidad de códigos de línea.

1.2.1. Clasificación según la polaridad.

a. Código unipolar.

Es aquel código en el cual la señal toma valores negativos ó valores positivos (uno solo de ellos), además del nivel cero, de modo que su signo algebraico no cambia¹.

b. Código polar.

Es aquel en el cual la señal toma valores positivos y negativos; en este tipo de código, los signos opuestos identifican los dos estados lógicos binarios, sin incluir el nivel cero. Si el nivel mayor se asigna al 1, se hablará de lógica positiva, en tanto que si con el nivel

¹ BLACK U., Op. Cit., p. 222.

mayor se designa al 0_L, se tendrá lógica negativa.

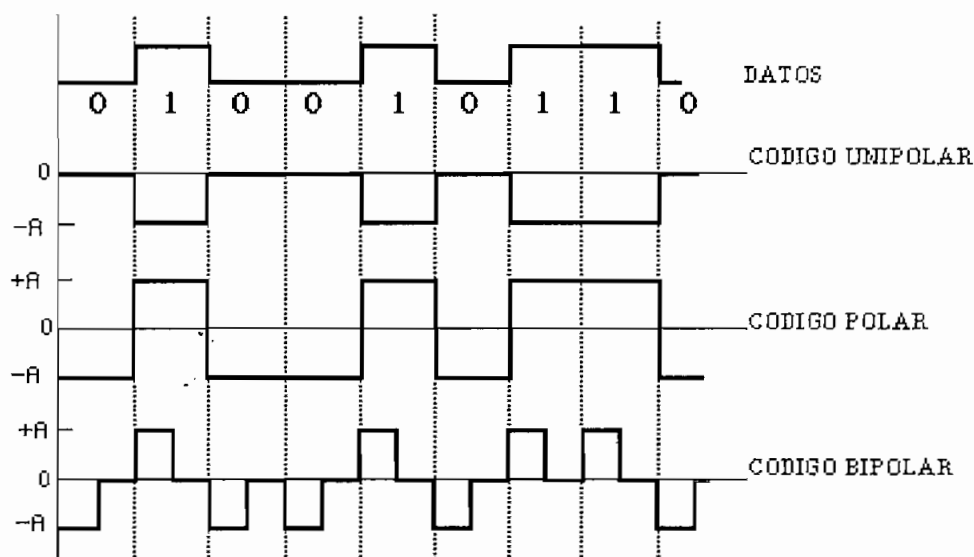


Fig. 1.4. Tipos de código según su polaridad

c. Código bipolar.

Cuando la señal varía entre tres niveles: uno positivo, el cero y otro negativo, se tiene un código bipolar.

1.2.2. Clasificación general.

En lo que sigue, se dará una clasificación general de los códigos para transmisión en banda base y una descripción resumida de los mismos; la explicación detallada del proceso de codificación/decodificación se la verá en el ítem 1.3 de este mismo capítulo.

a. Código AMI.

AMI (*Alternate Mark Inversion*), es un código en el

cual se emplean pulsos de polaridad alternada para codificar los unos binarios, en tanto que los ceros se codifican mediante la ausencia de pulsos (figura 1.5).

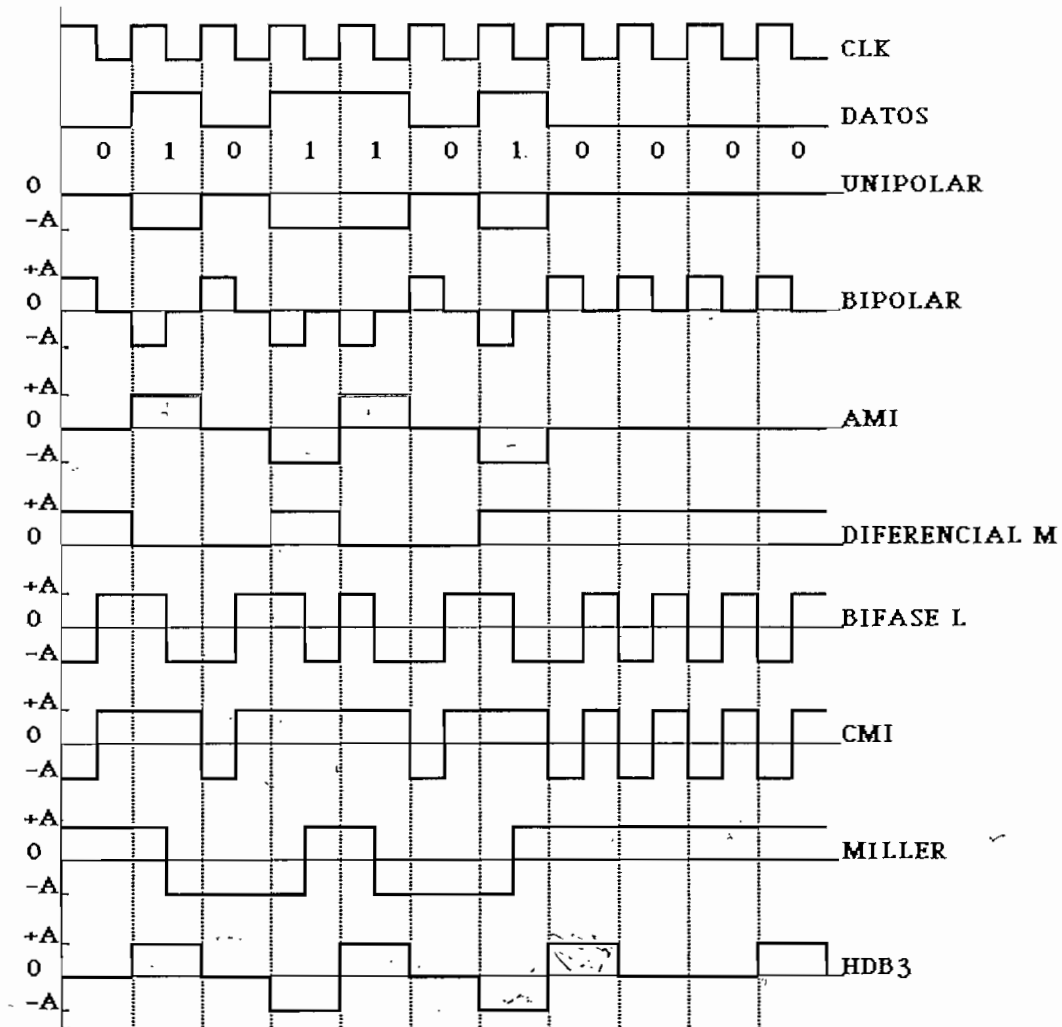


Fig. 1.5. Ejemplos de Codificación en banda base

b. Códigos diferenciales.

Son códigos que se fundamentan en que uno de los dos símbolos binarios cambia el nivel de la señal y el otro no; según sea el 1_L o el 0_L el que produzca el cambio de

nivel, se tendrán los códigos M o S, respectivamente¹ (figura 1.5).

c. Códigos bifase.

Son códigos que producen transiciones entre dos niveles, uno positivo y otro negativo; dependiendo de en dónde se den las transiciones, se puede tener: código bifase L o código Manchester, código bifase M, código bifase S y CMI (*Coded Mark Inversion*).

d. Modulación por retardo o código de Miller.

Se basa también en las transiciones entre dos niveles de polaridad opuesta, con diferentes reglas de codificación y con la particularidad de que se puede tener hasta el tiempo equivalente a dos períodos de reloj sin que se produzca una conmutación de nivel.

e. Códigos pseudoternarios.

"Se fundamentan en asignar a la secuencia binaria de entrada, una secuencia de salida a tres niveles"².

Se tiene una variedad de códigos pseudoternarios (PT) de características muy variadas, que se pueden clasificar en dos grandes grupos: códigos PT lineales y códigos PT no lineales.

¹ VIDALLER L. y OTROS, Op. Cit., p. 129.

² VIDALLER L. y OTROS, Op. Cit., p. 137.

Entre los códigos lineales se tiene:

- Básicos: Dicode, duobinario.
- Precodificados.

De los no lineales se puede encontrar:

- Alfabéticos:
 - Sin disminución de velocidad: PST, NST.
 - Con disminución de velocidad: 3B-2T, 4B-3T, MS43.
 - De longitud variable VL43.
- No alfabéticos: de todos los propuestos, sólo los códigos bipolares rellenos tienen aceptación práctica, entre estos se tiene:
 - Con sustitución de n ceros: B3ZS, B6ZS.
 - Bipolares de alta densidad: HDB, CHDB.

f. Códigos entrelazados.

No se trata precisamente de un tipo de código sino más bien de una técnica que alterna una codificación con otra, de modo que se altere la densidad espectral de potencia.

Además, es necesario anotar que cada uno de los códigos nombrados puede usar una técnica de retorno a cero (RZ) o de no retorno a cero (NRZ), según se tenga que en una parte del período de bit la señal vaya hasta cero o no.

1.3. ALGORITMOS DE CODIFICACION.

1.3.1. Código NRZ-neutral o unipolar.

Es el código más sencillo, ya que a cada dígito binario, 0_L ó 1_L , se le asigna uno de los niveles de señal, ya sea cero o un nivel A , dependiendo de la lógica utilizada. En la figura 1.6 se presenta un ejemplo de codificación de lógica positiva ya que el mayor nivel de señal (A) se lo ha asignado al 1_L^1 .

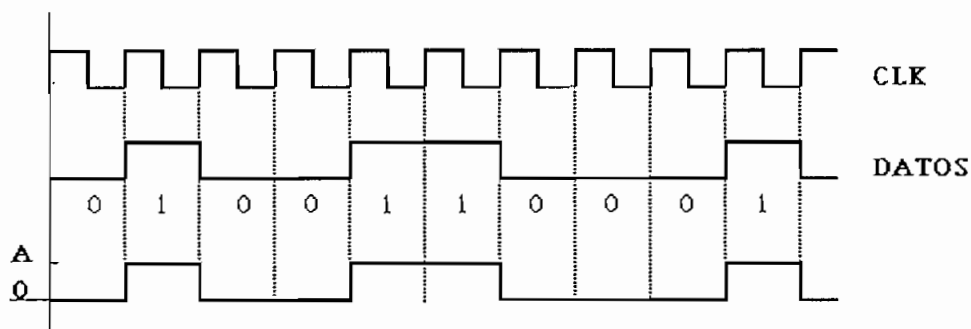


Fig. 1.6. Codificación NRZ-neutral

1.3.2. Código NRZ-polar.

La codificación se logra asignando un pulso positivo a uno de los dígitos binarios, y un pulso negativo al otro dígito binario. Igualmente, se puede tener una codificación de lógica positiva o negativa. En la figura 1.7. se muestra un ejemplo de codificación NRZ-polar con lógica negativa.

¹ VIDALLER L.Y OTROS, Op. Cit, p. 124.

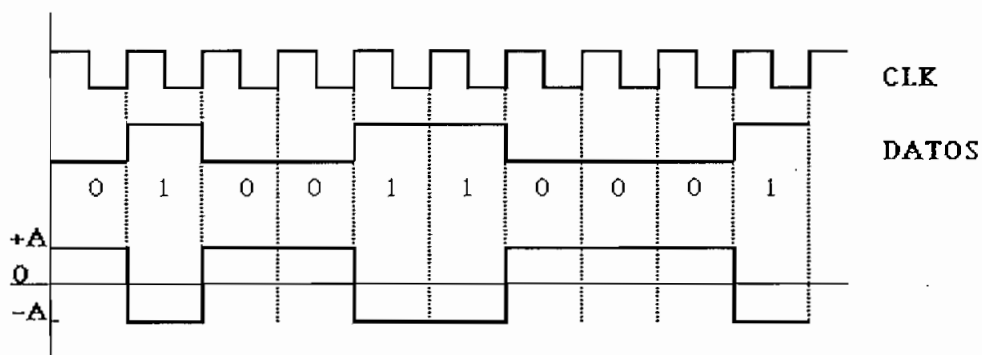


Fig. 1.7. Codificación NRZ-polar

1.3.3. Código AMI.

El código AMI (*Alternate Mark Inversion*) es un código de línea que emplea una señal ternaria para transportar dígitos binarios. Existen dos posibilidades, en la primera, los 'unos' binarios consecutivos están representados por elementos de señal cuya polaridad alterna normalmente entre positiva y negativa, teniendo la misma amplitud; los 'ceros' binarios están representados por elementos de señal de amplitud nula¹.

El 0_L es codificado como un nivel de amplitud cero, en tanto que para la codificación del 1_L es necesario conocer la polaridad del pulso que antecedió al actual. Si el pulso anterior fue positivo, el actual 1_L se codificará como un pulso negativo; en tanto que si el anterior pulso fue negativo, el presente se codificará como positivo.

Como segunda posibilidad, se tiene el código AMI invertido en el cual será el 0_L el que produzca la alternabilidad de pulsos, en tanto que el 1_L mantendrá la salida en un nivel de cero.

Este código puede ser además del tipo AMI-NRZ si el

¹ CCITT, Recomendaciones - Libro Rojo, tomo III, fascículo III.3, Rec. G.701, Málaga, 1984, p. 36.

pulso tiene la duración de un período de reloj, o AMI-RZ si el pulso tiene una duración menor que la de un período de reloj. Desde el punto de vista de la recuperación de la señal de reloj, el uno o el otro no presentan mucha diferencia, sin embargo, cuando se analiza los requerimientos de ancho de banda, se debe tomar muy en cuenta cual de los dos se adopta, ya que su densidad espectral de potencia es diferente. En la figura 1.8 se puede observar un ejemplo de codificación AMI.

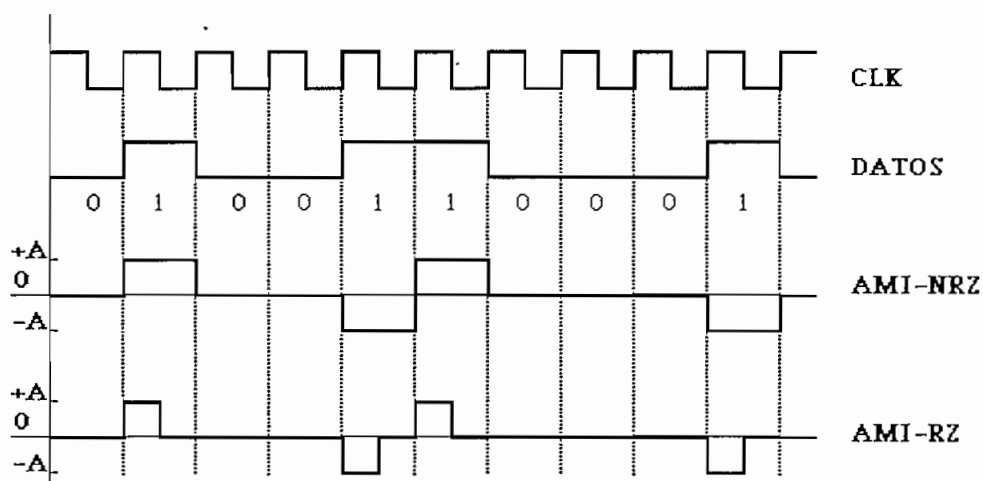


Fig. 1.8. Codificación AMI

Dado que se tendrá siempre una alternabilidad en la polaridad de los pulsos, se puede usar esta característica como una forma de control de errores, ya que si se tienen dos pulsos (separados o no por niveles de cero), que tengan la misma polaridad se determinará la ocurrencia de un error.

Existe sin embargo una desventaja en el uso del código AMI, la misma que se presenta cuando la ocurrencia de bits que mantienen la señal en el nivel cero es demasiado sostenida y por tanto no se tendrán suficientes transiciones que permitan una adecuada recuperación de la señal de reloj, esto puede ocasionar la pérdida de sincronismo.

1.3.4. Códigos diferenciales.

1.3.4.1. Código diferencial NRZ tipo M.

Para codificar un flujo de datos binarios según este código, cuando se tiene un 1, se deberá cambiar el nivel presente de la señal codificada, de modo que si estaba en alto se ponga en bajo y viceversa. Por el contrario, cuando se codifica un 0, se mantendrá el mismo nivel anterior. En la figura 1.9 se representa una secuencia binaria codificada con este método, tanto para el caso neutral como para el caso polar.

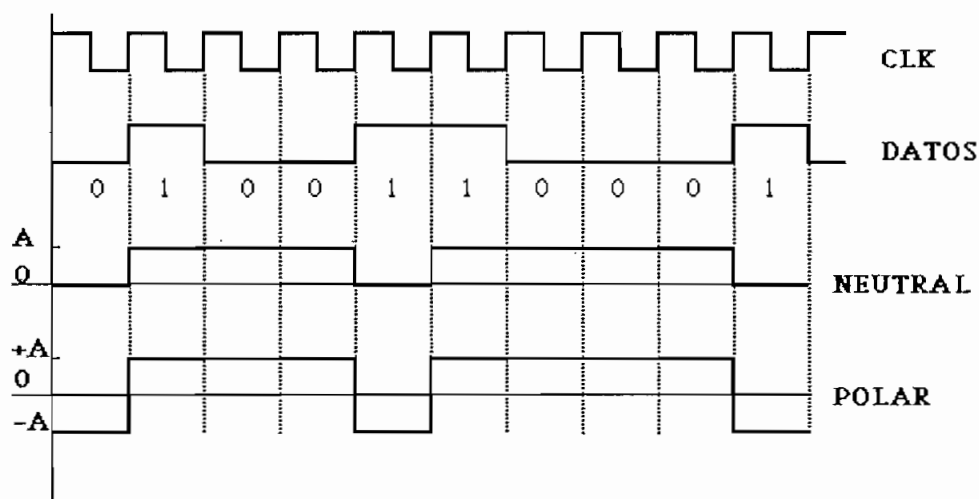


Fig. 1.9. Codificación diferencial NRZ tipo M

1.3.4.2. Código diferencial NRZ tipo S.

Es un código similar al anterior, con la única diferencia que es el 0, el que produce el cambio de nivel. En la figura 1.10 se ilustra el algoritmo de codificación mencionado, tanto para el caso unipolar como polar.

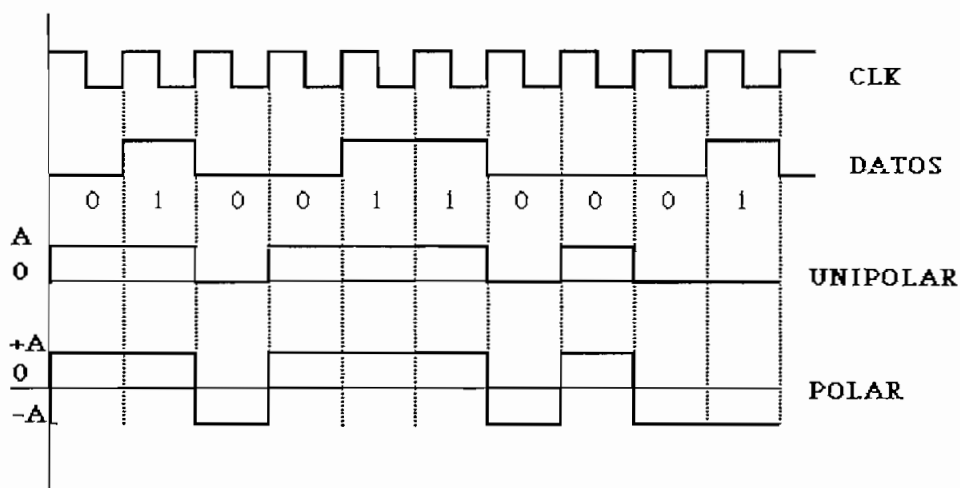


Fig. 1.10. Codificación diferencial NRZ tipo S

1.3.5. Códigos bifase.

1.3.5.1. Código bifase-L (*Biphase-Level*) o Manchester.

Este código basa su algoritmo en la transición que se produce entre dos niveles extremos, uno positivo ($+A$) y otro negativo ($-A$). Para ello, cada período de bit se divide en dos intervalos iguales. Un dígito binario con valor 1_L se envía con un nivel alto durante el primer intervalo, y bajo durante el segundo. Un bit binario de valor 0_L produce el efecto contrario, es decir, primero se tiene un nivel bajo y después uno alto. Con este esquema se asegura que todos los períodos de bit tengan una transición en la parte media, permitiendo así un mejor sincronismo entre el transmisor y receptor. En la figura 1.11(a) se muestra la codificación Manchester.

Existe además una variación de la codificación Manchester básica y se la conoce como **codificación Manchester diferencial** la cual se indica en la figura 1.11(b); en ésta, un bit 1_L se indica por la ausencia de transición al inicio del período, y un bit 0_L se indica por la presencia de una transición al inicio del intervalo. En

ambos casos, existe una transición en la mitad del intervalo de bit. El esquema diferencial exige un equipo más sofisticado, pero ofrece mayor inmunidad al ruido¹. Resumiendo esta regla de codificación se tiene que para un 0_L la fase del elemento de señal es la misma que la del elemento precedente, en tanto que cuando se tiene un 1_L, la fase del elemento de señal es opuesta.

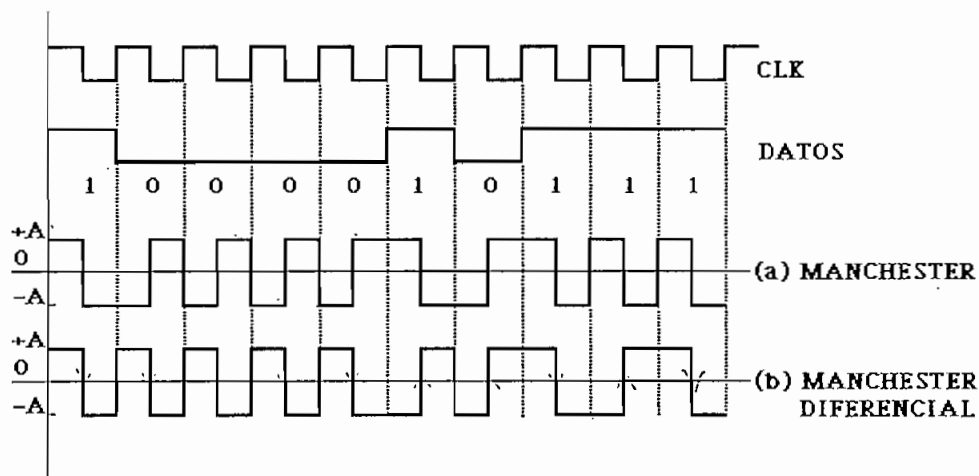


Fig. 1.11. Codificación Manchester y Manchester Diferencial

1.3.5.2. Código bifase-M (*Biphase-Mark*).

Este es un código de dos niveles, uno positivo y otro negativo, de la misma amplitud. Una transición aparece siempre al principio del intervalo. El símbolo 1_L produce otra transición medio período después, en tanto que el símbolo 0_L no produce transición. En la figura 1.12 se muestra esta forma de codificación.

¹ TANENBAUM A., Op. Cit., p. 68.

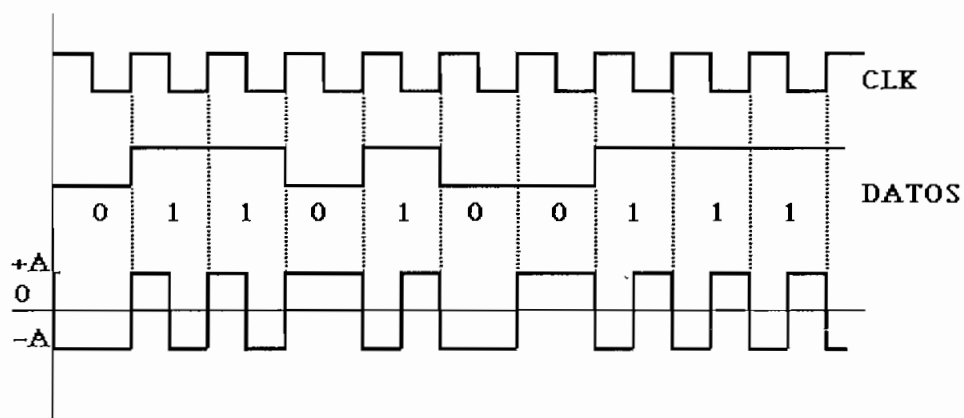


Fig. 1.12. Codificación bifase-M

1.3.5.3. Código bifase-S (*Biphase-Space*).

Es el código complementario al bifase-M, por tanto es el símbolo 0_L el que produce la transición a la mitad del periodo en tanto que el 1_L no produce transición, tal como se observa en la figura 1.13.

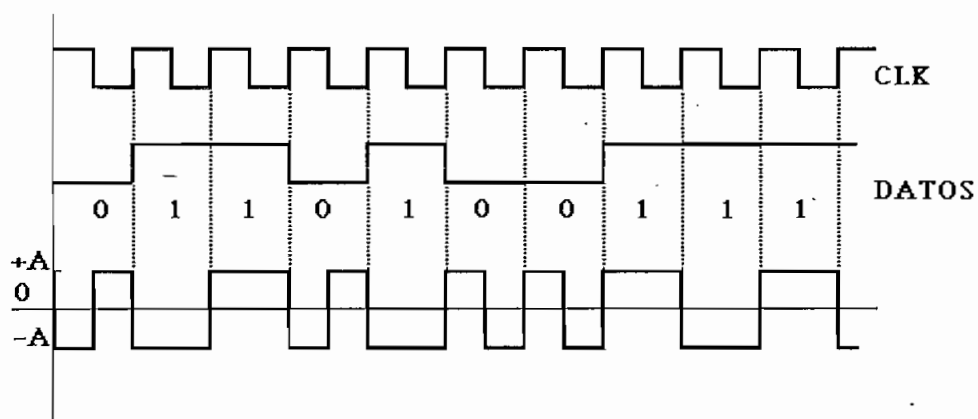


Fig. 1.13. Codificación bifase-S

1.3.5.4. Código CMI.

El código CMI (*Coded Mark Inversion*) o codificación por inversión de marca es similar a los códigos bifase

descritos anteriormente y está recomendado por el CCITT¹. En éste, el 0_L en lugar de ser representado por una ausencia de señal, es codificado con un cambio de polaridad de negativo a positivo, lo que ocurre a la mitad del período.

El símbolo 1_L es enviado de modo que los niveles positivo y negativo se obtienen alternadamente cada uno durante un período. En la figura 1.14 se da un ejemplo de codificación CMI.

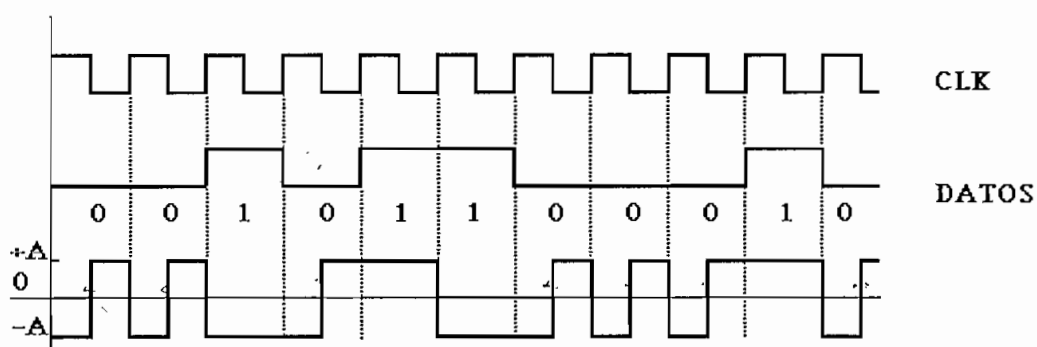


Fig. 1.14. Codificación CMI

1.3.6. Código de modulación por retardo o código Miller.

Se fundamenta en la existencia de transiciones entre dos niveles $+A$ y $-A$. El símbolo 1_L produce una transición en el punto medio del período. El símbolo 0_L no produce ninguna transición, a no ser que vaya seguido por otro 0_L en cuyo caso se produce una transición entre los dos ceros, al final del primer período².

¹ CCITT, Recomendaciones - Libro Rojo, tomo III, fascículo III.3, Rec. G.703, Málaga, 1984, p. 64.

² VIDALLER L. y OTROS, Op. Cit., p. 135.

Un ejemplo de codificación con este algoritmo se muestra en la figura 1.15.

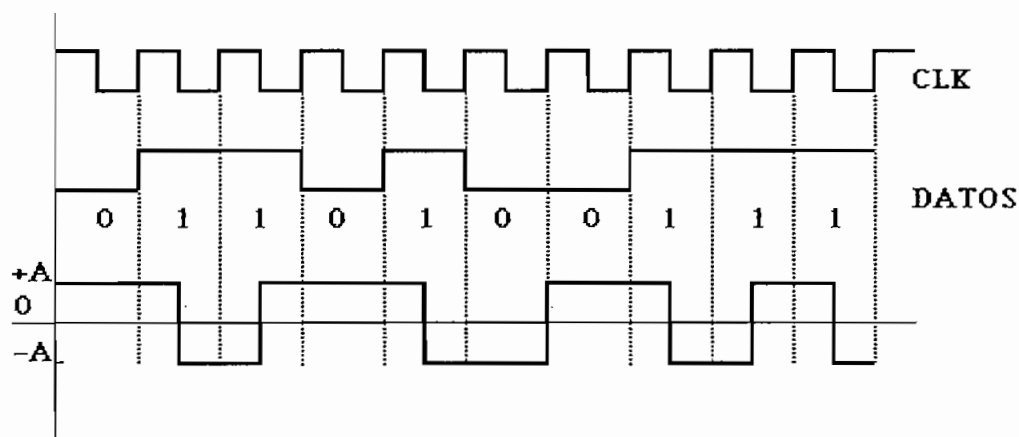
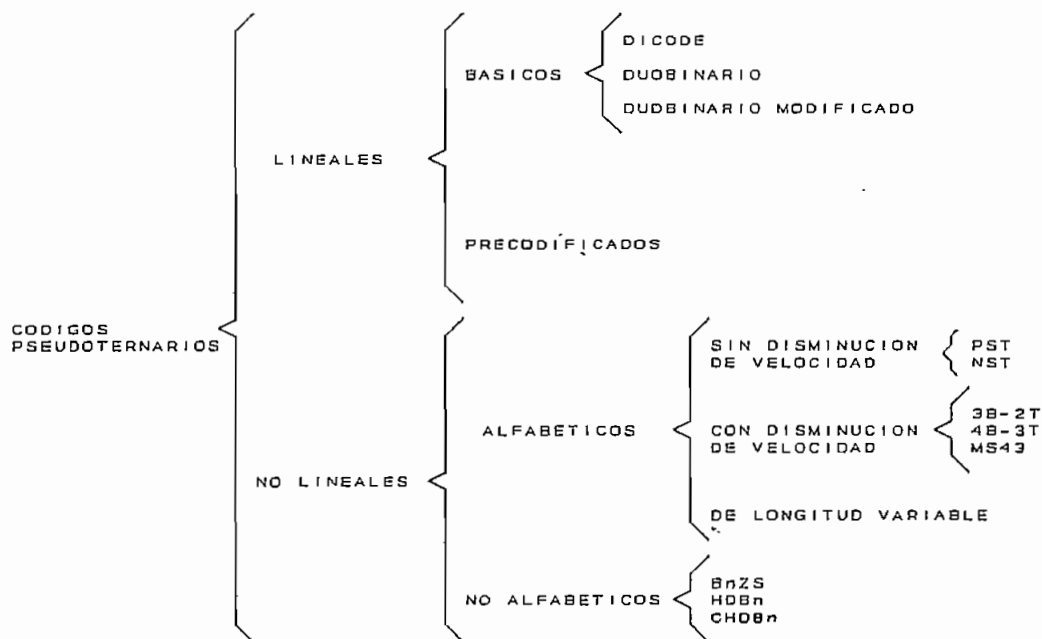


Fig. 1.15. Codificación Miller

1.3.7. Códigos pseudoternarios.

Estos códigos se caracterizan porque la secuencia binaria de entrada produce a la salida una secuencia dada, cuyos elementos de señal serán escogidos de un conjunto de señales de tres niveles. Esta secuencia está formada por elementos de señal que son calculados en base al dígito binario actual y a un número fijo de elementos binarios precedentes al dígito binario que se está codificando.

Para el tratamiento de los algoritmos de codificación de los esquemas pseudoternarios, es necesario tener presente su clasificación por lo que ésta se esquematiza de la siguiente manera:



1.3.7.1. Códigos pseudoternarios lineales.

En general, la codificación (y por tanto la decodificación), consiste en obtener una secuencia de elementos de señal de salida como consecuencia de una secuencia de entrada. Si se representa como $\{a_n\}$ la secuencia de entrada y como $\{b_n\}$ la secuencia de salida, se podrá establecer una transformación lineal para relacionar las dos secuencias de la siguiente manera:

$$b_n = \sum_{i=0}^N c_i a_{n-i} \quad [1.2]$$

donde $\{c_i\}$ son los coeficientes de ponderación que definen el código utilizado; N es el número de elementos de la secuencia de entrada, empleados en el cálculo del elemento de salida.

Esta expresión general permite convertir la secuencia binaria en secuencias de diferente número de

niveles, dependiendo de la cantidad y valores de los coeficientes utilizados. En lo que se refiere al caso ternario (o de tres niveles), sólo debe haber dos coeficientes distintos de cero, e iguales entre sí en módulo.

De entre los códigos pseudoternarios lineales se distinguen dos grupos, los básicos y los precodificados.

a. Códigos pseudoternarios lineales básicos.

Dentro de esta clasificación, los más importantes son:

1. Código dicode;
2. Código duobinario;
3. Código duobinario modificado.

1. Código dicode.

Para esta codificación se cumple que en la ecuación [1.2]:

$$N = 1, C_0 = A, C_1 = -A$$

donde $+A$ y $-A$ son los niveles de señal entre los cuales estará variando la secuencia de salida. Para estas condiciones, la ecuación [1.2] puede reescribirse como:

$$\begin{aligned} b_n &= Aa_n - Aa_{n-1} \\ b_n &= A(a_n - a_{n-1}) \end{aligned} \quad [1.3]$$

A partir de esta última expresión puede observarse que la codificación requiere del conocimiento del bit anterior y del actual que se está codificando, tal como se indica en el ejemplo de la figura 1.16.

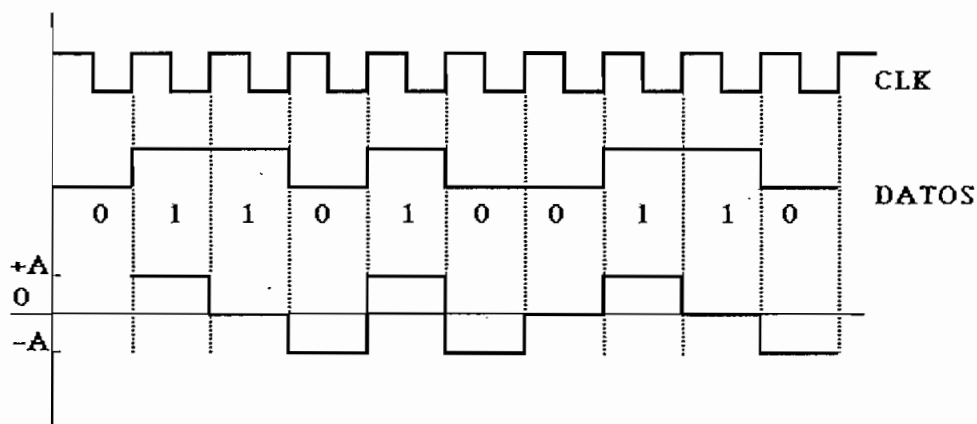


Fig. 1.16. Codificación dicode

De esta figura se deduce que aparecerá un pulso positivo o negativo cada vez que haya una transición de 0_L a 1_L ó de 1_L a 0_L , respectivamente; en el caso de dos bits del mismo valor no aparecerá ningún pulso.

2. Código duobinario.

También recibe el nombre de biternario o polibinario, aunque el más extendido es el de duobinario, que hace referencia al hecho de duplicar la velocidad del binario. Para este código, en la ecuación [1.2] se debe hacer:

$$N = 1, C_0 = A/2, C_1 = A/2$$

con lo cual la ecuación de codificación será la siguiente:

$$b_n = \frac{A}{2}a_n + \frac{A}{2}a_{n-1} \quad [1.4]$$

$$b_n = \frac{A}{2}(a_n + a_{n-1})$$

Para cumplir con el requerimiento de que la señal deba tener los niveles positivo (+A), negativo (-A) y nulo

(0), en la ecuación [1.4] se adopta el convenio de que cuando se trata de un 1_L , $a_n = 1$ y cuando se tiene un 0_L , $a_n = -1$. En la figura 1.17 se tiene un ejemplo de codificación duobinaria.

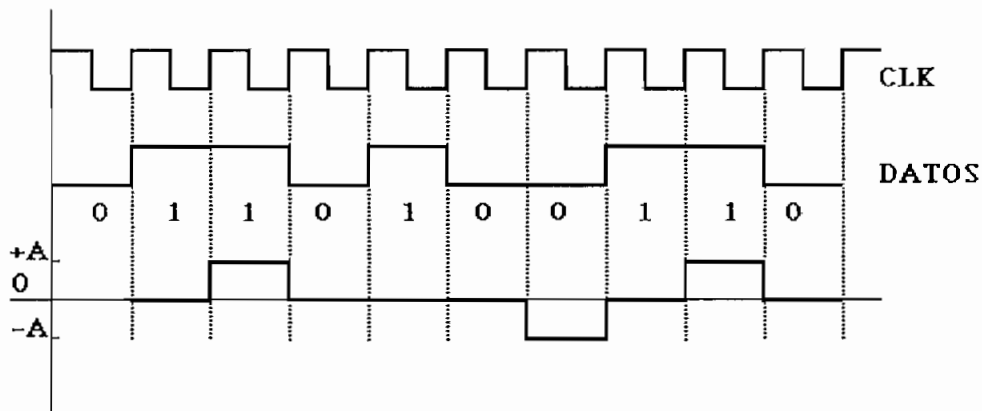


Fig. 1.17. Codificación duobinaria

Al igual que para la codificación dicode, el primer dígito binario no puede ser codificado ya que se desconoce el bit inmediatamente anterior.

3. Código duobinario modificado.

Se tiene este código, si en la ecuación [1.2] se hace:

$$N = 2, C_0 = A, C_1 = 0, C_2 = -A$$

y la ecuación resultante será:

$$b_n = Aa_n - Aa_{n-2} \quad [1.5]$$

$$b_n = A(a_n - a_{n-2})$$

La figura 1.18 muestra el resultado de una codificación usando la ecuación [1.5]. Se puede notar que debido a que se necesita conocer el segundo bit anterior, los dos primeros bits de la secuencia codificada no tendrán

como salida un elemento de señal definido.

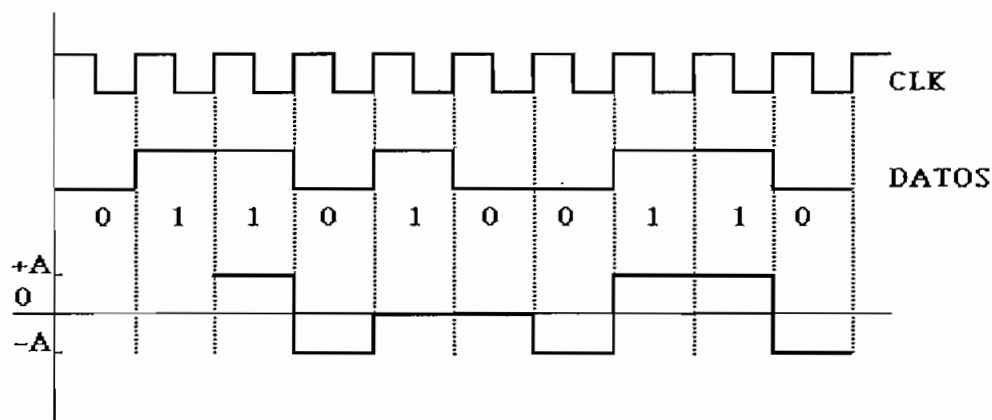


Fig 1.18. Codificación duobinaria modificada

b. Códigos pseudoternarios lineales precodificados.

El principal inconveniente que se encuentra en la utilización de los códigos básicos es la propagación de errores, ya que para obtener la secuencia de salida se requiere conocer el bit anterior y cuando se produce un error en la detección de un símbolo, este error se propaga hasta que vuelva a producirse otro error, lo cual puede deducirse de las ecuaciones [1.3], [1.4] y [1.5].

Se busca entonces establecer una relación directa e independiente entre la secuencia de datos a_k y la secuencia codificada b_k , de tal forma que para la detección sea suficiente muestrear la señal recibida $b(t)$, sin tener que utilizar los pulsos decodificados anteriormente. La solución a este problema es realizar una precodificación antes de codificar. Con este fin, a partir de la secuencia a_k se forma otra secuencia de bits d_k , tal que:

$$a_k = d_k \oplus d_{k-1} \quad [1.6]$$

donde $\oplus$ representa la operación suma módulo dos. Los

elementos de señal a transmitirse no serán los a_k sino los d_k , a los cuales se aplicará algún código pseudoternario lineal básico.

El precodificador debe cumplir con la ecuación [1.6] o su equivalente:

$$\begin{aligned} a_k &= d_k \oplus d_{k-1} \\ a_k \oplus d_{k-1} &= (d_k \oplus d_{k-1}) \oplus d_{k-1} \\ a_k \oplus d_{k-1} &= d_k \oplus 0 \end{aligned} \quad [1.7]$$

$$d_k = a_k \oplus d_{k-1} \quad [1.8]$$

Como ejemplo se propone el código dicode precodificado. Al aplicar los coeficientes c_n para este código, la ecuación de codificación se presenta como:

$$\begin{aligned} b_n &= Ad_n - Ad_{n-1} \\ b_n &= A(d_n - d_{n-1}) \end{aligned} \quad [1.9]$$

En la figura 1.19 se presenta un ejemplo de este tipo de codificación.

Es de notar, que cuando se usa el esquema dicode precodificado, en la respuesta $\{b_n\}$ aparece un pulso cuya polaridad alterna entre positivo y negativo, cada vez que hay un 1_L en la secuencia de entrada. De lo que antecede se concluye que la codificación dicode-precodificada es idéntica a la codificación AMI¹.

¹ VIDALLER L. y OTROS, Op. Cit., p. 137 - 143.

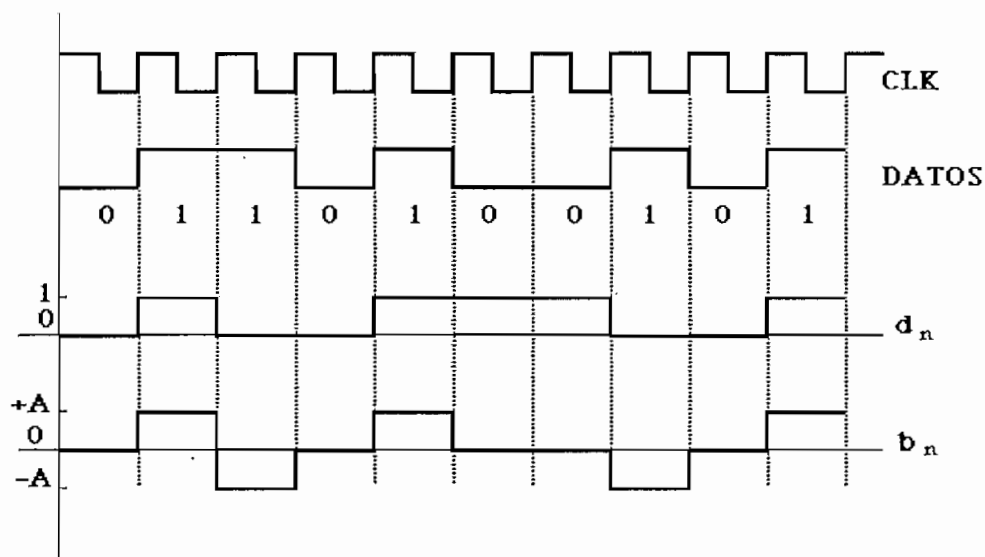


Fig. 1.19. Codificación dicode precodificada

El proceso de decodificación implicará el uso de una doble decodificación, el primer algoritmo permitirá una decodificación del código pseudoternario lineal aplicado; y el segundo, a partir de la regla de precodificación, permitirá obtener la secuencia binaria original.

1.3.7.2. Códigos pseudoternarios no lineales.

a. Códigos pseudoternarios no lineales alfabéticos.

A este grupo pertenecen los códigos basados en asignar a un grupo de m dígitos binarios, otro grupo de n dígitos ternarios. Estos códigos son generalmente conocidos como códigos $mB - nT$. A este conjunto de m bits, independientemente de su número, se le denomina caracter o palabra binaria. En todo momento se debe cumplir que el número de palabras binarias a codificar, sea menor o igual que el número de palabras ternarias, es decir:

$$2^m \leq 3^n \quad [1.10]$$

$$m \leq n \frac{\log 3}{\log 2} \quad [1.11]$$

$$m \leq 1.58n$$

Puesto que la relación existente entre la velocidad de transmisión codificada v_c y la velocidad de transmisión binaria v_t es:

$$\begin{aligned} v_t &= v_c \log_2 M \\ v_c &= v_t \frac{\log 2}{\log M} \end{aligned} \quad [1.12]$$

donde M es el número de niveles con que se va a codificar la señal binaria. Si $M=3$ se tendrá:

$$\begin{aligned} v_c &= 0.63 v_t \\ v_t &= 1.58 v_c \end{aligned}$$

Por lo que un código que use señales ternarias para codificar dígitos binarios puede alcanzar una disminución máxima de un 37% en la velocidad de transmisión codificada, respecto a la de transmisión binaria; o lo que es lo mismo, transmitiendo a tres niveles en lugar de dos, el ritmo de transmisión binaria puede incrementarse hasta en un 58%; caso ideal al que se llegaría si la relación del número de dígitos de la palabra binaria al número de dígitos de la palabra ternaria fuese de 1.58.

Debido a que es necesario dividir la secuencia de bits en caracteres, este tipo de códigos presentan el inconveniente de necesitar un sincronismo de carácter.

1. Códigos sin disminución de velocidad de transmisión.

Se tiene este caso cuando el número de elementos de señal codificados es igual al número de bits del caracter, es decir si $m = n$.

El código más conocido, de entre los de este tipo, es el denominado PST (*Pair Selected Ternary*) que se conoce también como **parejas ternarias seleccionadas**.

La codificación PST procesa pares de datos binarios de entrada para producir secuencias de caracteres de dos dígitos ternarios, los mismos que serán transmitidos.

Puesto que existen nueve códigos de dos dígitos ternarios y sólo cuatro caracteres de dos dígitos binarios, existe una considerable flexibilidad en la selección de los códigos. En la tabla 1.1 se muestra el formato más usual de todos los posibles.

Entrada binaria	Modo positivo	Modo negativo
00	-+	-+
01	0+	0-
10	+0	-0
11	+-	+-

Tabla 1.1. Equivalencia para el código PST

Este formato en particular, no sólo que asegura una gran cantidad de transiciones, sino que también disminuye la componente continua al producirse el cambio entre el modo positivo y negativo, lo cual permite mantener el balance entre pulsos positivos y negativos.

Para realizar la codificación, se selecciona inicialmente uno de los modos hasta que se transmita un pulso simple. En este punto, el codificador conmuta el

modo y selecciona códigos apropiados hasta que igualmente se transmita un pulso simple (de polaridad opuesta)¹.

Para ilustrar el algoritmo de codificación, se presenta un ejemplo en la figura 1.20.

Como se ve, una desventaja potencial de la codificación PST es que el flujo de bits debe ser arreglado en pares. Por lo tanto el decodificador PST debe reconocer los pares de elementos de señal para que pueda asignar los bits correctamente. Esto no es difícil ya que se puede reconocer que se tiene un error cuando ocurren códigos no permitidos (00, ++, --). Además, cuando se emplea la técnica de multiplexación por división de tiempo, se provee de caracteres de alineación que permiten mantener el sincronismo. Una generalización de estos códigos, tomando N dígitos binarios y N dígitos ternarios da origen a los denominados códigos NST (*N bits Selected ternary*).

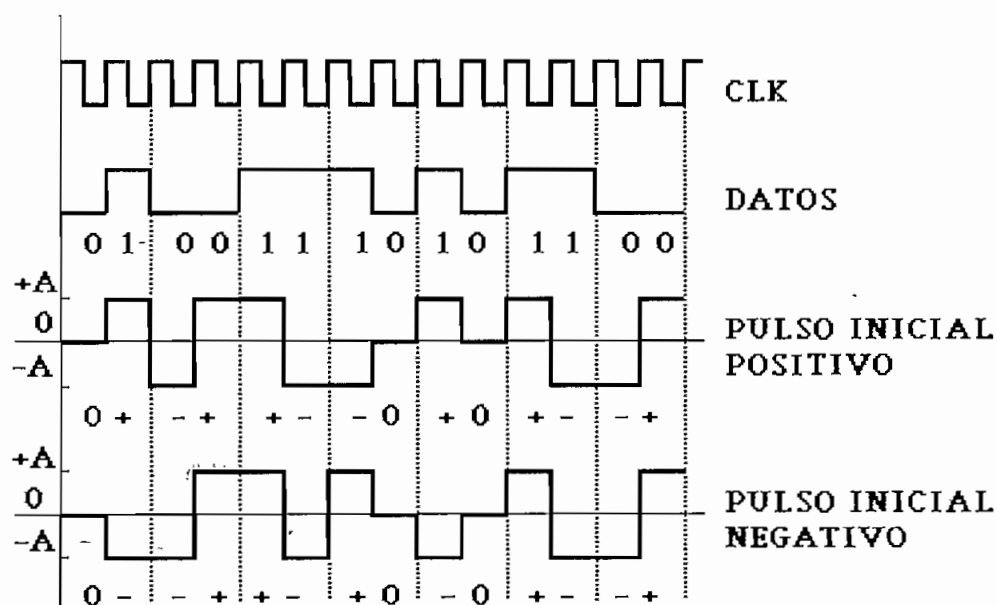


Fig. 1.20. Codificación PST

¹ Bellamy J., *Digital telephony*, John Wiley & Sons, New York, 1982, p. 179, 180.

2. Códigos con disminución de velocidad de transmisión.

• Código 3B - 2T.

Con la utilización de este tipo de codificación, se disminuye la velocidad de transmisión codificada a una relación de $2/3$ con respecto a la velocidad de transmisión binaria, lo que corresponde a una reducción del 90% (0.33) del límite máximo teórico (0.37).

Existen para este código ocho combinaciones binarias y nueve combinaciones ternarias, por lo cual, una de las combinaciones ternarias no se utiliza. Su elección depende de las características de diseño, siendo generalmente la 00.

• Código 4B-3T.

Este código produce una sustitución de grupos de cuatro bits por grupos de tres dígitos ternarios. Puesto que los grupos binarios de cuatro bits requieren sólo 16 de las 27 posibles combinaciones ternarias, existe gran flexibilidad en la selección de los códigos ternarios. En la tabla 1.2 se presenta un posible procedimiento de codificación propuesto por Jessop-Waters¹.

Cuando se trata de codificar una secuencia binaria usando el algoritmo 4B-3T, el objetivo será mantener la "disparidad" de componente positiva y negativa en cero, con ello se logrará que la componente continua sea mínima.

¹ OWEN F., PCM and digital transmission systems, McGRAW-HILL, New York, 1982, p. 223-224.

Entrada binaria (4B)	Palabra ternaria (3T)		Disparidad acumulada
	Modo positivo	Modo negativo	
0000	0-+	0-+	0
0001	-+0	-+0	0
0010	-0+	-0+	0
0011	+--+	+--+	1
0100	0++	0--	2
0101	0+0	0-0	1
0110	00+	00-	1
0111	-++	+--	1
1000	0+-	0+-	0
1001	+ -0	+ -0	0
1010	+0-	+0-	0
1011	+00	-00	1
1100	+0+	-0-	2
1101	++0	--0	2
1110	++-	---+	1
1111	+++	---	3

Tabla 1.2. Codificación 4B-3T

Para lograr este objetivo, se debe considerar la disparidad acumulada que se tiene al momento de realizar la codificación de un grupo de cuatro bits. Si la disparidad es positiva, se elegirá el modo negativo para la codificación; en tanto que si la disparidad es negativa, se usará el modo positivo para la codificación.

Con la finalidad de elegir el modo correcto, es preciso tener en cada momento el valor de la disparidad acumulada; para esto es útil el diagrama de estados que se presenta en la figura 1.21. En éste debe observarse que se trata en todo momento de equilibrar la disparidad existente, la cual debería estar fluctuando, al menos teóricamente, entre +3 y -3. Si se llegara a tener una disparidad de cero, sería indiferente el modo en el cual se codifiquen los siguientes cuatro bits, sin embargo, lo más acertado sería que se codifique con la polaridad opuesta a la última realizada.

Por esta razón, en la figura 1.21 nunca se llega a una disparidad de cero y siempre existe una disparidad acumulada que obligará a que la codificación siguiente se realice de acuerdo al criterio anteriormente expuesto.

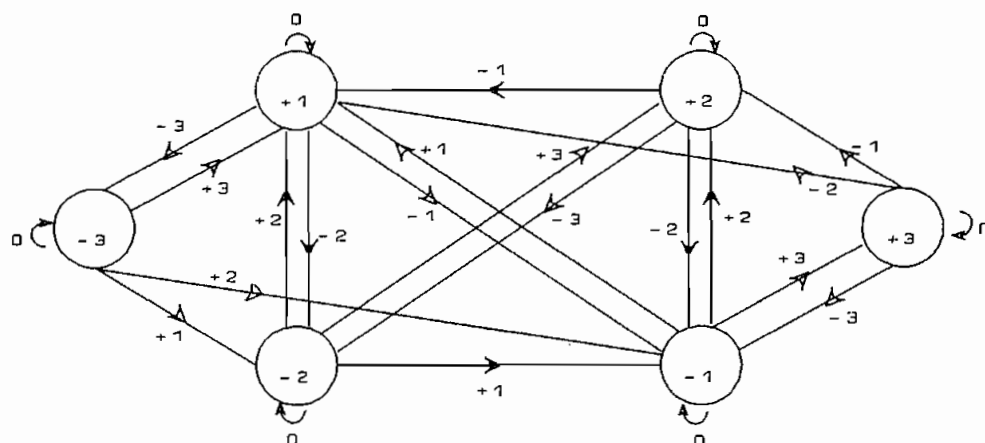


Fig. 1.21. Diagrama de transición de estados del código 4B - 3T

A continuación se expone un ejemplo de codificación en 4B-3T, en el cual se supone que la disparidad inicial acumulada es +2.

Entrada binaria	0000	1000	0110	0111	1011	0101	1111	0000	0000	0000	1101
Salida ternaria	0+	0+	00-	+-	+00	0-0	+++	0+	0+	0+	--0
Disparidad actual	+2	+2	+1	-1	+1	-1	+3	+3	+3	+3	+1

Ejemplo de codificación 4B-3T

Código MS43.

El código MS43, propuesto por Franaszek, tiene un concepto similar al 4B-3T descrito anteriormente. En este caso, cuatro dígitos binarios se codifican para dar una secuencia ternaria de tres dígitos, existen sin embargo más de dos modos de codificación.

En la estructura original del código propuesto por Franaszek se definían tres posibles modos de codificación, sin embargo se considera aquí el esquema modificado que ha sido adoptado ampliamente en los sistemas de transmisión digital. En el cuadro 1.3 se observa la tabla de codificación y en la figura 1.22 se presenta el diagrama de transición.

La composición de la codificación está estructurada de modo que cada grupo de tres símbolos ternarios es asociado unívocamente con un grupo de cuatro dígitos binarios, y por lo tanto no es necesario que el decodificador identifique el modo o el alfabeto empleado por el transmisor¹.

Entrada binaria	Palabra ternaria				Disparidad acumulada
	Modo 1	Modo 2	Modo 3	Modo 4	
0000	+++	-+-	-+-	-+-	+3, -1, -1, -1
0001	++0	00-	00-	00-	+2, -1, -1, -1
0010	+0+	0-0	0-0	0-0	+2, -1, -1, -1
0011	0-+	0-+	0-+	0-+	0
0100	0++	-00	-00	-00	+2, -1, -1, -1
0101	-0+	-0+	-0+	-0+	0
0110	-+0	-+0	-+0	-+0	0
0111	-++	-++	--+	--+	+1, +1, -1, -1
1000	+--	+--	+--	---	+1, +1, +1, -3
1001	00+	00+	00+	--0	+1, +1, +1, -2
1010	0+0	0+0	0+0	-0-	+1, +1, +1, -2
1011	0+-	0+-	0+-	0+-	0
1100	+00	+00	+00	0--	+1, +1, +1, -2
1101	+0-	+0-	+0-	+0-	0
1110	+ -0	+ -0	+ -0	+ -0	0
1111	++-	++-	+--	+--	+1, +1, -1, -1

Tabla 1.3. Codificación MS43

¹ BYLANSKY P. e INGRAM D., Digital transmission systems, IEE Telecommunications Series 4, England, 1979, p. 243, 244.

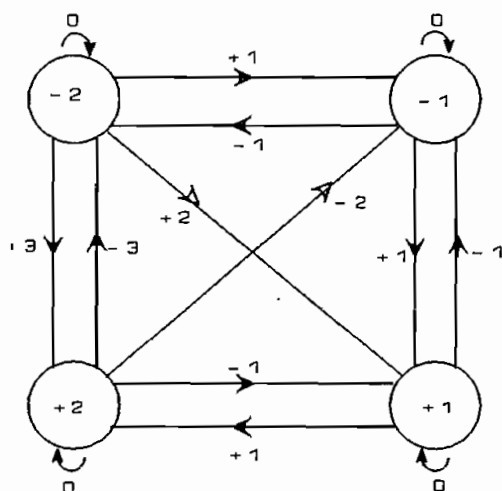


Fig. 1.22. Diagrama de transición MS43

Las características globales del código MS43 son en algún modo mejores que las del código 4B-3T, a pesar de que esto es pagado con un incremento en la complejidad de la implementación de los equipos terminales. En particular, se mejora la fluctuación de frecuencia en el reloj de muestreo (*jitter*) y la resistencia a errores ocasionales por pérdida de sincronismo. Además, la disparidad es limitada a un rango de ± 2 en lugar de ± 3 .

Como ejemplo, se plantea el ya expuesto para el esquema 4B-3T con una disparidad inicial de +2, su codificación con MS43 se expone a continuación.

Entrada binaria	0000 1000 0110 0111 1011 0101 1111 0000 0000 0000 1101
Salida ternaria	-+ +-+ -+0 -+ 0+- -0+ +- -+ +++ -+ +0-
Disparidad actual	+1 +2 +2 +1 +1 +1 -1 -2 +2 +1 +1

Ejemplo de codificación MS43

Como se puede observar, el hecho de disponer de cuatro alfabetos para codificar las señales binarias de entrada, permite mantener la disparidad acumulada en el rango de +2 y -2, sin embargo, se debe tener en cuenta que tanto en el código 4B-3T como en el MS43, la disparidad

acumulada puede irse del rango previsto por dos razones:

- Errores en la transmisión.
- Pérdida del alineamiento de la palabra.

Para solucionar este inconveniente, el contador de disparidad debe ser reseteado cada vez que el rango es superado¹.

Tanto para el código 4B-3T como para el MS43, la disminución del ritmo transmisión codificado está en relación 3/4 del ritmo de transmisión binario, esto corresponde a una disminución del 68% (0.25) del límite teórico (0.37).

• Códigos de longitud variable.

En estos códigos, tanto la longitud de los caracteres de la secuencia de entrada, como la de los de la secuencia de salida es variable.

Para que sean prácticos, la velocidad de transmisión de la fuente debe ser constante, y por lo tanto la longitud de los caracteres de salida debe ser proporcional a la de los de entrada. Un ejemplo de este tipo de codificación es el código VL43 de Franaszek en el que los caracteres de entrada son de 4 u 8 dígitos y los de salida² de 3 ó 6.

Las ventajas obtenidas con el código VL43, respecto al 4B-3T y MS43 son pocas, en cambio la dificultad de codificación (y decodificación) aumenta, por lo que no es considerado como un código práctico y por ello no se lo estudiará.

¹ KUSTRA R. y TUJSNAIDER O., Principios de comunicaciones digitales, Vol. II, Colección Técnica ARCIET-ICI; FICUM S.A., 1988, p. 401.

² VIDALLER L. y OTROS, Op. Cit., p.148 - 149.

b. Códigos pseudoternarios no lineales y no alfabéticos.

En estos códigos, no existe una división del alfabeto de entrada en caracteres a los que se les hace corresponder un caracter del alfabeto de salida.

Se han propuesto muchos tipos de códigos no lineales y no alfabéticos, pero solamente los "**códigos bipolares rellenos**" han tenido cierta aceptación práctica y serán éstos los que se estudien.

El problema que presenta la codificación bipolar del tipo AMI, es la pérdida de sincronismo cuando aparecen largas secuencias de ceros; por ello los códigos pseudoternarios no lineales y no alfabéticos (además del PST) se han propuesto, con la idea de superar este inconveniente.

El método general de codificación se basa en cambiar las secuencias formadas por más de n ceros, por otras de dígitos ternarios que faciliten la permanencia del sincronismo. En los receptores prácticos, la mínima secuencia de ceros que produce pérdida de sincronismo es de aproximadamente 15.

Dado que el receptor debe reconocer la secuencia de reemplazo y decodificarla, se incluyen en ésta pulsos con violación de polaridad (respecto a la alternancia en la polaridad de los pulsos).

De entre este tipo de códigos se puede encontrar los bipolares con sustitución de n ceros (BnZS) y los bipolares de alta densidad (HDBn).

1. Códigos bipolares con sustitución de n ceros.

Los códigos bipolares con sustitución de n ceros o BnZS (*Bipolar with n-Zero Substitution*) con mayor utilidad práctica son el B3ZS y el B6ZS.

• Código B3ZS.

El código B3ZS (*Bipolar with three-Zero Substitution*), conocido como código bipolar con sustitución de tres ceros, es una versión modificada del código AMI. Los dígitos binarios 1_r se codifican como pulsos que son positivos y negativos alternadamente con respecto al nivel cero. Las excepciones están constituidas por aquellos casos en que aparecen tres ceros lógicos consecutivos en el tren de bits.

En el formato B3ZS, cada bloque de tres ceros consecutivos se sustituye por B0V o 00V, donde B representa un pulso conforme a la regla AMI y V representa un pulso que viola la regla bipolar. Se elige entre B0V y 00V de tal manera que el número de pulsos B comprendidos entre dos pulsos de violación V consecutivos sea impar¹. Lo dicho anteriormente se puede resumir en la tabla 1.4.

Polaridad del pulso precedente	Nº de 1 _r desde la última sustitución	
	Impar	Par
-	00-	+0+
+	00+	-0-

Tabla 1.4. Regla de sustitución B3ZS

Con este código se aumenta la densidad mínima de

¹ CCITT, Recomendaciones - Libro Rojo, tomo III, fascículo III.3, Rec. G.703, Málaga, 1984, p. 57, 58.

pulsos en línea. La densidad mínima de marcas (positivas y negativas) es aproximadamente de 33% (una marca por cada dos ceros) en tanto que la densidad promedio está sobre el 60% (un cero por cada dos marcas). Por esta razón, el código B3ZS garantiza una temporización aceptable. Hay que notar que todos los códigos BnZS garantizan una información de temporización continua, sin considerar restricciones sobre la fuente de datos, por esta razón se aplica el código de una manera completamente transparente.

A continuación se plantea un ejemplo de codificación B3ZS, donde el caso par supone que el número inicial de l_e es par, en tanto que el impar supone lo contrario.

Entrada binaria	101 000 11 000 000 001 000 1
Caso par	+0- +0+ -+ -0- +0+ 00- 00- +
Caso impar	+0- 00- +- +0+ -0- 00+ 00+ -

Ejemplo de codificación B3ZS

• Código B6ZS.

El principio de codificación es el expuesto para B3ZS con la diferencia de que la secuencia a ser reemplazada debe estar conformada por seis ceros seguidos. La regla de sustitución se la da en la tabla 1.5.

Polaridad del pulso que precede inmediatamente a los 6 ceros a sustituirse	Sustitución
-	0-+0+-
+	0+-0-+

Tabla 1.5. Regla de sustitución B6ZS

Un ejemplo de codificación usando B6ZS,

considerando dos posibilidades, se presenta a continuación.

Entrada binaria	1 000000 1 0 11 000000 000000 000 1
Polaridad negativa	- 0-+0+- + 0 -+ 0+-0-+ 0+-0-+ 000 -
Polaridad positiva	+ 0+-0-+ - 0 +- 0-+0+- 0-+0+- 000 +

Ejemplo de codificación B6ZS

2. Códigos bipolares de alta densidad.

Propuestos por Croisier en 1970; estos códigos tratan de obtener una gran densidad de marcas (pulsos de polaridad alternada) con la finalidad de facilitar la sincronización. Entre éstos se encuentran los códigos HDBn (*High Density Bipolar n*) y los CHDB (*Compatible High Density Bipolar*).

• Código HDBn.

El código HDBn no admite un número superior a n 0_r consecutivos para una señal en la que los 1_r son codificados siguiendo la regla AMI. Cuando ocurre un número $n+1$ de ceros consecutivos, se coloca un pulso en esta posición.

Se debe considerar que este pulso suplementario debe ser eliminado en el receptor y para ello es necesario diferenciarlo de los pulsos normales. Este pulso es transmitido con una polaridad idéntica a la del pulso que lo precede y constituye por tanto una violación de paridad (bit V).

Para conservar una componente continua nula se

deben transmitir tantas violaciones positivas como negativas en forma alternada.

Esta condición de alternabilidad de las violaciones para mantener una componente continua nula obliga a colocar un pulso de relleno cuando el pulso que precede a la actual violación no tiene polaridad opuesta a la violación anterior. Este pulso de relleno (bit B) que sigue la regla AMI se coloca en lugar del primer cero del bloque de n+1 ceros. Resumiendo el algoritmo de codificación, la secuencia de reemplazo queda:

```
B 0 ..... 0 V ó
0 0 ..... 0 V
```

Donde la alternabilidad en la polaridad de los pulsos de violación se consigue eligiendo la una o la otra secuencia de forma que el número de pulsos B comprendidos entre dos pulsos de violación V consecutivos sea siempre impar. Esta alternabilidad en la polaridad de las violaciones permite además detectar errores simples.

• El código más usado, de entre los de este tipo, es el **HDB3**, recomendado por el CCITT¹. En este código se admite un máximo de tres ceros consecutivos, el cuarto cero da lugar a que se apliquen las reglas descritas anteriormente, y que se esquematizan en la tabla 1.6.

Polaridad de pulso precedente	Nº de 1, desde la última sustitución	
	Impar	Par
-	000-	+00+
+	000+	-00-

Tabla 1.6. Regla de sustitución HDB3

¹ CCITT, Recomendaciones - Libro Rojo, tomo III, fascículo III.3, Rec. G.703, Málaga, 1984, p. 69.

Finalmente, se debe anotar la existencia de los códigos AHDBn, similares a los HDBn, salvo que utilizan las secuencias B0.....0VB y 0.....0VB.

• Código CHDBn.

Es totalmente similar al HDBn con la diferencia de que las secuencias de reemplazo serán:

0 0B 0 V ó
0 0 0 V

Lo cual, aplicado al código CHDB3 produce la regla de sustitución que se muestra en la tabla 1.7.

Polaridad de pulso precedente	Nº de 1 _r desde la última sustitución	
	Impar	Par
-	000-	0+0+
+	000+	0-0-

Tabla 1.7. Regla de sustitución CHDB3

En la figura 1.23 se representa un ejemplo de codificación HDB3 y CHDB3, donde el primer reemplazo es siempre el correspondiente a 000V ya que el número de 1_r desde el último reemplazo es indeterminado.

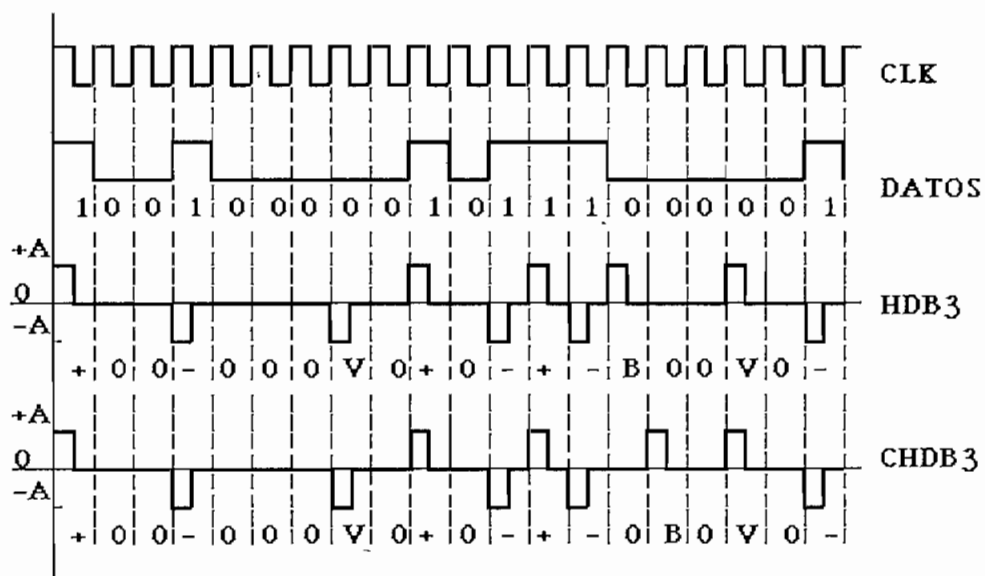


Fig. 1.23. Codificación HDB3 y CHDB3 RZ

1.3.8. Códigos entrelazados.

El entrelazado consiste en separar los símbolos pares de los impares en la secuencia de datos de entrada y codificarlos por separado, para volver a multiplexarlos a la salida, tal como se representa en la figura 1.24.

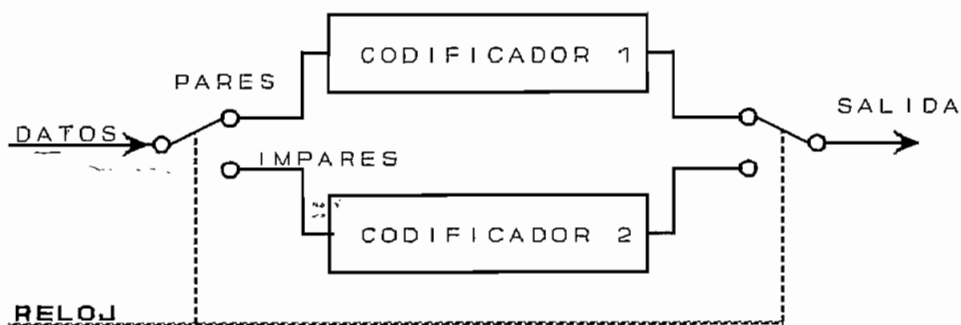


Fig. 1.24. Esquema de una codificación entrelazada

Los multiplexores de entrada y salida están gobernados por el reloj de datos y conmutan cada período. Para decodificarlos, en el extremo receptor habrá que separarlos otra vez, e introducirlos de forma independiente en cada decodificador.

En principio, los codificadores podrían ser distintos, sin embargo esta estructura no resulta práctica ya que en recepción habría que disponer de un sincronismo especial que nos indicará cuales son los que pertenecen a cada código, los pares o los impares.

Un tipo de código entrelazado es el **código bipolar transparente entrelazado (TIB)** que utiliza dos codificadores AMI. Posteriormente, luego del multiplexor de salida se realiza una codificación de reemplazo, similar a la HDBn, cambiando las secuencias de $n+1$ ceros seguidos, por la secuencia de $n+1$ dígitos:

00.....0XXVV

Las X se reemplazan por 0 ó por B de forma que el número de pulsos B (que siguen la regla AMI) entre dos V (violaciones) sucesivos sea impar en cada subcanal. El número n indica el orden del código que se nombra como TIBn. Tanto el detector de errores como el codificador son válidos¹ para cualquier orden n .

En la figura 1.25 se muestra un ejemplo de codificación TIB3, donde el máximo número de ceros seguidos admisibles es de tres. V_1 y V_2 constituyen los pulsos de violación para cada uno de los codificadores, los elementos X_1 y X_2 se usan para asegurar que el número de pulsos que siguen la regla AMI entre dos violaciones, en cada codificador sea impar (se asume que en un estado inicial

¹ VIDALLER L. y OTROS, Op. Cit., p. 154-156.

existió una violación).

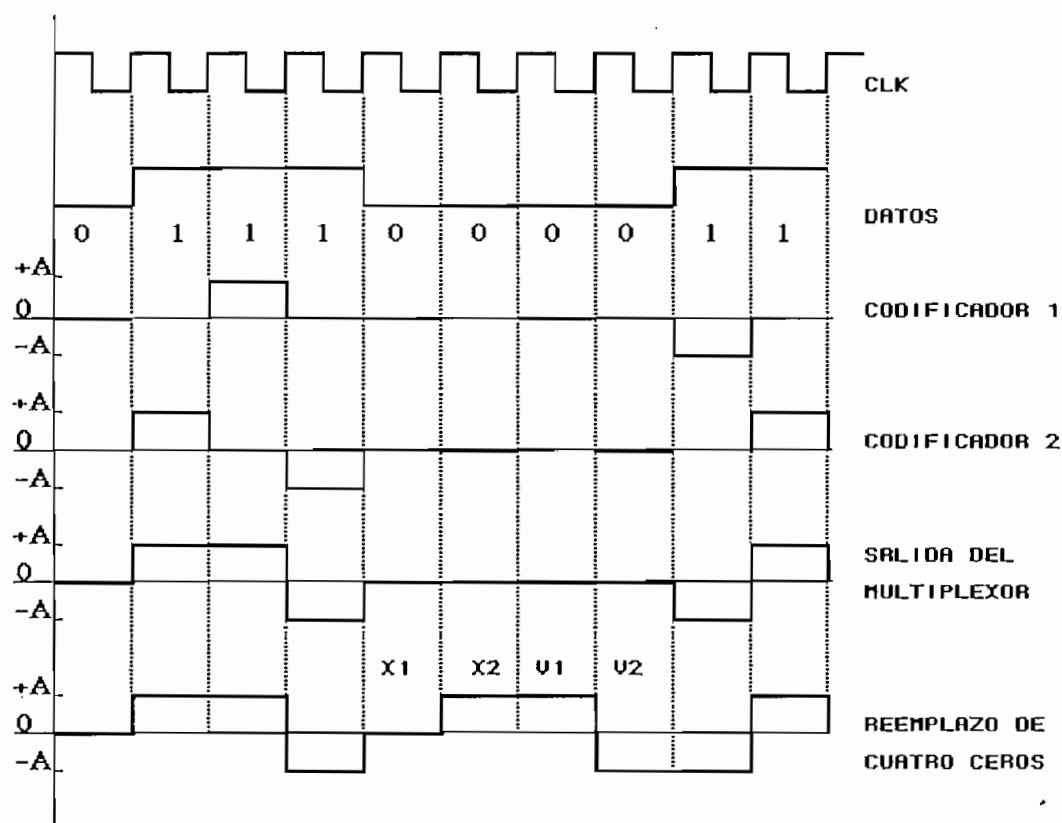


Fig. 1.25. Codificación TIB3

1.4. COMPARACION Y APLICACIONES.

El análisis precedente ha servido para estudiar la gran diversidad de códigos implementados para la transmisión de datos en banda base; sin embargo, no todos tienen la misma utilidad, por lo que es necesario establecer las características favorables y desfavorables de los códigos más usados en la práctica.

La transmisión en banda base es utilizada comúnmente en: redes locales de computadoras, usando ya sea

par trenzado o cable coaxial; en las conexiones, a través de centrales PBX digitales, de terminales, *hosts*, y teléfonos digitales; en el acceso digital a redes públicas de telecomunicaciones, sobre un "lazo digital local"; entre otras.

1.4.1. Criterios de evaluación.

Existen dos importantes tareas involucradas en la interpretación de señales digitales en el receptor. La primera es que el receptor debe conocer con cierta exactitud cuándo comienza y termina un bit. La segunda es que el receptor debe determinar si el nivel de señal para cada posición de bit es un 1_L o un 0_L .

Un cierto número de factores determina que tan exitosamente el receptor interpretará correctamente la señal de entrada, teniendo entre los más importantes a la relación señal a ruido (S/N), la velocidad de transmisión y el ancho de banda de la señal.

Si los demás factores permanecen constantes se tiene que un incremento en la velocidad de transmisión aumentará la tasa de bits erróneos o BER (**bit error rate**), mientras que un incremento en la relación señal a ruido disminuirá el BER. Además, si se incrementa el ancho de banda se puede aumentar también la velocidad de transmisión.

A todo esto hay que añadir que el esquema de codificación también determina en qué medida se mejora la calidad de la transmisión de datos, para lo cual es importante considerar los siguientes criterios de evaluación¹:

¹ IEEE Communications Magazine, Digital signaling techniques, STALLINGS W., Vol.22 (Nº 12, 1984), p. 21-25.

(a.) Espectro de la señal.

Es importante considerar la densidad espectral de potencia media de la señal a transmitirse, ya que será el indicador del ancho de banda necesario. Una ausencia de componentes de alta frecuencia implicará que el ancho de banda requerido no sea demasiado grande. Por otro lado, una ausencia de componente continua (DC) es deseable, ya que permitirá evitar un acoplamiento físico en los elementos de transmisión, y más bien utilizar un acoplamiento inductivo que provee un excelente aislamiento eléctrico reduciendo así la interferencia.

(b.) Capacidad de sincronización.

Se refiere a la capacidad que algunos códigos presentan para ayudar en la recuperación de la señal de reloj en el receptor; esto es importante ya que de otro modo sería necesario un reloj independiente para sincronizar la señal de transmisión con la señal de recepción.

(c.) Inmunidad a la interferencia y al ruido.

Algunos códigos presentan mejor comportamiento que otros, frente al ruido e interferencias, siendo en este caso necesario relacionar la tasa de bits erróneos (BER) con la relación señal a ruido (S/N).

(d.) Capacidad para detectar errores.

Hace relación a la capacidad inherente que presentan ciertos esquemas de codificación para detectar errores.

e. Costo y complejidad.

Es indudable que los mejores tipos de codificación serán más complejos y por lo tanto su implementación más costosa, se debe entonces analizar si esto es preciso para la aplicación en que se vaya a utilizar. A pesar de que el costo de los dispositivos digitales es cada vez menor, éste es un factor que no debe ser ignorado.

1.4.2. Códigos diferenciales NRZ.

Los códigos NRZ son los más sencillos de implementar y el más simple de ellos, el código NRZ unipolar, es por lo general el código usado para generar o interpretar datos digitales. Si se utiliza un código diferente, éste será usualmente generado a partir del código NRZ unipolar.

La ventaja que los códigos diferenciales (tanto M como S) presentan, es la de ser decodificados por comparación con el elemento de señal adyacente, antes que por comparación con un valor absoluto de señal, como es el caso del código NRZ unipolar.

Un beneficio de este esquema de codificación es que se puede tener mayor confiabilidad en detectar una transición en presencia de ruido, antes que realizar una comparación con un nivel umbral. Otra ventaja es que, en un sistema de transmisión, en el que accidentalmente se puedan invertir las polaridades de los conductores convirtiendo los 1_L en 0_L y viceversa, el esquema diferencial no será afectado en la decodificación.

La codificación NRZ diferencial presenta un adecuado uso del ancho de banda, esta propiedad es ilustrada en la figura 1.26, en la cual se compara la densidad espectral de potencia normalizada ($S(f)/A^2T$) de

varios esquemas de codificación. Como se puede ver, la mayor parte de la energía está comprendida entre 0 y $0.5fT$. En esta figura, $S(f)$ es la densidad espectral de potencia, A representa la magnitud de los elementos de señal, T el período de duración de un bit y f es la frecuencia a la cual se está analizando el espectro.

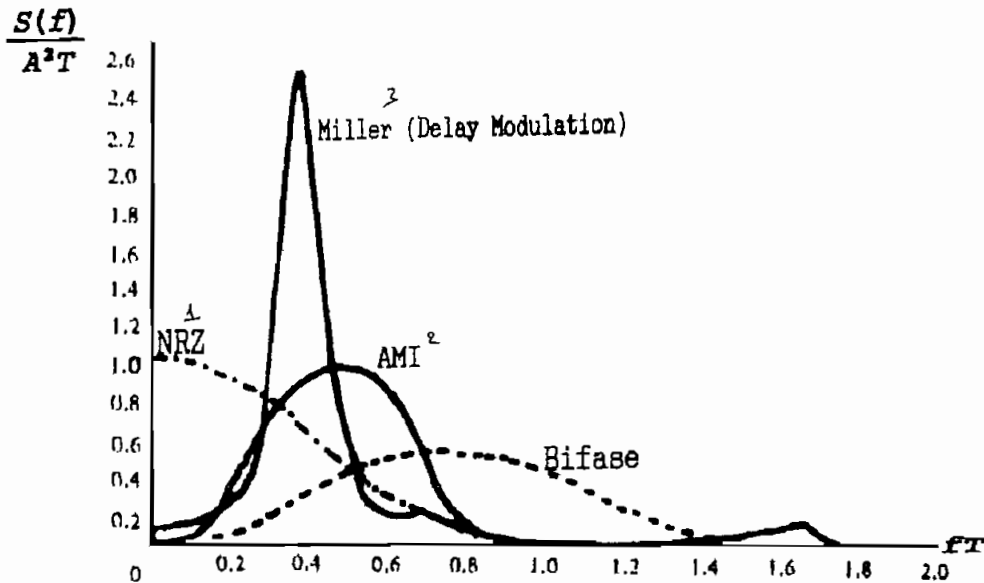


Fig. 1.26. Densidad espectral de varios esquemas de codificación

Las principales limitaciones de las señales NRZ son la presencia de una componente DC y la falta de capacidad de sincronización (recuperación de la señal de reloj). Esto se pone en evidencia cuando se tiene una larga cadena de 1_L para el código diferencial S o de 0_L para el diferencial M, ya que la salida permanecerá en un voltaje constante; para este caso, cualquier pérdida de sincronismo entre el transmisor y el receptor no podrá ser corregida usando sólo la señal de entrada.

Debido a su simplicidad y a su relativamente mediano ancho de banda, los códigos NRZ (diferenciales o no) son usados comúnmente en grabaciones digitales en

discos magnéticos. Sin embargo, sus limitaciones hacen que estos esquemas de codificación sean poco usuales en aplicaciones de transmisión digital.

1.4.3. Codificación con retorno a cero (RZ).

Con este código, la duración mínima de un pulso se reduce a la mitad, por lo cual la velocidad de transmisión codificada (v_c) será el doble de la velocidad de transmisión binaria (v_t).

Para realizar una comparación en este sentido, en la tabla 1.8 se muestra la velocidad de cambio de los elementos de señal en términos del número de transiciones por período. En ésta se establecen el número de transiciones máximo, mínimo y para una secuencia alternada de 0_L y 1_L . Es de notar que para el caso RZ, el máximo número de transiciones por período se presenta para una cadena de 1_L .

Código	Mínima	101010...	Máxima
NRZ-unipolar	0 (todos 0_L ó 1_L)	1.0	1.0 (1010...)
Diferencial-M	0 (todos 0_L)	0.5	1.0 (todos 1_L)
Diferencial-S	0 (todos 1_L)	0.5	1.0 (todos 0_L)
RZ	0 (todos 0_L)	1.0	2.0 (todos 1_L)
Manchester	1.0 (1010...)	1.0	2.0 (todos 0_L o 1_L)
Bifase-M	1.0 (todos 0_L)	1.5	2.0 (todos 1_L)
Bifase-S	1.0 (todos 1_L)	1.5	2.0 (todos 0_L)
Manchester diferencial	1.0 (todos 1_L)	1.5	2.0 (todos 0_L)
Miller	0.5 (1010...)	0.5	1.0 (todos 0_L o 1_L)
Bipolar	0 (todos 0_L)	1.0	2.0 (todos 1_L)

Tabla 1.8. Transiciones por período

Debido a que la tasa de cambio de los elementos de señal es mayor que en el caso NRZ, el ancho de banda de la señal es mayor. Además, se presentan los mismos problemas de la componente DC y la falta de sincronización para una secuencia larga de 0_L . Debido a su simplicidad, la

codificación RZ es usada en algunas transmisiones elementales y en equipo de grabación, pero no es una técnica muy empleada en la mayoría de las aplicaciones.

1.4.4. Codificación bifase.

Los esquemas bifase presentan al menos una transición por intervalo de bit (tabla 1.8) y puede tener como mucho dos transiciones. Así, la máxima velocidad de transmisión codificada es el doble que en el caso diferencial, por lo que el ancho de banda es mayor; para compensar esto, la codificación bifase presenta algunas ventajas.

La sincronización es una de estas ventajas debido a que existe una transición predecible durante cada intervalo de bit. Para la codificación Manchester y Manchester diferencial existe siempre una transición en la mitad del intervalo del bit. Para el código bifase-M y bifase-S, existe siempre una transición al comienzo del período. Por esta razón, los códigos bifase se conocen como códigos autosincronizados.

Otra ventaja es que no presentan una componente continua. Además, posibilitan la detección de errores ya que se puede usar la ausencia de una transición esperada para conocer que ha ocurrido un error. Sin embargo, el ruido en la línea podría invertir la señal antes y después de la transición, con lo cual el error no sería detectado. Por otra parte, como puede verse en la figura 1.26, el ancho de banda es medianamente estrecho y no contiene componente continua.

Los códigos bifase son técnicas populares para transmisión de datos. De entre éstas, la codificación Manchester es la más utilizada tanto en transmisión como en la grabación en cintas magnéticas y como señales de entrada

para sistemas de modulación en transmisión por fibras ópticas.

Adicionalmente, tanto el código Manchester como el Manchester diferencial son usados en los estándares para redes locales de computadores. El código Manchester, el más común, ha sido especificado en la norma IEEE 802.3 para cable coaxial de banda base y en la MIL-STD-1553B, que regula la transmisión por par trenzado en ambientes de alto ruido.

El código Manchester diferencial por su parte, es contemplado en la norma IEEE 802.5 para redes Token Ring, ya sea usando cable coaxial de banda base o par trenzado. Debido a su característica diferencial, éste es preferido en implementaciones con par trenzado.

El esquema de codificación CMI, considerado también como bifase, es recomendado por el CCITT (Rec. G.703) como el código que debe usarse en el interfaz digital a 139264 kbit/s. Se define el interfaz como el elemento que permite la conexión de los componentes de las redes digitales (secciones digitales, equipo multiplex, centrales) para formar un enlace o una conexión digital internacional.

1.4.5. Modulación por retardo o código Miller.

Con la codificación Miller existe al menos una transición en dos períodos de bit. De esta manera se proporciona alguna capacidad de sincronización, requiriendo menor velocidad de transmisión codificada y un ancho de banda menor que los códigos bifase.

La figura 1.26 muestra que el ancho de banda es significativamente menor que para los otros esquemas, sin embargo es necesario anotar que en el peor de los casos (una secuencia continua de 1010...) existirá una importante

componente continua y un ancho de banda mayor que el mostrado para la codificación NRZ.

En la figura 1.27 se compara el BER teórico en función de la relación señal a ruido (S/N), para varios códigos.

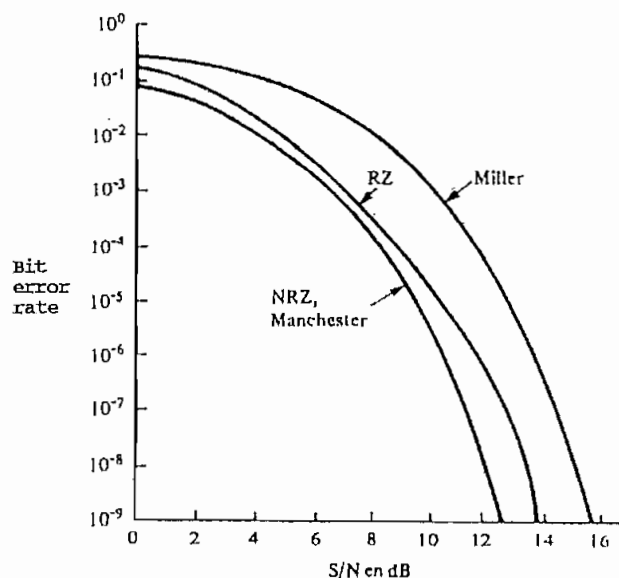


Fig. 1.27. BER teórico para varios esquemas de codificación

Es de notar que tanto el código Manchester como los NRZ (unipolar y diferencial) tienen el mismo comportamiento y éste es mejor en unos 3 dB respecto al código de Miller.

Si se evalúa para $S/N = 12$ dB, el BER para NRZ y Manchester es de dos órdenes de magnitud mejor que para RZ y este último dos órdenes mejor que el código de Miller. Este comportamiento se explica en los siguientes términos: el receptor debe distinguir entre un 1_L y un 0_L a partir de la señal de entrada; en el caso del código Manchester o de los NRZ existen sólo dos elementos de señal para discernir, en el caso de RZ esto es también así, pero el disminuir el ancho del pulso hace que el muestreo sea menos eficiente. El código Miller por su parte tiene cuatro elementos de

señal (transición positiva, transición negativa, pulso y espacio) entre los cuales discernir, lo cual hace que la decisión sea más difícil y por lo tanto que se tienda a cometer más errores.

1.4.6. Codificación bipolar.

La codificación bipolar básica lo constituye el código AMI, y es por ello muy usado. La capacidad de sincronización no es muy buena cuando se tiene una cadena de varios 0, seguidos, sin embargo, su espectro de potencia es tal que anula la componente continua (en base a la alternabilidad de la polaridad de los pulsos) como puede verse al graficar la función $C(f)$ ¹ en la figura 1.28.

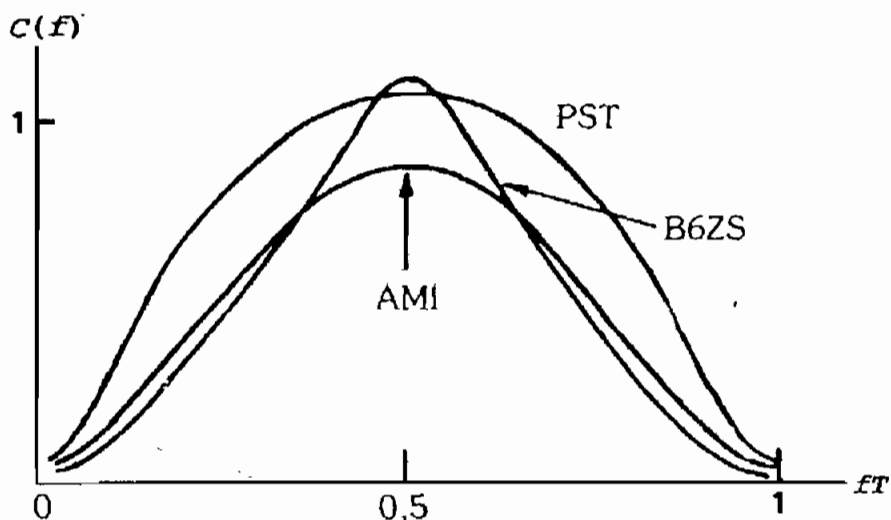


Fig. 1.28. Espectro de potencia para los códigos PST, B6ZS y AMI

El código AMI es recomendado por el CCITT (Rec. G.703) para los interfaces digitales a 64 kbit/s y a 1544 kbit/s. Otra aplicación muy importante del código AMI se encuentra en sistemas de ISDN (red digital de servicios

¹ El significado de $C(f)$ será explicado en el numeral 1.5.

integrados), en donde se lo usa como código de línea¹ a 192 kbit/s.

El código PST mejora la sincronización ya que provee de pulsos positivos y negativos en el caso de tener dos 0_L seguidos, sin embargo como puede verse en la figura 1.28, su ancho de banda es mayor que el código AMI. Un código que disminuye el ancho de banda es el B6ZS, aunque a costo de una mayor complejidad en la implementación. Es de observar también que B6ZS y PST tienen niveles de energía más altos que AMI, como resultado de densidades de pulso mayores. Esto se paga con el aumento de la diafonía en cables multipares; sin embargo la degradación por diafonía es compensada por la mejora en la exactitud de la recuperación del reloj.

Otros códigos BnZS son el B8ZS, recomendado por el CCITT para el interfaz digital a 1544 kbit/s y el B3ZS recomendado para el interfaz digital a 44736 kbit/s y usado por la empresa NEC en los sistemas de radio digital por microondas (serie 500 de 24 canales) a 44736 kbit/s.

El código 4B-3T es otro de los códigos bipolares que tiene un ancho de banda aceptable, pero tiene una gran cantidad de energía en frecuencias bajas (figura 1.29); sin embargo, su principal ventaja consiste en la disminución de la velocidad de transmisión codificada a un factor de 3/4 de la correspondiente a la velocidad de transmisión binaria a la entrada del codificador; este hecho lo hace muy atractivo para sistemas de alta capacidad. Es utilizado en sistemas de transmisión por fibra óptica a 565 Mbit/s tipo 8TR685 de ATT y Philips.

¹ NATIONAL SEMICONDUCTOR, Telecommunications Databook, Santa Clara CA., 1990, p. 2.51,

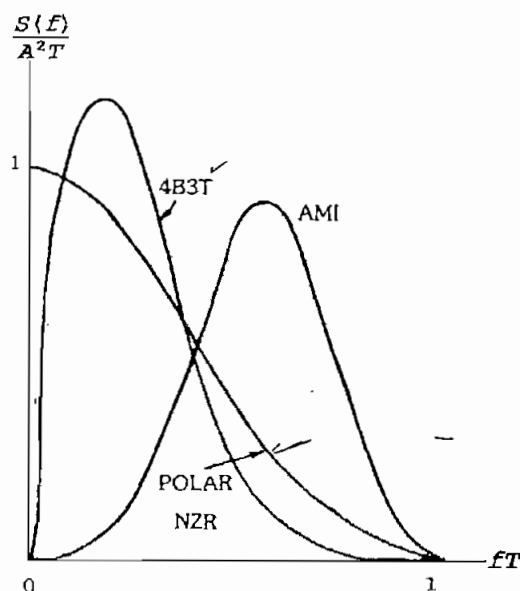


Fig. 1.29. Espectro de potencia para los códigos 4B3T, NRZ polar y AMI

La codificación HDB3 por su parte presenta una buena característica de sincronización, ya que no permite la existencia de más de tres ceros seguidos.

En la figura 1.30 se grafica la función $C(f)$ tanto para el código AMI como para el HDB3. De este gráfico se concluye que el esquema de codificación HDB3 presenta un espectro de potencia que tiene su máxima cantidad de energía alrededor de la mitad de la frecuencia normalizada ($0.5 fT$), lo cual garantiza una pequeña componente de bajas frecuencias y un ancho de banda razonablemente estrecho. Este código es especificado por el CCITT (Rec. G.703) para el interfaz digital a 34368 kbit/s.

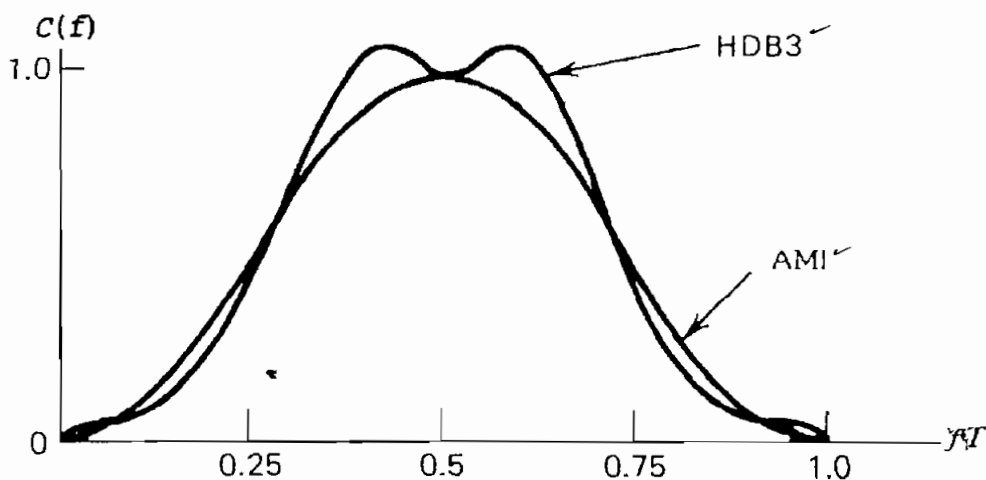


Fig. 1.30. Comparación entre el código HDB3 y AMI

1.5. ANALISIS DE LA DENSIDAD ESPECTRAL DE POTENCIA DE LOS CODIGOS DE LINEA.

La densidad espectral de potencia es una función que da la indicación de las contribuciones relativas de potencia, en las diversas frecuencias que abarca una señal. Puesto que las señales periódicas se caracterizan por su potencia promedio, el estudio que sigue supondrá un tratamiento de la potencia en estos términos.

El concepto de densidad espectral de potencia tiene particular importancia al analizar el proceso de transmisión de información, puesto que es necesario conocer la ubicación espectral del máximo contenido de energía de la señal, que será transmitida a través de un canal de comunicación; éste presentará características espectrales que deberán ajustarse a las de la señal a transmitir, lo cual garantizará una adecuada transferencia de la información.

La densidad espectral de potencia $S(w)$ tiene unidades de potencia por unidades de frecuencia y se define

matemáticamente como:

$$\boxed{P = \frac{1}{2\pi} \int_{-\infty}^{\infty} S(\omega) d\omega} \quad [1.13]$$

Donde P es la potencia media en el tiempo, considerando una carga de 1Ω .

1.5.1. Tratamiento estadístico de la densidad espectral de potencia.

Un flujo de señales binarias no puede ser tratado como una señal determinística sino que se la debe tratar como un proceso estadístico aleatorio. Este proceso aleatorio se lo considera estacionario, ya que las características estadísticas no varían en el tiempo, además se considera que los dígitos binarios a la entrada del codificador son equiprobables.

Supóngase que se tiene un proceso aleatorio $x(t)$ tal que en $t = t_1$, el proceso $x(t)$ se caracteriza por la variable aleatoria $X_1 = x(t_1)$ y en $t = t_2$ se caracteriza por $X_2 = x(t_2)$. Ahora surge la posibilidad de caracterizar el proceso aleatorio $x(t)$ por la función densidad probabilística conjunta de estas variables aleatorias.

Puesto que el proceso $x(t)$ es estacionario, la esperanza matemática (valor medio m_x) es constante:

$$E\{x(t)\} = m_x \quad [1.14]$$

El proceso aleatorio $x(t)$ se caracteriza en términos de una densidad conjunta bidimensional $p_{x_1x_2}(x_1, x_2)$. Un parámetro estadístico de particular importancia es la

esperanza matemática $E\{X_1, X_2\}$, la que se expresa como¹:

$$E\{x(t_1)x(t_2)\} = R_{xx}(t_1, t_2)$$

pero,

$$E\{X\} = \int_{-\infty}^{\infty} xp(x) dx = m_x$$

luego,

$$R_{xx}(t_1, t_2) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} x_1 x_2 p_{x_1 x_2}(x_1, x_2) dx_1 dx_2 \quad [1.15]$$

La función $R_{xx}(t_1, t_2)$ se llama **función autocorrelación** del proceso aleatorio $x(t)$ evaluada en los tiempos t_1 y t_2 . Puesto que se tiene un proceso estacionario, esta función sólo depende de la diferencia entre t_1 y t_2 , por lo que se puede escribir:

$$R_{xx}(t_1, t_2) = R_{xx}(t_2 - t_1)$$

Si $\tau = t_2 - t_1$, $t_2 = t_1 + \tau$, por lo que:

$$R_{xx}(\tau) = E\{x(t)x(t+\tau)\} \quad [1.16]$$

La función autocorrelación no describe ni define completamente el proceso aleatorio $x(t)$, pero da mucha información sobre éste. En particular, da una medida de que tan dependiente es un valor particular de una función muestra, de otro valor desplazado τ unidades de tiempo.

Esta función de autocorrelación es importante, ya que también permite calcular la densidad espectral de

¹ STREMLER F., Sistemas de comunicación, Fondo Educativo Interamericano, México, 1991, p. 472-482.

potencia a partir de la transformada de Fourier de la siguiente manera:

$$S_x(\omega) = \mathcal{F}\{R_{xx}(\tau)\} = \int_{-\infty}^{\infty} R_{xx}(\tau) e^{-j\omega\tau} d\tau \quad [1.17]$$

La potencia media de $x(t)$, para una carga unitaria (1Ω) será:

$$P = \lim_{T \rightarrow \infty} \frac{1}{T} \int_{-T/2}^{T/2} x^2(t) dt = E\{x^2(t)\} = R_{xx}(0) \quad [1.18]$$

Para continuar con el análisis, conviene en este punto establecer un **modelo para un sistema de transmisión en banda base**. El modelo usado es el lineal y se lo representa por el esquema general de bloques de la figura 1.31.

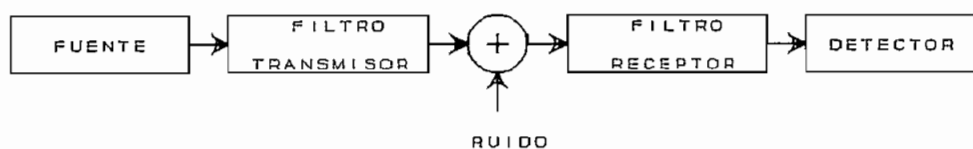


Fig. 1.31. Esquema de un sistema en banda base

Si en lugar de enviar directamente al filtro $H(f)$, la secuencia de datos $\{a_n\}$ (que se supone es una variable aleatoria discreta perteneciente a un alfabeto binario de realizaciones independientes y equiprobables) se introduce a un codificador, éste convertirá la secuencia $\{a_n\}$ en otra $\{b_n\}$, tal como se muestra en la figura 1.32.



Fig. 1.32. Transmisión con codificación

El filtro $H(f)$ es un filtro formador de onda conocido como **filtro transversal**, que determina la forma de onda que se transmitirá. Su implementación se realiza como se indica en la figura 1.33 y de ahí que la señal a la salida del filtro se escribe como:

$$s(t) = \sum_{i=0}^W b_i X(t-iT) \quad [1.19]$$

Donde $X(t)$ es una variable determinística que representa la forma de un pulso.

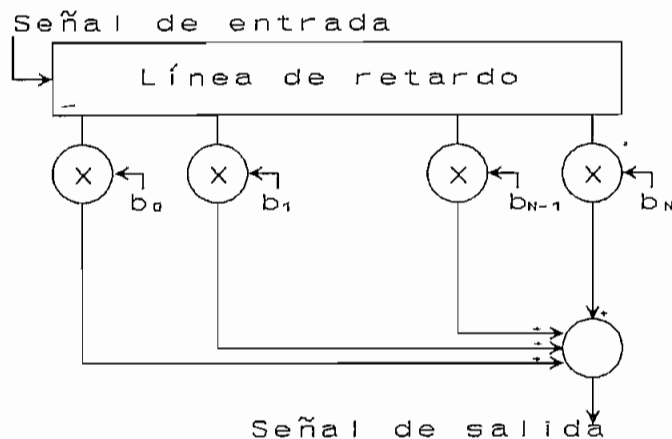


Fig. 1.33. Filtro transversal

La secuencia de datos $\{b_n\}$ representa un proceso estacionario, con una media m_b y una función de correlación $k(i)$ tal que:

$$m_b = E \{b_i\} = cte$$

$$k(i) = k(n-j) = E \{b_n \cdot b_j\} \quad i = n-j \quad [1.20]$$

Puesto que la salida del filtro está determinada por la ecuación [1.19], la función de autocorrelación será:

$$R_{ss}(t+\tau, t) = E \{s(t+\tau) s(t)\} \quad [1.21]$$

$$R_{ss}(t+\tau, t) = E \{ \sum_n \sum_j b_n b_j x(t+\tau-nT) x(t-jT) \}$$

$$R_{ss}(t+\tau, t) = \sum_n \sum_j E \{b_n b_j\} x(t+\tau-nT) x(t-jT)$$

$$R_{ss}(t+\tau, t) = \sum_n \sum_j k(n-j) x(t+\tau-nT) x(t-jT)$$

y finalmente:

$$R_{ss}(t+\tau, t) = \sum_i k(i) \sum_j x(t+\tau-iT-jT) x(t-jT) \quad [1.22]$$

Es necesario independizar a la función de autocorrelación R_{ss} de la variable t , para lo cual se procede a promediar la función autocorrelación $R_{ss}(t+\tau, t)$ sobre un período, o sea:

$$R_{ss}(\tau) = \frac{1}{T} \int_0^T R_{ss}(t+\tau, t) dt$$

$$R_{ss}(\tau) = \frac{1}{T} \int_0^T \sum_i k(i) \sum_j x(t+\tau-iT-jT) x(t-jT) dt$$

$$R_{ss}(\tau) = \frac{1}{T} \sum_i k(i) \sum_j \int_{-jT}^{-jT+T} x(\xi-iT+\tau) x(\xi) d\xi$$

donde $\xi = t - jT$

$$R_{ss}(\tau) = \frac{1}{T} \sum_i k(i) \int_{-\infty}^{+\infty} x(\xi-iT+\tau) x(\xi) d\xi$$

$$R_{ss}(\tau) = \frac{1}{T} \sum_i k(i) z(\tau-iT) \quad [1.23]$$

donde $z(\tau)$ es la función de autocorrelación del pulso $x(t)$, que es una señal determinística, siendo por tanto definida en el sentido determinístico como:

$$z(\tau) = \int_{-\infty}^{\infty} x(t+\tau)x(t) dt \quad [1.24]$$

y $k(i)$ es la función de correlación estadística de la secuencia $\{b_n\}$.

Se calculará ahora la densidad espectral de potencia media, según la ecuación [1.17]:

$$\begin{aligned} S(f) &= \mathcal{F}\{R_{ss}(\tau)\} = \frac{1}{T} \sum_i k(i) \mathcal{F}\{z(\tau - iT)\} \\ S(f) &= \left(\frac{1}{T}\right) \sum_i k(i) [X(f)]^2 e^{-j2\pi f iT} \\ S(f) &= [X(f)]^2 \cdot C(f) \end{aligned} \quad [1.25]$$

donde $X(f)$ es la transformada de Fourier de $x(t)$, en tanto que $C(f)$ está dado por la expresión:

$$C(f) = \frac{1}{T} \sum_i k(i) e^{-j2\pi f iT} \quad [1.26]$$

$C(f)$ es una función periódica en f^1 , de período $1/T$, que depende de la función de correlación entre los elementos de la secuencia $\{b_n\}$ y cuyos valores $k(i)$ son los coeficientes de su desarrollo en serie de Fourier². En el caso de que los elementos de la secuencia $\{b_n\}$ no estén correlacionados, $k(i)$ será una constante para $i = 0$ y se anulará para $i \neq 0$; por lo cual la densidad espectral de potencia dependerá sólo de la forma determinística del pulso $x(t)$, es decir:

$$S(f) = cte. [X(f)]^2 \quad [1.27]$$

¹ Se debe tener presente que T es una constante igual a la duración del bit en tanto que f es una variable.

² VIDALLER L. y OTROS, Op. Cit., p. 111-115.

1.5.2. Densidad espectral de potencia de los códigos de línea.

En lo que sigue se detallará la densidad espectral de potencia de los más importantes códigos de línea, utilizando para ello las consideraciones expuestas en el numeral 1.5.1.

a. Código NRZ unipolar.

La codificación se realiza con una variación entre los niveles 0 y A. Se supone que los símbolos de entrada son independientes y con probabilidades:

$$p(1_L) = p$$

$$p(0_L) = q$$

$$p + q = 1$$

con lo cual la función de correlación será:

$$k(0) = A^2 pq$$

$$k(i) = A^2 p^2 \quad i \neq 0$$

de donde:

$$C(f) = \frac{1}{T} \sum_i k(i) e^{-j2\pi f i T}$$

$$C(f) = \frac{A^2}{T} [pq + p^2 \sum_i e^{-j2\pi f i T}] \quad [1.28]$$

$$C(f) = \frac{A^2}{T} pq + \frac{A^2}{T^2} p^2 \sum_i \delta(f - \frac{i}{T})$$

donde $\delta(t)$ es la función impulso (delta de Dirac). En el desarrollo posterior, se supone que $x(t)$ es un pulso cuadrado, tal que:

$$x(t) = \Pi\left(\frac{t}{T} - \frac{1}{2}\right)$$

$$[X(f)]^2 = T^2 \left[\frac{\text{sen}(\pi fT)}{\pi fT} \right]^2 \quad [1.29]$$

por lo que,

$$S(f) = A^2 T p q \left[\frac{\text{sen}(\pi fT)}{\pi fT} \right]^2 + A^2 p^2 \left[\frac{\text{sen}(\pi fT)}{\pi fT} \right]^2 \sum_i \delta\left(f - \frac{i}{T}\right)$$

$$[1.30]$$

Un gráfico de la densidad espectral de potencia se muestra en la figura 1.34, para dos probabilidades distintas.

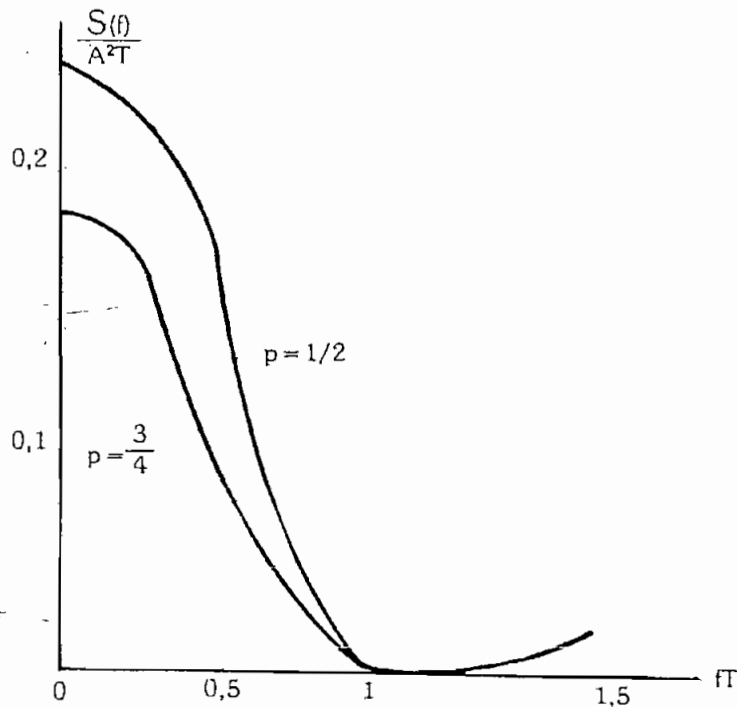


Fig. 1.34. Densidad espectral de potencia para el código NRZ unipolar

b. Código NRZ polar.

Es similar al NRZ unipolar, cambiando el nivel 0 por $-A$; en este caso el valor de $C(f)$ está dado por:

$$C(f) = \frac{4A^2}{T} p q + \frac{A^2}{T^2} (2p-1)^2 \sum_i \delta(f - \frac{i}{T}) \quad [1.31]$$

La densidad espectral de potencia será:

$$S(f) = 4A^2 T p q \left[\frac{\text{sen}(\pi f T)}{\pi f T} \right]^2 + A^2 (2p-1)^2 \left[\frac{\text{sen}(\pi f T)}{\pi f T} \right]^2 \sum_i \delta(f - \frac{i}{T}) \quad [1.32]$$

que se gráfica en la figura 1.35, igualmente para dos valores de probabilidad.

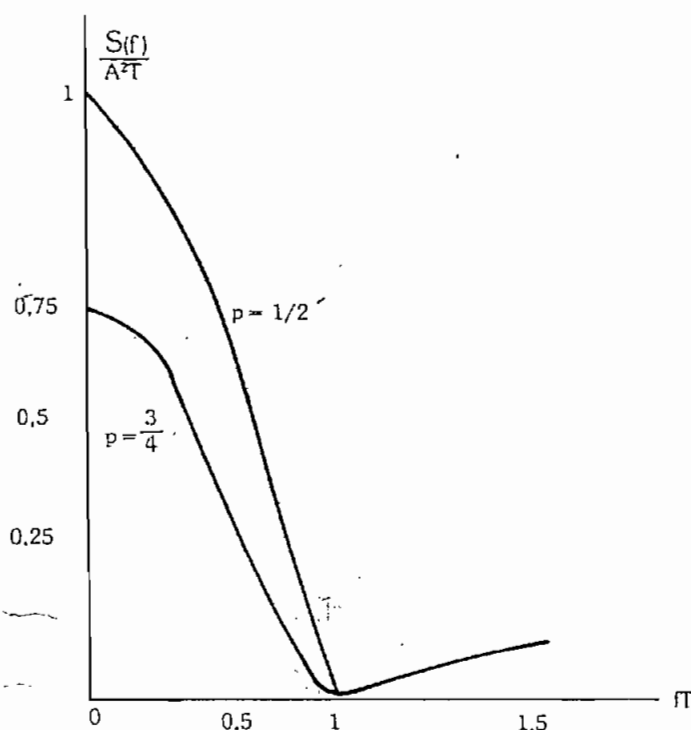


Fig. 1.35. Densidad espectral de potencia del código NRZ polar

c. Código AMI-NRZ.

La función $C(f)$ está dada por:

$$C(f) = \frac{A^2}{T} \frac{4pq \operatorname{sen}^2(\pi fT)}{1 + (2p-1)^2 + 2(2p-1) \cos(2\pi fT)} \quad [1.33]$$

La densidad espectral de potencia se hallará multiplicando la ecuación anterior con $[X(f)]^2$:

$$S(f) = A^2 T \left[\frac{\operatorname{sen}^2(\pi fT)}{\pi fT} \right]^2 \frac{4pq}{1 + (2p-1)^2 + 2(2p-1) \cos(2\pi fT)} \quad [1.34]$$

En la figura 1.36 se representa la densidad espectral de potencia para tres valores de probabilidad.

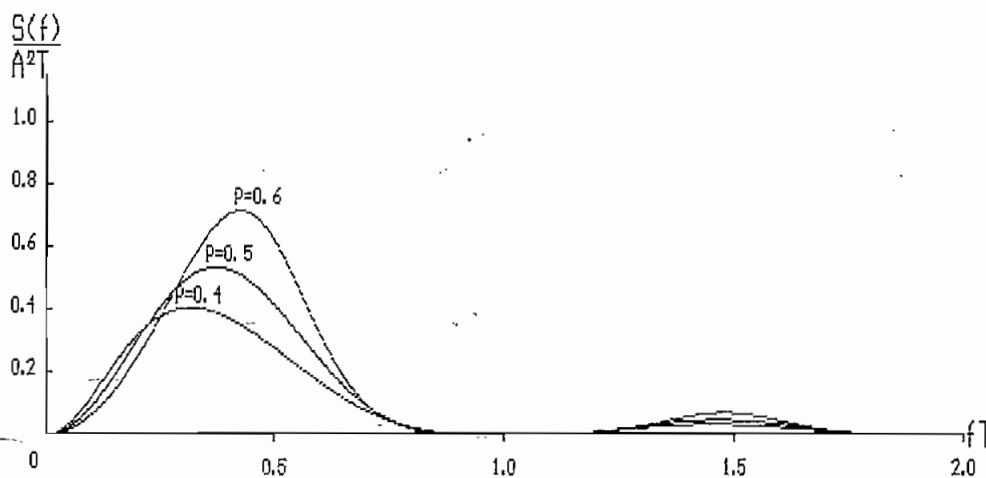


Fig. 1.36. Densidad espectral de potencia para el código AMI 2

d. Códigos diferenciales NRZ.

La función $C(f)$ para el código diferencial NRZ-M unipolar es:

$$C(f) = \frac{A^2}{4T} \frac{1+(1-2p) [(3-2p) - 4q \cos(2\pi fT)]}{1+(1-2p)^2 - 2(1-2p) \cos(2\pi fT)} + \frac{A^2}{8T^2} \delta(f)$$

$$\Rightarrow S(f) = \frac{A^2 T}{4} \left[\frac{\sin(\pi fT)}{\pi fT} \right]^2 \frac{1+(1-2p) [(3-2p) - 4q \cos(2\pi fT)]}{1+(1-2p)^2 - 2(1-2p) \cos(2\pi fT)} + \frac{A^2}{8} \left[\frac{\sin(\pi fT)}{\pi fT} \right]^2 \delta(f)$$

[1.35]

En tanto que para el caso polar se tiene:

$$C(f) = \frac{A^2}{T} \frac{1+(1-2p) [(3-2p) - 4q \cos(2\pi fT)]}{1+(1-2p)^2 - 2(1-2p) \cos(2\pi fT)}$$

$$\Rightarrow S(f) = A^2 T \left[\frac{\sin(\pi fT)}{\pi fT} \right]^2 \frac{1+(1-2p) [(3-2p) - 4q \cos(2\pi fT)]}{1+(1-2p)^2 - 2(1-2p) \cos(2\pi fT)}$$

[1.36]

En la figura 1.37 se muestra la densidad espectral de potencia para dos valores de probabilidad en el caso unipolar, y en la figura 1.38 se tiene estos mismos gráficos para el caso polar.

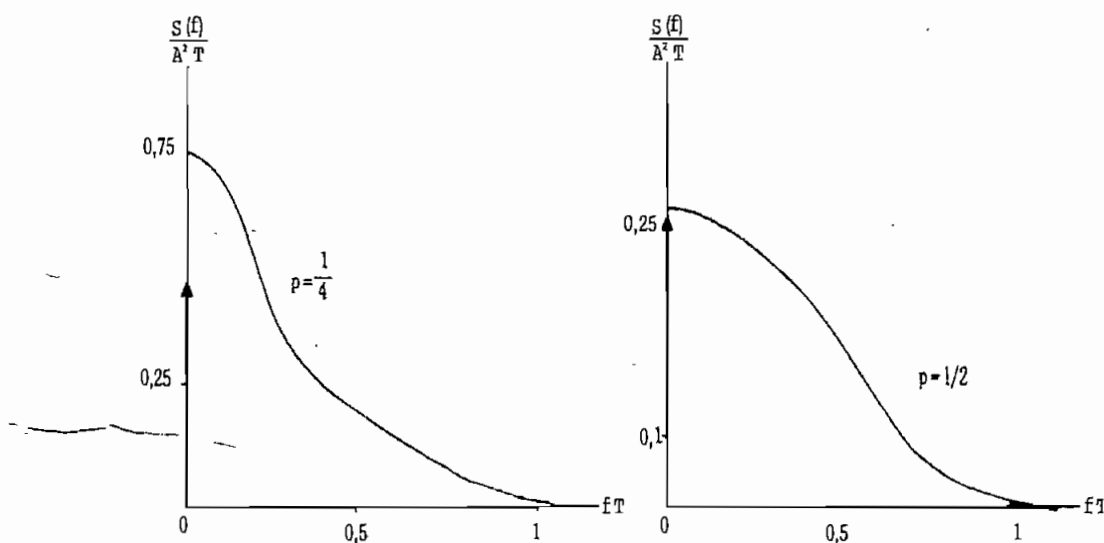


Fig. 1.37. Densidad espectral de potencia para el código diferencial NRZ unipolar

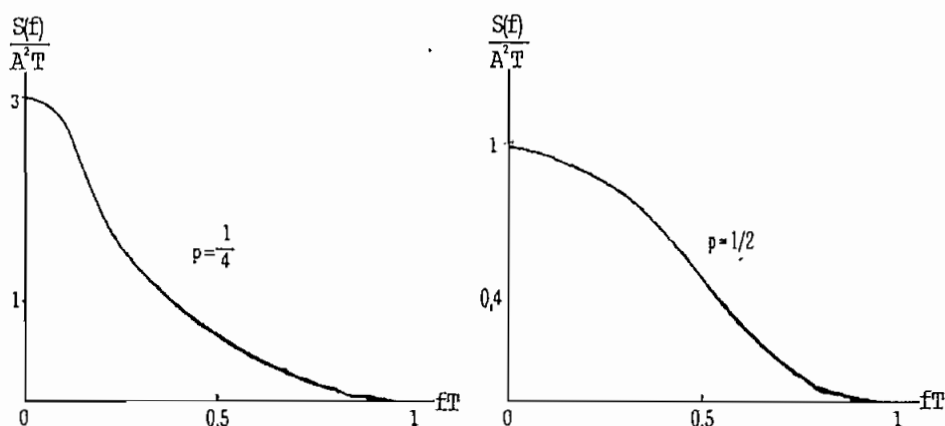


Fig. 1.38. Densidad espectral de potencia para el código diferencial NRZ-M polar

Para el caso del código diferencial S, las ecuaciones anteriores son válidas, si se intercambian las probabilidades p y q .

e. Códigos con retorno a cero (RZ).

Hasta ahora se ha tratado el caso de los códigos sin retorno a cero. Cuando se tienen códigos RZ, las correspondientes expresiones de $C(f)$ serán idénticas a las anteriores, según el tipo de código. Lo que varía es la función $X(f)$, ya que el ancho del pulso se reduce a la mitad, por lo que:

$$x(t) = \Pi\left(2\frac{t}{T} - \frac{1}{2}\right)$$

$$[X(f)]^2 = \frac{T^2}{4} \left[\frac{\text{sen}\left(\frac{\pi f T}{2}\right)}{\frac{\pi f T}{2}} \right]^2 \quad [1.37]$$

Como un ejemplo de este tipo de codificación, en la figura 1.39 se grafica la densidad espectral de potencia para el código RZ-polar; es de notar que el ancho de banda prácticamente se ha duplicado respecto al caso NRZ

(figura 1.35).

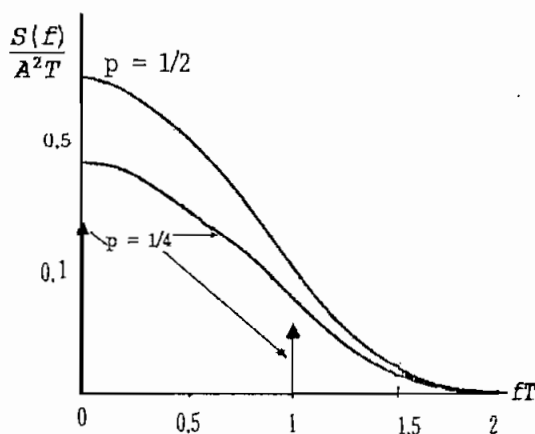


Fig. 1.39. Densidad espectral de potencia para el código RZ-polar

f. Código bifase-L o Manchester.

La función $C(f)$ es la misma que para el código AMI-NRZ (ecuación 1.33). Lo que varía es la función $X(f)$ puesto que el pulso tiene un cambio de nivel en el centro del intervalo unitario T , por lo tanto:

$$x(t) = -\Pi\left(2\frac{t}{T} - \frac{1}{2}\right) + \Pi\left(2\frac{t}{T} - \frac{3}{2}\right)$$

$$X(f) = \frac{T^2}{4} \left[\frac{\sin\left(\pi f \frac{T}{2}\right)}{\pi f \frac{T}{2}} \right]^2 \quad [1.38]$$

En la figura 1.40 puede verse la densidad espectral de potencia para dos probabilidades diferentes.

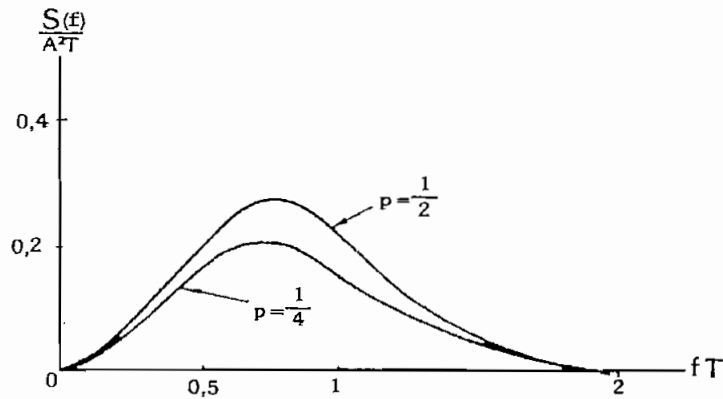


Fig. 1.40. Densidad espectral de potencia para el código Manchester

g. Código de modulación por retardo o código Miller.

El cálculo de la densidad espectral de potencia es algo más complejo que los casos anteriores, por lo que el estudio se restringe a plantear la ecuación correspondiente.

$$S(f) = \frac{1}{\theta^2 T (17 + 8 \cos \theta)} [23 - 2 \cos \theta - 22 \cos (2\theta) - 12 \cos (3\theta) + 5 \cos (4\theta) + 12 \cos (5\theta) + 2 \cos (6\theta) - 8 \cos (7\theta) + 2 \cos (8\theta)]$$

[1.39]

donde $\theta = \pi fT$.

La distribución de potencia que produce este código puede verse en la figura 1.41, donde se ha incluido también el código polar NRZ y el de Manchester.

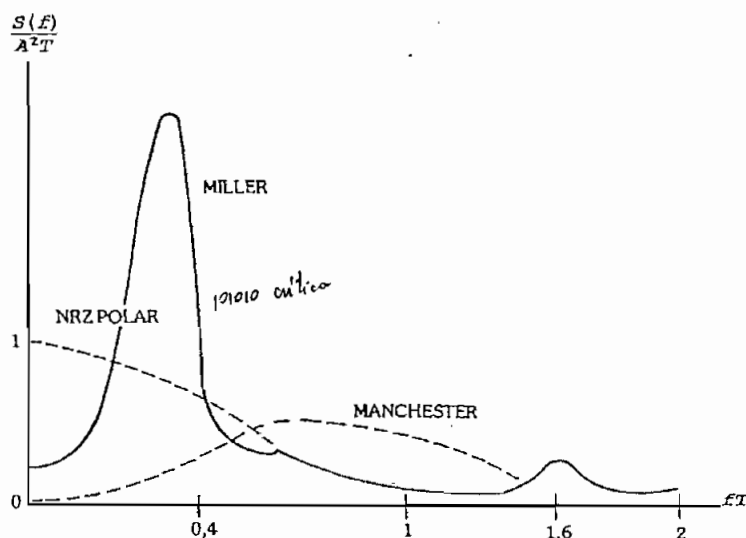


Fig. 1.41. Densidad espectral de potencia del código de Miller

h. Códigos pseudoternarios.

Para el código **dicode**, la densidad espectral de potencia es:

$$S(f) = 4A^2T pq \operatorname{sen}^2(\pi fT) \left[\frac{\operatorname{sen}(\pi fT)}{\pi fT} \right]^2 \quad [1.40]$$

El gráfico de su densidad espectral de potencia se muestra en la figura 1.42.

El código **duobinario** presenta su densidad espectral de potencia como:

$$S(f) = 4A^2T pq \cos^2(\pi fT) \left[\frac{\operatorname{sen}(\pi fT)}{\pi fT} \right]^2 \quad [1.41]$$

En la figura 1.43 se grafica su densidad espectral de potencia para dos valores de probabilidad.

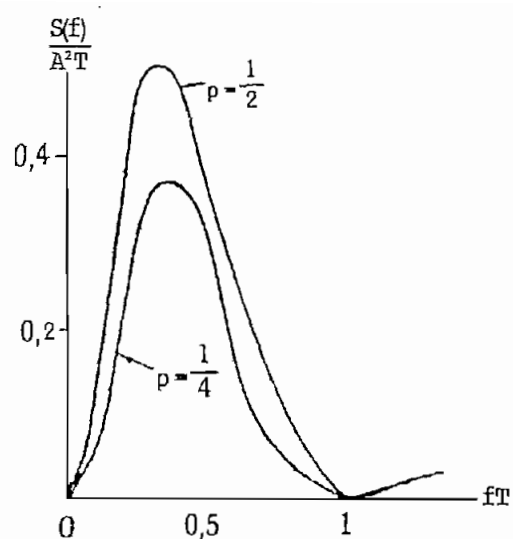


Fig. 1.42. Espectro de la codificación dicode

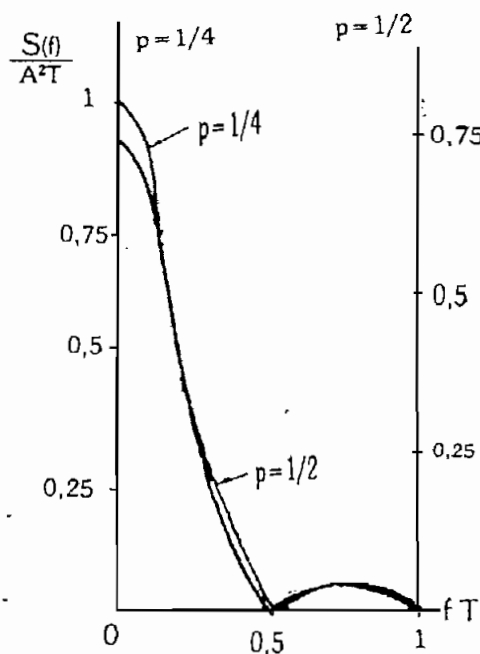


Fig. 1.43. Espectro duobinario

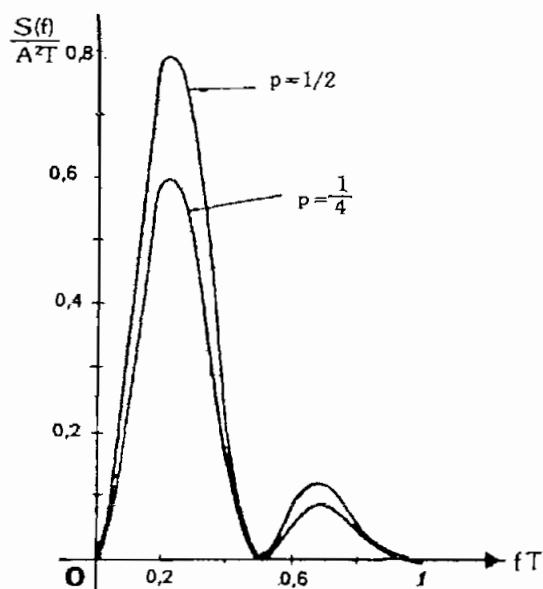


Fig. 1.44. Duobinario modificado

En tanto que para el código **duobinario modificado** la densidad espectral de potencia será:

$$S(f) = 4A^2T pq \operatorname{sen}^2(2\pi fT) \left[\frac{\operatorname{sen}(\pi fT)}{\pi fT} \right]^2 \quad [1.42]$$

La forma del espectro de este código está en la figura 1.44.

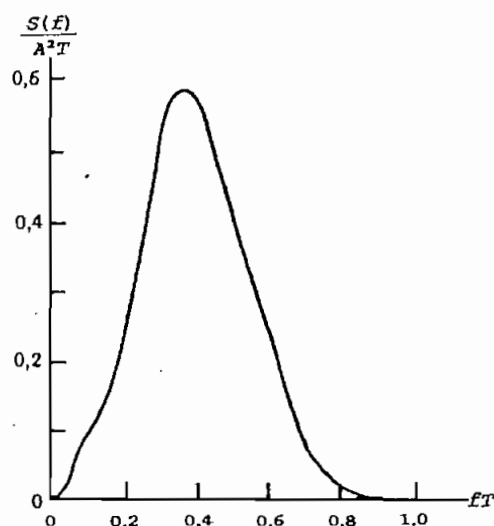


Fig. 1.45. Densidad espectral de potencia del código HDB3

Para el código **HDB3** (figura 1.45), la expresión de su densidad espectral de potencia es mucho más compleja que para los casos anteriores y para una entrada equiprobable de 0_L y 1_L se expresa en los siguientes términos¹:

$$S(f) = \frac{40-32\cos\Phi}{465T(1025-64\cos5\Phi)} \cdot [7258.5-1929\cos\Phi-1424\cos(2\Phi) + \\ -160\cos(3\Phi)+32\cos(4\Phi) + \\ -\frac{131288.5-41399\cos\Phi-86112\cos(2\Phi)}{85-44\cos\Phi-24\cos(2\Phi)-16\cos(3\Phi)}] [X(f)]^2 \quad [1.43]$$

donde $\Phi = 2\pi fT$.

¹ BYLANSKY P. e INGRAM D., Op. Cit., p. 242, 243.

1.6. DESCRIPCION DEL SOFTWARE PARA EL ANALISIS.

Con la finalidad de analizar la densidad espectral de potencia (d.e.p.) de los esquemas de codificación más importantes, se desarrolla un programa para computador personal. Debido a que el programa en cuestión necesita de un buen manejo de la pantalla en modo gráfico, para su desarrollo se usa el lenguaje de programación PASCAL, que presenta características favorables en el manejo del modo gráfico. La versión utilizada es **TURBO PASCAL 6.0** de Borland International Inc. Los códigos para los cuales se grafica la d.e.p. son:

- a. Código NRZ unipolar..
- b. Código NRZ polar.
- c. Código AMI.
- d. Código diferencial NRZ-M polar.
- e. Código RZ polar.
- f. Código de modulación por retardo o código de Miller.
- g. Código bifase-L o código de Manchester.
- h. Código HDB3.

Tanto para el código de Miller como para el HDB3, se puede graficar la d.e.p. únicamente para la probabilidad de bit $p = 0.5$, ya que las ecuaciones correspondientes se obtuvieron para el caso en que la ocurrencia de 1_L y 0_L son equiprobables.

El programa es básicamente un graficador que usa las ecuaciones para la d.e.p. descritas en el numeral 1.5. Los parámetros de tales ecuaciones son: la amplitud de los elementos de señal que se usan en la codificación (amplitud **A**), el ritmo de entrada de los dígitos binarios al codificador (período **T**) y la probabilidad de ocurrencia de los bits que entran en el codificador.

Tanto la amplitud **A** como el período **T** son parámetros poco determinantes en la forma del espectro de potencia y constituyen solamente factores de escala. El parámetro realmente importante es la probabilidad de ocurrencia de los bits. Por ello, lo conveniente es graficar la densidad espectral de potencia normalizada respecto a la amplitud y el período, es decir $\frac{S(f)}{A^2 T}$.

Es importante aclarar que **T** es un parámetro de los bits de entrada, en tanto que **f** es la variable independiente que representa la frecuencia puntual en la cual se está evaluando la d.e.p. y por tanto no equivale al inverso del período **T**.

De acuerdo a la convención establecida anteriormente, **p** representa la probabilidad de ocurrencia de un 1_L en los bits de entrada al codificador, en tanto que **q** será la probabilidad de que ocurra un 0_L . Dado que $p + q = 1$, será suficiente ingresar el parámetro **p** para graficar la d.e.p.

Con estos antecedentes, el programa se lo estructura con dos opciones, a saber: la comparación de las densidades espectrales de potencia de varios códigos para un mismo valor de probabilidad **p**, y la visualización de la variación de la d.e.p. en función de la variación del parámetro **p**, para un esquema de codificación. A continuación se detalla la estructuración del programa.

1.6.1. Programa principal.

El programa principal en esencia consiste en una repetición continua del menú principal, el cual se encargará de enrutar el flujo del programa hacia una de las dos opciones de graficación, o de finalizar la sesión de

trabajo, tal como se indica en el flujograma de la figura 1.46.

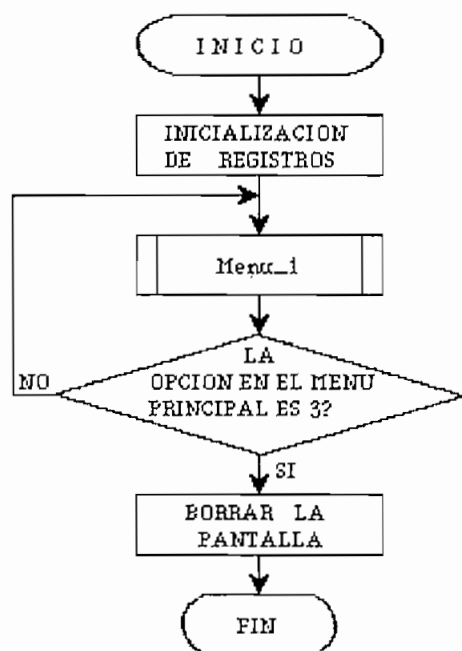


Fig. 1.46. Programa principal

El menú principal se lo implementa en el procedimiento **Menu_1**, este menú contiene tres opciones para analizar la d.e.p.:

- a. 'Según los Códigos'
- b. 'Según las Probabilidades'
- c. 'Salir al D. O. S.'

Un cursor se desplaza verticalmente para posicionarse en cada una de las opciones, permitiendo la aceptación de una de ellas con la tecla 'ENTER' (figura 1.47).

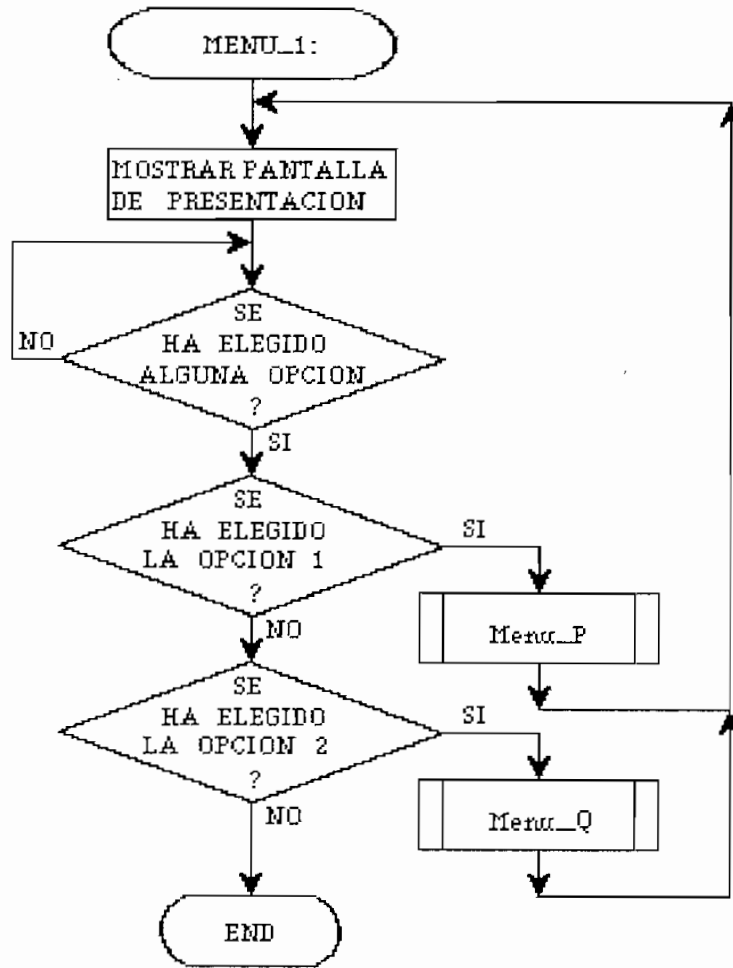


Fig. 1.47. Menú principal

1.6.2. Comparación de la d.e.p. de diferentes códigos.

Si se ha elegido la primera opción del menú principal, se ingresará a comparar de la d.e.p. de los códigos que se seleccionen. Para ello se deberán escoger los códigos a compararse y la probabilidad p . Con esta finalidad, las opciones que se presentan en pantalla son:

- 'Seleccionar los parámetros'
- 'Ver espectros de potencia'
- 'Regresar al menú principal'

En la figura 1.48 se muestra el flujograma de este menú estructurado en el procedimiento **Menu_P**.

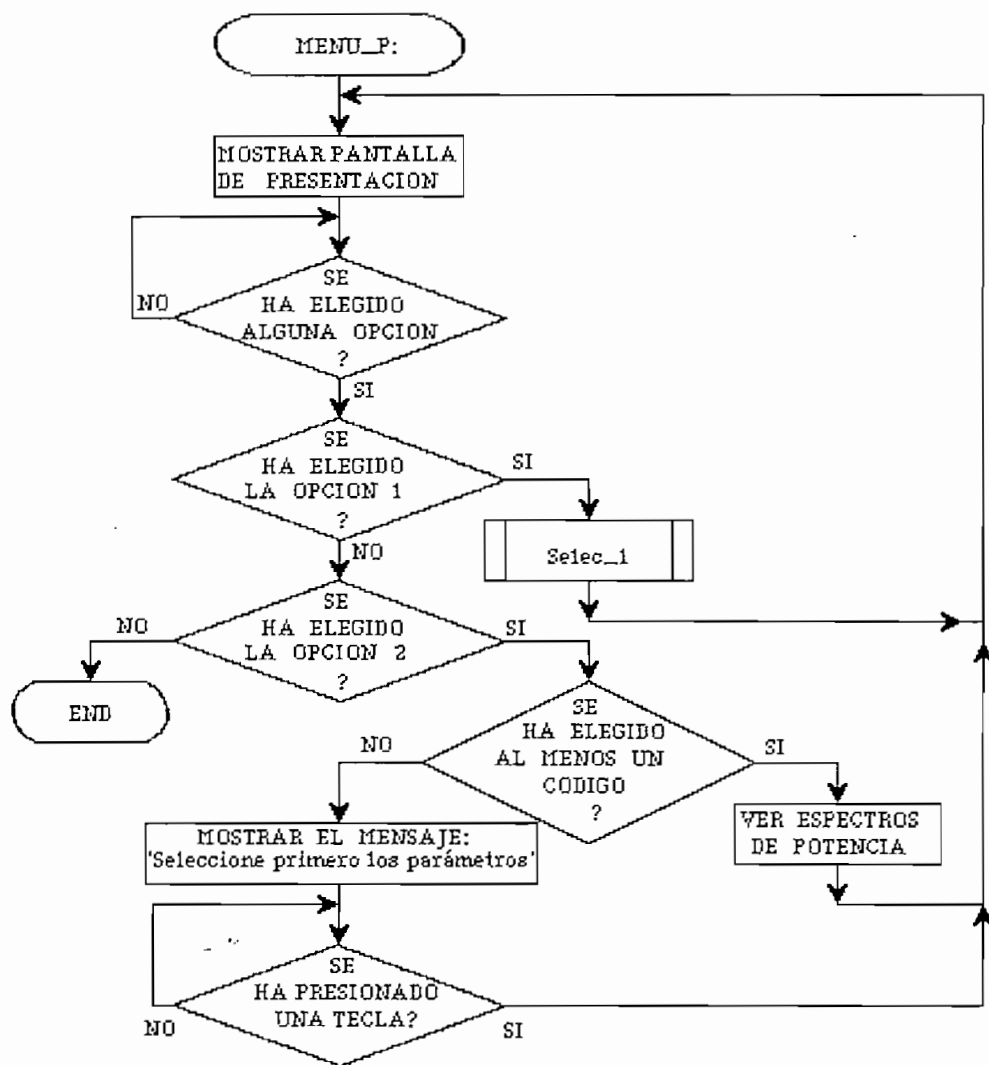


Fig. 1.48. Análisis de la d.e.p. según los códigos

La opción 'Seleccionar parámetros' permite escoger los códigos cuya d.e.p. se va a comparar y el valor de la probabilidad p que será un parámetro común en la comparación. Para ello, en la pantalla se muestra un listado de los códigos disponibles, de modo que se pueda navegar a través de esta lista mediante las teclas de desplazamiento vertical ($\uparrow$ y $\downarrow$). Para escoger uno de los

esquemas de codificación se debe llevar el cursor hasta él y presionar 'ENTER' con lo cual se activa una marca al lado izquierdo del nombre. Para eliminar un código del conjunto seleccionado, bastará con presionar nuevamente 'ENTER'; de este modo se puede seleccionar todos los códigos de línea que se desee comparar simultáneamente.

Existe además una opción para escoger el valor de la probabilidad de entrada de los 1_L al codificador; este valor es por defecto $p = 0.5$; para cambiar este valor, se deberá presionar 'ENTER' e ingresar un valor que deberá estar comprendido entre 0 y 1, si no se cumple con este limitante, se volverá a pedir que se ingrese un valor adecuado.

Una vez ingresados los parámetros se deberá regresar al menú anterior, con el fin de visualizar los espectros de los códigos seleccionados. Una ilustración de este proceso de selección se muestra en el flujograma de la figura 1.49, descrito como el procedimiento **Selec_1**.

La opción 'Ver espectros de potencia' permite visualizar los gráficos de la d.e.p. para los códigos escogidos. Si no se ha seleccionado ningún esquema de codificación, se mostrará un mensaje indicando este hecho y cualquier tecla que se presione llevará el cursor hasta la opción 'Seleccionar primero los parámetros'. Si el monitor es de color, cada código aparecerá de diferente coloración y al lado derecho se enlistará los nombres de los esquemas de codificación graficados con su correspondiente color. Un número ubicado cada gráfico permitirá establecer la correspondencia entre la curva y el nombre del respectivo código de línea.

Para ilustrar lo dicho anteriormente, en la figura 1.50 se muestra un ejemplo en el cual se compara la densidad espectral de potencia de los códigos AMI, Miller,

y Manchester para $p = 0.5$.

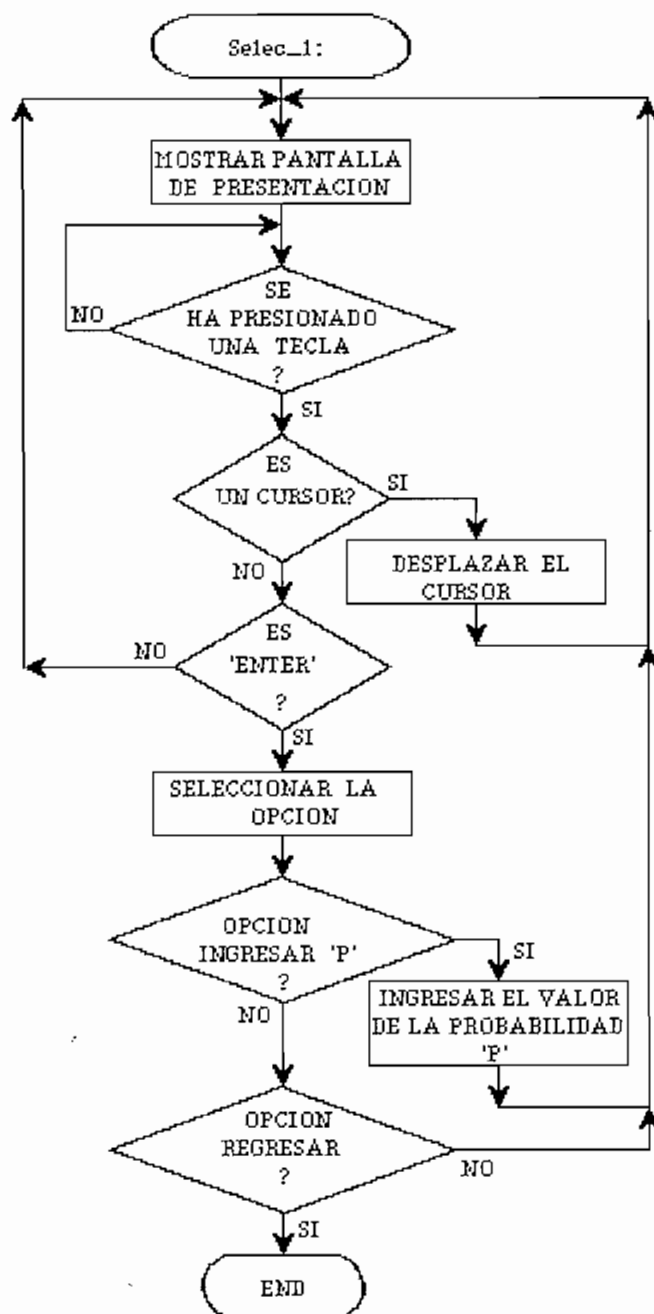


Fig. 1.49. Selección de los códigos y la probabilidad

Finalmente, la opción 'Volver al menú principal' permitirá regresar a escoger la opción para analizar la d.e.p.

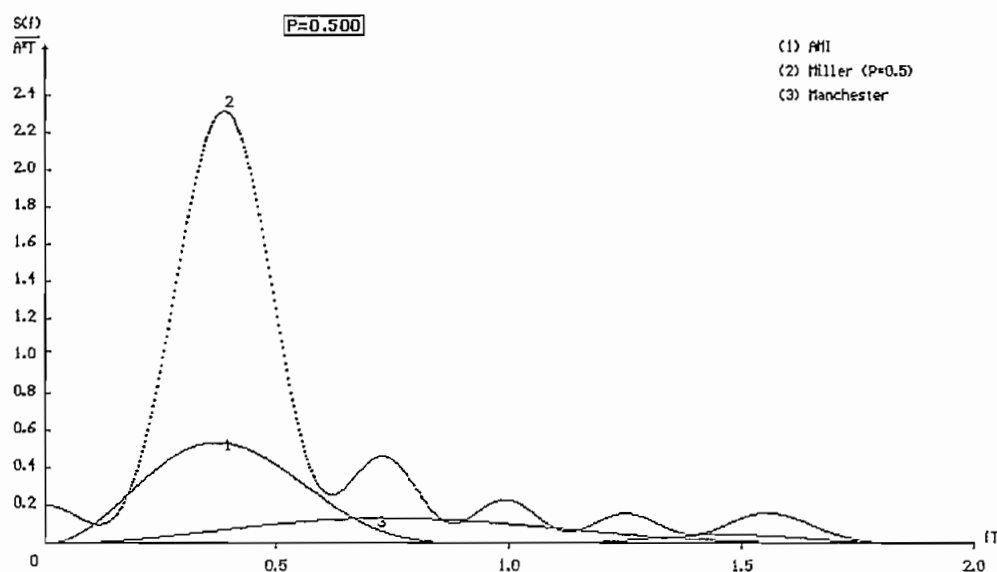


Fig. 1.50. Ejemplo de comparación de d.e.p.

1.6.3. Variación de la d.e.p. en función de la probabilidad p .

Este ítem corresponde a la segunda opción del menú principal y permite escoger un código del cual se observará la d.e.p. para tres valores de probabilidad p . Debido a que la probabilidad de ocurrencia de bits (1_L y 0_L) igual a 0.5 es la más cercana a la realidad, la comparación incluirá este caso, por lo que el usuario tendrá la posibilidad de ingresar sólo dos valores de probabilidad como parámetros.

La pantalla de presentación muestra cuatro opciones, las mismas que son:

- a. 'Seleccionar código'
- b. 'Seleccionar probabilidades'
- c. 'Ver espectros de potencia'
- d. 'Regresar al menú principal'

La selección de las opciones se la implementa

mediante el procedimiento **Menu_Q**, cuyo flujograma se presenta en la figura 1.51.

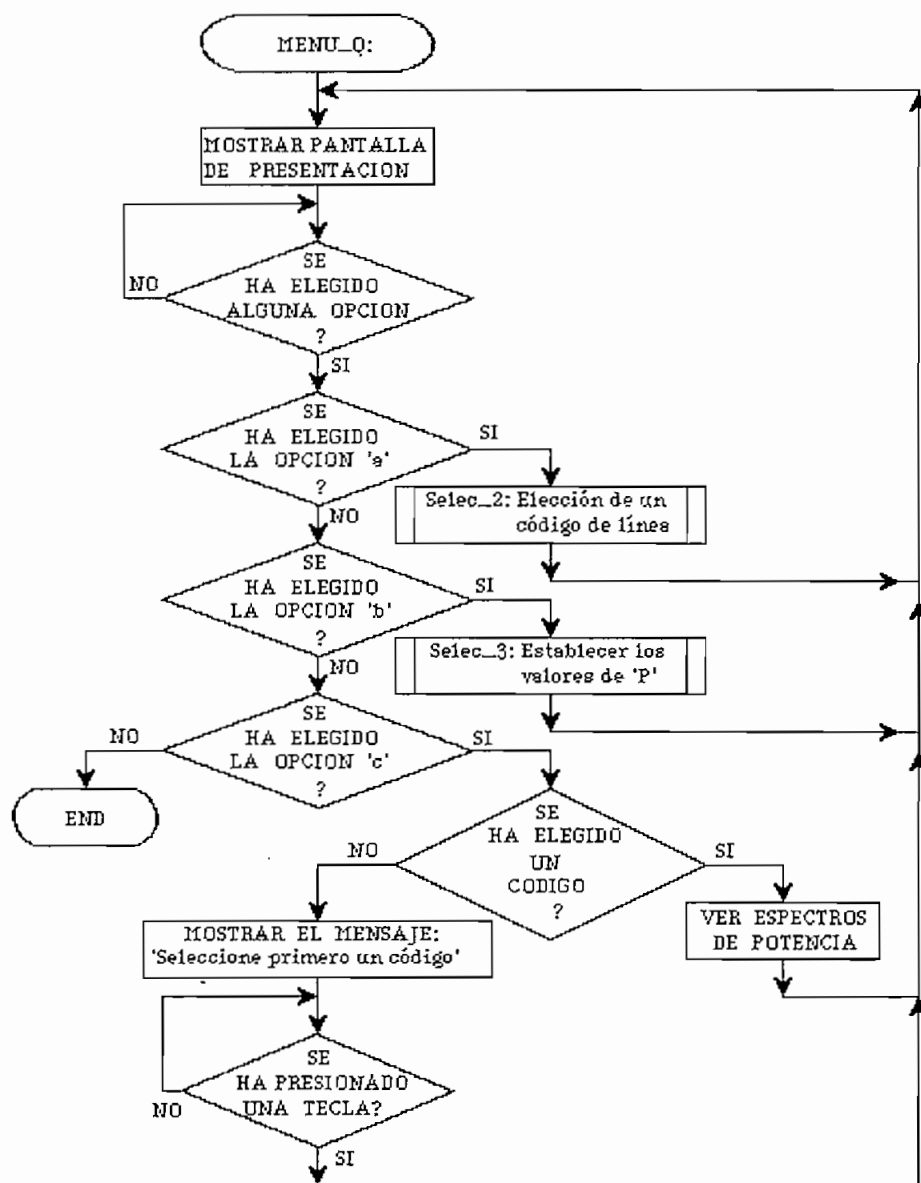


Fig. 1.51. Análisis de la d.e.p. según las probabilidades

El procedimiento **Selec_2** permite seleccionar el esquema de codificación del cual se graficará la d.e.p. para diferentes valores de probabilidad. Del listado de códigos que aparece, se selecciona uno, luego de lo cual se regresa al menú anterior.

El procedimiento **Selec_3** está diseñado para escoger los valores de probabilidad con los que variará la d.e.p. del código seleccionado. Si no se escoge ningún valor adicional, sólo se graficará para $p = 0.5$. Si en el procedimiento **Selec_3**, la opción escogida es regresar, se volverá al menú anterior y se estará listo para visualizar los espectros de potencia. Si se escoge esta opción ('Ver espectros de potencia') antes de haber elegido un código de línea, un mensaje dará esta indicación y cualquier tecla que se presione llevará el cursor a la opción de elegir primeramente el código de línea.

El proceso de visualización de los gráficos de la d.e.p. es similar al descrito en el numeral 1.6.3 con la diferencia de que ahora se indica en el gráfico el tipo de código escogido y se hace corresponder a cada gráfico un valor de probabilidad. En la figura 1.52 se tiene un ejemplo para el código RZ polar. El listado del programa fuente aquí descrito se lo puede encontrar en el ANEXO D.

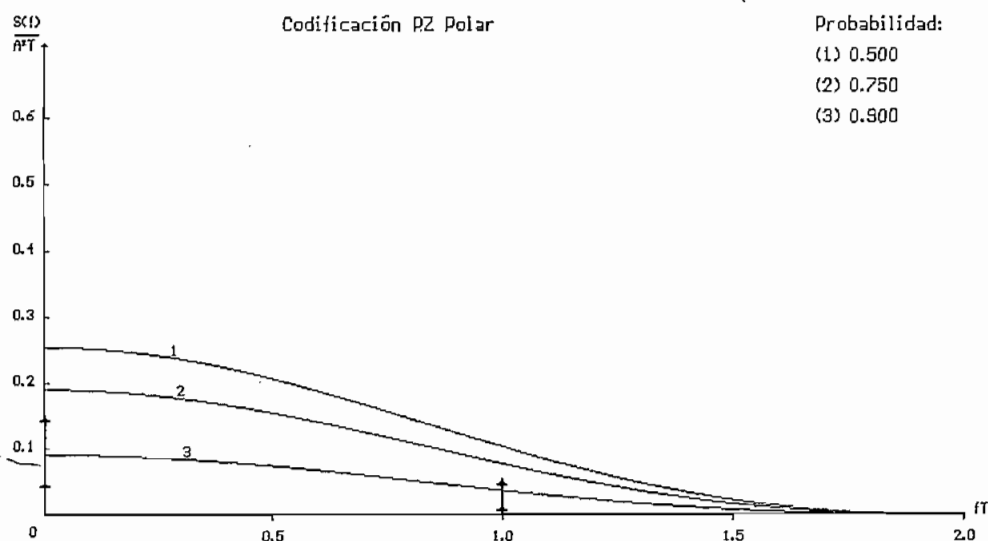


Fig. 1.52. Ejemplo del gráfico de la variación de la d.e.p.

CAPITULO II

DISEÑO Y CONSTRUCCION DEL CODEC

2.1. ESPECIFICACIONES Y REQUERIMIENTOS DEL EQUIPO.

El CODEC (codificador/decodificador) para transmisión digital en banda base se lo ha concebido como un dispositivo que permita el ingreso de señales digitales binarias (1_L y 0_L), tanto de niveles normalizados RS-232 como niveles TTL, las cuales serán tratadas por el sistema microprocesado y a la salida se tendrá una señal correspondiente al código seleccionado previamente.

Por otro lado, la decodificación permite la entrada de elementos de señal correspondientes al código en cuestión, para que sean traducidos a dígitos binarios, que tendrán en su salida niveles tanto RS-232 como TTL.

De esta forma se permite que para el estudio de los códigos de línea, se pueda ingresar al CODEC directamente una secuencia determinada de dígitos binarios de niveles TTL o acceder con señales de niveles RS-232 (convertidos posteriormente a niveles TTL) y proporcionados en general por un dispositivo de comunicación serial, como es el pórtico de un terminal de comunicaciones o un computador personal.

La figura 2.1 muestra un esquema en el que se presenta la ubicación del CODEC dentro de un sistema de transmisión de datos. De esta figura se puede concluir que el CODEC debe ser un dispositivo transparente para el sistema de comunicación, de tal forma que mejore la calidad de la comunicación, pero que no se constituya en un elemento más a ser supervisado por el controlador de la

comunicación.



Fig. 2.1. Esquema de transmisión de datos en banda base

Los dos parámetros básicos que presenta el CODEC son: los tipos de códigos y las velocidades de transmisión estandarizadas a las que le es posible trabajar. Al CODEC se lo puede describir globalmente como un sistema microprocesado, en el que para establecer los parámetros mencionados, se debe disponer de un medio que permita realizar la configuración inicial. Con esta finalidad se dispondrá de un teclado y de un sistema de visualización alfanumérico que permita una comunicación entre el operador y el microcontrolador.

Los códigos a implementarse son:

- 1) El código NRZ polar, de gran importancia por ser el usado en la norma EIA RS-232-C y en la recomendación V.24 del CCITT;
- 2) El código AMI que es también de gran difusión sobre todo en sistemas de transmisión PCM;
- 3) El código RZ polar, con la finalidad de comprobar sus bondades en la recuperación de la señal de reloj;
- 4) El código Manchester diferencial utilizado ampliamente en redes de computadores de área local;
- 5) El código bifase-M, por ser uno de los códigos bifase más representativos;
- 6) El código de modulación por retardo o código de Miller;

- 7) El código cuatro binario tres ternario (4B-3T);
- 8) El código MS43 que junto con el 4B-3T son los más importantes entre los códigos con variación de la velocidad de transmisión codificada;
- 9) El código B3ZS y;
- 10) El código HDB3 muy utilizado en sistemas jerárquicos PCM y que como tal es recomendado por el CCITT.

La complejidad en la implementación de los codificadores/decodificadores para cada uno de estos 10 códigos de línea, varía de acuerdo a la complicación que presente su algoritmo. Se buscará que el hardware sea lo más general posible para todos los códigos y que sea el software el que realice cada tarea de codificación y decodificación.

Esto implica que la diferencia de complejidad en los algoritmos de codificación/decodificación será trasladada al software, lo cual se reflejará en la variación del tiempo necesario para que el sistema microprocesado realice una tarea determinada con cada bit o elemento de señal que le llega, según el código de línea implementado. Por lo tanto, el ritmo máximo de transmisión que se alcance con cada uno de los esquemas de codificación será diferente, teniendo como límite máximo una velocidad de 19200 bit/s para aquellos algoritmos simples.

En todo caso, las velocidades normalizadas que se usen para transmitir serán las siguientes: 150, 300, 600, 1200, 2400, 4800, 9600 y 19200 bit/s como máximo. Se ha tomado estos valores por ser parte de la normalización de facto para comunicación serial y por ser algunos de los ritmos de transmisión establecidos por el CCITT en la transmisión digital usando modems¹.

¹ CCITT, Recomendaciones - Libro Rojo, tomo VIII, fascículo VIII.1, Rec. V.5 - V.6, Málaga, 1984, p. 11.

En cuanto a la amplitud de los pulsos de señal que se usan en la transmisión, el CCITT los ha normalizado en la recomendación G.703 "Características Físicas y Eléctricas de los Interfaces Digitales", estableciendo valores que van desde 1 voltio hasta 3.4 voltios nominales, pasando por 2.37 y 3 voltios. La selección del valor pico de la señal se lo hace dependiendo del medio de transmisión (cable coaxial o par trenzado), de los diferentes ambientes de ruido y de las diferentes longitudes entre los equipos implicados¹.

En el diseño del CODEC, sin embargo, no se empleará ninguno de los valores normalizados de amplitud antes indicados, ya que está concebido básicamente como un dispositivo didáctico para la demostración del proceso de transmisión en banda base y no como un equipo que vaya a realizar alguna tarea específica en un sistema de comunicación existente. Se usarán entonces los niveles de tensión más comunes para los pulsos; éstos tendrán como valor pico los 5 voltios, de modo que la señal de transmisión tendrá valores que pueden ser de +5, 0 y -5 voltios.

Un diagrama de bloques general sobre las unidades que constituyen el CODEC se lo presenta en la figura 2.2. Tanto el codificador como el decodificador se realizan sobre la misma unidad central de procesamiento, y sus parámetros son inicializados y visualizados usando el mismo módulo; sin embargo el hardware de sincronización y el correspondiente a la unidad de entrada/salida es diferente para cada una de las funciones, a pesar de ello se lo considera como una sola unidad funcional que será manejada por la unidad central de procesamiento.

¹ CCITT, Recomendaciones - Libro Rojo, tomo III, fascículo III.3, Rec. G.703, Málaga, 1984, p. 44-68.

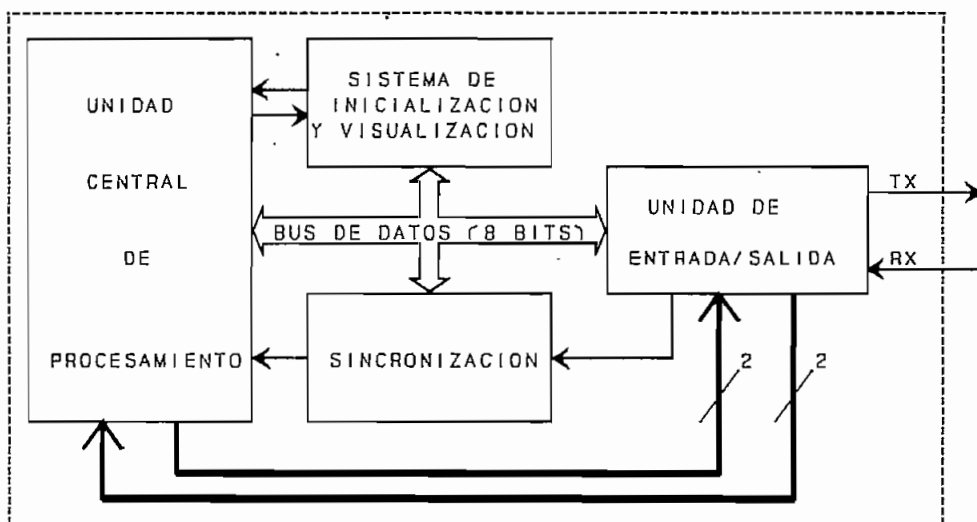


Fig. 2.2. Diagrama de bloques del CODEC

2.2. DISEÑO DEL CODIFICADOR.

La configuración del hardware del codificador está determinado por el diagrama de bloques de la figura 2.3.

Antes que el sistema entre en funcionamiento, se deberá establecer los parámetros de transmisión en cuanto al tipo de código a usarse y la velocidad de transmisión. Además se tendrá que elegir si la transmisión es *full duplex* o *half duplex* y si la señal de entrada será de niveles TTL o de niveles RS-232. Con este fin se precisa de un sistema de inicialización, que consta de un teclado y un *display* alfanumérico, los mismos que permiten que este proceso sea interactivo con el usuario.

El circuito de ingreso de datos acondiciona las señales, que pudiendo ser de niveles TTL o RS-232 deben ser procesadas en formato TTL. Estos datos, que no constituyen otra cosa que un flujo de dígitos binarios (bits), se deberán sincronizar con una señal de reloj interna que es

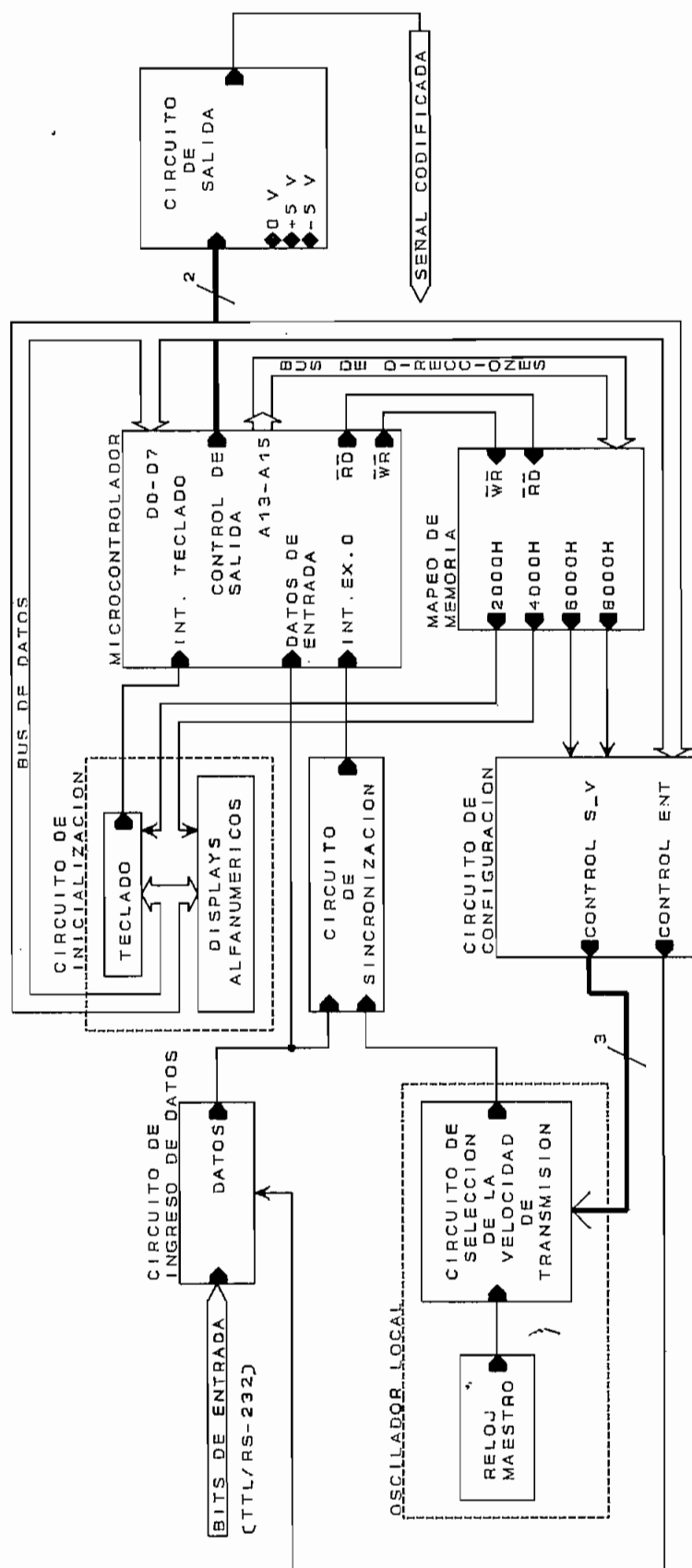


Fig. 2.3. Diagrama de bloques del codificador

la que va a indicar al microcontrolador que acepte cada bit para ser procesado.

El sistema microprocesado está basado en el uso de un microcontrolador (μC) al cual se conectan, entre otros, circuitos periféricos de entrada/salida los mismos que son tratados como localidades de memoria RAM externa, con esta finalidad se establece un circuito de mapeo de memoria.

La inicialización del sistema se realiza de manera interactiva mediante un sistema constituido por un teclado y un *display* alfanumérico. Como resultado de esta inicialización, se almacena la configuración del sistema tanto en la memoria RAM interna del microcontrolador como en el circuito externo de configuración.

Para el establecimiento del ritmo de transmisión, el microcontrolador actuará sobre el circuito de selección de velocidad a través del circuito de configuración, ejecutándose este proceso como uno de los pasos en la inicialización del sistema. El circuito de selección de velocidad usa la señal de un reloj maestro para establecer el ritmo de transmisión, al conjunto de estos dos elementos se le denomina oscilador local.

Tanto la señal de reloj como la señal de datos ingresan en un circuito que permite la sincronización de estos dos flujos de bits. La señal de reloj resultante deberá conseguir que el microcontrolador muestree o tome el dato aproximadamente a la mitad de la duración del bit.

Luego de que el bit ha sido muestreado por el microcontrolador, éste deberá encargarse de procesarlo según el esquema de codificación que se trate y emitirá las señales de control para que un circuito de salida transmita los niveles apropiados de señal (+5 V, -5 V o 0 V). El microcontrolador establecerá además la duración del nivel,

de modo que se conforme la señal codificada en amplitud y en tiempo. Finalmente un interfaz con la línea de transmisión permitirá emitir la señal codificada para que sea transmitida al otro extremo del sistema de comunicación.

2.2.1. Configuración para el microcontrolador.

Tanto la tarea de codificación como la de decodificación implican el manejo de elementos de señal que permitirán transportar los datos digitales en función de su magnitud y su duración; consecuentemente, la temporización es un factor determinante en el desarrollo del sistema. Por esta razón, el uso de un microcontrolador que además de la unidad central de proceso (CPU) disponga de un método de temporización resulta una elección óptima para constituirse en el elemento inteligente del equipo en cuestión.

Por otra parte, es necesario establecer que el procesamiento de los datos se lo realizará individualmente para cada dígito binario entrante al codificador y para cada elemento de señal que ingresa al decodificador, por lo que es de suponer que la velocidad de procesamiento de la unidad inteligente debe ser lo suficientemente grande para no permitir la pérdida de datos, de tal suerte que cuando llegue el siguiente elemento, el anterior ya haya sido procesado.

Con estos antecedentes, un microcontrolador de la familia INTEL MCS-51, con una alta velocidad de procesamiento resulta ser el adecuado. Este último requerimiento lleva a utilizar el microcontrolador P83C652-03 de Philips¹, que es equivalente al INTEL 80C52 tanto en *hardware* como en *software* pero que permite un procesamiento de hasta 20 MHz, velocidad que no es alcanzada por los

¹ Las correspondientes hojas de especificaciones se encuentran en el Anexo B.

microcontroladores estándar de la familia INTEL.

El microcontrolador es de tecnología CMOS y posee una CPU de 8 bits; 256 bytes de RAM interna; 32 líneas bidireccionales de entrada/salida, con direccionamiento individual para cada línea; dos temporizadores/contadores de 16 bits; un receptor/transmisor universal asincrónico (UART) tipo *full duplex*; dos interrupciones externas; cinco vectores de interrupción correspondientes al reset, interrupción externa 0 (EX0), interrupción externa 1 (EX1), interrupción por desbordamiento del temporizador/contador 0 (ET0) y la interrupción por desbordamiento del temporizador/contador 1 (ET1); posee además la capacidad para realizar procesamiento booleano (lógica de simple bit).

Este microcontrolador posee 8 kbytes de memoria ROM interna, pero debido a la imposibilidad de medios para grabarla no será empleada, por lo que es necesario usar una memoria externa del tipo EPROM. La configuración básica del sistema microprocesado se muestra en la figura 2.4 en donde el *buffer* 74LS244 está presente para que el microcontrolador de tecnología CMOS pueda manejar a circuitos integrados de tecnología TTL.

El circuito de reloj para el funcionamiento del microcontrolador, está provisto externamente de un cristal de cuarzo (XTAL1). Puesto que el límite máximo para el funcionamiento del microcontrolador P83C652-03 es de 20 MHz, se escoge un cristal de 18.432 MHz que es un valor estándar para aplicaciones de comunicación y que permitirá alcanzar las velocidades de transmisión serial antes indicadas. Los condensadores C2 y C3 son recomendados por el fabricante para filtrar ruido en el reloj.

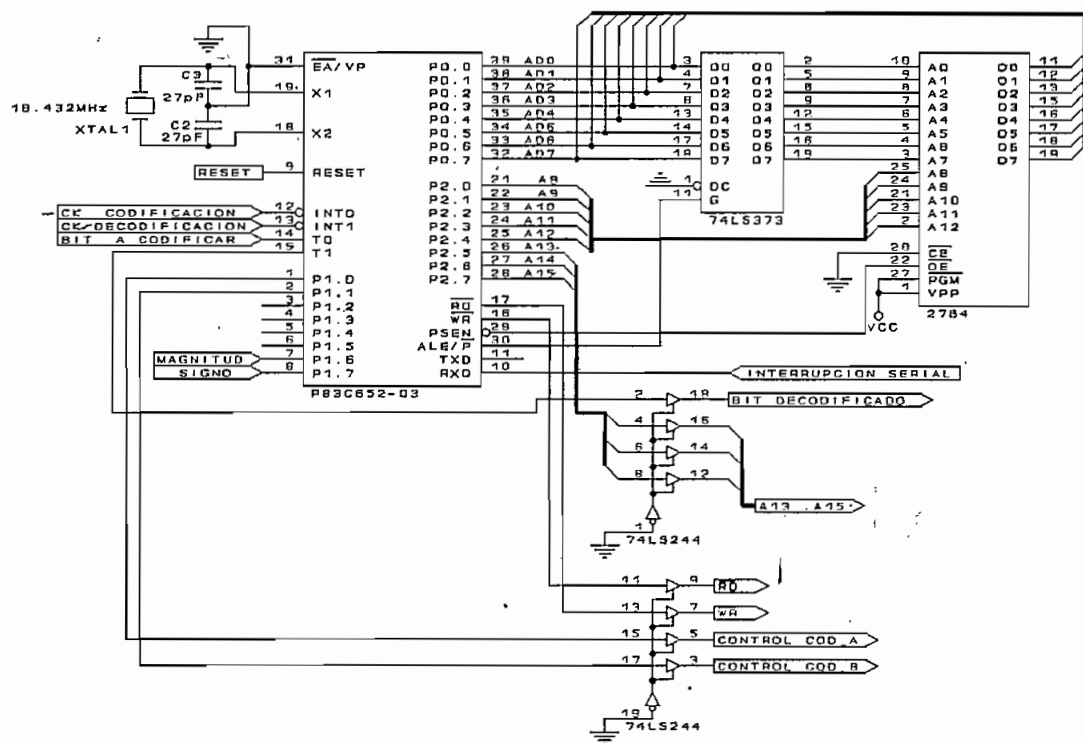


Fig. 2.4. Configuración del microcontrolador

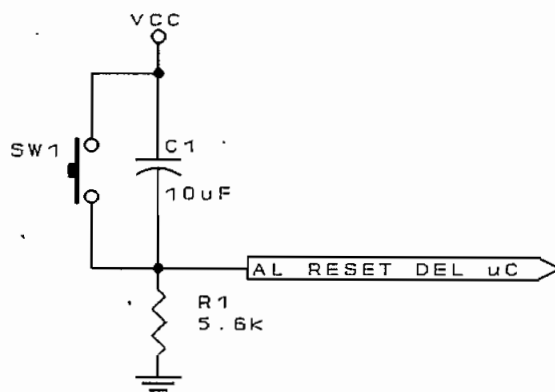


Fig. 2.5. Circuito de reset

La señal de reset se la ejecuta en su correspondiente circuito (figura 2.5) mediante un pulsador SW1, el cual al ser accionado pone un nivel alto en el pin de inicialización; en estado de reposo el nivel en este pin es bajo, manteniendo el funcionamiento del sistema. El condensador C1 y la resistencia R1 proveen el retardo necesario para que el microcontrolador sea inicializado

luego de que todo el circuito ha sido polarizado.

Puesto que la memoria de programa es externa, el pin de control $\overline{EA}$ debe ser conectado a un nivel bajo. La memoria EPROM a usarse es una 2764 de 8 kbytes (8k x 8), en consecuencia posee 13 líneas de direcciones ($A_0 - A_{12}$). El microcontrolador puede direccionar hasta 65536 localidades de memoria (64 kbytes), para lo cual hace uso de los pórtricos P0 y P2 con lo que se consigue las 16 líneas de direcciones. El bus de datos, sin embargo está multiplexado con los 8 bits menos significativos del bus de direcciones, por lo que es necesario un *latch* o retenedor (C.I. 74LS373) para almacenar los 8 bits menos significativos del bus de direcciones, mediante la activación de la señal ALE (*Adress Latch Enable*) del microcontrolador, luego de esto se liberará el pórtrico P0 que puede actuar entonces como bus de datos. Los 8 bits más significativos del bus de direcciones estarán siempre presentes en el pórtrico P2.

Luego de tener la dirección completa, se activa la señal PSEN (*Program Storage Enable*) en el microcontrolador para habilitar la salida del código del programa desde la memoria, este código entra en el microcontrolador a través del bus de datos (pórtrico P0).

Puesto que sólo se utilizan los 13 bits menos significativos para direccionar la memoria de programa, se utilizan los 3 bits más significativos para direccionar periféricos del microcontrolador, los mismos que serán tratados como localidades de memoria RAM externa. Estos periféricos son el teclado para ingreso de datos, el *display* alfanumérico, y dos retenedores (*latch*) en donde se almacenará la configuración inicial del sistema.

Tanto el *display* alfanumérico como los *latch* de configuración serán elementos de escritura, mientras que el

teclado será un periférico de lectura para el microcontrolador; la distinción entre una y otra función se realiza mediante las habilitaciones de lectura ($\overline{RD}$) y escritura ($\overline{WR}$) del microcontrolador.

Para la sincronización del transmisor y el receptor del CODEC se proveen dos relojes (CK), uno para cada función.

El funcionamiento del CODEC es básicamente el siguiente: el reloj de codificación ordena el muestreo del bit a ser codificado activando para ello la interrupción externa 0 (EX0), la misma que permitirá el ingreso del bit a través de la línea T0 (P3.4) del microcontrolador. La rutina de atención a esta interrupción controlará los pines P1.0 y P1.1 que realizan el envío de las señales de control de codificación determinada en amplitud y duración. Como se mencionó anteriormente, la amplitud de la señal de salida puede tomar los valores de +5, -5 y 0 voltios, por lo cual es suficiente disponer de dos bits para el control. En la figura 2.6 se muestra un ejemplo para la codificación AMI, en donde se observa el proceso de codificación.

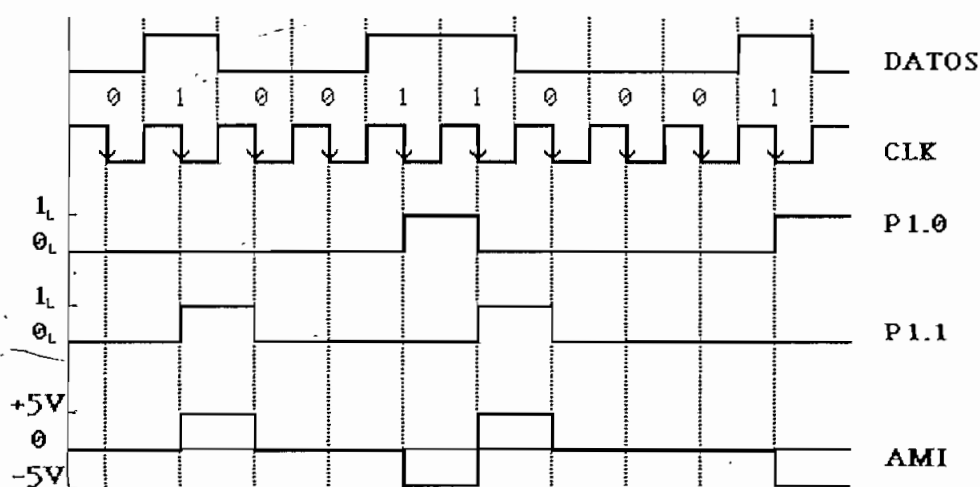


Fig. 2.6. Ejemplo del proceso de codificación

Para la decodificación el proceso es similar, sólo que en lugar de ingresar un bit para la decodificación, se requieren de dos, uno que indique la magnitud y el otro el signo de la señal que está ingresando al decodificador. Estos datos serán ingresados al microcontrolador a través de las líneas P1.6 y P1.7, cuando se activa la interrupción externa 1 (EX1) a través del reloj de decodificación. La rutina de atención a esta interrupción ocasionará que el bit original aparezca en la línea T1 (P3.5) del microcontrolador con lo que la transmisión habrá concluido.

2.2.2. Mapeo de memoria.

Como se ha manifestado antes, los dispositivos periféricos al microcontrolador se los trata como localidades de memoria RAM externa y por lo tanto deben ser habilitados para lectura o para escritura. Esta función se la realiza mediante el circuito mostrado en la figura 2.7.

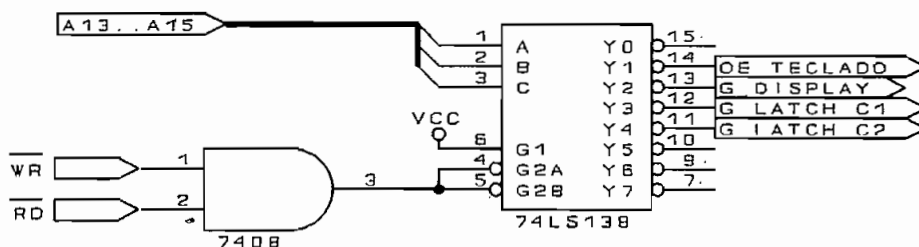


Fig. 2.7. Circuito de mapeo de memoria

Cuando se trata de leer los datos de un dispositivo externo (teclado), o escribir datos en dispositivos de salida (*display* y *latch* de configuración), se activarán las señales $\overline{RD}$ o $\overline{WR}$ del microcontrolador con lo cual se habilita el decodificador 74LS138 y de acuerdo a los bits de direcciones (A13-A15) se seleccionará uno de los dispositivos. Cada uno de los dispositivos tiene una sola función, de lectura o escritura, así que el programa que ejecuta el microcontrolador será el que discierna si se

Con la finalidad de realizar una temporización adecuada en el ciclo de escritura del *display*, las líneas de datos y las señales de control son entregadas a través de un *latch*, de modo que el microcontrolador asegure que el tiempo de permanencia de cada señal es la apropiada.

EL dispositivo requiere de una fuente de alimentación de +5 V para la polarización de los ASIC's y de una fuente negativa (-5 V) para el control de contraste del display de LCD; dicho control se lo realiza con la ayuda de un potenciómetro de 10 k Ω tal como se observa en la figura 2.8.

El teclado está conformado por tres pulsantes (SW2, SW3 y SW4), cuando cualquiera de ellos es presionado, el codificador de teclado (C.I. 74C922) activa la señal DA (*Data Available*) que ingresando por el pin de recepción serial del microcontrolador (RXD) producirá una interrupción correspondiente al p $\acute{o$ rtico serial y permitirá que la CPU ejecute una rutina de atención a esta interrupción.

Para leer el código que identifica a la tecla presionada, el microcontrolador, a través del circuito de mapeo, activa la señal **OE_TECLADO** que habilita para que el C.I. 74C922 coloque este código en el bus de datos. Con esta lectura el microcontrolador ejecuta una rutina que muestra en el display alfanumérico el mensaje correspondiente para que el usuario vaya configurando interactivamente los parámetros del CODEC. En funcionamiento normal (luego de la inicialización), el display alfanumérico mostrará el código de línea y la velocidad de transmisión que están siendo utilizados.

2.2.4. Circuito de configuración.

Una vez escogidos los parámetros con los que funcionará el CODEC, se requiere que éstos se traduzcan en códigos que controlarán algunos circuitos integrados, para ello se dispone de dos *latch* de configuración en los que se escribirán los bits de control; cada *latch* funciona como una localidad de memoria RAM externa y su diagrama circuital se muestra en la figura 2.9.

Las señales de **SELECCION DE VELOCIDAD** permiten establecer el ritmo de transmisión, en tanto que **CONTROL T/R** establece si los datos que ingresan al codificador son de niveles TTL o RS-232; las señales **CONTROL S/M**, **CONTROL V_A**, **CONTROL V_B**, **CONTROL W_A** y **CONTROL W_B** se utilizan en el proceso de decodificación y serán explicadas en el numeral 2.3.

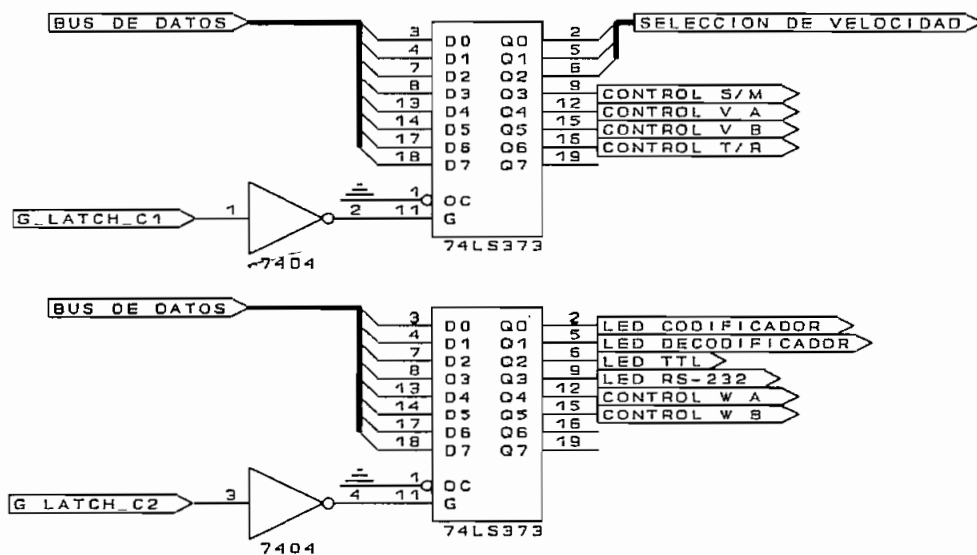


Fig. 2.9. Circuito de configuración

Las señales correspondientes a LED's, manejan diodos emisores de luz que indicarán al usuario si los niveles de los datos con los que debe alimentar al CODEC son TTL o RS-232, y si el equipo actúa como codificador, decodificador, o cumple las dos funciones.

2.2.5. Circuito de ingreso de datos.

Para el circuito de ingreso de datos que se muestra en la figura 2.10, se tiene un C.I. MAX232, el cual a través de su *receiver* convertirá niveles RS-232 en niveles TTL.

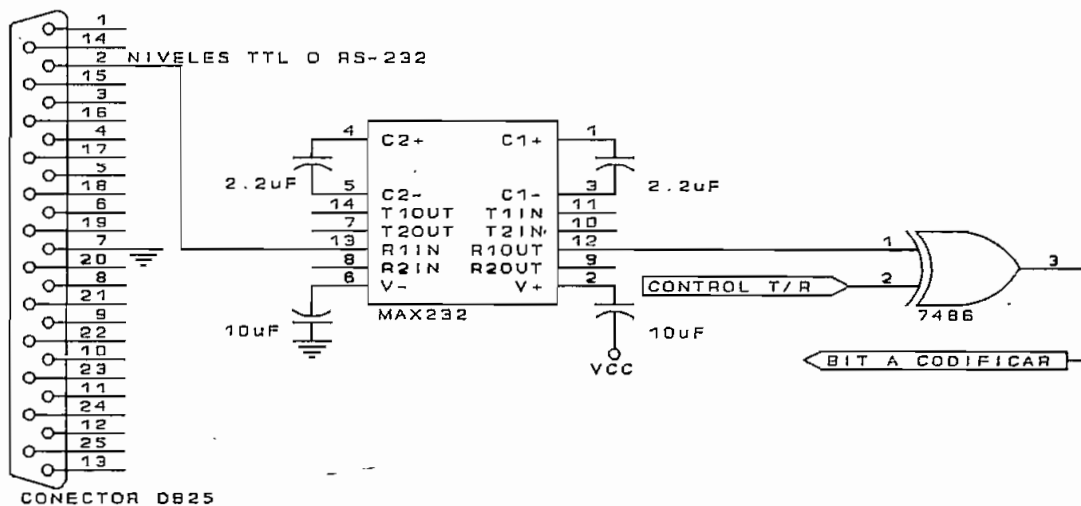


Fig. 2.10. Circuito de entrada de datos

En general, el *receiver* convertirá un nivel de voltaje negativo de máximo -30 V o un nivel cero (0 V) en un estado alto (+ 5 V) y un nivel positivo de máximo +30 V, en un estado bajo (0 V). Esto permitirá que la entrada de datos sea común tanto para niveles TTL como para RS-232, con la diferencia de que se deberá invertir los bits que entran al sistema microprocesado cuando se tenga una entrada TTL (ya que el *receiver* invertirá la lógica

positiva TTL). Con esta finalidad, el microcontrolador proporciona una señal de control (T/R) que invertirá la señal de datos, a través de una compuerta OR-Exclusiva (EX-OR), cuando ésta provenga de una fuente de niveles TTL, y la dejará pasar inalterada cuando se trate de datos con niveles de entrada RS-232.

El hecho de que exista una sola entrada de datos impide que se pueden conectar simultáneamente dos fuentes de dígitos binarios.

2.2.6. Reloj maestro.

En el sistema de transmisión que se implementa, la sincronización en la captación de bits es sumamente importante, por ello es necesario la presencia de un reloj maestro de muy buena estabilidad. Esta consideración justifica la implementación del reloj utilizando un cristal de cuarzo, el cual provee una estabilidad aceptable¹ ya que su desviación de frecuencia está en alrededor de 0.015 ppm en un mes.

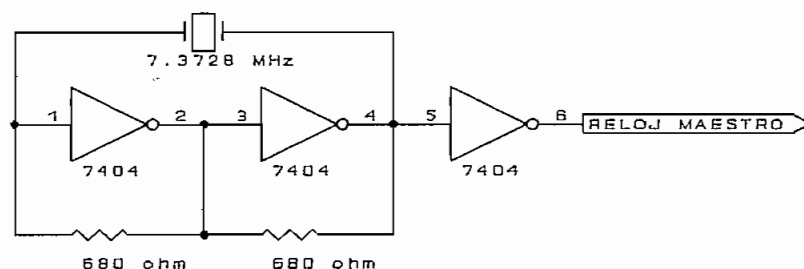


Fig. 2.11. Reloj maestro

El circuito del reloj maestro se muestra en la figura 2.11; se usan dos inversores cuyas salidas se realimentan mediante una resistencia a sus respectivas entradas, y la salida del segundo inversor se realimenta a

¹ FREEMAN R., Sistemas de telecomunicaciones, LIMUSA, México, 1991, p. 486.

la entrada del primero mediante el cristal que fijará la frecuencia de oscilación del reloj maestro. Un tercer inversor permitirá tener una mejor conformación de la forma de onda cuadrada.

El valor de la resistencia de realimentación debe garantizar que cuando se tenga un nivel alto en la entrada del inversor (y por tanto un nivel bajo a la salida), la corriente de salida en bajo (I_{OL}) no sobrepase los 16 mA que corresponden al máximo admitido para compuertas estándar, por lo tanto:

$$\begin{aligned}\frac{5V}{R} &< 16 \text{ mA} \\ R &> 312.5 \Omega\end{aligned}\tag{2.1}$$

De esto se concluye que una resistencia de realimentación de 680 Ω será adecuada.

La industria provee cristales destinados a sistemas de comunicación, cuya frecuencia de oscilación es un múltiplo de las velocidades de transmisión más usadas; en el presente caso se emplea un cristal de 7.3728 MHz, cuya frecuencia dividida adecuadamente proporciona las velocidades de transmisión requeridas.

2.2.7. Circuito de selección del ritmo de transmisión.

La frecuencia de la señal que proviene del reloj maestro, deberá ser dividida de acuerdo al ritmo de transmisión seleccionado para constituirse en la frecuencia del oscilador local. Sin embargo, esta división no se la hace directamente hasta obtener el ritmo de transmisión, sino que se divide hasta una frecuencia igual a 16 veces la velocidad de transmisión; esto debido a que la señal de reloj debe primero entrar en un circuito de sincronización que requiere este múltiplo de la velocidad. En la tabla

2.1 se detalla el factor de división que se requiere para bajar de 7.3728 MHz del reloj maestro, a 16 veces el ritmo de transmisión.

Velocidad de Tx v_t (bit/s)	$16v_t$ (kHz)	Factor de división
19200	307.2	24
9600	153.6	48
4800	76.8	96
2400	38.4	192
1200	19.2	384
600	9.6	768
300	4.8	1536
150	2.4	3072

Tabla 2.1. Factor de división

El máximo factor de división es $3072 = 16 \times 16 \times 12$. Puesto que la división de frecuencia se la realiza usando contadores, el máximo factor de división se lo obtiene de la salida más significativa de dos contadores módulo 16 (C.I. 7493) y un contador módulo 12 (C.I. 7492) conectados en cascada; el resto de velocidades se lo puede obtener si las salidas menos significativas de los contadores binarios se alimentan al contador módulo 12 a través de un multiplexor (C.I. 74151).

Lo indicado anteriormente se encuentra implementado en la figura 2.12.

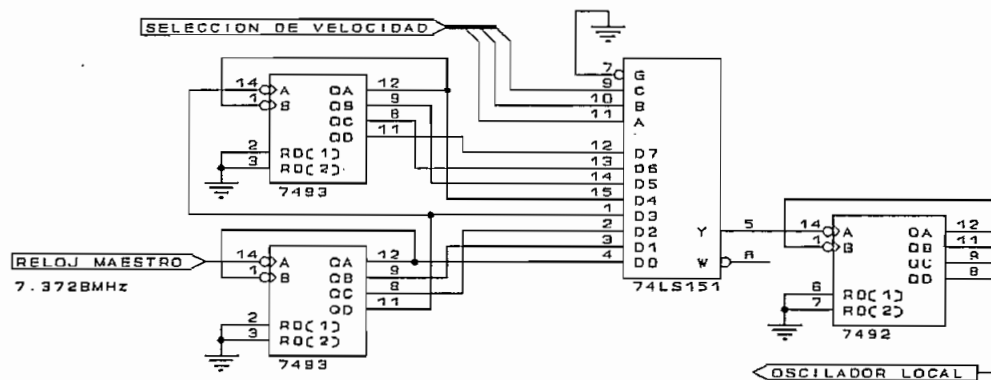


Fig. 2.12. Circuito de selección del ritmo de transmisión

El multiplexor 74LS151, controlado por tres líneas de selección, enrutará el reloj apropiado para que luego de pasar por el contador módulo 12, entre en el circuito de sincronización. Las tres líneas de selección serán provistas por el microcontrolador a través del circuito de configuración. La tabla 2.2 muestra la correspondencia entre los bits de control y el ritmo de transmisión.

C	B	A	Velocidad de transmisión (bit/s)
0	0	0	19200
0	0	1	9600
0	1	0	4800
0	1	1	2400
1	0	0	1200
1	0	1	600
1	1	0	300
1	1	1	150

Tabla 2.2. Selección de velocidad

2.2.8. Circuito de sincronización.

Para realizar la sincronización entre los bits de entrada y el reloj de codificación, se aprovechan las transiciones (positivas y negativas) en la señal de entrada

The circuit diagram shows a 4-bit counter implemented using a 7474 D flip-flop, a 7404 inverter, and a 7493 counter. The 7474 flip-flop is configured with its D input to the 'BIT DE ENTRADA' (Input Bit), its PR (Preset) input to VCC, and its CLK (Clock) input to the output of a 7404 inverter. The 7404 inverter's input is connected to the 'OSCILADOR LOCAL' (Local Oscillator) and its output is connected to the 'CK CODIFICACION' (Encoding Clock) and the clock input of the 7493 counter. The 7493 counter is configured with its A input to the 'OSCILADOR LOCAL', its B input to the output of the 7404 inverter, and its RDC(1) and RDC(2) inputs to the output of the 7404 inverter. The 7493 counter's QA, QB, QC, and QD outputs are connected to the 7485 comparator. The 7485 comparator's inputs are connected to the outputs of the 7474 flip-flop (Q and P) and the output of the 7404 inverter. The 7485 comparator's output Y is connected to the output of the 7493 counter. The 7493 counter's output Z is connected to the output of the 7485 comparator. The 7474 flip-flop's Q output is connected to the output of the 7485 comparator. The 7474 flip-flop's P output is connected to the output of the 7485 comparator. The 7474 flip-flop's C output is connected to the output of the 7485 comparator. The 7474 flip-flop's L output is connected to the output of the 7485 comparator. The 7474 flip-flop's O output is connected to the output of the 7485 comparator. The 7474 flip-flop's output is connected to the output of the 7485 comparator.

Los diagramas de tiempo correspondientes al circuito de sincronización se muestran en la figura 2.14.

-114-

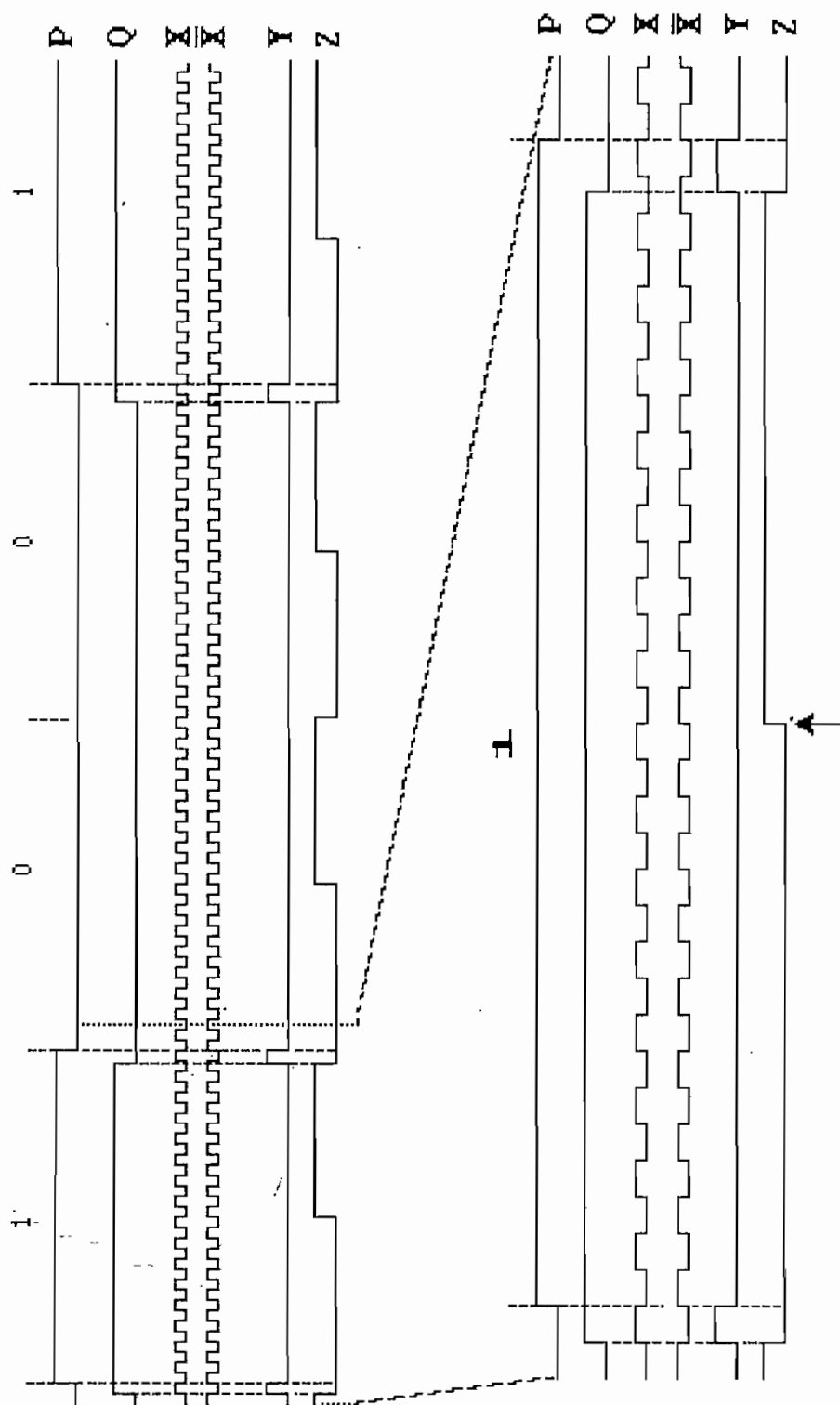


Fig. 2.14. Diagramas de tiempo en la sincronización

La salida más significativa del contador (señal Z), corresponderá al reloj sincronizado cuya frecuencia se ha dividido para 16 ya que a la entrada del circuito de sincronización se tiene este múltiplo de frecuencia; de este reloj sincronizado se observa que su flanco positivo se encuentra aproximadamente en la mitad de la duración del bit. Sin embargo, se debe usar el flanco negativo porque es ésta la transición reconocida por la interrupción externa 0, de allí la necesidad de usar un inversor que entrega finalmente la señal del reloj de codificación.

2.2.9. Circuito de selección del nivel de salida.

La señal codificada correspondiente al tren de bits de entrada, se la obtiene mediante el envío o la ausencia de pulsos de +5 y -5 voltios, según el esquema de codificación en uso. Estos niveles de salida son seleccionados mediante dos bits que el microcontrolador envía a un multiplexor analógico (C.I. 4052) como se observa en la figura 2.15.

Las entradas correspondientes a 0 V, +5 V y -5 V aparecerán a la salida por espacios de tiempo correspondientes al menos a la mitad del período de bit, en tanto que la entrada X3 correspondiente a la presencia del oscilador local, tendrá una duración de 1/8 del tiempo de bit. Esta entrada entregará pulsos cuya frecuencia corresponde a 16 veces el reloj de transmisión, de modo que ubicados al inicio del período permitan la sincronización a nivel de bit para los códigos bifase, en donde la información transmitida está contenida en la fase de las señales digitales.

La presencia de estos pulsos de sincronismo no es arbitraria ya que debido a la dificultad de extraer la información de la fase de la señal en banda base, es conveniente utilizar algunos ciclos de sincronismo que

permitan establecer claramente el inicio de un elemento de señal¹.

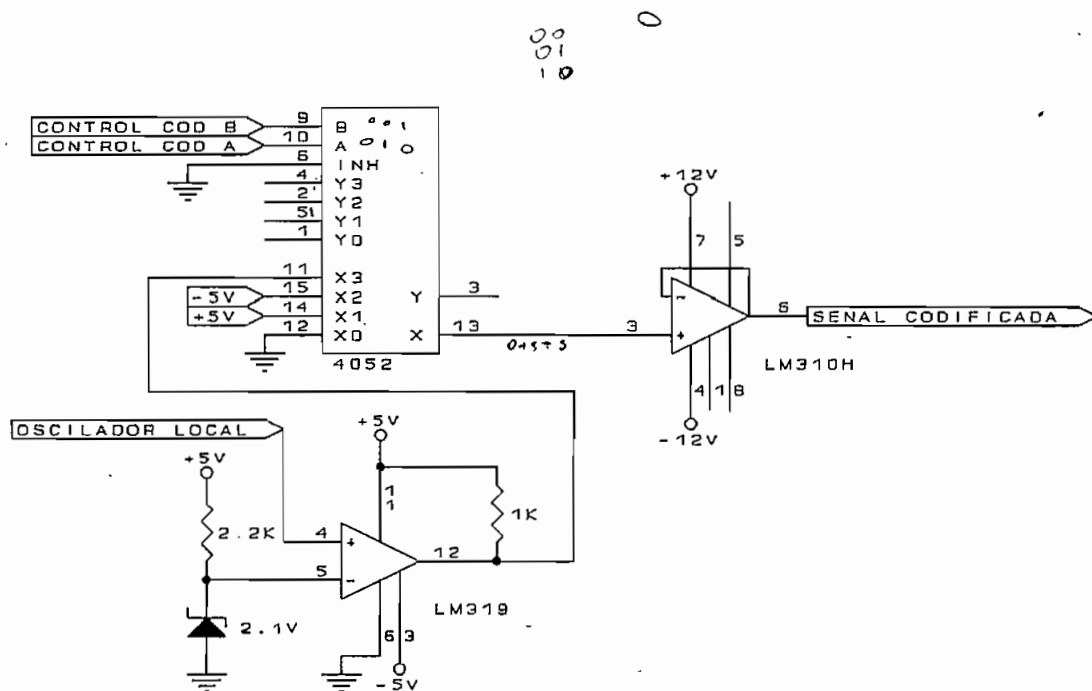


Fig. 2.15. Selección del nivel de salida

Estos pulsos de sincronismo se forman de modo que alternen entre +5 V y -5 V evitando la presencia de una componente continua; para obtener estos niveles se usa un comparador de voltaje de alta velocidad (C.I. LM319) de manera que cuando en su entrada se tenga un 0_L se transmitirá un pulso de -5 V en tanto que si entra un 1_L se transmitirá un pulso de +5 V.

La señal de salida se la obtiene a través de un circuito de tecnología CMOS por lo que es necesario proveer a la salida de un buffer que permita manejar niveles de corriente correspondientes al lazo de transmisión. Considerando que el diseño del receptor será tal que presente en su etapa de entrada una alta impedancia, será

¹ NATIONAL SEMICONDUCTOR, Interface, bipolar LSI, bipolar memory, programmable logic databook, Santa Clara, CA, 1983, p. 9-6.

suficiente usar un seguidor de señal con la característica que tenga un gran ancho de banda para transmitir de manera adecuada los pulsos de señal. Con estas consideraciones, se emplea el C.I. LM310 que permite manejar una corriente de 4 mA con un ancho de banda de 20 MHz y que requiere de un máximo de 10 nA a su entrada.

2.3. DISEÑO DEL DECODIFICADOR.

El diseño del hardware para el decodificador es muy similar al del codificador, tendiendo a efectuar el proceso inverso de la codificación. En la figura 2.16 se observa el diagrama de bloques para el decodificador.

La configuración del microcontrolador es la misma que se ha realizado para el proceso de codificación, las señales para los dos procesos son independientes y estarán presentes simultáneamente en el microcontrolador.

Por otra parte, los parámetros que se inicializan para la codificación sirven también para la decodificación, por lo cual, el proceso de inicialización a través del teclado y del display es uno solo.

Se utiliza además la misma salida del oscilador local antes analizado y sólo se tendrá una variación en la selección de la frecuencia empleada para la sincronización del reloj de decodificación.

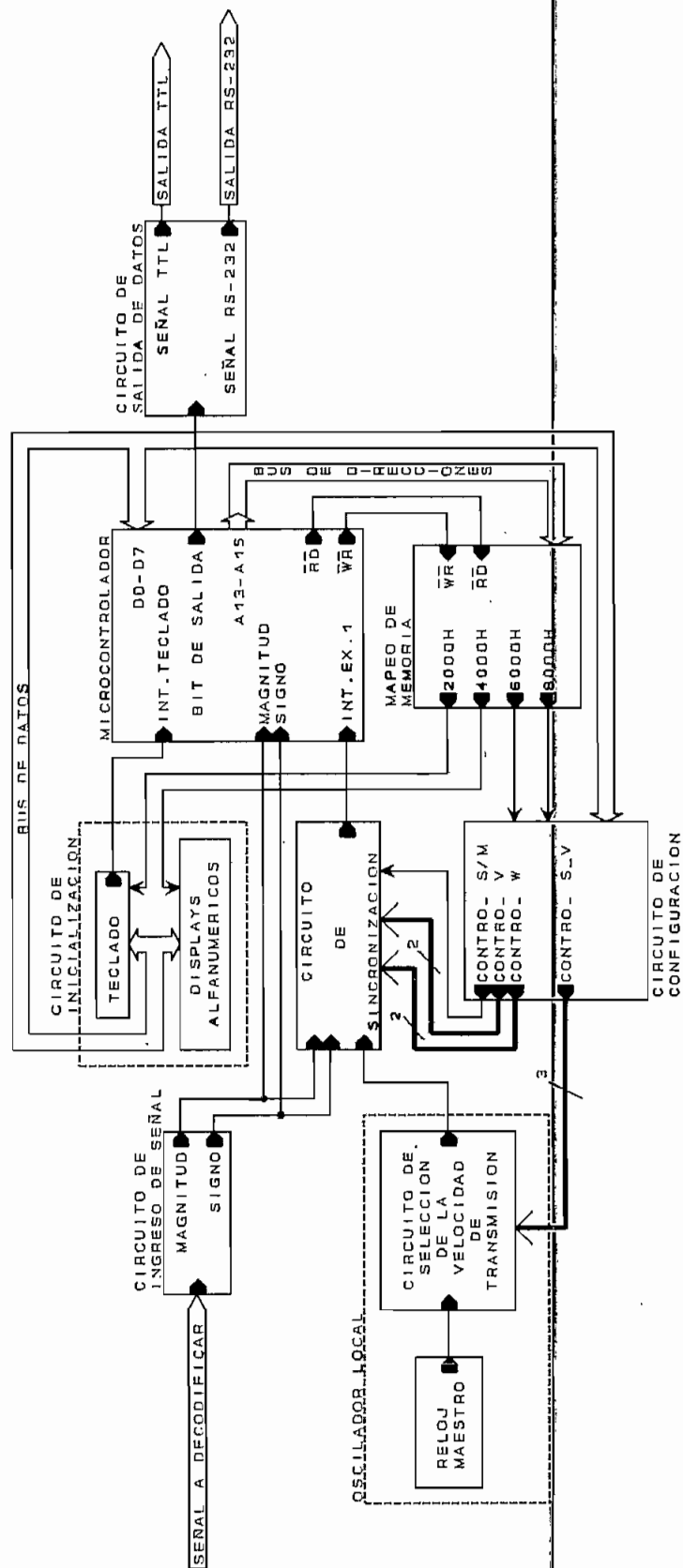


Fig. 2.16. Diagrama de bloques del decodificador

2.3.1. Circuito de ingreso de señal.

La señal a ser decodificada puede tener un valor positivo, negativo o cero, por lo que ésta quedará completamente definida si se conoce su magnitud (valor absoluto) y el signo de su polaridad. El obtener esta información es la función básica del circuito de ingreso de señal; este circuito proveerá de dos bits que serán ingresados al microcontrolador y servirán a la realización de la decodificación. En la figura 2.17 se tiene el circuito empleado para obtener la magnitud y el signo.

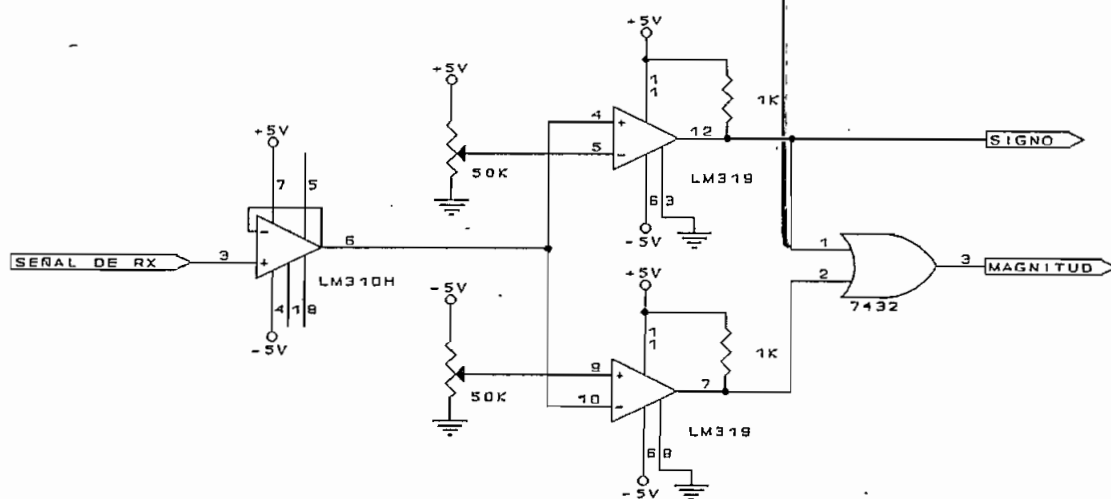


Fig. 2.17. Circuito de ingreso de señal

Como primer elemento de este circuito se tiene un seguidor de tensión que permite acoplar la señal de entrada, éste es un C.I. LM310 que al ser empleado también como interfaz de salida, garantiza un adecuado manejo de los niveles de corriente ya que el nivel de entrada no requiere más de 10 nA para su funcionamiento. La salida de este circuito será alimentada por dos comparadores (C.I. LM319), uno para niveles positivos y otro para niveles negativos, los voltajes de referencia se establecen mediante dos potenciómetros que se ajustarán de acuerdo a los niveles de entrada, de modo que se

pueda obtener la magnitud y el signo de las señales entrantes.

Cuando exista un voltaje positivo o negativo, uno de los comparadores pondrá un nivel alto en su salida; si se alimentan estas dos señales a una compuerta OR se tendrá el valor absoluto o magnitud de la señal a decodificar.

El signo se lo obtiene directamente del comparador de valores positivos, de modo que cuando exista un voltaje positivo a la entrada, este bit tendrá un nivel alto, en tanto que si el voltaje es negativo, este bit estará en bajo. Cuando la señal de entrada esté en cero, tanto la magnitud como el signo tendrán en su salida un nivel bajo.

2.3.2. Circuito de selección del ritmo de transmisión.

Para la mayoría de esquemas de codificación, la frecuencia del reloj de codificación es la misma que se debe usar en la decodificación; sin embargo, para los códigos 4B-3T y MS43 la velocidad de transmisión codificada es disminuida a $3/4$ de la velocidad de ingreso de bits; esto implica que el reloj de decodificación, responsable de muestrear la señal de entrada debe operar también a $3/4$ del ritmo de transmisión nominal.

Por lo dicho anteriormente se ve la necesidad de disponer para la decodificación, tanto del ritmo de transmisión nominal como del ritmo disminuido a $3/4$ del nominal. En la tabla 2.3 se muestra el factor en el que se debe reducir la frecuencia del reloj maestro de 7.3728MHz para obtener 16 veces la velocidad de transmisión reducida.

Ritmo de Tx v_t (bit/s)	$\frac{3}{4}v_t$ v_c (baudios)	$16v_c$ (kHz)	Factor de división
19200	14400	230.4	32
9600	7200	115.2	64
4800	3600	57.6	128
2400	1800	28.8	256
1200	900	14.4	512
600	450	7.2	1024
300	225	3.6	2048
150	112.5	1.8	4096

Tabla 2.3. Factor de división

Como se puede observar, el máximo factor de división es $4096 = 2^{12}$, que se consigue conectando en cascada tres divisores de frecuencia módulo 16 (2^4). Los demás factores se obtendrán conectando las salidas menos significativas de los dos primeros divisores de frecuencia al tercero a través de un circuito multiplexor (C.I. 74151) tal como se indica en la figura 2.18.

A la salida del multiplexor 8 a 1 se tiene la señal del reloj maestro dividida para 2^8 y se requiere de un nuevo divisor de frecuencia de 4 bits (C.I. 7493) para obtener el máximo factor de 2^{12} , con esto se consigue la señal necesaria para obtener la velocidad de transmisión reducida. Pero además se necesita para la mayoría de códigos, la velocidad de transmisión nominal; para ello se usa el mismo sistema de selección que se implementó para el codificador, éste consta de los dos divisores módulo 16 mencionados anteriormente y de un divisor de frecuencia módulo 12 adicional.

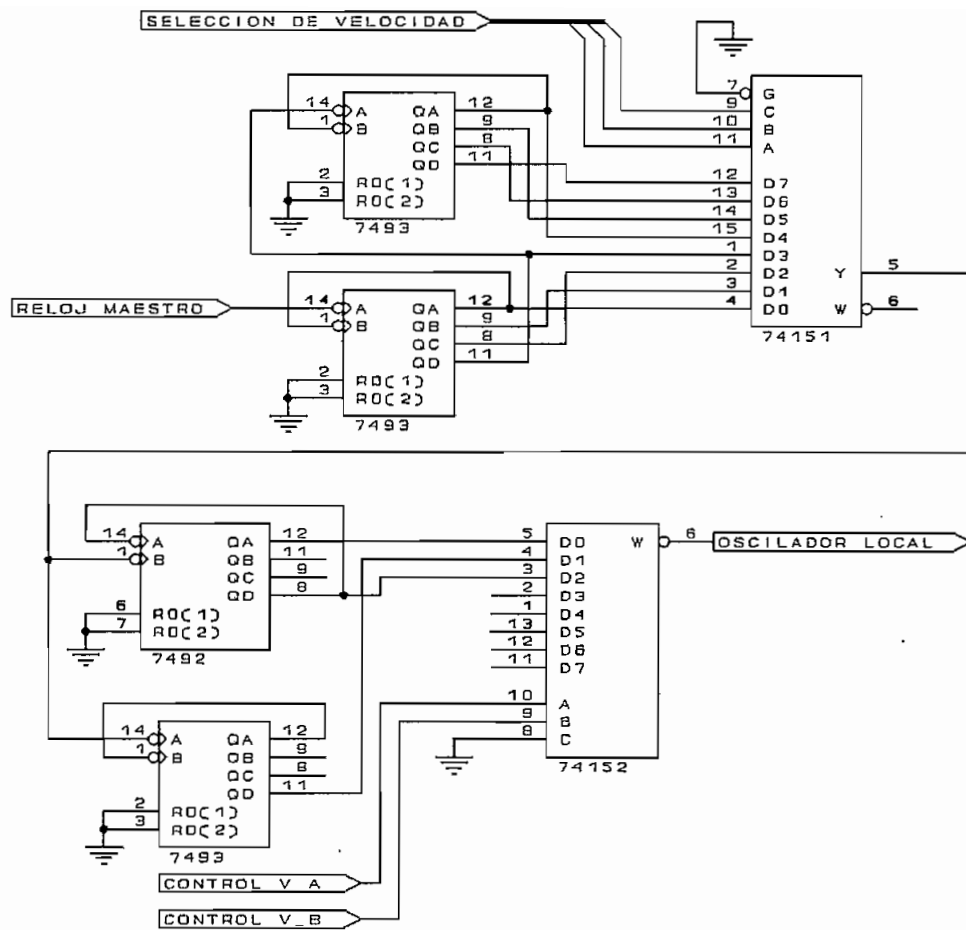


Fig. 2.18. Selección del ritmo de transmisión

Luego de tener estas dos señales de temporización, es necesario un medio para escoger la que corresponda a una aplicación particular. Esto se lo hace mediante un multiplexor (C.I. 74152) que con dos líneas de control enrutará la señal adecuada para el circuito de sincronización, el mismo que entregará el reloj de decodificación correspondiente.

2.3.3. Circuito de sincronización.

La realización del proceso de decodificación requiere conocer donde empieza y donde termina cada estado

de la señal recibida para efectuar el muestreo de la misma. Con esta finalidad se pueden usar dos métodos.

El primero consiste en enviar por una línea independiente a la de datos, una señal de reloj que indique el centro de la duración de los estados de la línea de datos. La segunda opción es la de usar las transiciones de los datos de entrada para recuperar la señal de reloj, ya sea empleando estas transiciones para generar una señal de reloj o empleando un reloj local de alta estabilidad que será puesto en fase con los datos ingresados.

En el caso del presente trabajo se usan las dos posibilidades del segundo método, la primera en el caso de los códigos bifase y la segunda en el resto de casos.

La implementación para el caso en que se usan las transiciones de los datos para generar la señal de reloj, se la aplica específicamente para los códigos bifase (Manchester diferencial, bifase M y Miller); se emplea para ello una circuitería adicional para recuperar los pulsos de sincronismo que se transmiten junto con los datos.

Esta circuitería, implementada para ritmos de transmisión de hasta 2400 bit/s, usa circuitos monoestables (C.I. 74122) cuya duración programable permite obtener pulsos de duración igual a $1/8$ del tiempo de bit y cuyo flanco negativo dará la orden para que el microcontrolador muestree el elemento de señal entrante (figura 2.19).

Para el resto de códigos de línea, la recuperación de la señal de reloj se la realiza poniendo en fase con los datos de llegada un reloj local, que en ausencia de transiciones mantendrá su frecuencia estable e igual a la frecuencia del reloj requerido para la decodificación, mientras que su fase será concordante con la obtenida en la última transición en la que se produjo la sincronización.

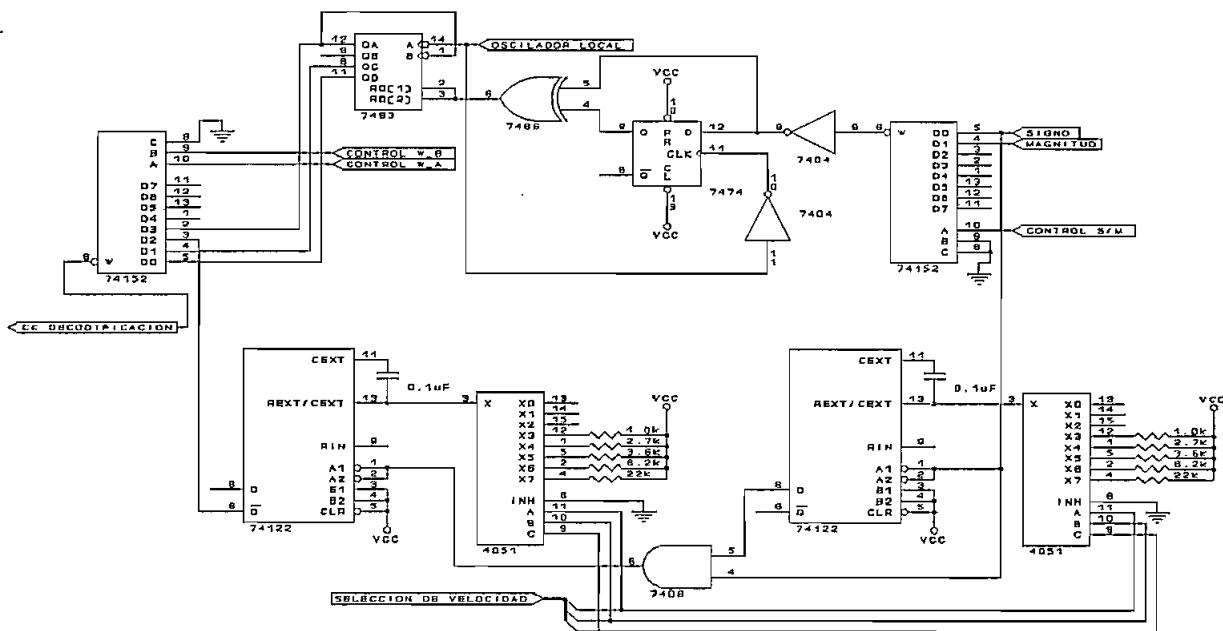


Fig. 2.19. Recuperación de la señal de reloj

El circuito empleado con este fin es el mismo que se utilizó para la sincronización de los datos con el reloj de codificación, con la diferencia que en el presente caso se tienen dos opciones para realizar la sincronización, usando la señal del signo o de la magnitud de la señal entrante.

El escoger la una o la otra opción dependerá de cuál de las dos señales presente el mayor número de transiciones, ya que serán éstas las que permitan realizar la sincronización. Con este criterio, en la tabla 2.4 se muestra la correspondencia entre el tipo de código y la señal utilizada en la recuperación de la señal de reloj.

CODIGO	SEÑAL UTILIZADA
NRZ polar	Magnitud
AMI	Signo
RZ polar	Magnitud
4B-3T	Signo
MS43	Signo
B3ZS	Magnitud
HDB3	Magnitud

Tabla 2.4. Señales usadas en la recuperación de la señal de reloj

La figura 2.19 muestra el circuito usado para la recuperación de la señal de reloj, en donde el multiplexor (C.I. 74152) es el encargado de escoger la señal adecuada que se emplea como reloj de decodificación.

Se debe realizar además otra consideración, cuando uno de los esquemas de codificación posee transiciones tanto al inicio como en la mitad del período de reloj se requerirá de un reloj del doble del ritmo de transmisión nominal para muestrear el estado de la señal tanto en la primera como en la segunda mitad del período. Además, para ciertos esquemas como el NRZ unipolar y AMI, es conveniente realizar al menos dos muestreos en cada período de bit con lo cual se tendrá mayor certeza en la decodificación del elemento de señal.

Con esta finalidad en el multiplexor de salida (C.I. 74152) se puede escoger, según las señales de control W_A y W_B, una señal de reloj de frecuencia simple o doble de la del ritmo de transmisión y que se constituirá definitivamente en el reloj de decodificación.

2.3.4. Circuito de salida de datos.

Luego de que al microcontrolador han ingresado la magnitud y el signo de la señal, éste ejecutará el

procesamiento necesario (programa correspondiente) para sacar por la línea de transmisión respectiva el bit decodificado. El nivel de la señal que entrega el microcontrolador es TTL y luego de pasar por un *buffer* será una de las salidas que presente el CODEC. Para obtener la señal en niveles RS-232, la señal TTL se ingresará al *driver* contenido en el CI MAX232, como se indica en la figura 2.20.

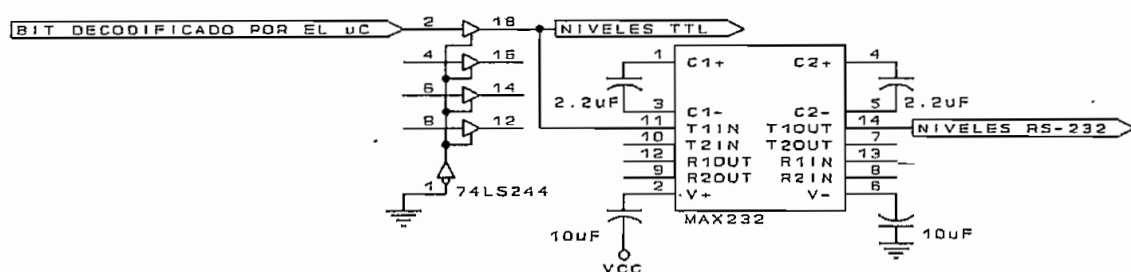


Fig. 2.20. Salida de datos luego de la decodificación

La circuitería descrita en los numerales 2.2 y 2.3 constituyen en conjunto el CODEC en banda base y su implementación usando un solo microcontrolador se la puede encontrar en la figura 2.21.

2.4. CONSTRUCCION DEL EQUIPO.

La construcción del codificador/decodificador en banda base, cuyo diagrama circuital completo se ilustra en la figura 2.21, se la realiza empleando la técnica de cableado (*wire wrap*).

Esta técnica se basa en el uso de zócalos de patas largas que facilitan la colocación y reemplazo de los circuitos integrados; estos zócalos se colocan sobre una tarjeta perforada de modo que las conexiones pertinentes se realizan en la parte inferior de la misma. La tarjeta

donde se realiza la implementación completa del circuito mide 6" x 8" (15.24 cm x 20.32 cm), el cable empleado para las conexiones es el 30 AWG.

La realización del equipo se concibe de tal manera que la tarjeta principal disponga de conectores de entrada por donde ingresarán los datos a procesarse así como también la alimentación, y conectores de salida que permiten monitorear las señales procesadas por el equipo.

En la figura 2.22 se observa la distribución de los elementos en la tarjeta del CODEC. En ésta no se contemplan los LED's, los pulsadores (teclado), el *display* alfanumérico, ni los conectores DB9 y DB25 ya que todos estos elementos serán montados en la caja del equipo por lo que las señales pertinentes serán obtenidas de los conectores que con este propósito tiene la tarjeta.

Se debe destacar que con la finalidad de realizar pruebas de transmisión *half duplex* y *full duplex*, se construyen dos equipos independientes. El listado de los elementos utilizados en la construcción de la circuitería de cada una de las tarjetas principales de los equipos es el que se detalla a continuación.

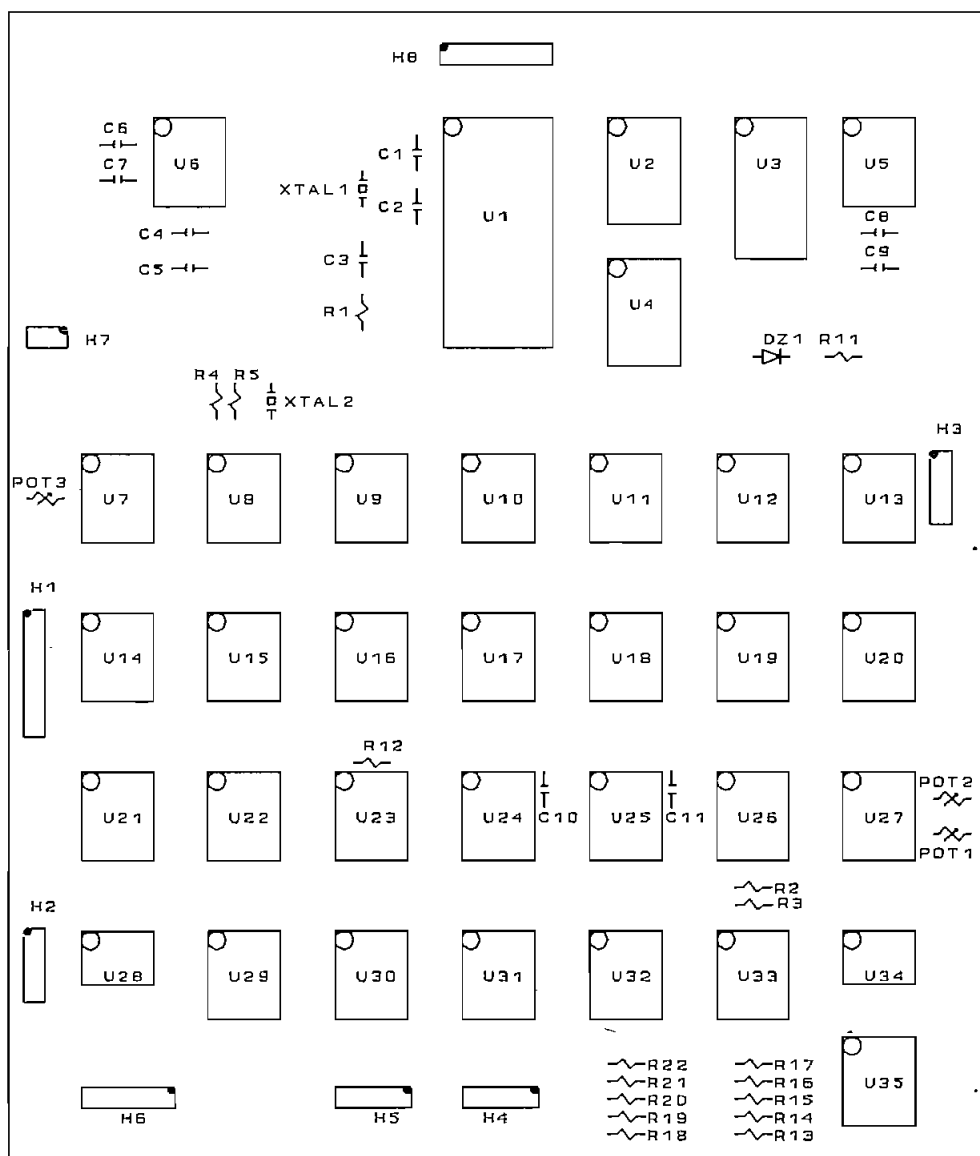


Fig. 2.22. Distribución de componentes de la tarjeta

Listado de elementos utilizados para cada tarjeta

Item	Cantidad	Referencia	Parte
1	2	C1,C2	27 pF
2	3	C3,C4,C5	10 μ F
3	2	C6,C7	2.2 μ F
4	1	C8	1 μ F
5	3	C9,C10,C11	0.1 μ F
6	5	D1,D2,D3,D4,D5	LED

7	1	DZ1	Zener 2.4 V
8	1	P1	CONECTOR DB25
9	1	P2	CONECTOR DB9
10	2	POT1,POT2	50 k Ω
11	1	POT3	10 k Ω
12	4	Q1,Q2,Q3,Q4	TRANS. 2N2222
13	1	R1	5.6 k Ω
14	3	R2,R3,R12	1 k Ω
15	2	R4,R5	680 Ω
16	5	R6,R7,R8,R9,R10	220 Ω
17	1	R11	2.2 k Ω
18	2	R13,R18	1.0 k Ω
19	2	R14,R19	2.7 k Ω
20	2	R15,R20	3.6 k Ω
21	2	R16,R21	6.2 k Ω
22	2	R17,R22	22 k Ω
23	4	SW1,SW2,SW3,SW4	SW pulsador
24	1	U1	μ C P83C652-03
25	4	U2,U14,U21,U22	C.I. 74LS373
26	1	U3	C.I. 2764
27	1	U4	C.I. 74LS244
28	1	U5	C.I. 74C922
29	1	U6	C.I. MAX232
30	1	U7	C.I. 7408
31	2	U8,U12	C.I. 7404
32	5	U9,U10,U17,U20,U29	C.I. 7493
33	1	U11	C.I. 74151
34	1	U13	C.I. 7474
35	1	U15	C.I. 74LS138
36	1	U16	C.I. 4052
37	1	U18	C.I. 7486
38	1	U19	C.I. 7492
39	2	U23,U26	C.I. LM319
40	2	U24,U25	C.I. 74122
41	3	U27,U30,U31	C.I. 74152
42	2	U28,U34	C.I. LM310
43	2	U32,U33	C.I. 4051
44	1	U35	C.I. 7432
45	1	U36	Disp. DMC16207
46	1	XTAL1	18.432MHz
47	1	XTAL2	7.3728MHz

La ejecución de las tarjetas principales de los dos equipos demandará el doble de la cantidad de elementos. Una fotografía de éstas se muestra en la figura 2.23.

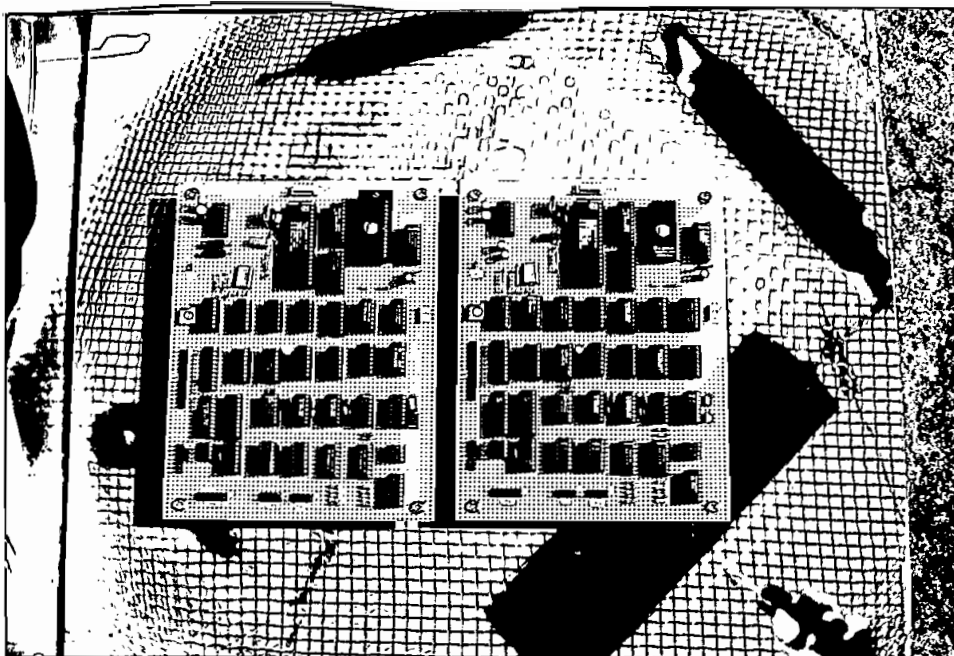


Fig. 2.23. Tarjetas construidas para los equipos

A continuación, en la figura 2.24, se detalla la asignación de pines de cada uno de los conectores que contempla la tarjeta, la marca en una de las esquinas del conector (figura 2.22) indicará el pin 1 del mismo, de acuerdo a la siguiente asignación.

- H1: Conector para el display.
- H2: Conector para los LED's.
- H3: Conector para el teclado.
- H4: Conector para la línea de transmisión.
- H5: Conector para el DTE.
- H6: Conector para el monitoreo de señales.
- H7: Conector para el reset (SW1).
- H8: Conector para la alimentación.

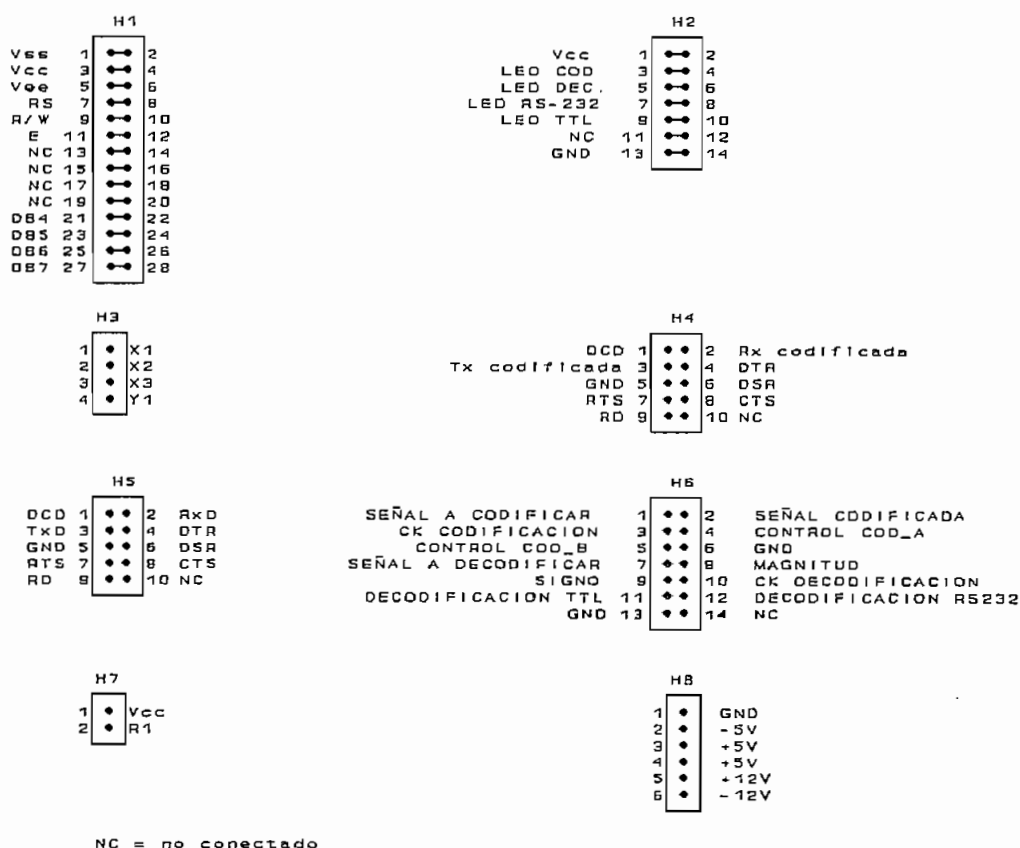


Fig. 2.24. Esquema de los conectores

Para la alimentación del circuito se emplea una fuente tipo switching TECTROL TC245-0795 de 41 W, cuyas especificaciones son las siguientes (Ver Anexo B):

- Salida +5 V a 4.8 A
- Salida -5 V a 0.12 A
- Salida +12 V a 1.1 A
- Salida -12 V a 0.34 A

2.5. DESCRIPCION DEL SOFTWARE DE CONTROL DEL EQUIPO.

En los numerales anteriores de este capítulo se ha descrito el hardware que se usa para el CODEC y que será manejado por el microcontrolador P83C652-03; debe por tanto exponerse ahora una descripción del software que el microcontrolador usará para realizar tanto la tarea de codificación como la de decodificación.

2.5.1. Rutina de inicialización.

En la rutina de inicialización del equipo se establecerá el código de línea a usarse, el tipo de transmisión, así como la velocidad de trabajo, parámetros que serán usados en el funcionamiento normal y que por tanto deberán ser retenidos hasta que no se los cambie en un proceso de reinicialización. La figura 2.25 ilustra el flujograma correspondiente a la rutina de inicialización.

Como primer punto en la rutina de inicialización se tiene la visualización de mensajes de presentación mediante el display alfanumérico; luego de ello se realiza la selección de los parámetros necesarios para establecer el funcionamiento normal del CODEC.

El proceso de selección de los parámetros mencionados en el diagrama de flujo se lo realiza de modo que las opciones de selección aparecerán en el display alfanumérico; una tecla (SW2 en la figura 2.21) permitirá avanzar en el listado de posibilidades, otra tecla (SW4) permitirá retroceder en dicha lista, en tanto que la selección de la opción mostrada se la realiza presionando una tercera tecla (SW3). Para evitar errores en la elección de los parámetros, antes de aceptar la opción elegida, se pregunta si ésta es correcta, presionando la

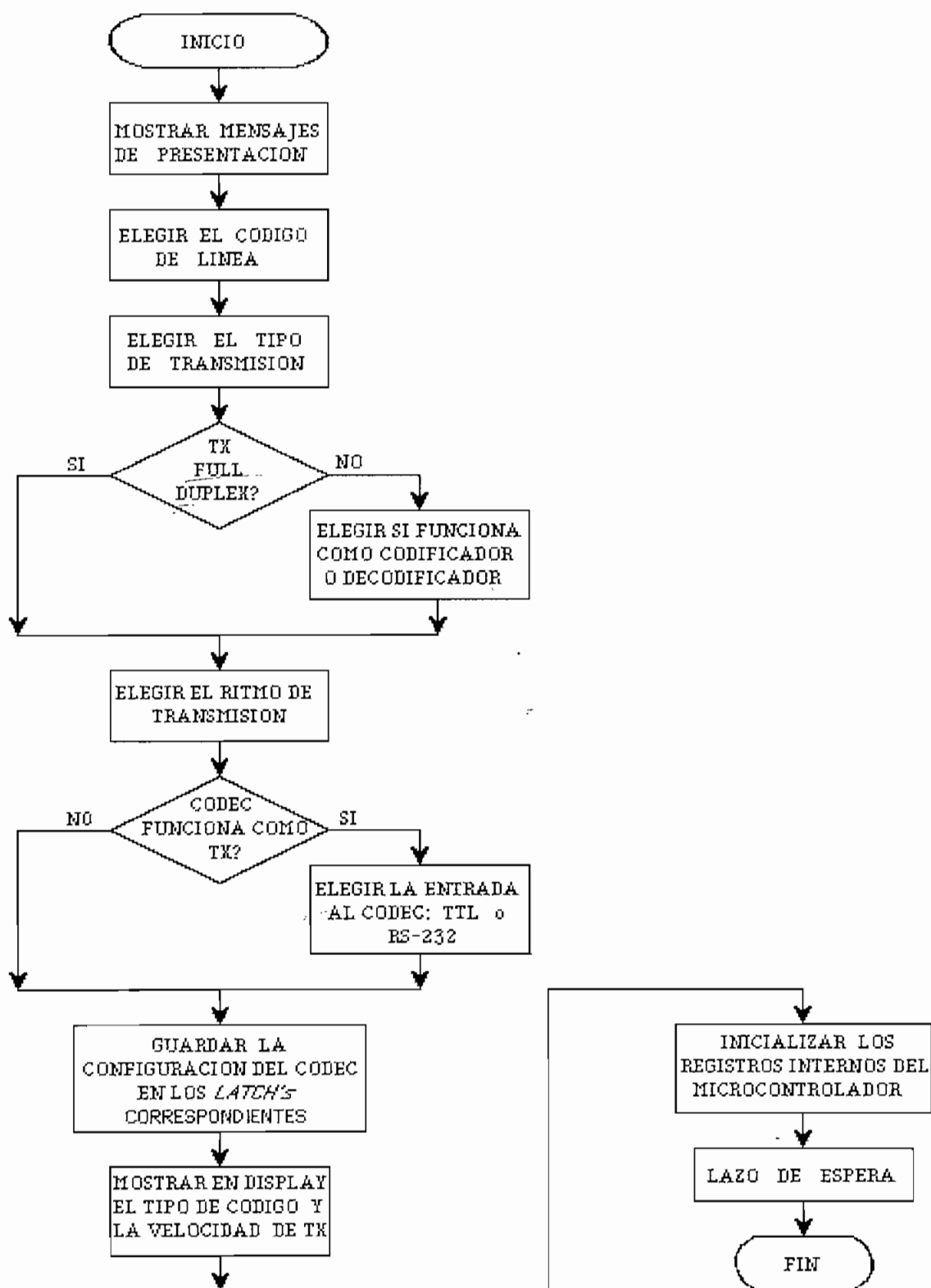


Fig. 2.25. Rutina de inicialización

tecla (SW3) en caso afirmativo, en tanto que si se presiona otra tecla se vuelve al proceso de selección.

Los parámetros seleccionados y que han quedado almacenados en la memoria RAM interna del microcontrolador deben ser escritos en los *latch's* de configuración correspondientes, lo que permitirá establecer el funcionamiento normal del CODEC. En su operación normal, es decir mientras que se está realizando permanentemente la tarea de codificación, decodificación o las dos, según se lo haya configurado, el CODEC muestra en el display alfanumérico el esquema de codificación utilizado y la velocidad de transmisión binaria.

Luego de ejecutar la rutina de inicialización aquí expuesta, el microcontrolador queda en un lazo de espera del cual sólo puede salir si hay un requerimiento de atención a una interrupción externa o a la interrupción serial.

Si ocurre una interrupción externa 0 se realiza el proceso de codificación a nivel de bit, en tanto que la interrupción externa 1 produce la tarea de decodificación de un elemento de señal. La ocurrencia de una interrupción serial mediante una transición negativa será un artificio que se use para que el microcontrolador atienda un requerimiento del teclado. Mediante esta última se interrumpirá el funcionamiento normal del CODEC para llevar a cabo una reinicialización del sistema, este proceso conlleva una nueva ejecución de la rutina de inicialización, luego de lo cual el CODEC vuelve a su funcionamiento normal.

Respecto a la organización de la memoria del sistema, en la figura 2.26 se tiene un mapa de memoria en el que se ha incluido tanto la memoria externa al microcontrolador como la RAM interna del mismo. La memoria

externa lo constituyen tanto la memoria de programa como los dispositivos que son considerados como RAM externa. Del bloque de memoria asignado a cada dispositivo externo, es suficiente una localidad para direccionarlo, esta localidad está indicada en la figura en mención junto al nombre de cada dispositivo.

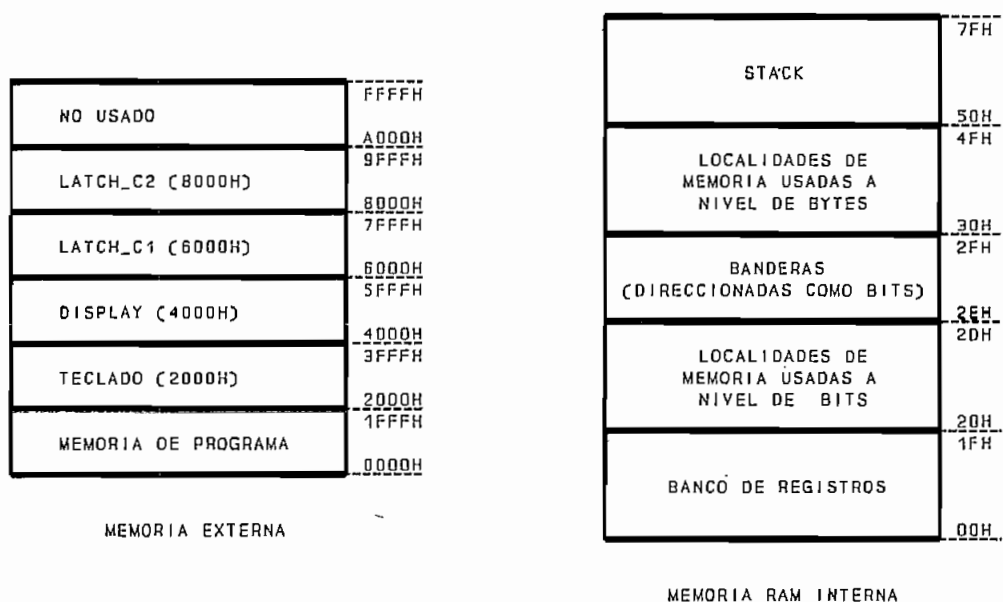


Fig. 2.26. Mapa de memoria del sistema

Con relación a la memoria RAM interna, los 128 bytes disponibles para el usuario (00H a 7FH) se estructuran de acuerdo al mapa mostrado en la figura, en el que se puede notar que su parte alta se la ha reservado para la pila (*stack*) que requiere el microcontrolador en instrucciones de llamado a subrutinas o de atención a interrupciones; luego del *stack* se encuentran las localidades de memoria usadas a nivel de bytes.

Las banderas de propósito específico que se usan en el programa estarán almacenadas en las localidades 2EH y 2FH que son las más altas en el área de memoria

direccionable a nivel de bits, en tanto que las localidades usadas a nivel de bits se ubican entre las direcciones 20H y 2DH.

La asignación de etiquetas a las localidades de memoria y banderas se puede encontrar en el listado del programa (Anexo D).

El programa de control se lo ha diseñado de modo que cuando se produzca una interrupción externa de codificación o decodificación, se busque en la localidad de RAM marcada como 'CODIGO' un byte que se ha escrito durante la inicialización y que indique el esquema de codificación que se está usando (tabla 2.5).

'CODIGO' (*)	ESQUEMA DE CODIFICACION
00H	NRZ POLAR
01H	AMI
02H	RZ POLAR
03H	MANCHESTER DIFERENCIAL
04H	BIFASE M
05H	MODULACION POR RETARDO (MILLER)
06H	4B-3T
07H	MS43
08H	B3ZS
09H	HDB3

(*) 'CODIGO' representa el contenido de la localidad de memoria.

Tabla 2.5. Numeración de los esquemas de codificación

Cuando se tenga una interrupción externa cero, se deberá producir la codificación del bit ingresado. Para ello se interroga qué tipo de codificación se está utilizando, mediante una pregunta secuencial fija que concluye cuando el número de la interrogación coincide con el esquema de codificación elegido.

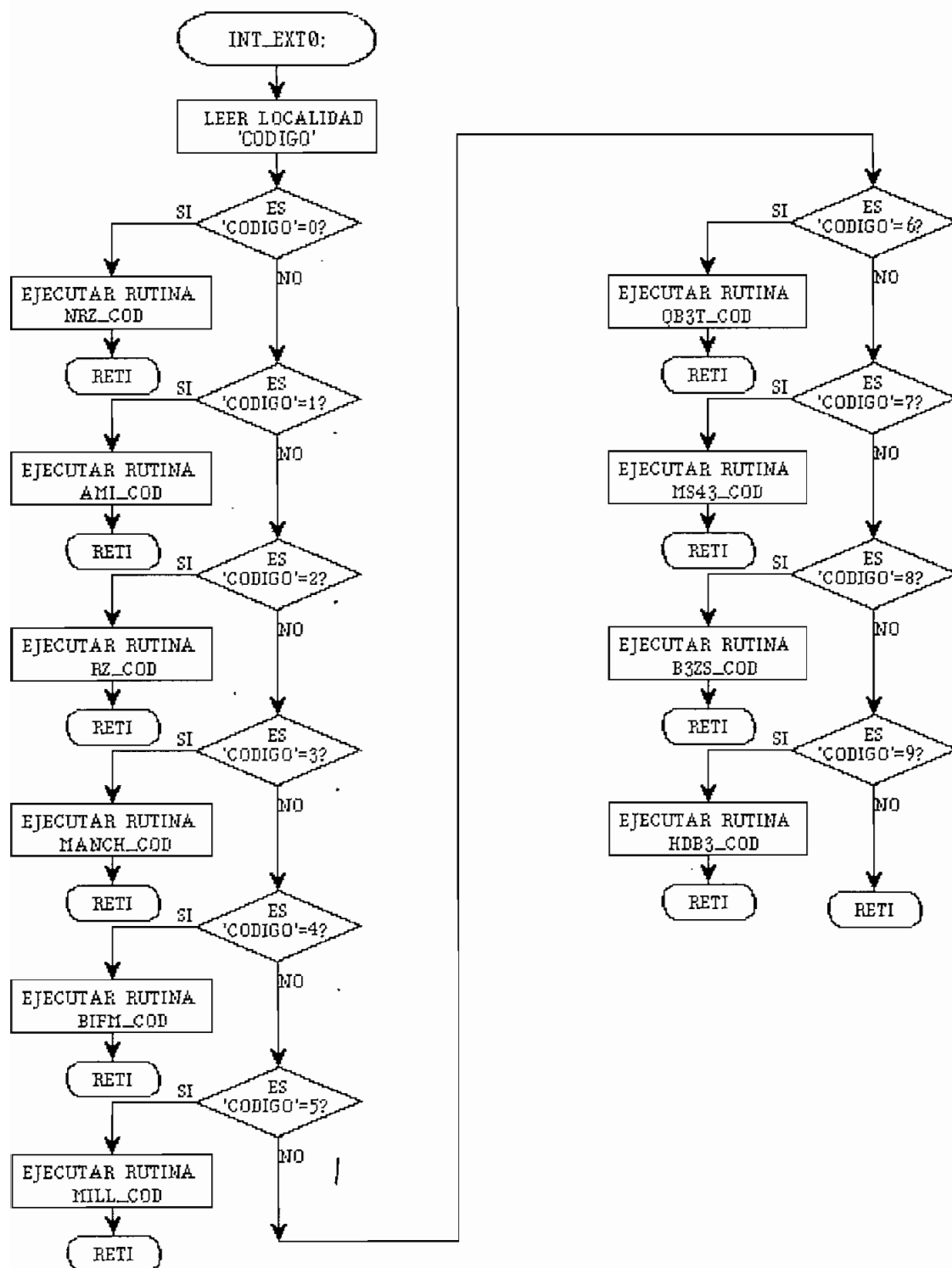


Fig. 2.27. Atención a la interrupción externa 0

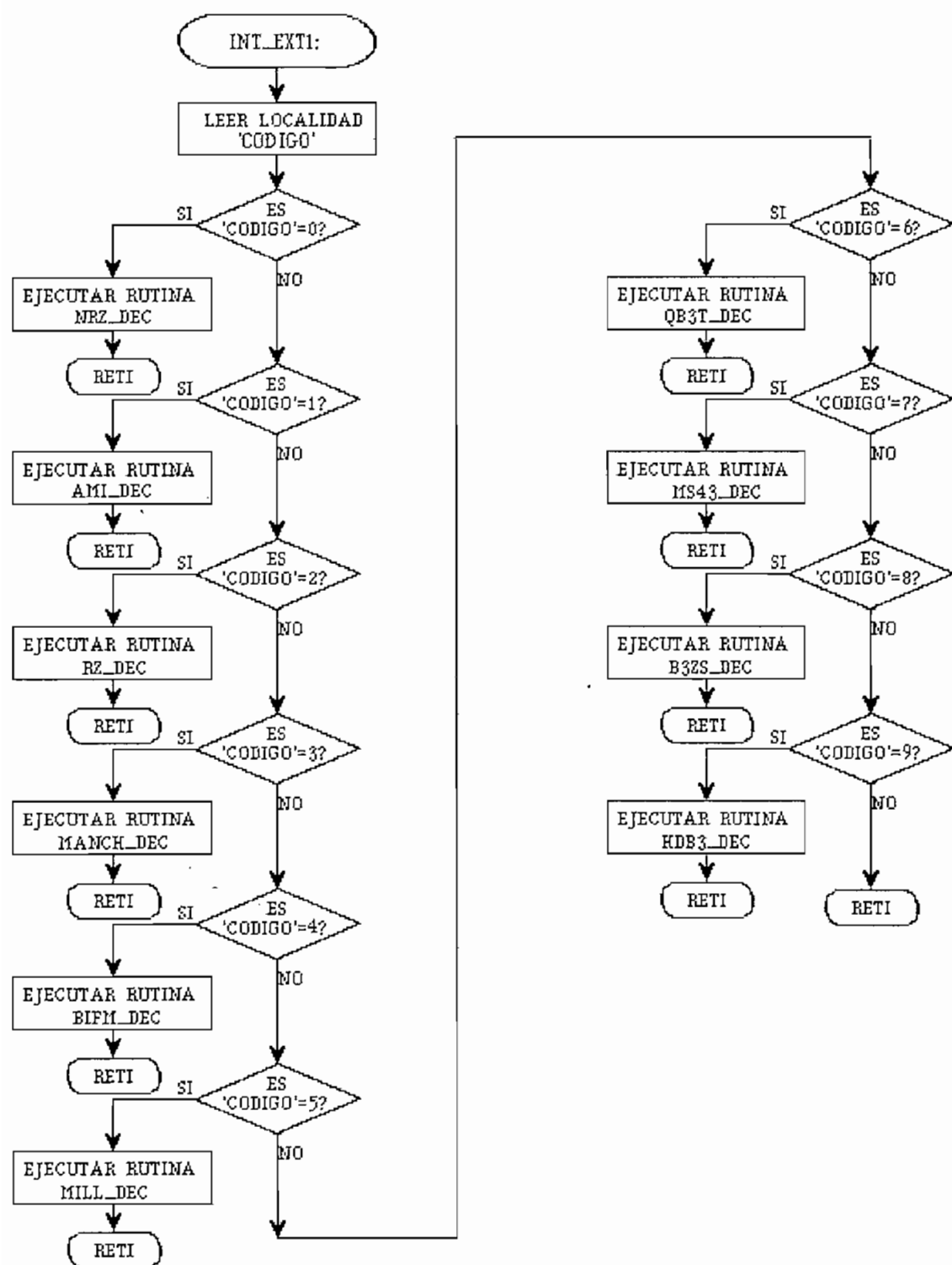


Fig. 2.28. Atención a la interrupción externa 1

A continuación se ejecuta una rutina de atención a esta interrupción y se retorna el control del programa al punto en donde estaba antes de producirse la misma (figura 2.27). Los bloques que marcan la ejecución de una rutina constituyen los algoritmos de codificación implementados, los mismos que serán detallados posteriormente en este mismo capítulo.

El aparecimiento de una interrupción externa 1, producirá en cambio una tarea de decodificación, pero que en esencia será tratada de la misma manera que para la interrupción externa cero, tal como se puede observar en la figura 2.28.

2.5.2. Rutinas para el código NRZ polar.

Antes de analizar el *software* empleado por el CODEC, conviene recordar que cada bit que ingresa al codificador lo hace por la ocurrencia de una interrupción externa 0, esta interrupción deberá ser deshabilitada hasta que se haya terminado de procesar el bit; igual cosa sucede cuando se decodifica un elemento de señal mediante la ocurrencia de una interrupción externa 1.

a. Codificación (NRZ_COD).

La rutina de codificación tiene como primer paso la deshabilitación de las interrupciones para no atender a ninguna otra hasta que no se haya procesado la presente tarea. Se utiliza lógica negativa de modo que el nivel más alto de señal corresponde a un 0_L y el nivel más bajo de señal representará un 1_L.

De esta manera, luego de ingresar el bit a codificar se envía un pulso negativo de 5 voltios si el bit es 1_L, o un pulso positivo si el bit es 0_L. Este pulso permanecerá en el mismo nivel hasta que exista otra

interrupción que requiera el realizar una nueva codificación.

La tarea de muestrear el bit de entrada se realiza aproximadamente en la mitad de la duración del bit, y sólo entonces se lo procesa para obtener una salida que presenta una demora de algo más de medio período de bit respecto a la señal de datos entrante.

Luego de que se produzca esta codificación, la siguiente interrupción ocurrirá después de que transcurra un período de bit, por lo que la duración del pulso estará de acuerdo con el ritmo de transmisión. En la figura 2.29 se muestra el algoritmo de codificación.

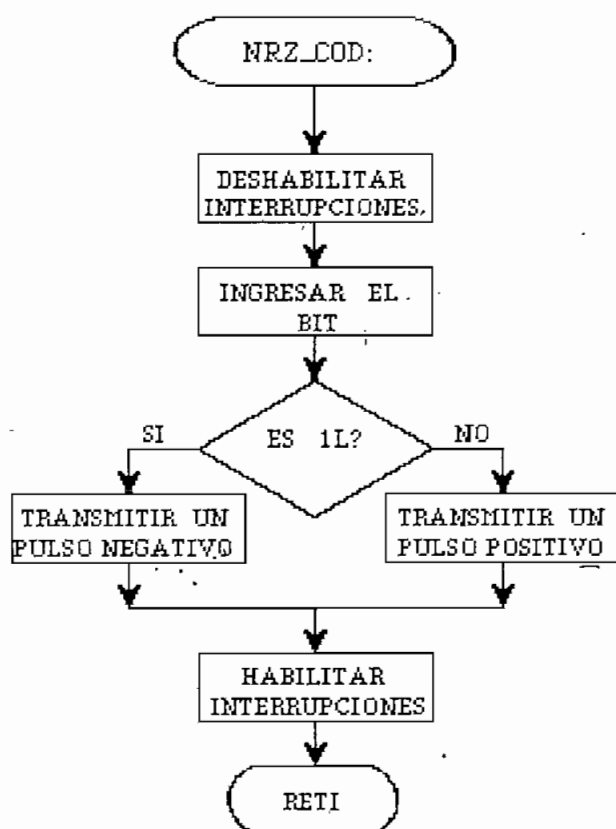


Fig. 2.29. Codificación NRZ polar

b. Decodificación (NRZ_DEC).

En el proceso de decodificación de la señal entrante, se toma como información el signo de la señal, ya que al ser un código polar, la señal está variando entre +5 y -5 voltios. La señal de magnitud presentará un nivel alto permanentemente y no facilitará información para la decodificación.

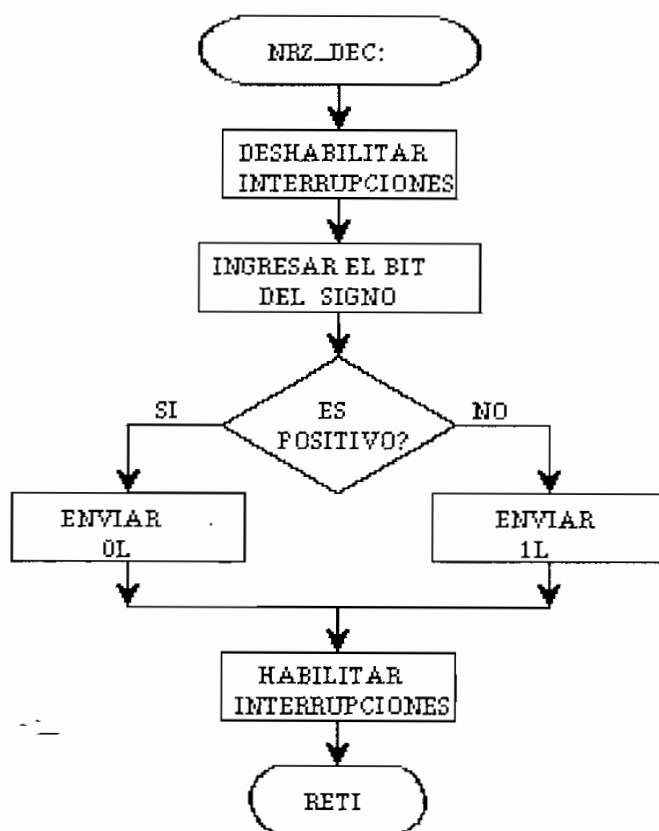


Fig. 2.30. Decodificación NRZ polar

Si la señal del signo presenta un nivel alto se tratará de un 0_L , bit que será enviado por el microcontrolador (a través del circuito de selección del nivel de salida), en tanto que si la señal del signo presenta un nivel bajo corresponderá a un 1_L que será igualmente entregado por el microcontrolador como señal de salida. Este desarrollo se lo ilustra en la figura 2.30.

Como se ha indicado, la magnitud permanece en un nivel alto por lo que se usa la señal del signo (que cambia permanentemente) en la recuperación de la señal de reloj.

2.5.3. Rutinas para el código AMI.

a. Codificación (AMI_COD).

La primera etapa será similar al caso de codificación anterior, debiendo indicar además que la deshabilitación de interrupciones será una etapa previa y común a todos los esquemas de codificación.

El algoritmo de este código, ilustrado en la figura 2.31, se basa en que se tiene almacenado en una bandera ('BIT_SIG') la polaridad del último pulso que se envió, de modo que si el bit a codificar es un 1_L se invierte la polaridad del pulso y se lo envía, guardándose entonces la última polaridad enviada.

Si el bit a codificar es 0_L, se envía una señal de 0 voltios. El retardo en la codificación es al igual que en el caso anterior de medio período.

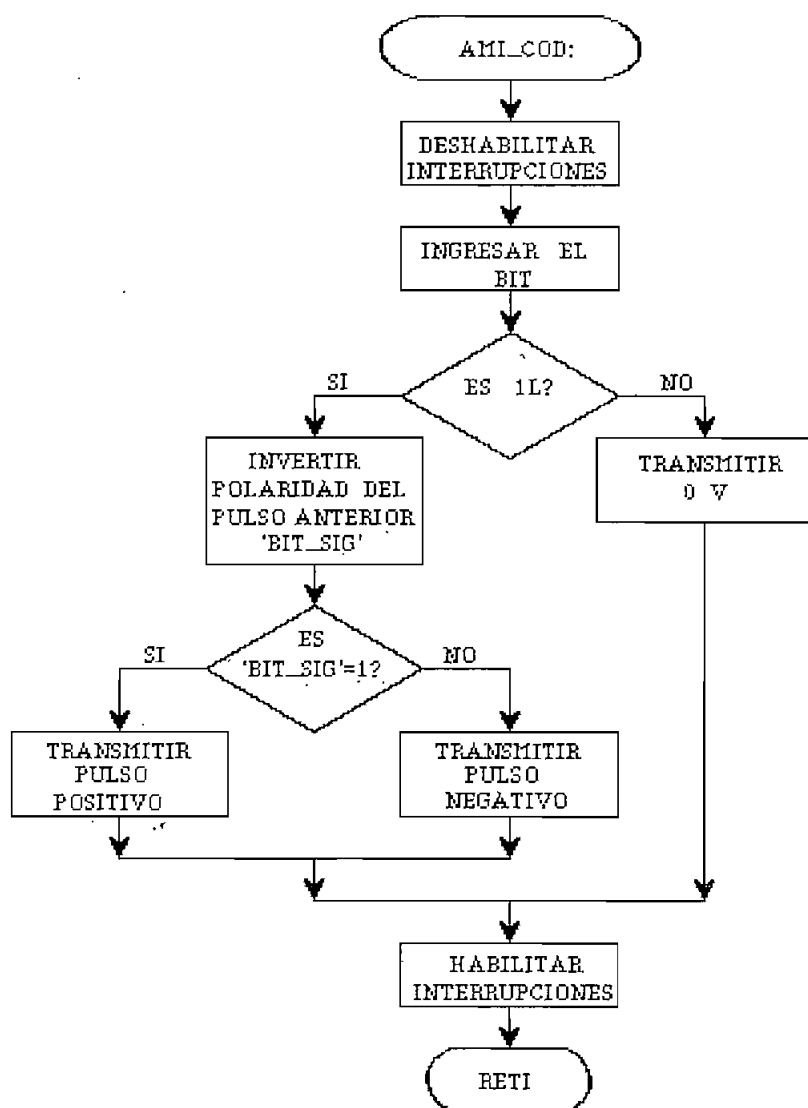


Fig. 2.31. Codificación AMI

b. Decodificación (AMI_DEC).

Para realizar la decodificación (figura 2.32), es suficiente con muestrear la magnitud de la señal entrante, ya que la codificación envía un pulso (positivo o negativo) cuando se tenga un 1_L , en tanto que el nivel de señal entrante es de cero voltios cuando el bit es 0_L . De esta manera, un nivel alto en la magnitud permitirá al microcontrolador decodificar un 1_L , mientras que un nivel

bajo indicará que se tiene un 0_L.

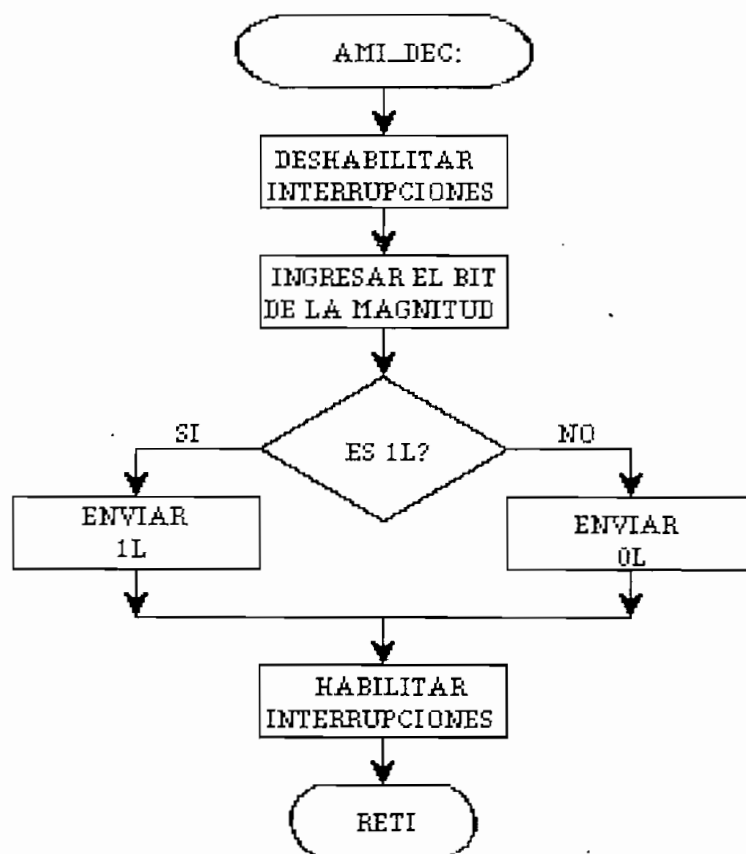


Fig. 2.32. Decodificación AMI

2.5.4. Rutinas para el código RZ polar.

a. Codificación (RZ_COD).

La implementación de la codificación RZ polar se la realiza con un ciclo de trabajo del 50%, es decir que la mitad del período de bit la señal obedecerá al dígito binario codificado mientras que en el resto del período permanecerá en un nivel cero. Cuando se ha ingresado el bit a codificar, y luego de haber deshabilitado las interrupciones, se activa el temporizador 1 (timer 1), el cual se encargará de mantener en un estado definido la salida de transmisión durante el primer semiperíodo, luego

de lo cual la señal irá a cero. Este procedimiento se ilustra en la figura 2.33.

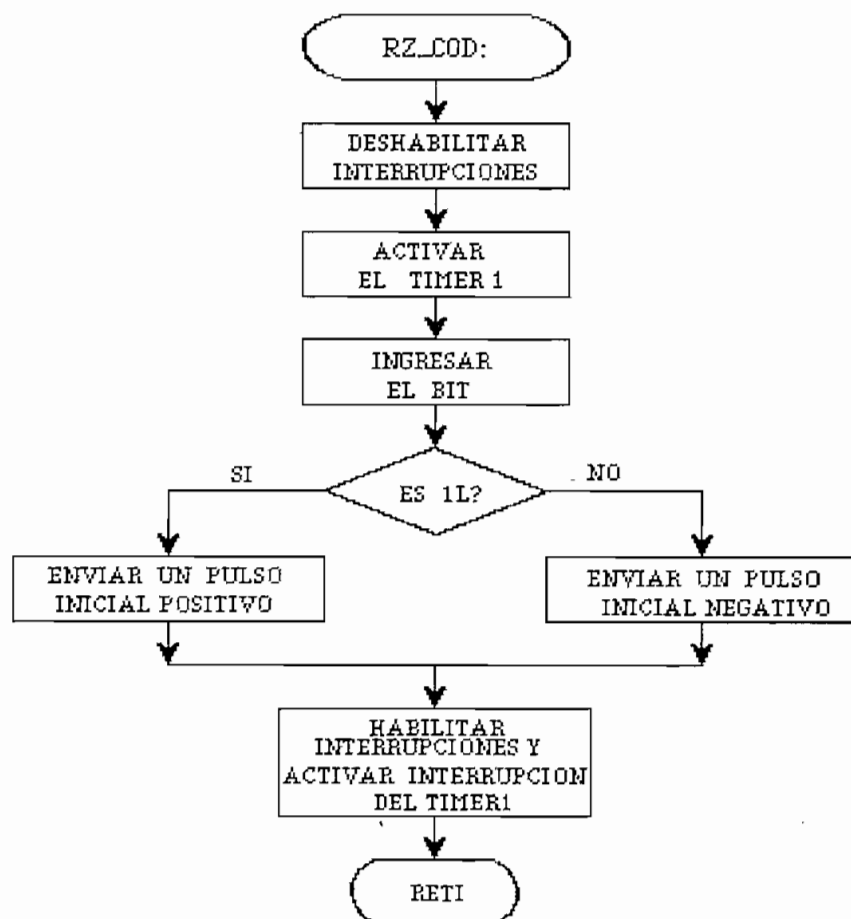


Fig. 2.33. Codificación RZ

El proceso se inicia transmitiendo un pulso positivo si el bit de entrada es 1_L y un pulso negativo si el bit es 0_L (la lógica usada en el resto de esquemas de codificación será positiva). Se retorna de la rutina de atención a la interrupción externa 0, habilitando las interrupciones y activando la interrupción del *timer* 1. Esto ocasiona que cuando se produzca la interrupción por desbordamiento del *timer* 1 (medio período después), la señal transmitida retorne a cero, estado en el que se

mantendrá hasta que ocurra la interrupción externa 0 (al iniciar la codificación de un nuevo bit de entrada). El proceso que ocurre cuando se da la interrupción por desbordamiento del *timer* 1 se muestra en la figura 2.34.

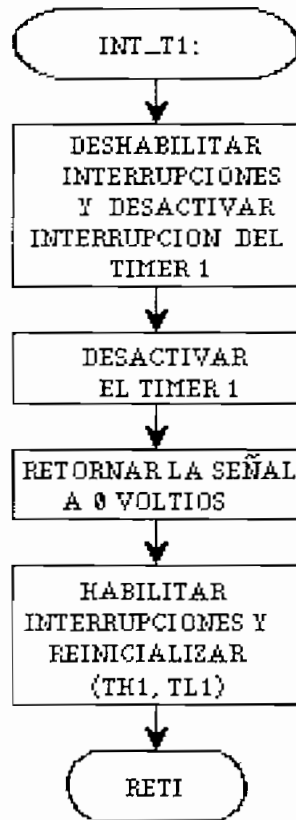


Fig. 2.34. Interrupción del *timer* 1 para la codificación RZ

Para lograr una temporización igual a la mitad de la duración de un bit, en los registros de temporización del *timer* 1 denominados TH1 y TL1, que en conjunto proporcionan 16 bits, se carga un número. Este número se incrementa en uno por cada doce ciclos de reloj del microcontrolador, de modo que cuando se llegue al valor máximo (FFFFH) habrá transcurrido el tiempo requerido.

Al activarse la interrupción del *timer* 1 por desbordamiento, lo primero que se hace es deshabilitar

todas las interrupciones y chequear el número que se encuentra en la localidad de memoria 'CODIGO', de modo que si ésta es igual a 02H (codificación RZ) se pondrá en cero la salida de transmisión, caso contrario se buscará el algoritmo de codificación al cual debe servir y retornará devolviendo el control al flujo normal del programa.

Para terminar con la función de esta rutina, se vuelve a cargar en los registros TH1 y TL1 el valor inicial que permitirá una temporización posterior, y se habilitan las interrupciones que estaban vigentes excepto la correspondiente al timer 1, que será habilitada sólo cuando se produzca la codificación de un nuevo bit.

Como se puede notar, la señal transmitida varía en un ritmo igual al doble de la velocidad de transmisión de la señal que entra al codificador.

b. Decodificación (RZ_DEC).

Durante un período de bit, sólo la primera parte de la señal contiene información sobre el dígito binario que se ha enviado, en tanto que en la segunda mitad siempre existirá un nivel de cero. Por esta razón, para la decodificación se ingresa tanto la magnitud como el signo de la señal de entrada y se interroga sobre el valor de la magnitud.

Si la magnitud es cero, no se puede determinar el valor del bit por lo cual se retorna a esperar otra interrupción sin tomar ninguna decisión en este punto; pero si la magnitud es diferente de cero, se considera el signo de la señal de entrada de modo que si es positiva se decodificará como 1, en tanto que si es negativa se tendrá un 0. Este algoritmo se ilustra en la figura 2.35.

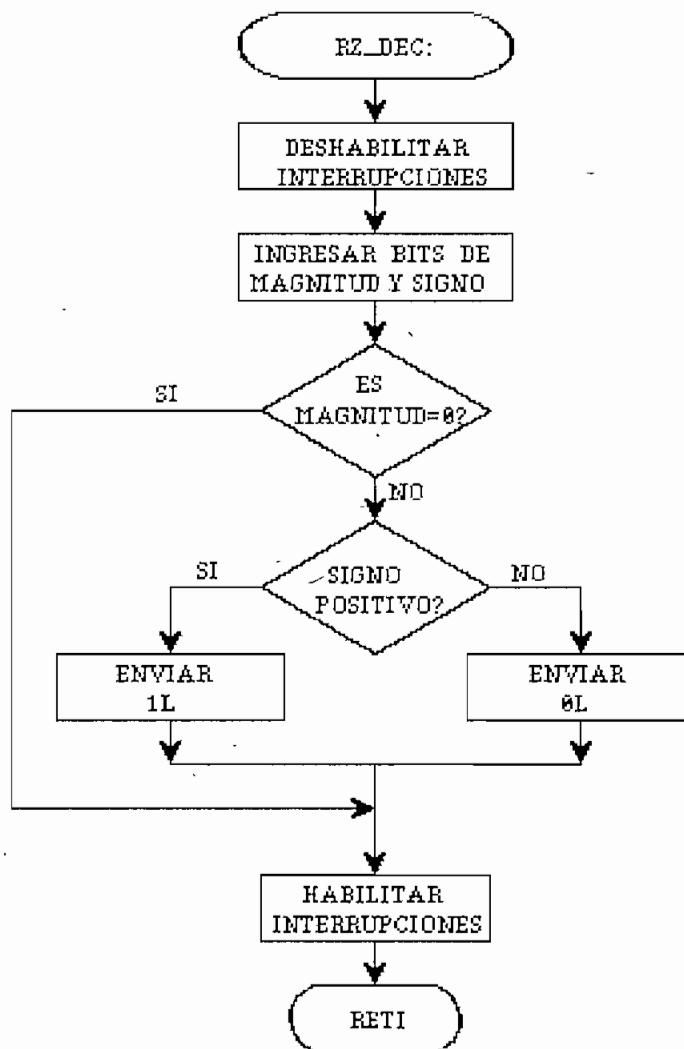


Fig. 2.35. Decodificación RZ

2.5.5. Rutinas del código Manchester diferencial.

a. Codificación (MANCH_COD).

La codificación Manchester diferencial (figura 2.36), de un bit cualquiera inicia con el envío, durante $1/8$ del tiempo de bit, de pulsos de sincronismo cuya frecuencia de 16 veces el ritmo de transmisión asegura que existan al menos dos transiciones negativas que permitan obtener una indicación del inicio del período. Terminada

esta temporización, que se la ejecuta mediante un lazo de retardo, se procede a interrogar si el bit ingresado es 1_L o 0_L , si se trata de un 1_L se mantiene el nivel correspondiente al estado anterior y se arranca el *timer 1* para demorar un retardo igual a la mitad de la duración del bit luego de lo cual se invertirá la polaridad del elemento de señal y se mantendrá este estado por el resto del período.

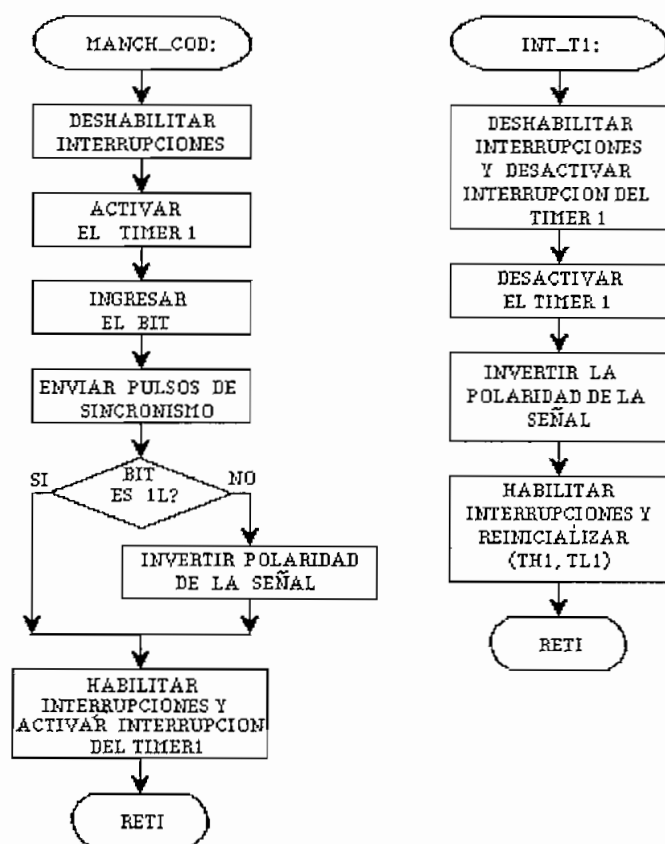


Fig. 2.36. Codificación Manchester diferencial

Si el bit es un 0_L se invierte la polaridad del pulso de codificación anterior y se arranca el *timer 1* para proceder de manera similar a la indicada anteriormente.

b. Decodificación (MANCH_DEC).

La decodificación de señales bifase entraña mayores

dificultades que para el resto de esquemas de codificación, ya que la información misma viaja en la fase de la señal y es necesario detectar el cambio de fase para discernir entre un 1_L y un 0_L .

Se habrá transmitido un 1_L si la polaridad del elemento de señal en el primer semiperíodo es igual a la polaridad del segundo semiperíodo del bit anterior, en caso contrario se tendrá un 0_L .

La señal de entrada estará completamente decodificada si se muestrea en la mitad de cada uno de los semiperíodos de bit para establecer la comparación antes indicada. En teoría esto es simple, sin embargo se debe considerar que en la práctica, una fluctuación ocasional de la fase, producirá un error permanente hasta que se produzca una nueva fluctuación de fase.

Con la ayuda de los pulsos de sincronismo que se transmiten junto con los datos, se procede a obtener una señal que muestree el signo de entrada en la primera mitad de la duración del bit y que no aparezca en la segunda mitad.

Se realiza entonces la comparación con el dato almacenado en la segunda mitad del período de bit anterior, con lo cual se ha decodificado el bit actual. El signo en la segunda mitad del período será contraria a la actual y su almacenamiento permitirá realizar la decodificación del siguiente elemento de señal, este proceso se lo ilustra en la figura 2.37.

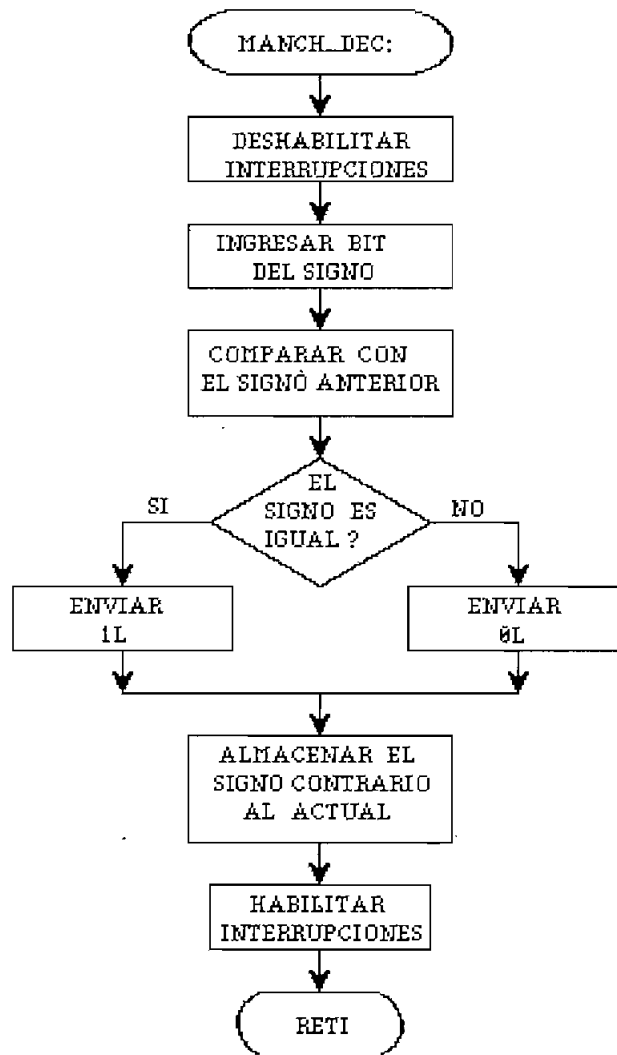


Fig.-2.37. Decodificación Manchester

2.5.6. Rutinas para el código bifase-M.

a. Codificación (**BIFM_COD**).

Para la realización de este código de línea (figura 2.38), al inicio de cada período de bit se envían los pulsos de sincronismo con las mismas consideraciones expuestas para la codificación Manchester.

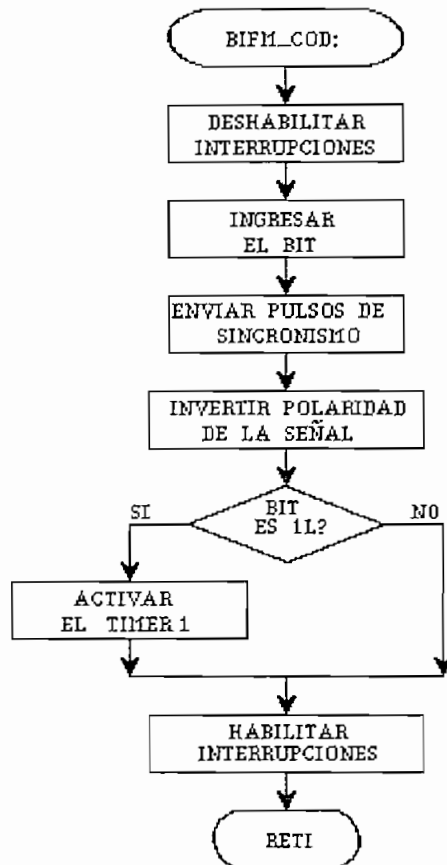


Fig. 2.38. Codificación bifase-M

El estado permanente de la salida luego de los pulsos de sincronismo es de signo opuesto al que se tenía al final del período de bit anterior. En este punto, luego de haber ingresado el bit a codificar, si el bit es 1, se activará el *timer 1* de modo que se pueda realizar una transición a la mitad del período, en tanto que si el bit es 0, la codificación para este bit habrá terminado ya que el estado de la señal se mantendrá durante todo el período.

La parte correspondiente a la interrupción del *timer 1* para este esquema de codificación es la misma que para la codificación Manchester ilustrada en la figura 2.36. El retardo producido en la codificación es de alrededor de medio período de reloj, igual que en el caso del código Manchester.

b. Decodificación (BIFM_DEC).

El reloj de decodificación es idéntico al utilizado por el esquema Manchester diferencial, pero en este caso luego de muestrear el elemento de señal en la primera mitad del período de bit, es necesario muestrear también en la segunda mitad, ya que la comparación de estos dos valores indicará si el bit es 1_r (en el caso de que sean diferentes) o 0_r (si son iguales).

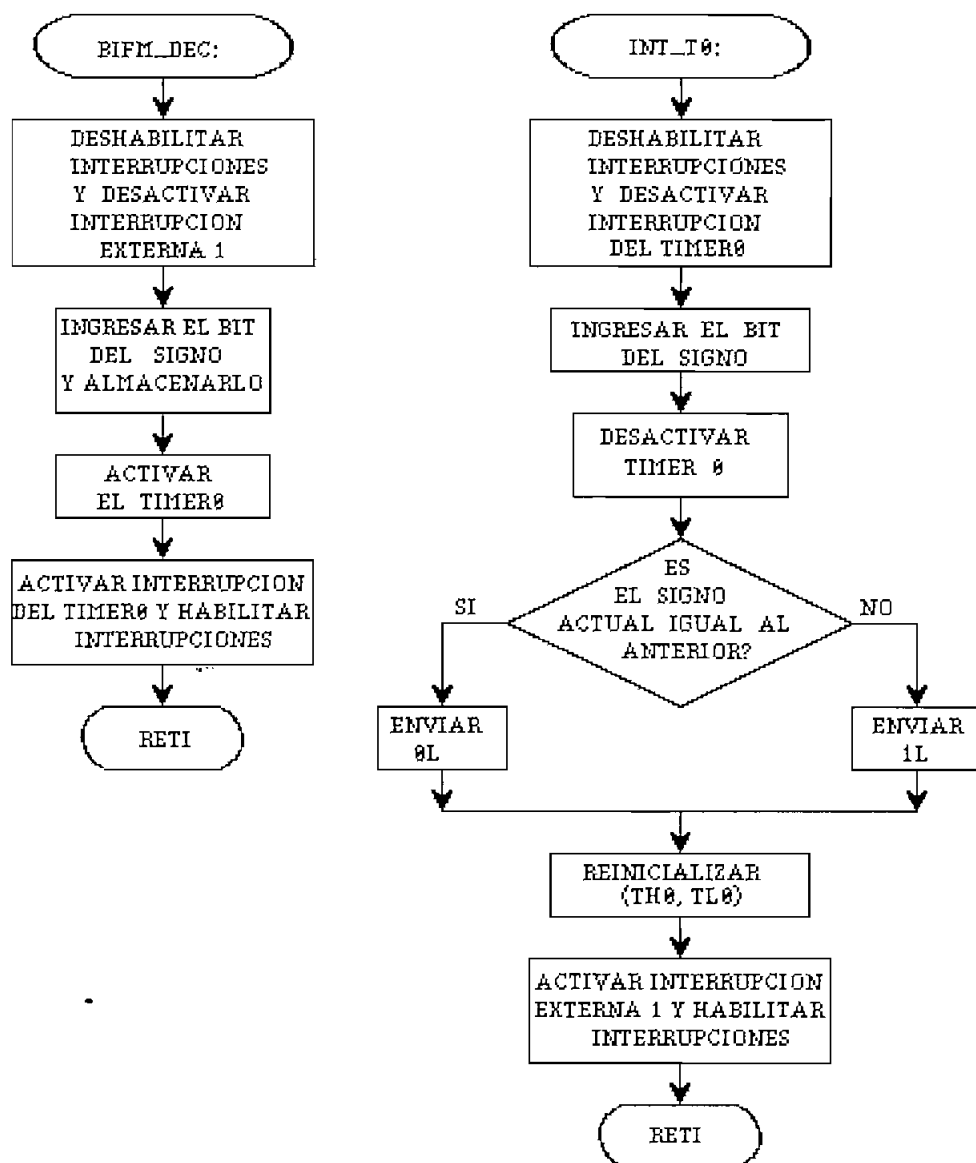


Fig. 2.39. Decodificación bifase-M

La segunda muestra es tomada luego de que transcurra un medio período de bit el cual es temporizado con la ayuda del *timer* 0; este proceso se ilustra en la figura 2.39.

2.5.7. Rutinas para el código de modulación por retardo (Miller).

a. Codificación (**MILL_COD**).

El esquema de codificación Miller hace que la señal a enviar varíe entre un nivel positivo y otro negativo sin retorno a cero. Luego de que se ha deshabilitado las interrupciones y se ha ingresado el bit a codificar, lo primero que se hace es enviar los pulsos de sincronismo de la misma forma que se ha descrito para los dos códigos bifase precedentes.

Si se tiene un 1_L , durante el primer semiperíodo deberá mantenerse la polaridad de la señal que se tenía antes de la codificación, para lo cual se activa el *timer* 1 que se encargará de mantener el estado anterior.

Cumplido este tiempo se activa la interrupción por desbordamiento del *timer* 1, la que ejecutará como acción el que se cambie la polaridad de la señal de salida, situación que ocurrirá a mitad del período, desactivándose entonces el *timer* 1 y deshabilitando la interrupción relacionada con éste.

Si el bit ingresado es 0_L , sólo es necesario interrogar sobre si el bit inmediatamente anterior fue 1_L ó 0_L . En el caso de que haya sido 0_L se cambiará la polaridad de la señal de salida y se retornará a esperar otro bit; pero si el bit anterior fue 1_L , no se ejecuta ninguna acción y se retorna a esperar un nuevo bit.

Este procedimiento se ilustra en la figura 2.40, en

tanto que la acción tomada al producirse la interrupción del *timer* 1 se observa en la figura 2.41.

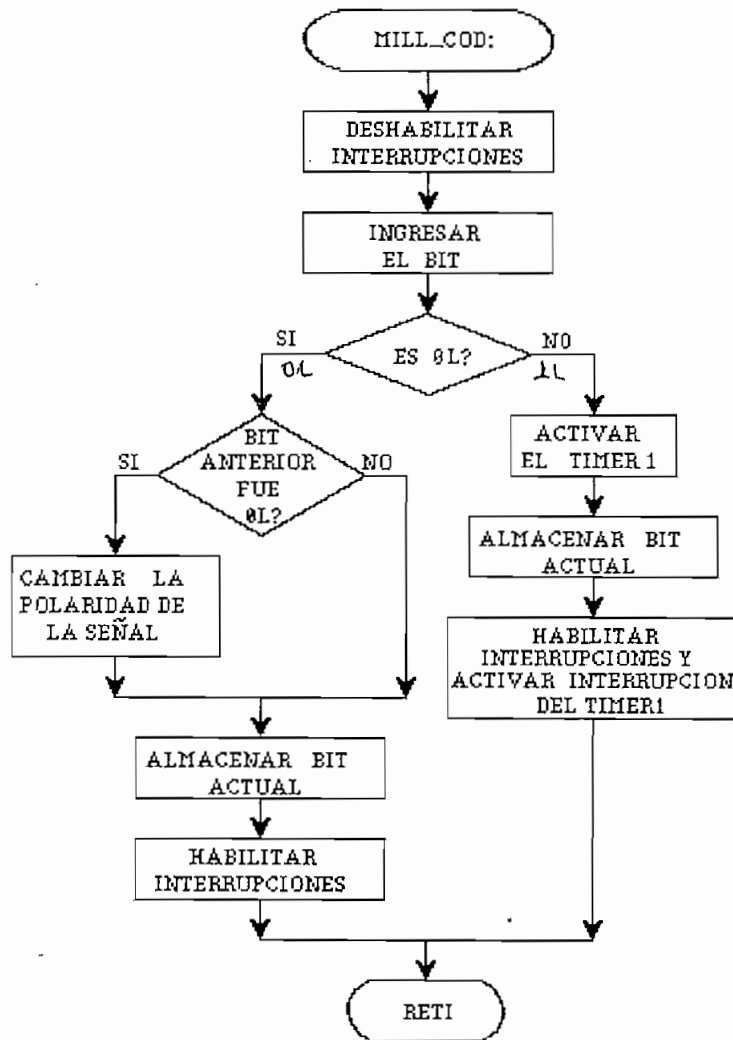


Fig. 2.40. Codificación Miller

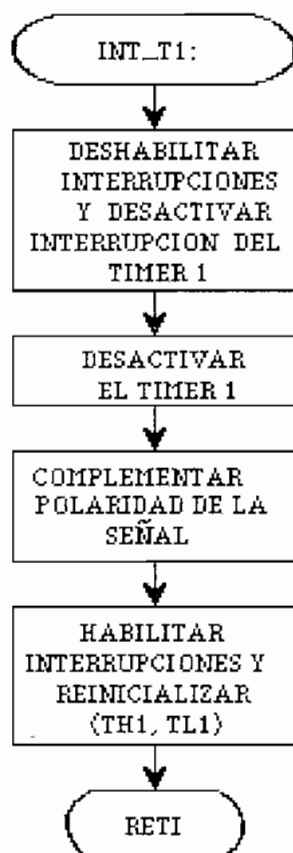


Fig. 2.41. Interrupción del *timer* 1 para el código Miller

b. Decodificación (MILL_DEC).

La decodificación implica el uso de la misma señal de reloj empleada en los dos anteriores esquemas bifase. Con este reloj se obtiene el signo que presenta la señal en el primer semiperíodo, pero es necesario conocer además el signo del segundo semiperíodo.

Para obtener el signo de la señal en el segundo semiperíodo, se pone en marcha el *timer* 0, que temporizará el paso de medio ciclo, luego de lo cual se producirá la interrupción de desbordamiento que da lugar a ingresar el signo del segundo semiperíodo.

Una vez que se tienen los signos de los dos

semiperíodos se compara; si el signo de la señal es el mismo en todo el período, no se ha producido una transición a la mitad del período y por tanto se tiene un 0_L , por el contrario, si cambia el signo de la señal se tienen un 1_L . Este algoritmo se muestra en la figura 2.42.

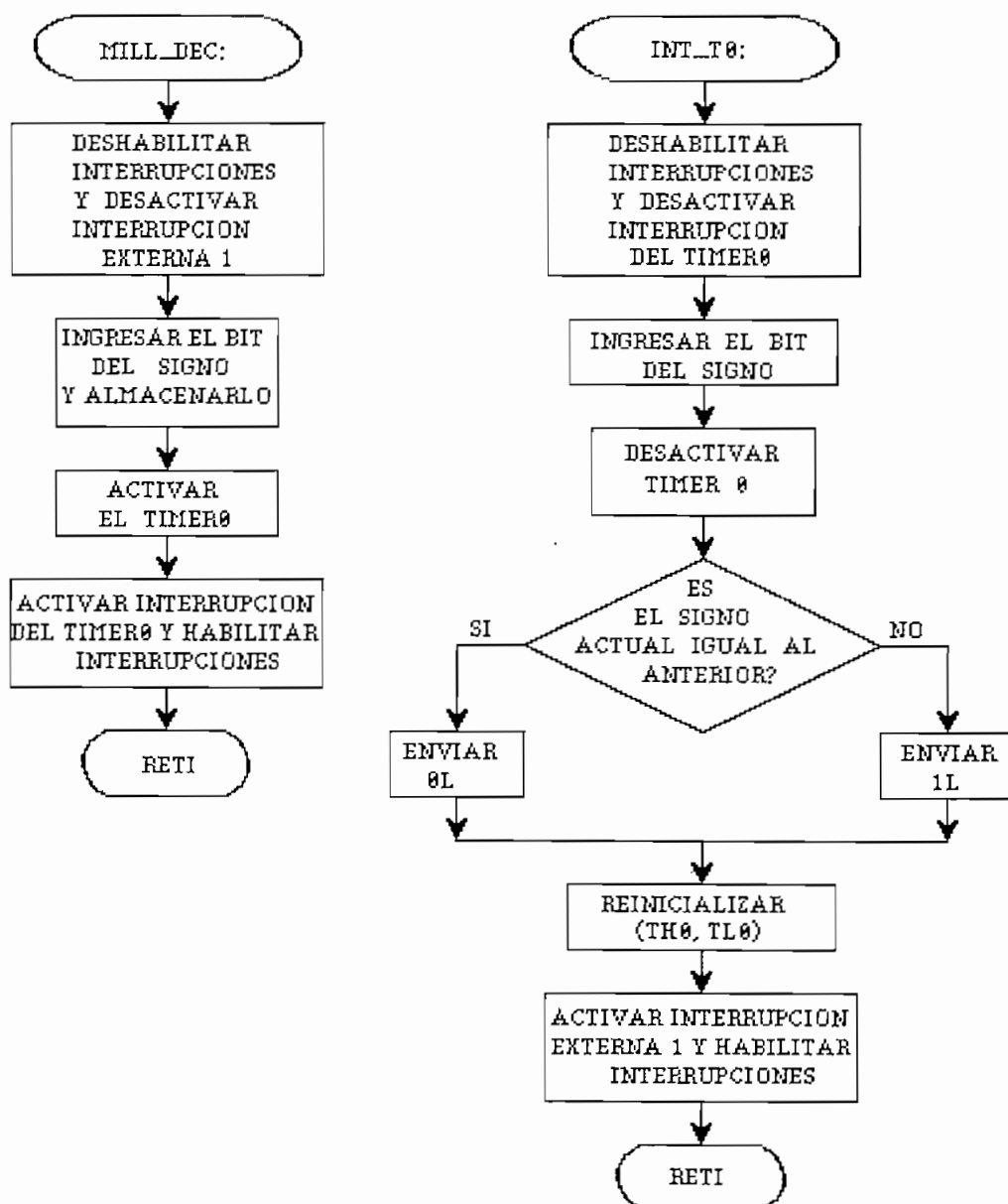


Fig. 2.42. Decodificación Miller

En la realización de este proceso es necesario tener en cuenta que la interrupción por desbordamiento se activa luego de ingresar el primer signo y se desactiva al finalizar la tarea de decodificación, reiniciándose el valor de temporización del *timer* 0. La habilitación de interrupciones se refiere a que pueda ejecutarse cualquiera de las que esté activa, mas no que se activen todas las interrupciones.

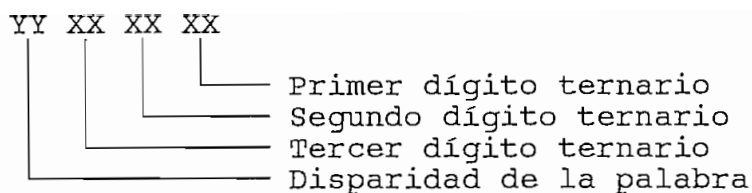
2.5.8. Rutinas para el código 4B-3T.

a. Codificación (QB3T_COD).

El código cuatro binario-tres ternario está basado en el reemplazo de una palabra binaria de cuatro dígitos por una palabra ternaria de tres dígitos, según lo expuesto en la tabla 1.2. Con tal finalidad se elabora secuencialmente una tabla de los códigos que deben ser emitidos para obtener cada una de las palabras ternarias.

Una vez que se han recibido cuatro dígitos binarios, se está en capacidad de encontrar la palabra ternaria a ser transmitida; existen sin embargo dos posibilidades, un modo de polaridad positiva y otro de polaridad negativa, por lo que la elección de la una o la otra se realizará de forma que la polaridad del elemento de señal sea contraria a la que presenta la disparidad acumulada.

La tabla de palabras ternarias será almacenada en 8 bits, dos bits para cada dígito ternario (seis en total) y los dos bits restantes contendrán la disparidad que la palabra presenta.



La equivalencia entre dígitos binarios y ternarios es la que se indica en la tabla 2.6. Estos dígitos binarios serán manejados por el microcontrolador para representar los dígitos ternarios involucrados en la transmisión y corresponden a los representados como XX. La disparidad del conjunto de tres dígitos ternarios puede ser 3 como máximo por lo que serán necesarios dos bits (YY) para representarla.

Dígitos binarios	Dígitos ternarios
00	0
01	+
10	-
11	no usado

Tabla 2.6. Dígitos binarios y ternarios

Con esta equivalencia y tomando en cuenta la tabla 1.2, se establecen las palabras ternarias que serán escritas en el *buffer* de transmisión tal como se indica en la tabla 2.7.

El flujograma que explica esta codificación se lo observa en la figura 2.44.

Palabras binarias	Palabras ternarias	
	Modo positivo	Modo negativo
0000	00 00 10 01 = 09H	00 00 10 01 = 09H
0001	00 10 01 00 = 24H	00 10 01 00 = 24H
0010	00 10 00 01 = 21H	00 10 00 01 = 21H
0011	01 01 10 01 = 59H	01 10 01 10 = 66H
0100	10 00 01 01 = 85H	10 00 10 10 = 8AH
0101	01 00 01 00 = 44H	01 00 10 00 = 48H
0110	01 00 00 01 = 41H	01 00 00 10 = 42H
0111	01 10 01 01 = 65H	01 01 10 10 = 5AH
1000	00 00 01 10 = 06H	00 00 01 10 = 06H
1001	00 01 10 00 = 18H	00 01 10 00 = 18H
1010	00 01 00 10 = 12H	00 01 00 10 = 12H
1011	01 01 00 00 = 50H	01 10 00 00 = 60H
1100	10 01 00 01 = 91H	10 10 00 10 = A2H
1101	10 01 01 00 = 94H	10 10 10 00 = A8H
1110	01 01 01 10 = 56H	01 10 10 01 = 69H
1111	11 01 01 01 = D5H	11 10 10 10 = EAH

Tabla 2.7. Palabras binarias y ternarias

Como se puede notar, el código 4B-3T es un código bloque, por lo que existe un retardo entre la entrada de bits y la transmisión de los elementos de señal que los representa. Para ilustrar este concepto se grafican las señales de un ejemplo de codificación en la figura 2.43. Como se ve, una vez que han ingresado los cuatro dígitos binarios al *buffer*, se busca la palabra ternaria a transmitir, la cual será almacenada en el *buffer* de transmisión.

La transmisión se la realiza a una velocidad codificada que es $3/4$ del ritmo nominal y cuya temporización es controlada por el *timer* 0. Cuando se ha producido el ingreso del cuarto bit, se "arranca" el *timer* 0 para que contabilice un tiempo igual a la mitad del período de transmisión (velocidad disminuida) y se envíe entonces el primer elemento de señal de la palabra ternaria. Antes de que se contabilicen 4 nuevos bits, el *timer* 0 habrá provocado tres interrupciones que permitan la salida de los tres dígitos ternarios correspondientes a los

4 dígitos binarios ingresados anteriormente. De lo anterior se concluye que el retardo en la transmisión es de algo más de $4\frac{1}{6}$ de la duración del bit de entrada.

El retardo adicional se produce por la demora que el microcontrolador tenga en ejecutar la codificación.

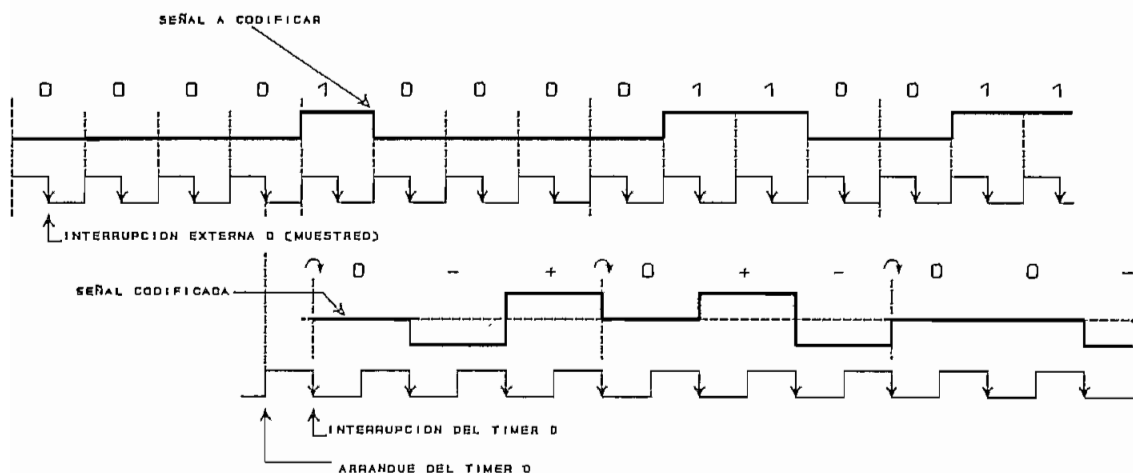


Fig. 2.43. Ejemplo de codificación 4B - 3T

Volviendo a la explicación del proceso de codificación (figura 2.43) se tiene como primer paso el envío de una secuencia de sincronismo que consta de 12 dígitos binarios (9 dígitos ternarios), todos los cuales son 0_L . Esta secuencia se envía para sincronizar el transmisor con el receptor, ya que al tratarse de un código bloque, es necesario que el decodificador recobre la estructura que envió el codificador. Cualquier deslizamiento de un elemento de señal producirá una decodificación errónea.

La secuencia escogida de 12 bits 0_L se lo hace debido a que la palabra codificada (0-+) es de disparidad cero. Se envían tres de estas palabras ternarias para dar oportunidad al decodificador a una correcta sincronización, ya que el máximo deslizamiento que puede haber es de dos

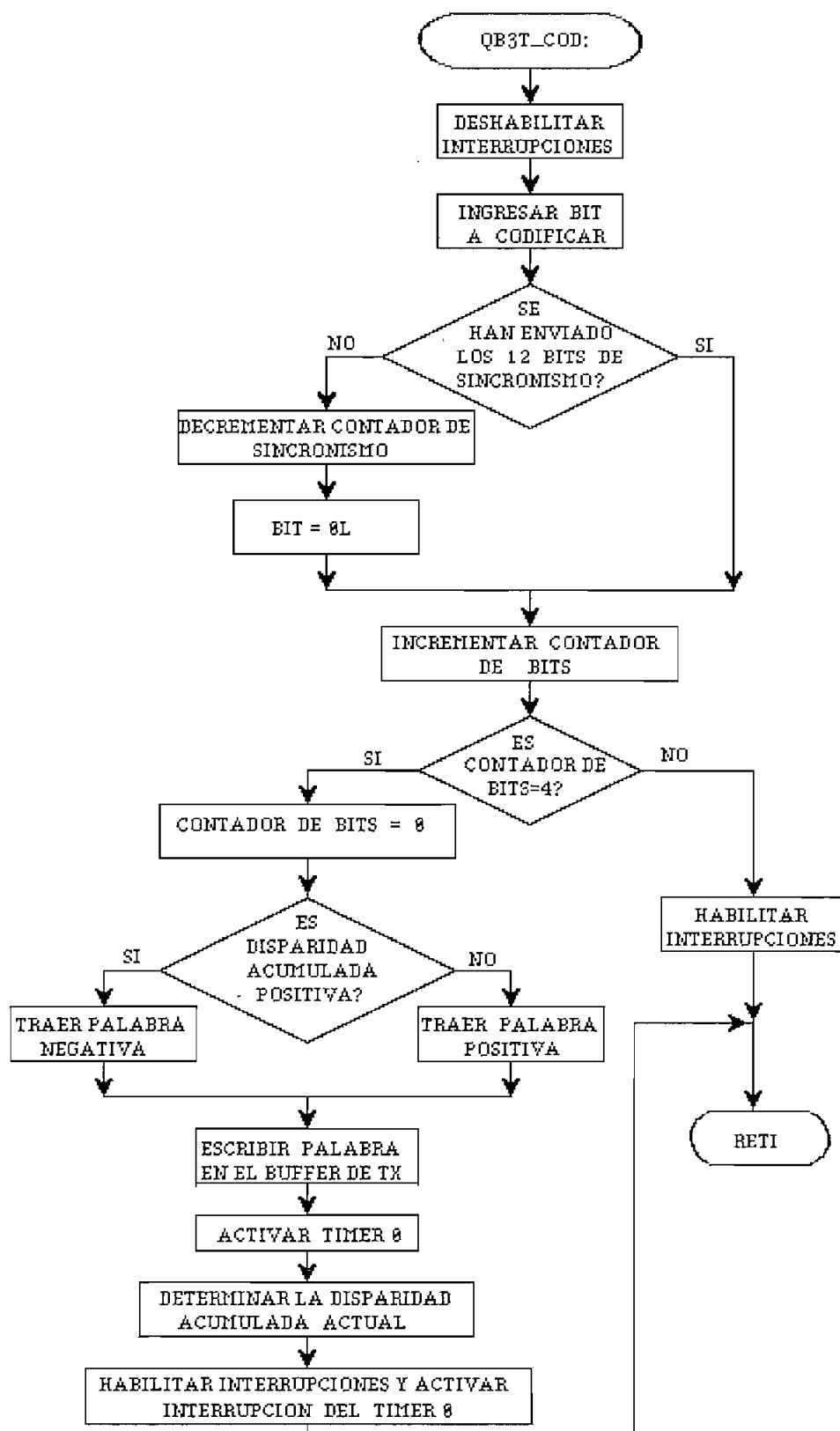


Fig. 2.44. Codificación 4B-3T

elementos de señal con lo cual en la tercera palabra se habrá recobrado el sincronismo.

Posterior al envío de la secuencia de sincronismo, se realiza la codificación de la manera antes indicada, es decir activando la interrupción del *timer* 0, cuya función se observa en la figura 2.44, este procedimiento se repite por cada dígito ternario que se transmite. A continuación se procede a calcular la nueva disparidad acumulada, la cual servirá para la codificación de los siguientes 4 dígitos binarios y se vuelve al lazo de espera no sin antes habilitar las interrupciones que estuvieron previamente activas así como también activar la interrupción del *timer* 0.

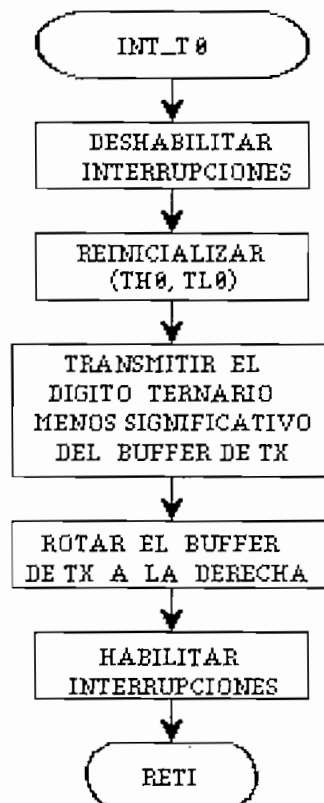


Fig. 2.45. Interrupción del *timer* 0 en la codificación 4B-3T

b. Decodificación (QB3T_DEC).

En la decodificación es necesario ingresar como información tanto la magnitud como el signo del elemento de señal entrante.

Como primer paso se debe verificar la sincronización del transmisor con el receptor, sólo así se garantizará que los cuatro dígitos binarios que se obtengan de un conjunto de tres ternarios corresponda exactamente a la información enviada.

Con esta finalidad existe una bandera (marcada como 'ATEND'), de modo que si su valor es 1, se atenderá a la interrupción externa 1, caso contrario se retornará al lazo de espera luego de complementar el bit; con ello no se atiende a la presente interrupción y se habrá deslizado un dígito ternario. Esto permite que cuando se reciban los tres primeros dígitos ternarios que no correspondan a la secuencia de sincronismo, no se realizará ninguna decodificación sino que se deslizará un dígito ternario hasta que se pueda ajustar la palabra ternaria de modo que se reconozca la secuencia de sincronismo.

Cuando se han recibido tres elementos de señal ternaria cuyo valor corresponde al establecido (0-+), están sincronizados el transmisor y receptor lo cual se indica mediante la activación de otra bandera (registro R2).

Para los tres dígitos siguientes (y para todos los sucesivos), se atenderá la interrupción externa 1 y se procederá a la decodificación. Los tres elementos de señal son almacenados binariamente en la localidad de memoria marcada como 'BUFRX3'.

La decodificación se basa en el establecimiento de una tabla de palabras esperadas 'DEC_Q1', cada una de las

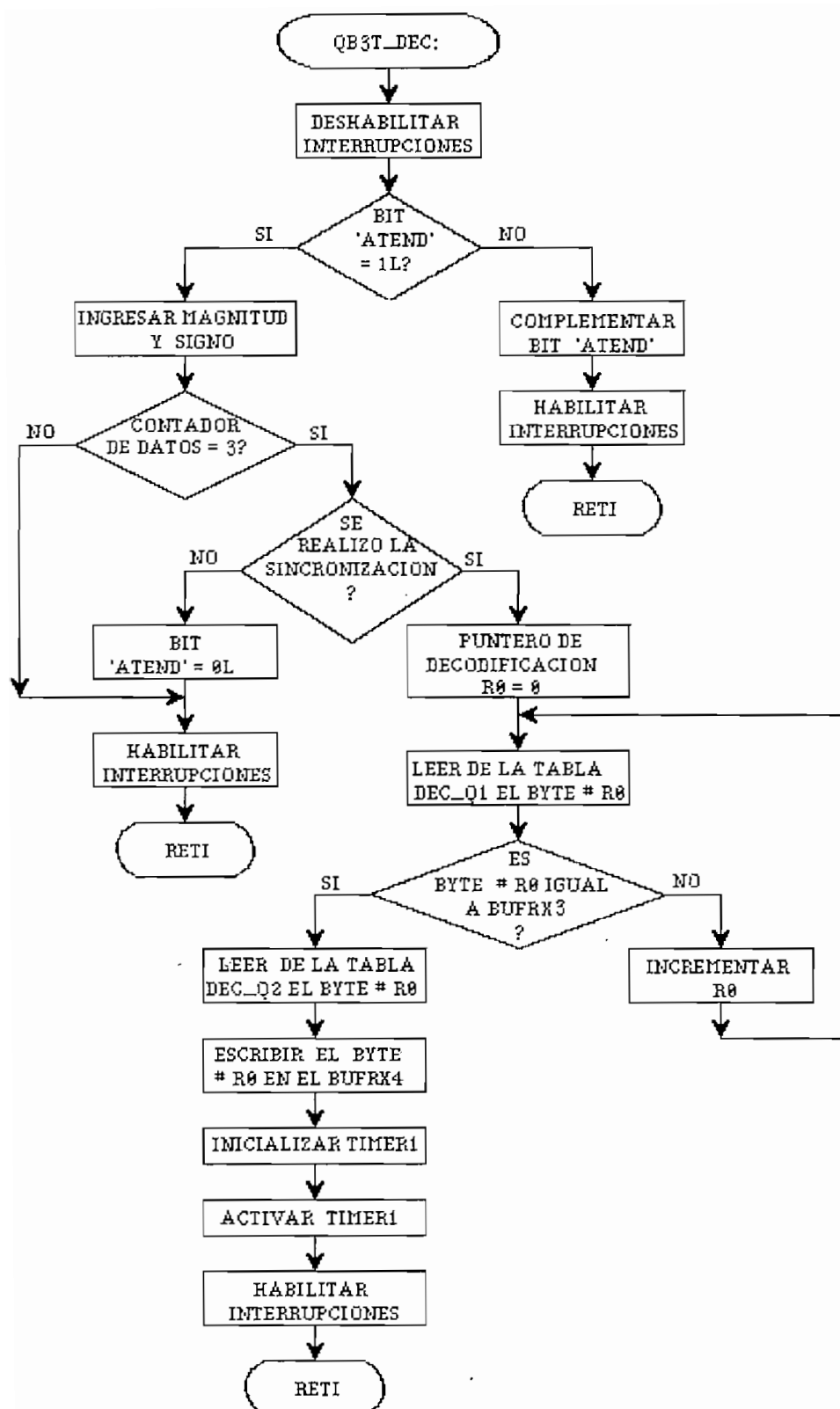
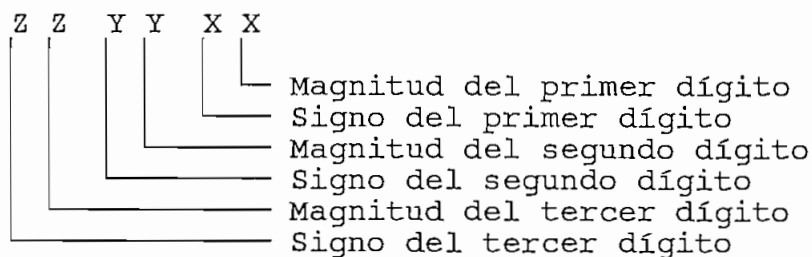


Fig. 2.46. Decodificación 4B-3T

cuales es comparada con la palabra ingresada hasta obtener una igualdad. Cuando se ha alcanzado la igualdad, se busca en una tabla paralela 'DEC_Q2', los cuatro dígitos binarios que corresponden y se los almacena en el *buffer* de transmisión marcado como 'BUFRX4', para ser enviados al equipo terminal receptor de datos. Este procedimiento se ilustra en forma esquemática en la figura 2.46 en tanto que en la tabla 2.8 se tiene la estructura de palabras usadas en la decodificación; un elemento de señal ternario consta de un signo (1_L si es positivo y 0_L si es negativo) y una magnitud (valor absoluto) que se almacenarán en el *buffer* de recepción según se indica:



Palabras binarias	Palabras ternarias	Tabla de decodificación 'DEC_Q1'			
0000	0-+	00	01	11	= 07H
0001	-+0	01	11	00	= 1CH
0010	-0+	01	00	11	= 13H
0011	++- +-+	11	01	11	= 37H
0100	0++ 0--	00	11	11	= 0FH
0101	0+0 0-0	00	11	00	= 0CH
0110	00+ 00-	00	00	11	= 03H
0111	-++ +-+	01	11	11	= 1FH
1000	-+-	00	11	01	= 0DH
1001	+ -0	11	01	00	= 34H
1010	+0-	11	00	01	= 31H
1011	+00 -00	11	00	00	= 30H
1100	+0+ -0-	11	00	11	= 33H
1101	++0 --0	11	11	00	= 3CH
1110	++- --+	11	11	01	= 3DH
1111	+++ ---	11	11	11	= 3FH
		01	11	01	= 1DH
		00	01	01	= 05H
		00	01	00	= 04H
		00	00	01	= 01H
		11	01	01	= 35H
		01	00	00	= 10H
		01	00	01	= 11H
		01	01	00	= 14H
		01	01	11	= 17H
		01	01	01	= 15H

Tabla 2.8. Tablas de decodificación

Una vez que se ha establecido la palabra binaria que será enviada al receptor de datos, se procede a la transmisión del *buffer* 'BUFRX4' mediante la interrupción del *timer* 1. Este temporizador se encargará de contar el tiempo necesario para que se envíen los datos binarios a la velocidad nominal de transmisión, por tanto se producirá una interrupción por desbordamiento del *timer* 1 cada período de bit. Esto se ve ilustrado en la figura 2.47.

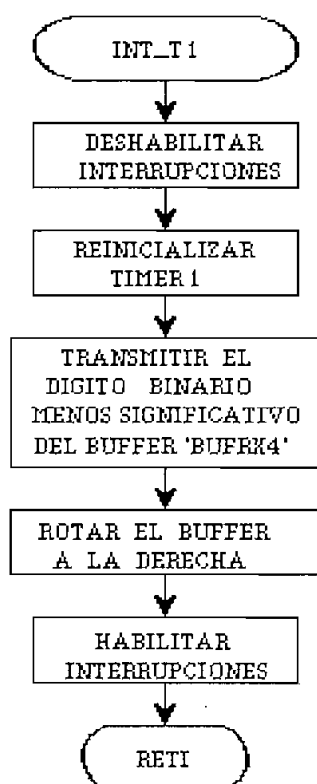


Fig. 2.47. Interrupción del *timer* 1 en la decodificación 4B-3T

2.5.9. Rutinas para el código MS43.

a. Codificación (MS43_COD).

El código MS43 al igual que el 4B-3T es un código de bloque, en el que se busca que la disparidad acumulada no exceda de una magnitud igual a dos. Al igual que en el

caso del esquema 4B-3T, su tabla de codificación (tabla 1.3) puede reducirse a un modo positivo y uno negativo. Igualmente, atendiendo al diagrama de estados de la figura 1.22 se podrá escoger uno de los modos cuya polaridad sea opuesta a la disparidad acumulada, de tal manera que se tienda a eliminar la componente continua.

Con este objetivo se elabora la tabla 2.9 en la que se resume la codificación a un modo positivo y otro negativo.

Palabras binarias	Palabras ternarias	
	Modo positivo	Modo negativo
0000	+++	-+-
0001	++0	00-
0010	+0+	0-0
0011	0-+	0-+
0100	0++	-00
0101	-0+	-0+
0110	-+0	-+0
0111	--+	--+
1000	+--	---
1001	00+	--0
1010	0+0	-0-
1011	0+-	0+-
1100	+00	0--
1101	+0-	+0-
1110	+ -0	+ -0
1111	+- -	+ - -

Tabla 2.9. Palabras binarias y ternarias

Para realizar la codificación, se parte del hecho de que existen cuatro estados posibles de disparidad acumulada (+1, -1, +2 y -2), por lo tanto para cada uno de estos estados se establece una tabla de la codificación ternaria que se puede tener para cada entrada de cuatro dígitos binarios. Por tanto, al establecer cuatro tablas de 16 elementos ($2^4 = 16$) cada una habrá contemplado la totalidad de casos binarios. Este es el origen de los cuatro modos de codificación a los que se hace referencia en la tabla 1.3 del capítulo 1. Con esto se logra

simplificar el proceso de codificación que consistirá en elegir la palabra correcta en una de los cuatro modos establecidos en la tabla 1.3, tal como se indica en la tabla 2.10.

Disparidad acumulada	Palabra ternaria
-2	Modo 1
-1	Modo 2
+1	Modo 3
+2	Modo 4

Tabla 2.10. Codificación MS43

La disparidad actual (luego de la codificación) está totalmente determinada para cada uno de los cuatro casos y se puede por tanto almacenar en una tabla paralela, a cada uno de los modos de codificación, con lo que el proceso está completo. La elaboración de las tablas usadas por el programa de codificación se la realiza similarmente a la manera descrita anteriormente para el código 4B-3T. Respecto a la demora de codificación, es la misma que se estableciera en el anterior código pseudoternario, de algo más de $4\frac{1}{6}$ períodos de bit de la señal de entrada.

El algoritmo de codificación hasta ahora descrito se lo resume en la figura 2.48. La secuencia de sincronismo que se envía para establecer una correcta temporización entre el transmisor y el receptor es el correspondiente a la secuencia binaria 0101 (-0+), que posee disparidad cero y que por tanto no altera la disparidad inicial. El proceso de transmisión de la palabra ternaria, desde el microcontrolador hacia la línea de transmisión, se lo hace mediante la temporización establecida por el timer 0, de la manera ya expuesta en la figura 2.45.

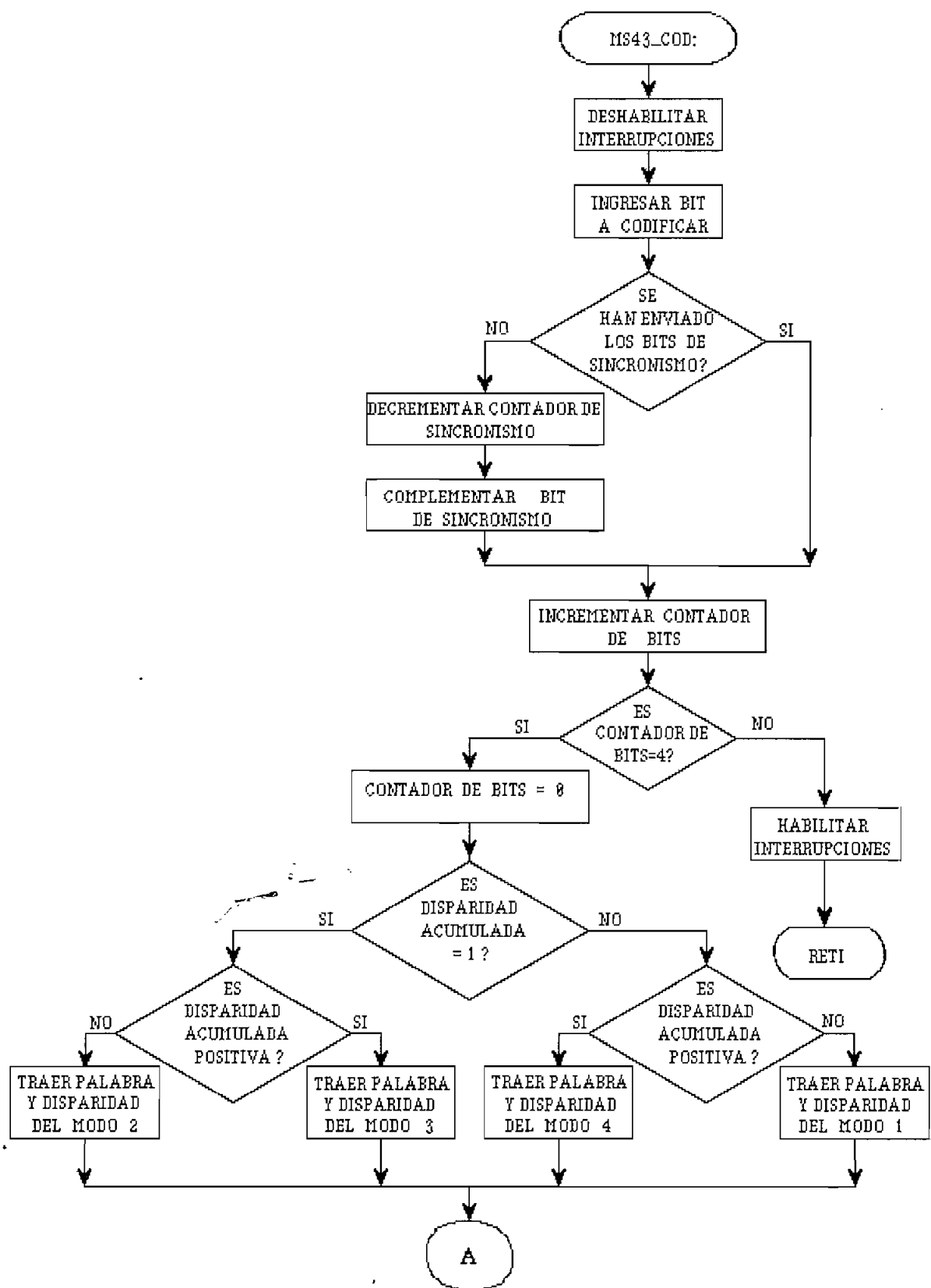


Fig. 2.48. Codificación MS43

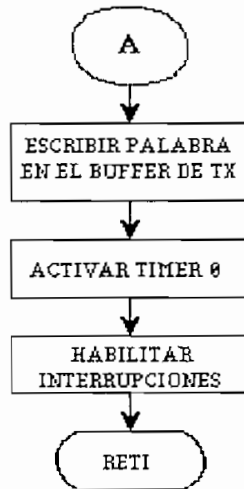


Fig. 2.48. Continuación

b. Decodificación (MS43_DEC).

Para el desarrollo de la decodificación MS43, se sigue el mismo algoritmo establecido en el esquema 4B-3T.

En primer lugar se verifica la sincronización entre transmisor y receptor mediante la recepción de la palabra de sincronización establecida (-0+). Luego de ello se procede a la decodificación de cada uno de los arreglos de tres elementos de señal ternarios que van siendo almacenados en el en *buffer* 'BUFRX3'.

La tabla de palabras esperadas se marca como 'DEC_M1', cada una de ellas se va comparando con la palabra ternaria ingresada hasta obtener la igualdad. En este punto se trae desde una tabla paralela marcada como 'DEC_M2' la palabra binaria correspondiente a los cuatro bits que se enviaron originalmente. Este proceso se repite continuamente cada vez que se ingresan tres dígitos ternarios.

Para el envío de los bits desde el microcontrolador

hasta el equipo receptor de datos se utiliza la temporización del *timer* 1 que los envía a un ritmo igual al nominal establecido.

Los diagramas de flujo son los mismos que para el esquema 4B-3T (figuras 2.46, 2.47) donde sólo cambia la denominación de las tablas de decodificación.

2.5.10. Rutinas para el código B3ZS.

a. Codificación (B3ZS_COD).

Para realizar esta codificación, cada bit que es recibido es almacenado previamente, pues primero debe realizarse la transmisión del bit codificado que llegó tres períodos de bit antes. La demora que se produce en el codificador es por lo tanto de tres duraciones de bit. Este procedimiento no se realiza durante la codificación de los tres primeros bits ya que no existe información válida para ser transmitida, es a partir del cuarto bit ingresado que se empieza a recibir y a transmitir simultáneamente.

Cuando el bit ingresado es 1_L es suficiente con escribir en el *buffer* de transmisión el código correspondiente a un pulso de polaridad opuesta al anterior transmitido, según la regla de codificación AMI. Cuando se ha recibido un 0_L será necesario interrogar si éste es el tercer 0_L ya que en este punto se debe aplicar la regla de codificación B3ZS; de no ser así, se escribirá el código de cero en el *buffer* y se esperará al siguiente bit, al cual se aplicará nuevamente estos criterios.

Al recibirse el tercer 0_L se debe tener en cuenta el número de 1_L 's desde la última sustitución y la polaridad de la última marca, de acuerdo con lo que se indica en la figura 2.49. Este procedimiento se repite indefinidamente por cada bit que ingresa en el codificador.

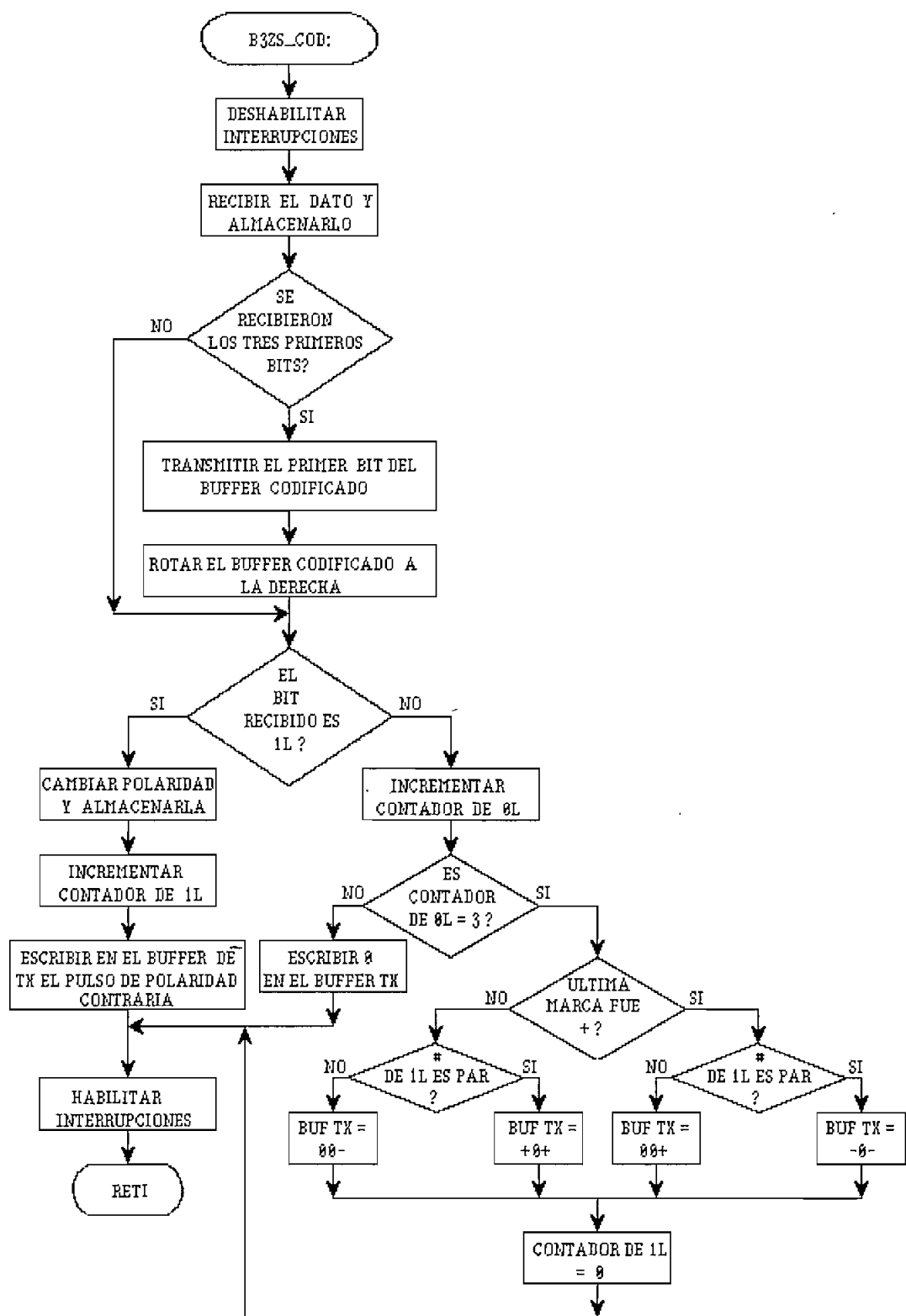


Fig. 2.49. Codificación B3ZS

b. Decodificación (B3ZS_DEC).

Para la decodificación B3ZS se utiliza el siguiente algoritmo: cuando el dígito ternario que se ingresa es cero, se escribe en el *buffer* de recepción marcado como 'BUFRX3' un 0_L.

Si el dígito ternario que ingresa es un pulso (positivo o negativo), es necesario interrogar si este constituye una violación a la regla AMI, es decir si tiene la misma polaridad que el pulso precedente. En caso afirmativo tanto el bit actual como los dos anteriores serán 0_L.

Si el pulso que se ha ingresado no constituye una violación a la regla AMI, se escribe un 1_L binario en la posición correspondiente del *buffer* de recepción.

Como un proceso paralelo a la decodificación, luego de haber recibido los tres primeros dígitos ternarios, se transmite uno tras otro los bits que se almacenan en el *buffer* de recepción, cada vez que se produzca la decodificación de un elemento de señal. De esto se concluye que el proceso de decodificación aporta con un retardo de 3 períodos nominales de bit. El proceso de decodificación se lo ilustra en la figura 2.50.

2.5.11. Rutinas para el código HDB3.

a. Codificación (HDB3_COD).

Cada bit que es ingresado es codificado, al mismo tiempo que se transmiten los dígitos ternarios producto de codificaciones anteriores, con un retraso de cuadro bits. Un 1_L producirá un pulso de polaridades alternadas (similar a la codificación AMI), en tanto que un 0_L origina la transmisión de un elemento de señal de cero voltios,

siempre y cuando no sea el cuarto 0_L consecutivo.

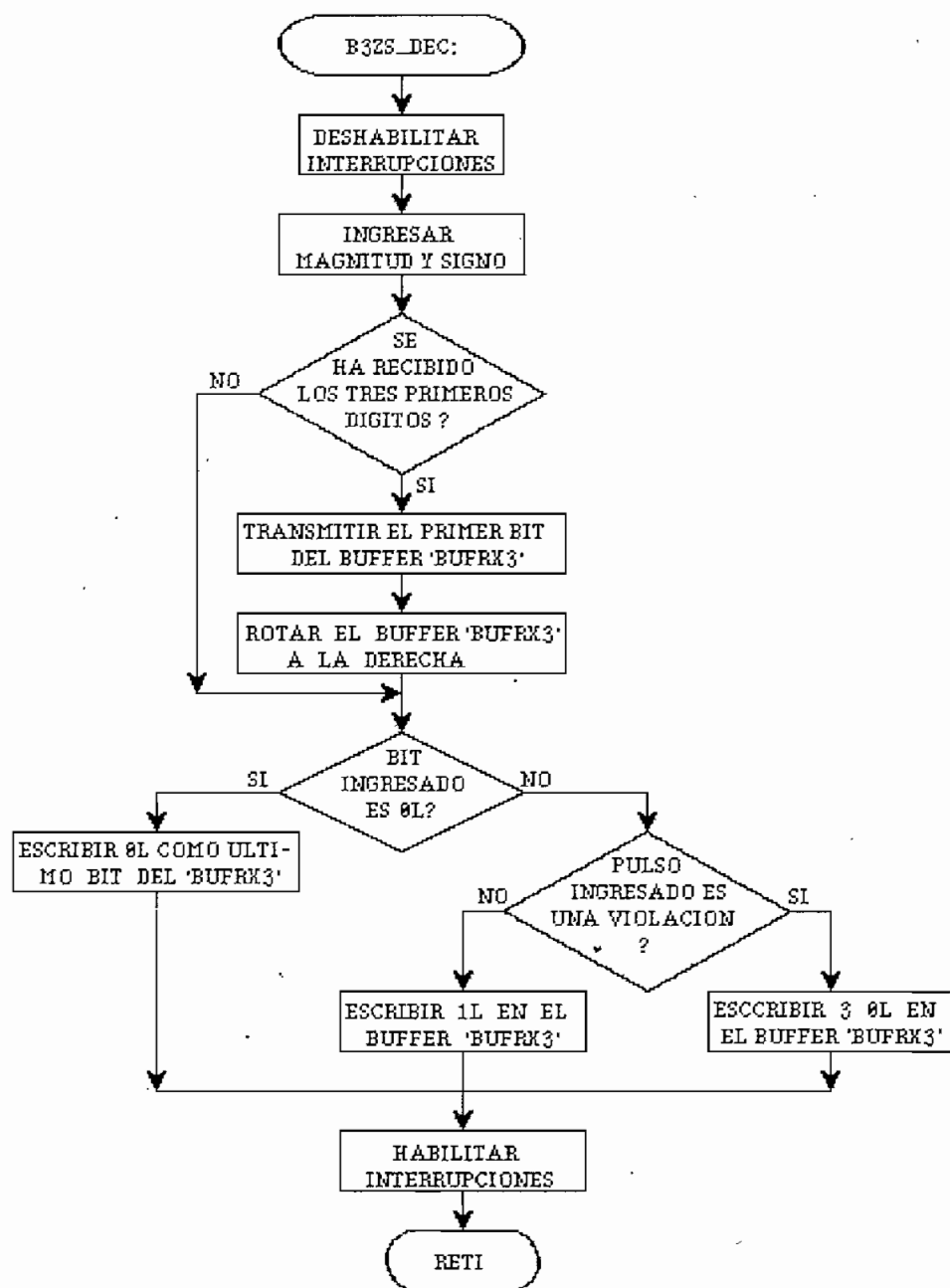


Fig. 2.50. Decodificación B3ZS

En el caso de que existan cuatro ceros seguidos, no se transmitirán cuatro elementos de señal de nivel cero sino que se debe realizar un reemplazo (de los 4 0_L) cuyo valor depende del número de 1_L que ocurrieron desde la

última vez que se produjo esta situación y de la polaridad del último pulso que se envió, tal como se indica en la figura 2.51.

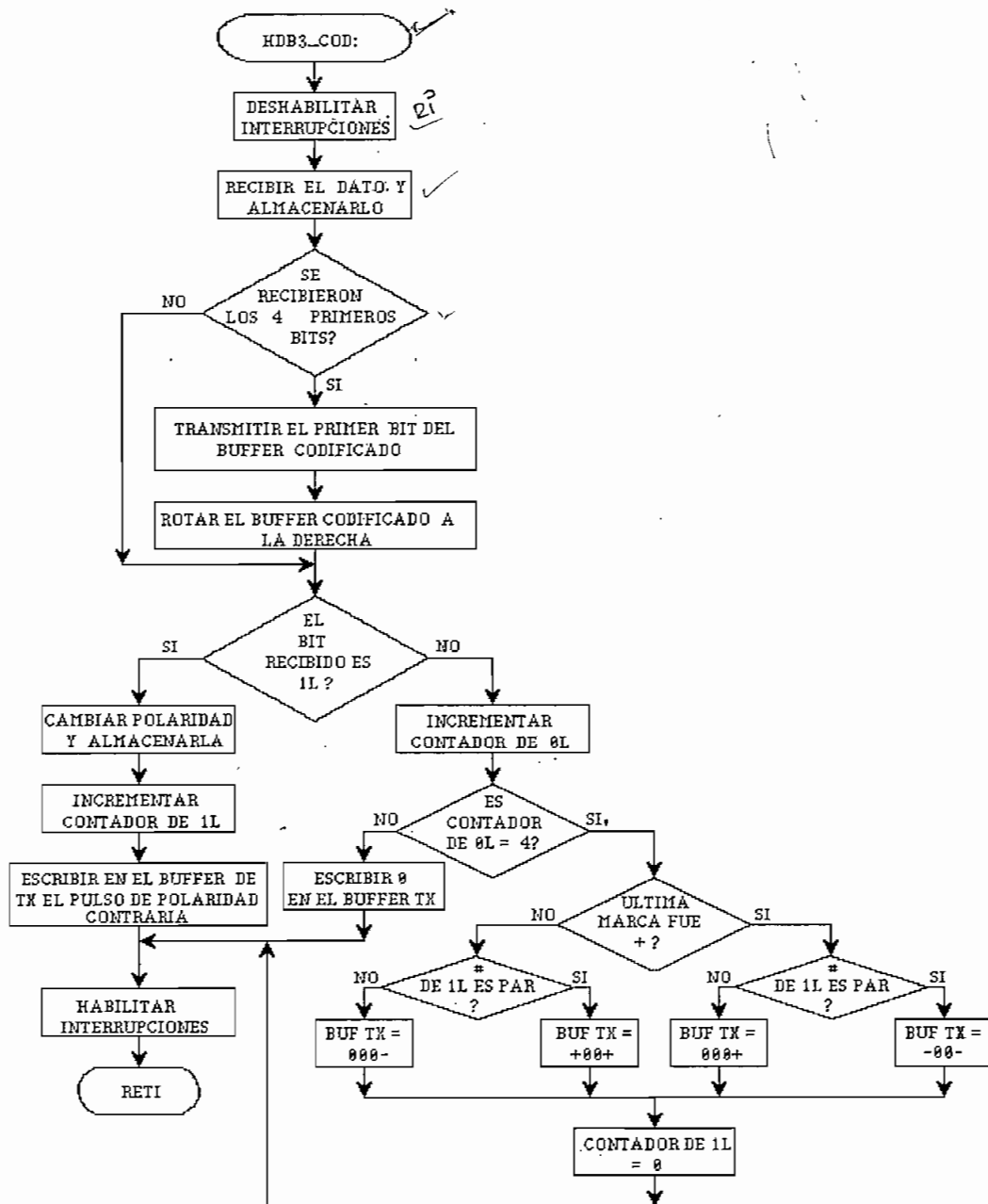


Fig. 2.51. Codificación HDB3

De lo que antecede se deduce que el retardo que origina el proceso de codificación en el caso del código HDB3 es de al menos cuatro períodos de duración de bit.

b. Decodificación (HDB3_DEC).

La decodificación es un proceso sencillo, similar al establecido para el esquema de codificación B3ZS. Se basa en que al recibir el elemento de señal ternario, si éste es cero se escribe un 0_L en el *buffer* de recepción, el cual consta de 4 elementos binarios.

Por otra parte, si el elemento de señal recibido es un pulso (positivo o negativo), se compara su polaridad con la del último pulso recibido, si ésta es diferente, se ha cumplido con la regla AMI de alternabilidad en la polaridad y se escribe un 1_L en el *buffer*.

Si las polaridades de los pulsos son iguales, se ha producido una violación a la regla AMI y por tanto se escribirán cuatro 0_L en el *buffer*, bits que son el producto de haber codificado cuatro 0_L seguidos en el transmisor.

Este procedimiento continuo de decodificación se produce con el ingreso de cada elemento ternario de señal; al mismo tiempo se realizará el envío de un bit del *buffer* hacia el receptor de datos, es decir, se tiene simultáneamente para cada interrupción externa el proceso de decodificación y el de transmisión con un retardo de 4 bits. La figura 2.52 ilustra el diagrama de decodificación.

CAPITULO III

EVALUACION EXPERIMENTAL DEL EQUIPO

3.1. PRESENTACION DEL EQUIPO.

El CODEC didáctico para transmisión digital en banda base está montado en un gabinete metálico, como se puede observar en la figura 3.1, en el que se distinguen claramente la tarjeta del CODEC y la fuente de alimentación. La tarjeta del CODEC posee conectores que permiten ingresar o monitorear las señales involucradas en el proceso de codificación o decodificación.

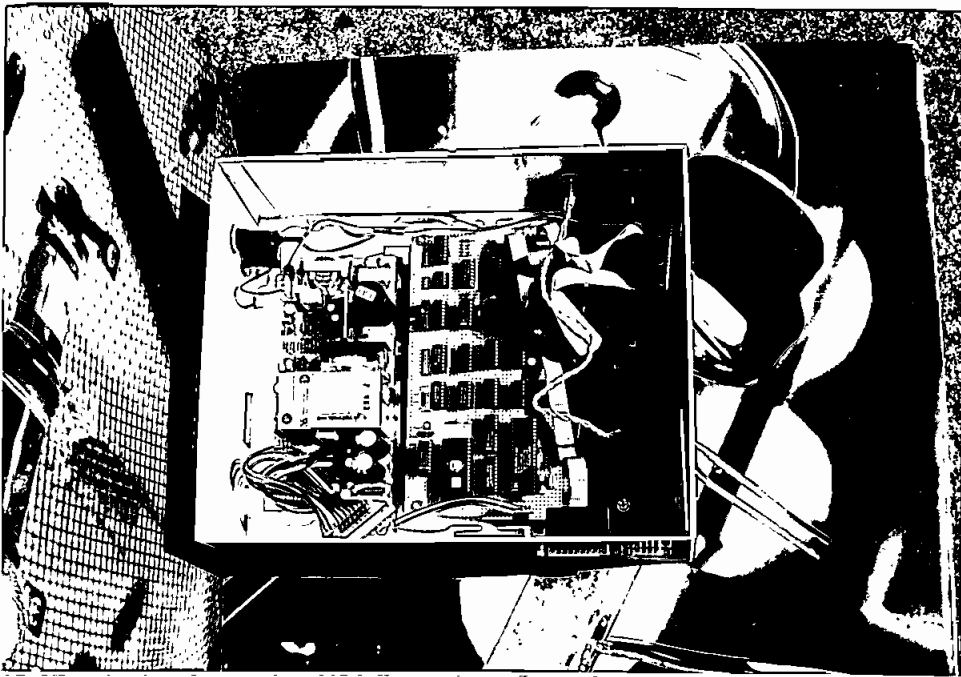


Fig. 3.1. Vista interior del equipo

En la parte superior del gabinete se encuentra el "display" alfanumérico y el teclado que permiten realizar el control sobre los parámetros del equipo (figura 3.2). Estos dos periféricos están conectados internamente a la

tarjeta del CODEC mediante los conectores descritos en el numeral 2.4 del capítulo anterior.

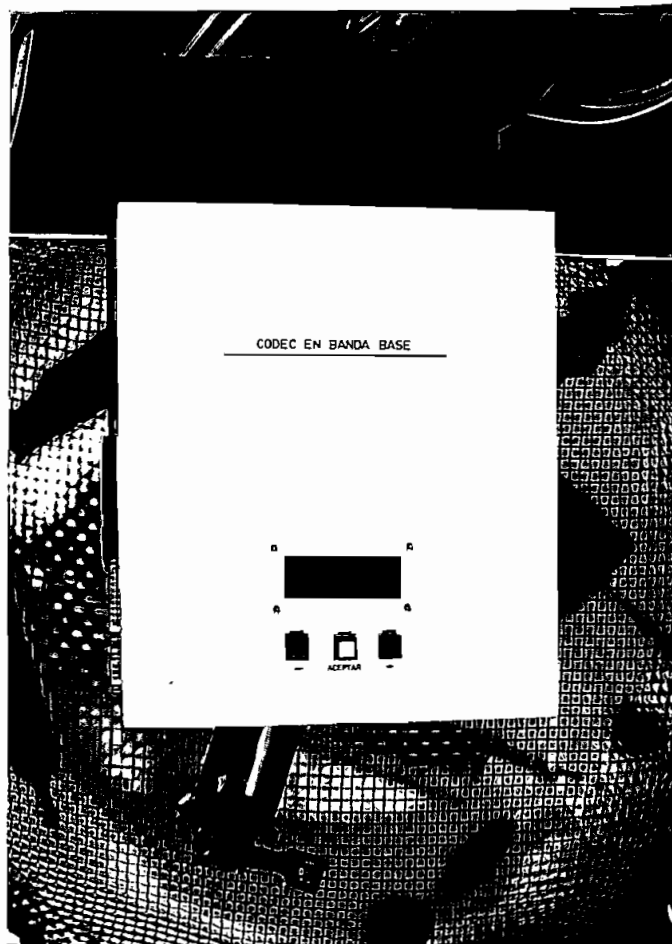


Fig. 3.2. Vista superior del equipo

En la parte frontal del equipo se tienen los puntos de prueba, de donde se podrán tomar las señales para monitorear el funcionamiento del CODEC. Las señales a medirse son:

- a. Señal a codificar.
- b. Señal codificada.
- c. Reloj de codificación.
- d. Control de codificación A.
- e. Control de codificación B.
- f. Nivel de referencia (GND).

- g. Señal a decodificar.
- h. Reloj de decodificación.
- i. Señal decodificada TTL.
- j. Señal decodificada RS-232.
- k. Nivel de referencia (GND).

Es de notar que tanto para el proceso de codificación como el de decodificación, la referencia de voltaje (GND) es la misma, pero se provee de dos salidas para facilitar el monitoreo de señales. En la figura 3.3 se ilustra la vista frontal del equipo.

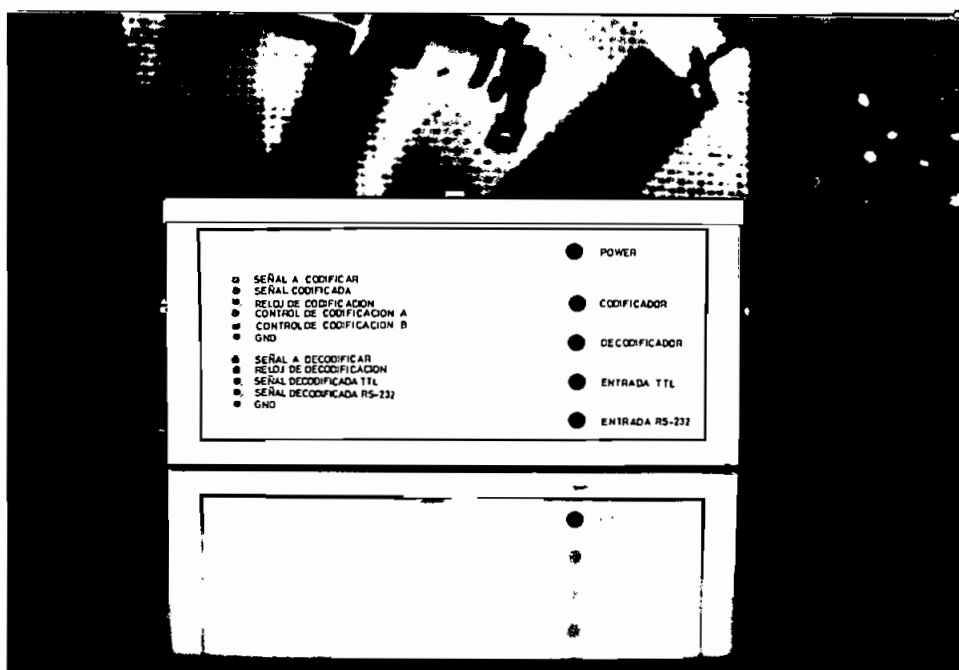


Fig. 3.3. Vista frontal del equipo.

Adicionalmente, el panel frontal presenta los LED's de configuración, los mismos que permiten conocer el modo de funcionamiento del CODEC, a saber:

- a. Encendido/Apagado (Power).
- b. Codificador.
- c. Decodificador.
- d. Entrada TTL.

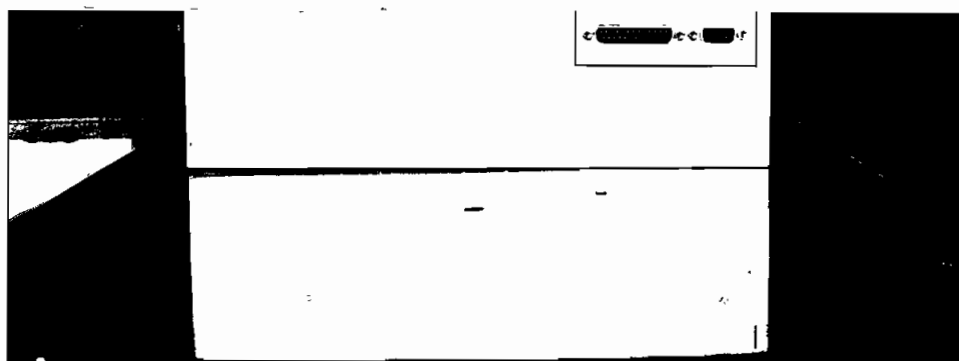


Fig. 3.4. Vista lateral izquierda del equipo

Los controles de encendido del equipo y de un reset para el microcontrolador se encuentran en la parte lateral derecha del gabinete (figura 3.5). El conector para la

alimentación AC (110 V) y el fusible de protección se ubican en la parte posterior, tal como se muestra en la figura 3.6.

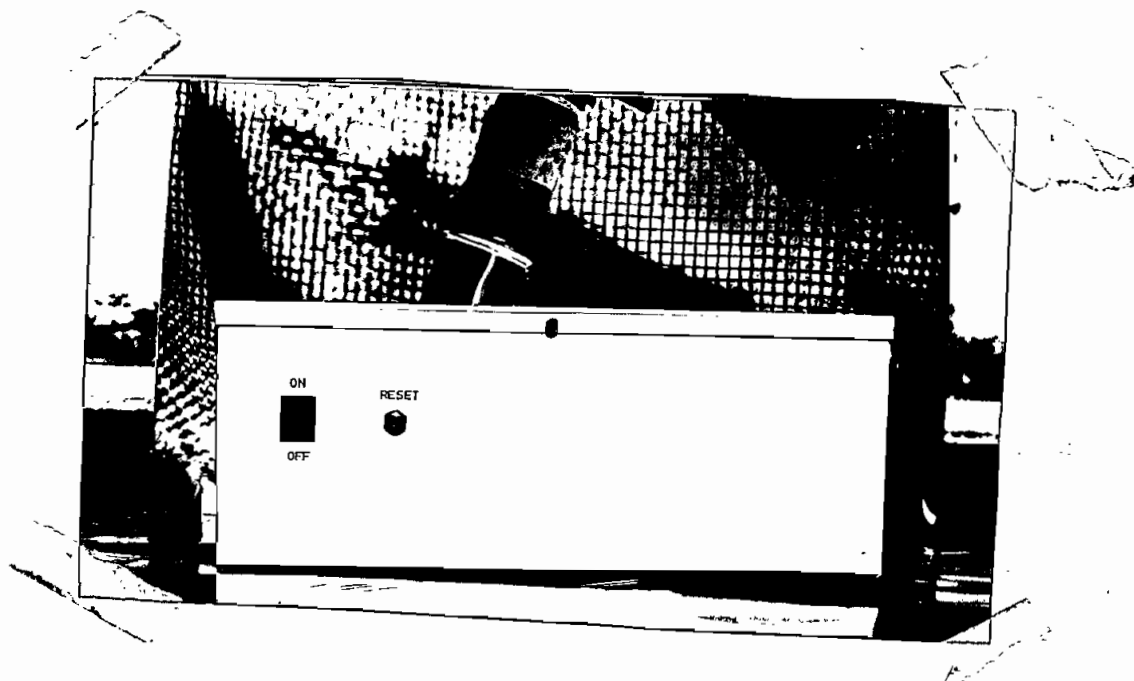


Fig. 3.5. Vista lateral derecha del equipo

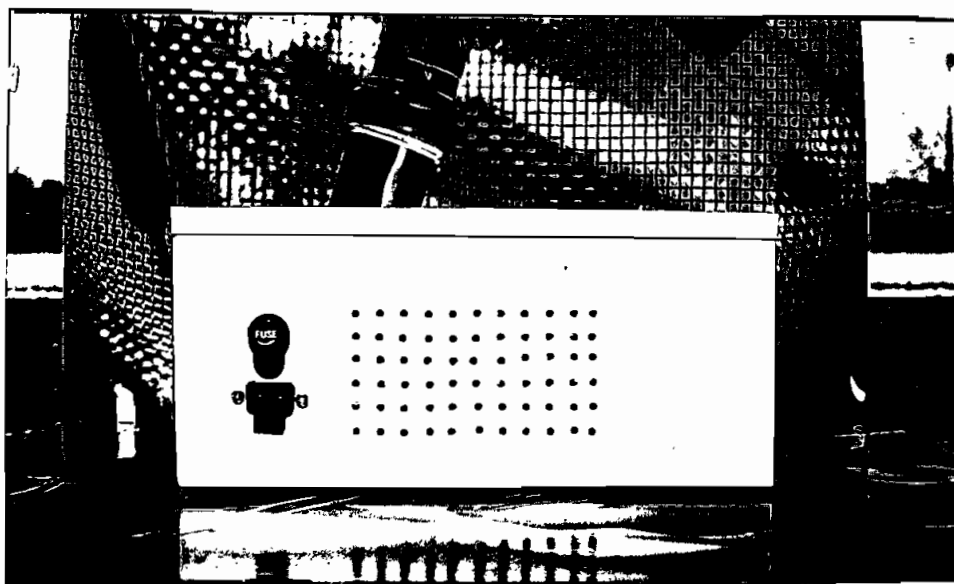


Fig. 3.6 Vista posterior del equipo

Una ilustración de la conexión demostrativa de dos terminales a través de los dos equipos de codificación/decodificación se observa en la figura 3.7.

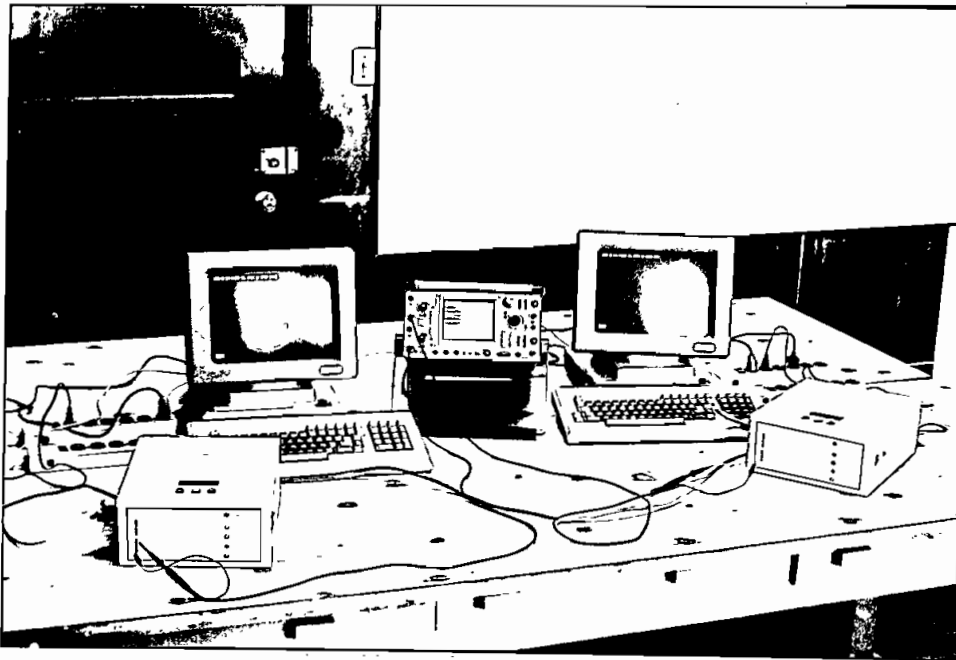


Fig. 3.7. Conexión demostrativa del equipo

3.2. PRUEBAS DE FORMATO.

El equipo construido se lo ha diseñado con fines didácticos, por lo cual la visualización de los formatos de codificación y la tarea de decodificación es una de las pruebas más importantes que se realiza con el codec en banda base.

Para la realización de esta prueba se llevará a cabo la codificación y decodificación de una secuencia de bits, que ingresados en niveles TTL permitirán observar de manera óptima los procesos de codificación y decodificación. La secuencia elegida es **01101100**, que

corresponde al carácter ASCII de la letra 'l' minúscula (6CH).

Este flujo repetitivo de bits es obtenido de un generador de secuencias binarias a una velocidad de 1200 bit/s y se alimenta a uno de los equipos que actuará como codificador. La señal de salida se alimentará al segundo equipo, que configurado como decodificador permitirá obtener la secuencia original de bits transmitidos. Debe señalarse sin embargo, que estas pruebas podrían realizarse también con un solo equipo si este actúa como codificador y decodificador.

Para el registro de los datos se emplea un osciloscopio Tektronix, modelo 2220, que posee memoria; un conector DB9 del osciloscopio provee la salida para comandar a un *plotter* analógico; con esta finalidad se usa el *plotter* Hewlett Packard modelo 7044A.

El primer paso en las pruebas de formato es la observación de la señal de datos entrante y el reloj de codificación. En la figura 3.8 se tiene la presentación de estos resultados en los cuales se aprecia que la sincronización con los datos entrantes ha permitido que el muestreo se lo realice en aproximadamente la mitad de la duración del bit (flanco negativo de la señal de reloj). Estas señales serán las mismas para cualquier esquema de codificación, excepto para el HDB3 en el que la secuencia de bits ingresado es **01010000** (código ASCII de la letra **P**, 50H), lo que permite observar el principio de sustitución (codificación de cuatro 0, consecutivos) de este esquema de codificación.

En la figura 3.9 se tiene el resultado de la codificación NRZ polar con lógica negativa y se aprecia un retardo de algo más de medio período de bit, debido al muestreo de codificación que se lo realiza en la mitad de

la duración del bit entrante, además del tiempo que requiere el microcontrolador para realizar la codificación. La decodificación correspondiente se la observa en la figura 3.10, en donde se indica la señal recuperada de reloj, la misma que permite muestrear cuatro veces la señal entrante, asegurando de esta manera una decodificación óptima. Este proceso se observa con mayor detalle en la figura 3.11.

Lo dicho anteriormente para el código NRZ se lo obtiene para el código AMI en las figuras 3.12, 3.13 y 3.14 y para el código RZ en las figuras 3.15, 3.16 y 3.17. Respecto al código RZ se debe manifestar que no se lo implementa exactamente con un ciclo de trabajo del 50% sino de un 55%, debido a que de esta manera se permite que el muestreo en la primera mitad del período de bit sea más preciso y con ello se aumenta la confiabilidad de la decodificación.

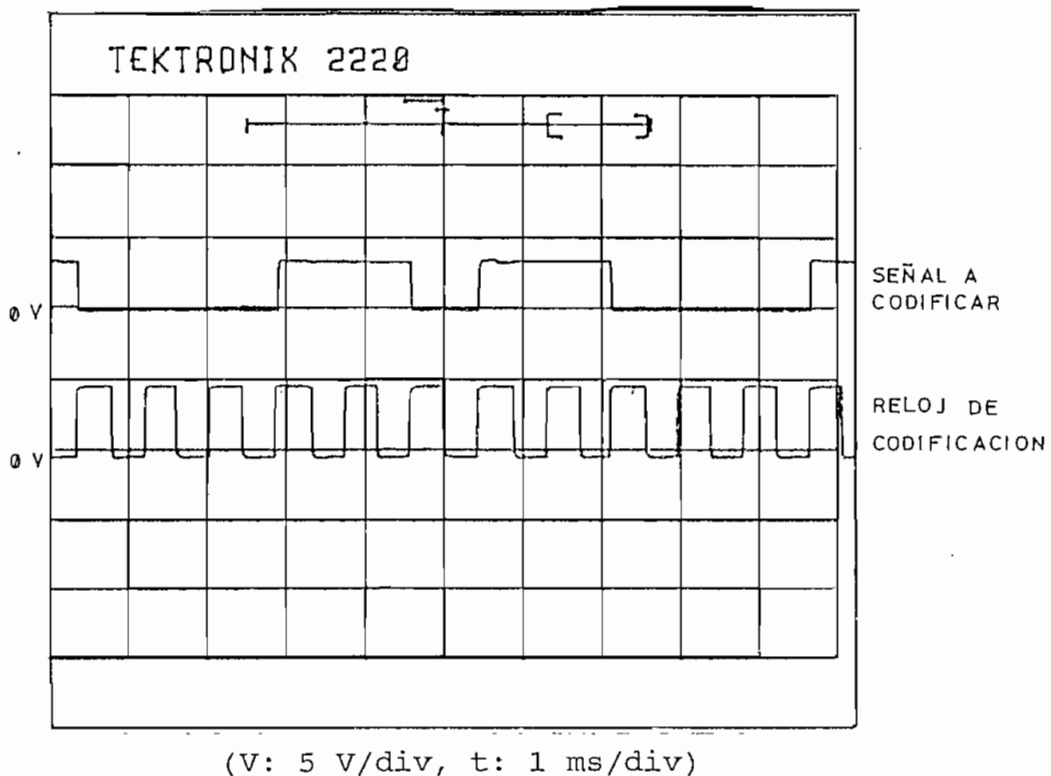
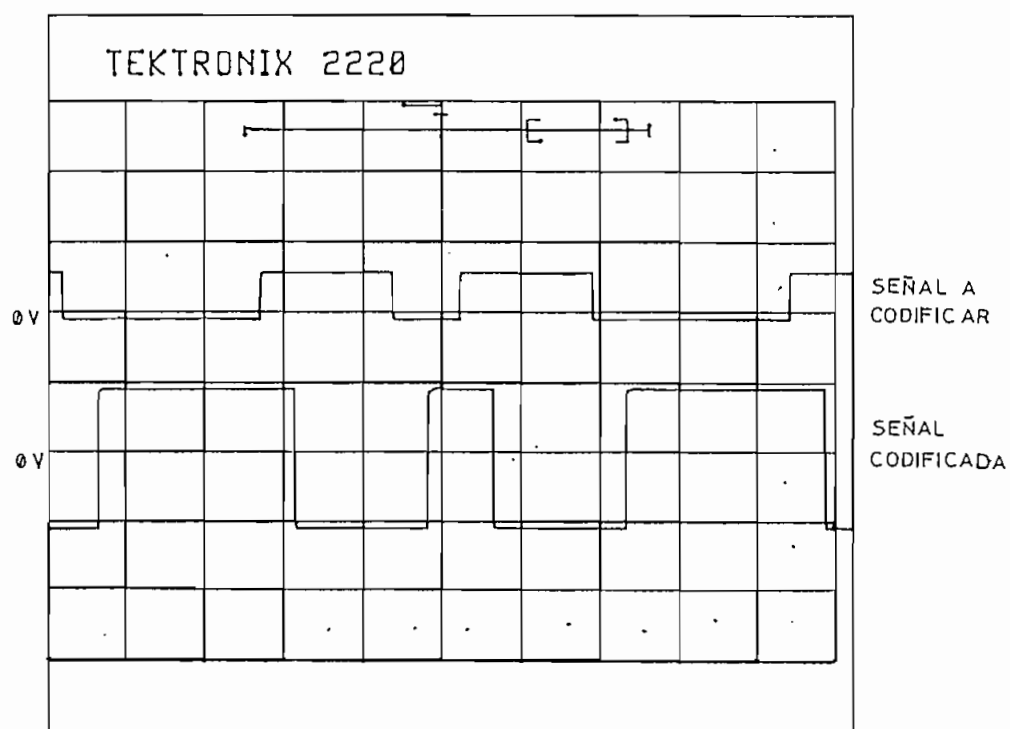
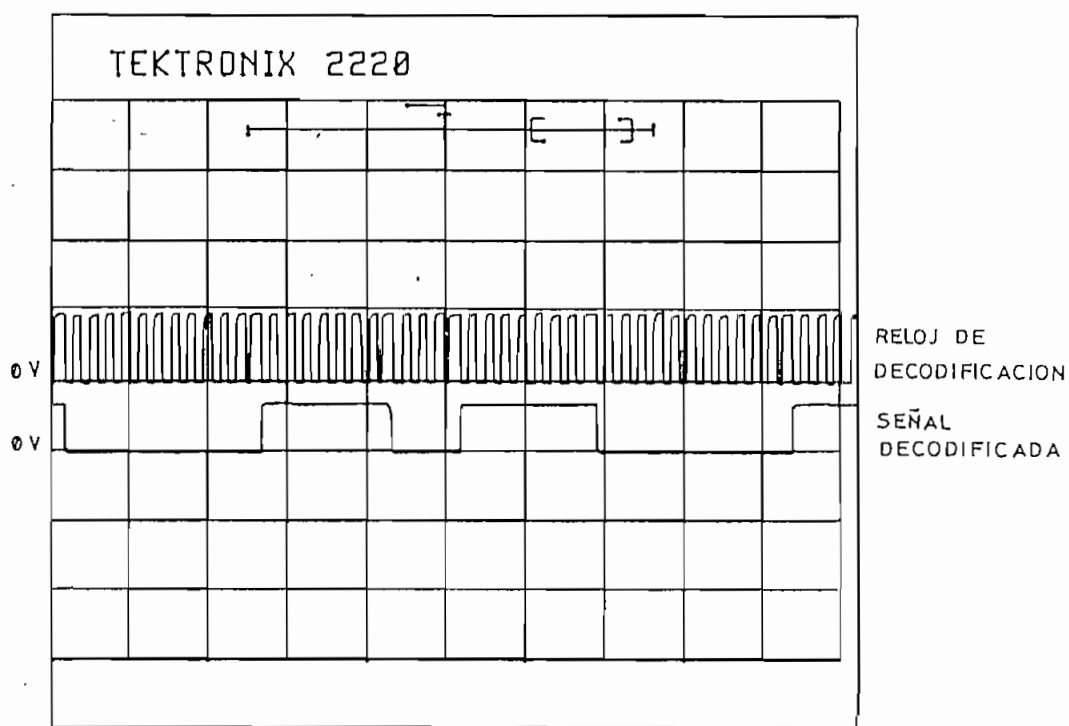


Fig. 3.8. Señal entrante al codificador



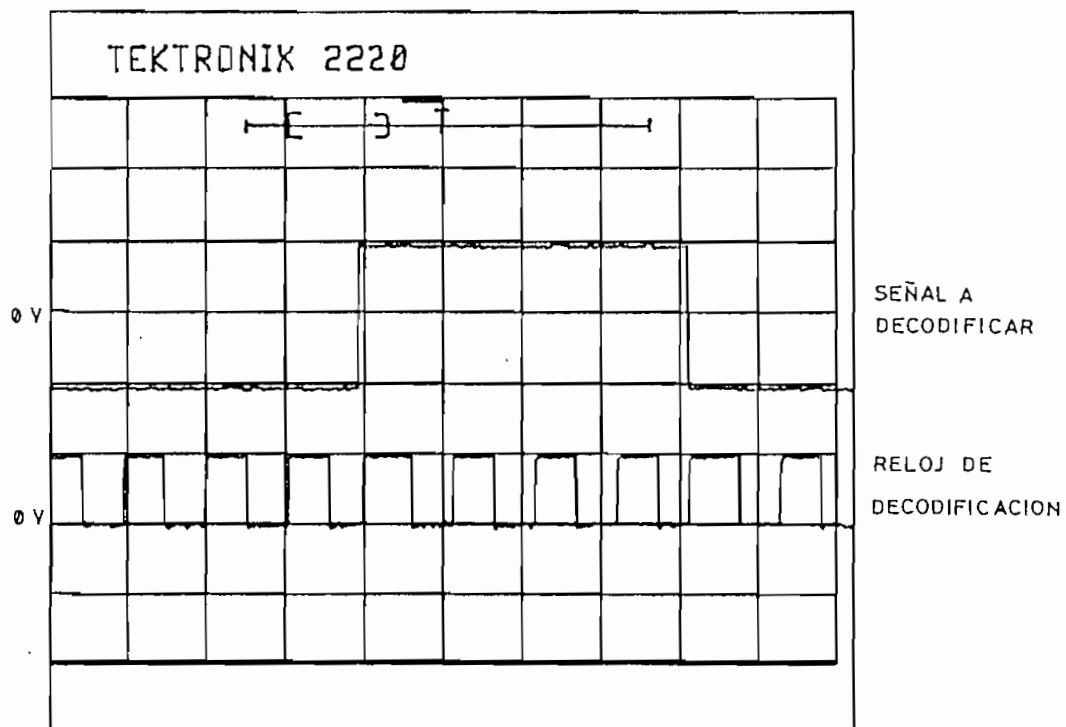
(V: 5 V/div, t: 1ms/div)

Fig. 3.9. Codificación NRZ polar



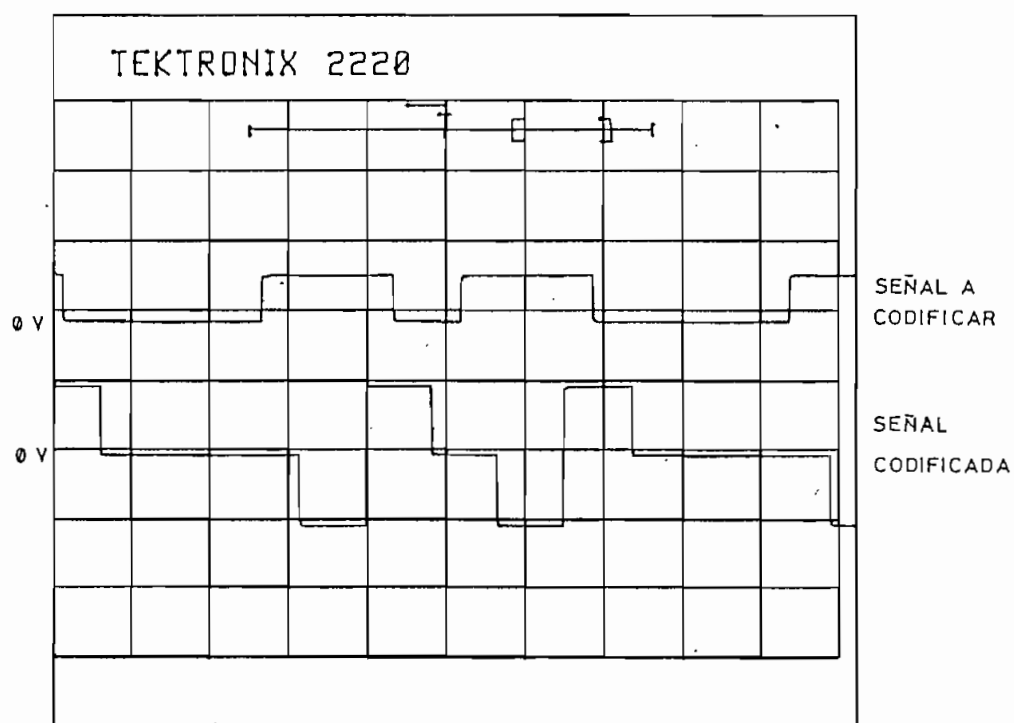
(V: 5 V/div, t: 1 ms/div)

Fig. 3.10. Decodificación NRZ polar



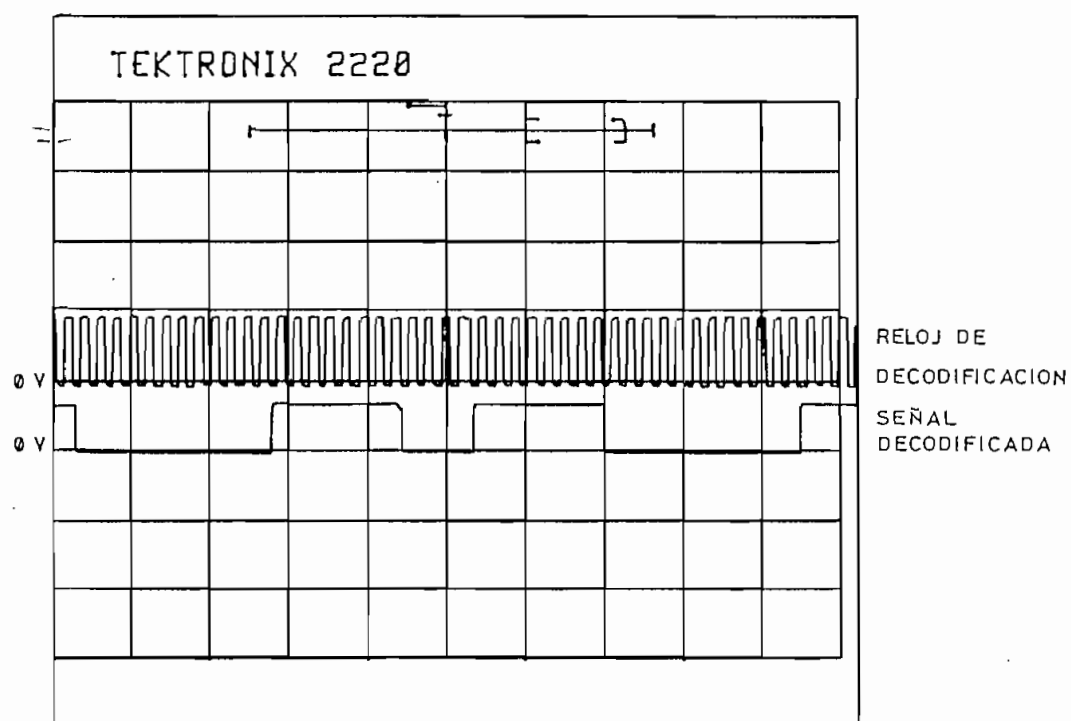
(V: 5 V/div, t: 0.2 ms/div)

Fig. 3.11. Reloj de decodificación (NRZ polar)



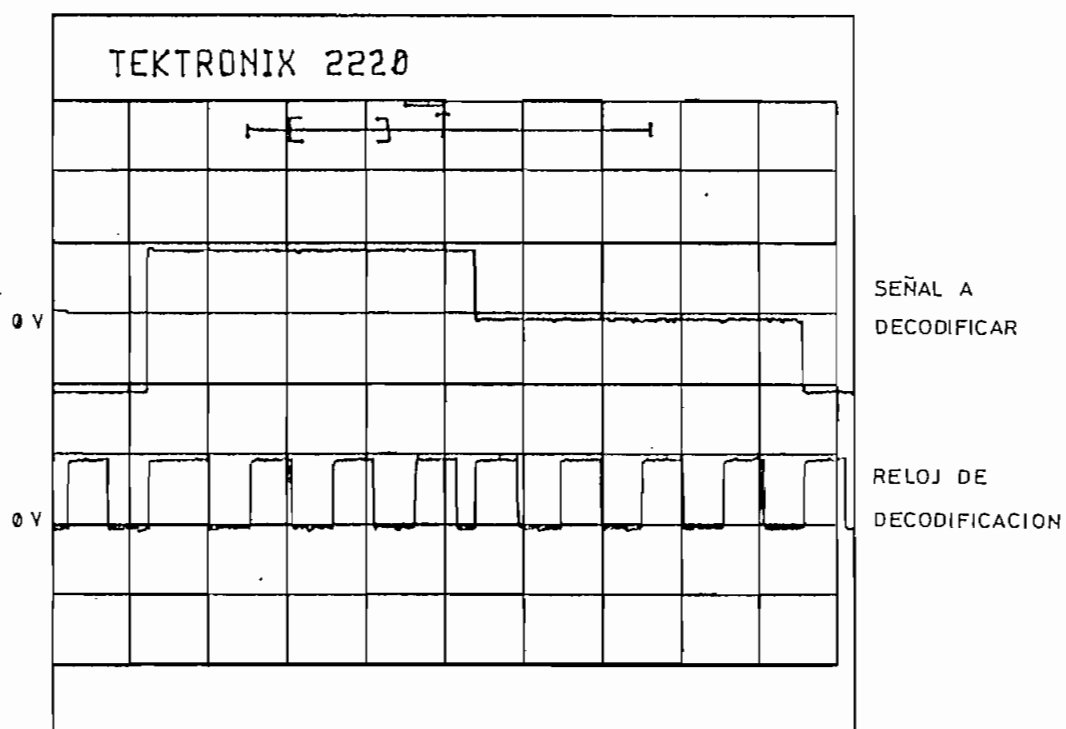
(V: 5 V/div, t: 1 ms/div)

Fig. 3.12. Codificación AMI



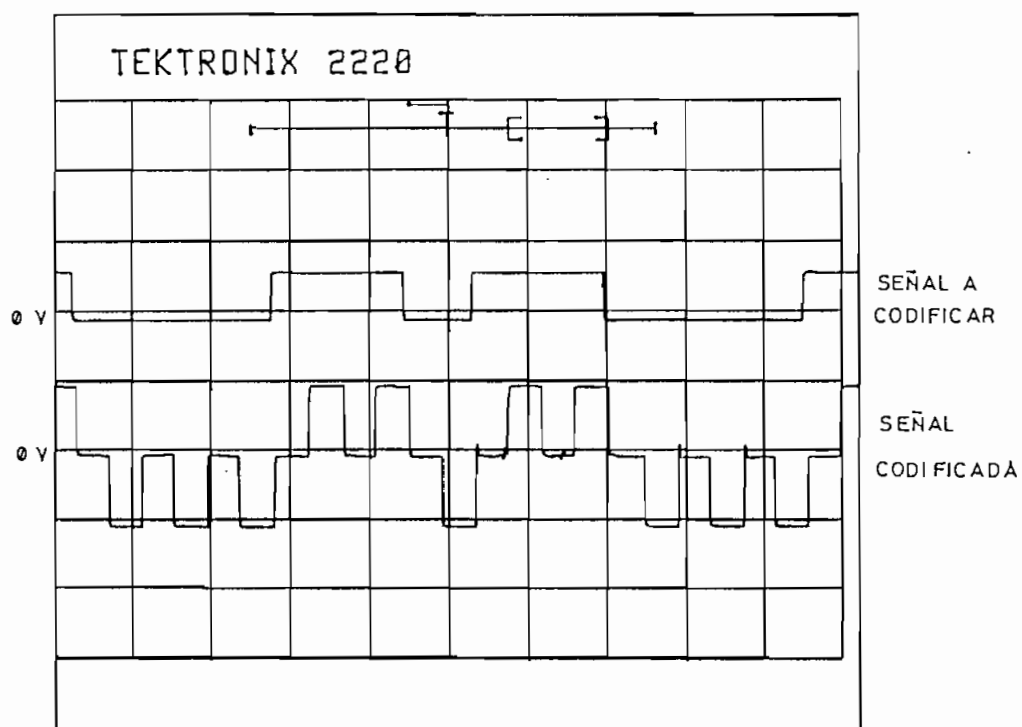
(V: 5 V/div, t: 1 ms/div)

Fig. 3.13. Decodificación AMI



(V: 5 V/div, t: 0.2 ms/div)

Fig. 3.14. Reloj de decodificación (AMI)



(V: 5 V/div, t: 1 ms/div)

Fig. 3.15. Codificación RZ polar

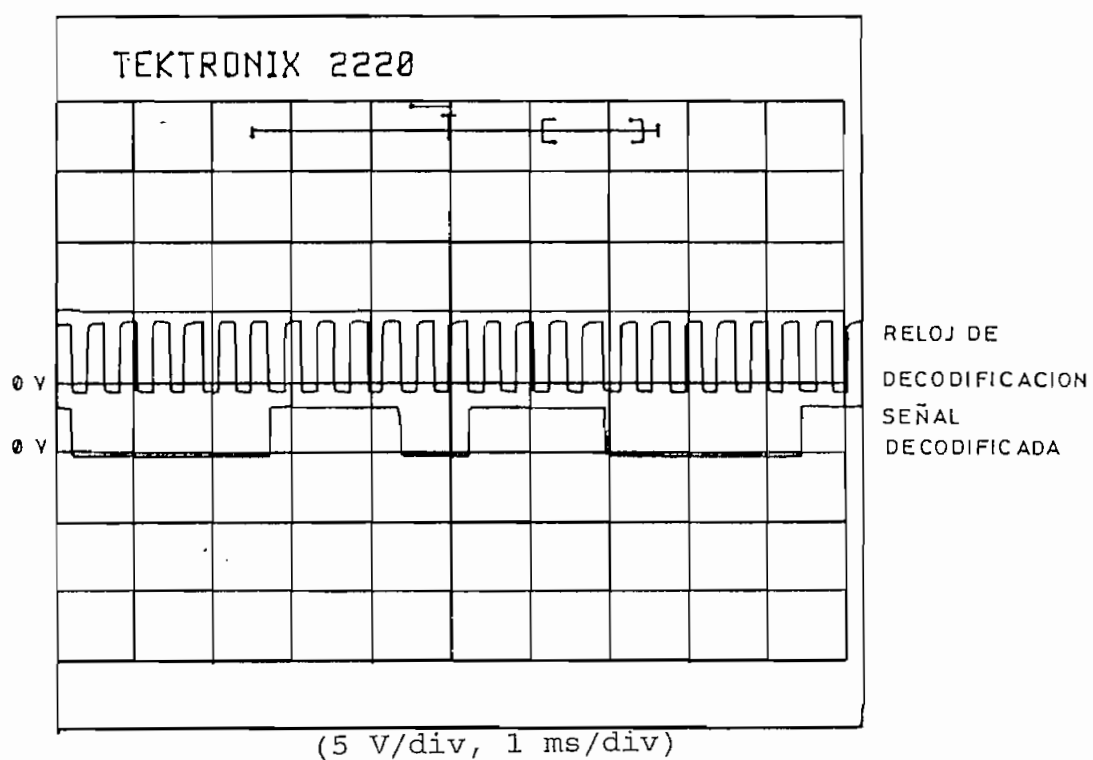
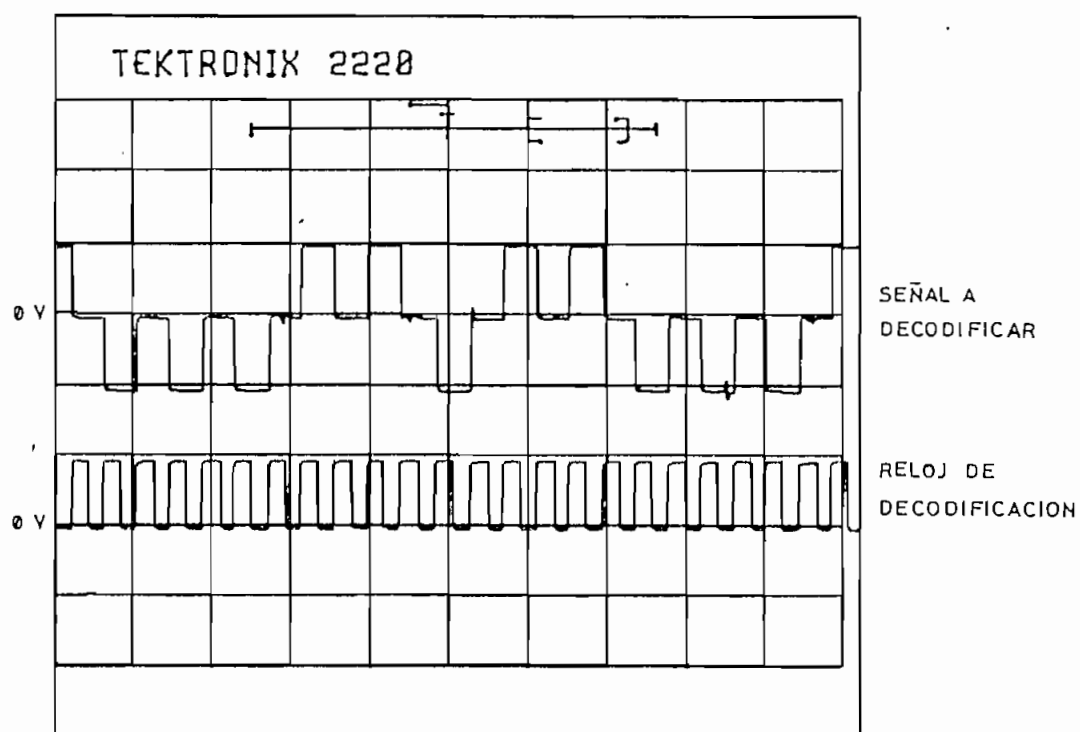


Fig. 3.16. Decodificación RZ polar



(V: 5 V/div, t: 0.2 ms/div)

Fig. 3.17. Reloj de decodificación (RZ polar)

Los códigos bifase (Manchester diferencial, bifase M y Miller) merecen una atención particular, ya que en ellos se tendrá la presencia de pulsos de sincronismo, que se han añadido para permitir la recuperación de la señal de reloj tal como se indicó en el numeral 2.5.5. Estos pulsos se emplean además como marcas que permiten advertir el inicio de cada período de bit, con la finalidad de realizar la correspondencia entre la señal entrante y la señal codificada. En la figura 3.18 se identifica claramente que un 0_L hace que se mantenga la fase de la señal digital en tanto que un 1_L ocasiona que la fase cambie en 180° (código Manchester diferencial). La figura 3.19 ilustra la correspondiente señal decodificada con su respectivo reloj de decodificación; un detalle de la recuperación de este reloj se tiene en la figura 3.20.

Para el código bifase M (figura 3.21) se tiene que siempre existe una transición al inicio del período de bit (lo que coincide con los pulsos de sincronismo), dándose otra transición en la mitad de la duración del bit si éste es un 1_L . La decodificación se la presenta en la figura 3.22 y el detalle del reloj de decodificación en la figura 3.23.

El proceso de codificación Miller se lo ilustra en la figura 3.24, en donde se nota que existen cambios de nivel al iniciar el período de bit sólo si se tiene dos o más 0_L seguidos, caso contrario sólo el 1_L producirá una transición a la mitad de duración del bit. Lógicamente que en este proceso es transparente la presencia de los pulsos de sincronismo, que luego de ocurrir retornan siempre el nivel de señal al punto correspondiente a la regla de codificación. El proceso de decodificación y su correspondiente reloj se ven en las figuras 3.25 y 3.26.

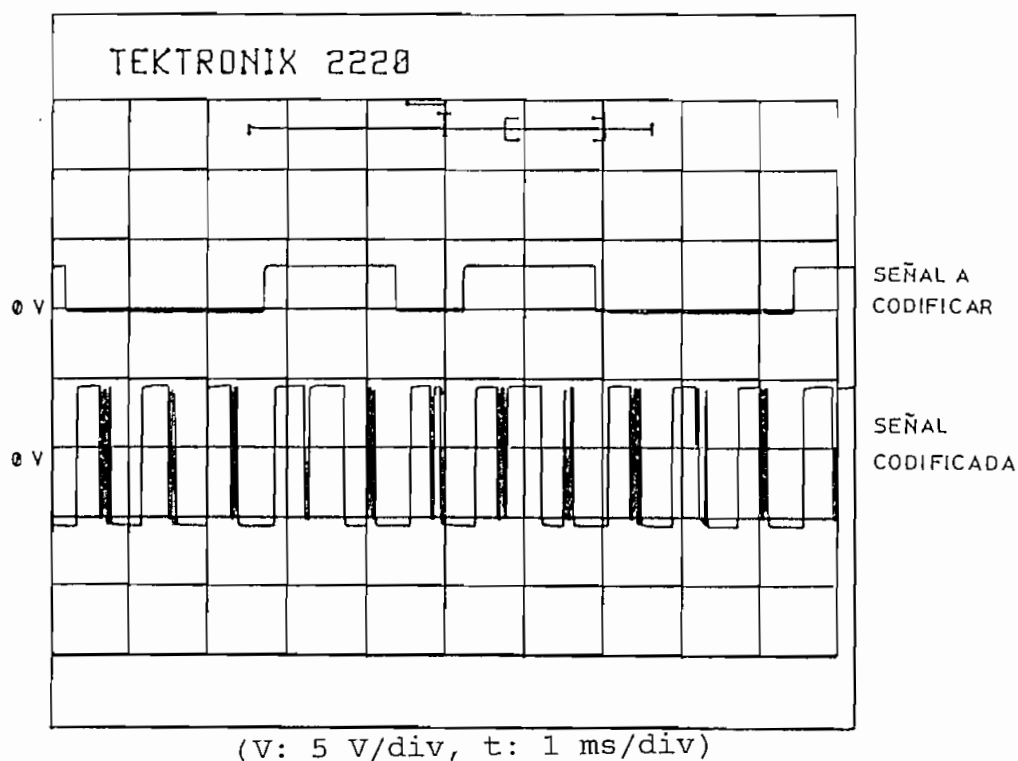


Fig. 3.18. Codificación Manchester diferencial

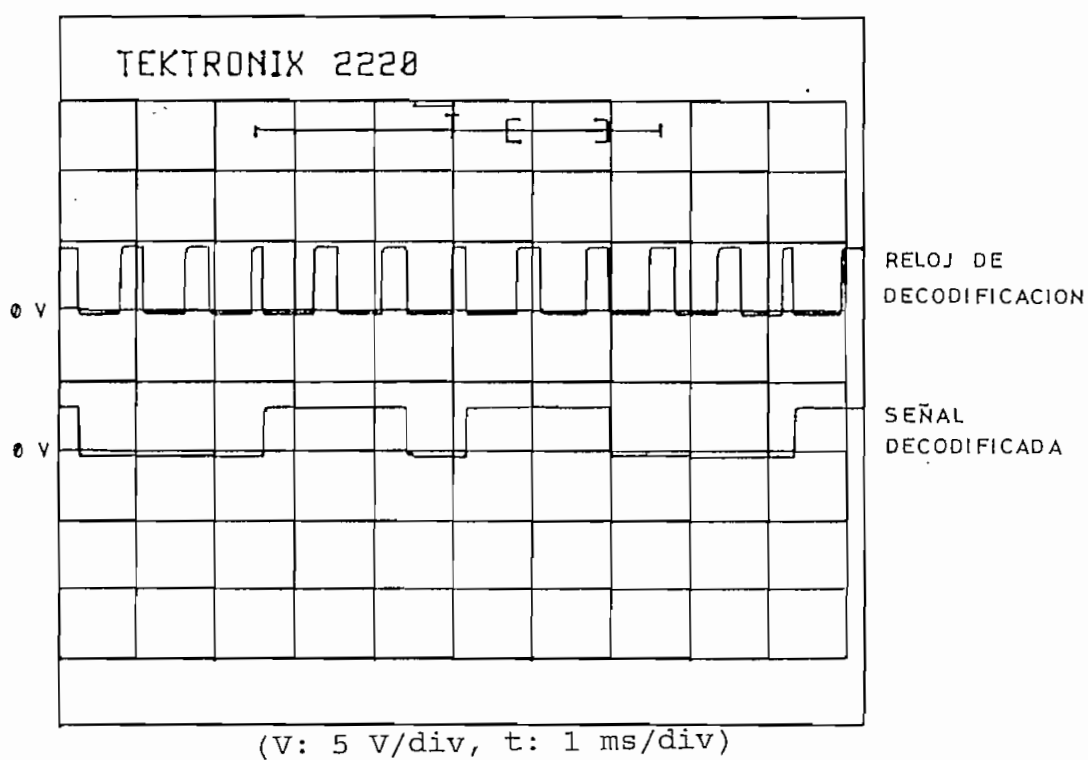
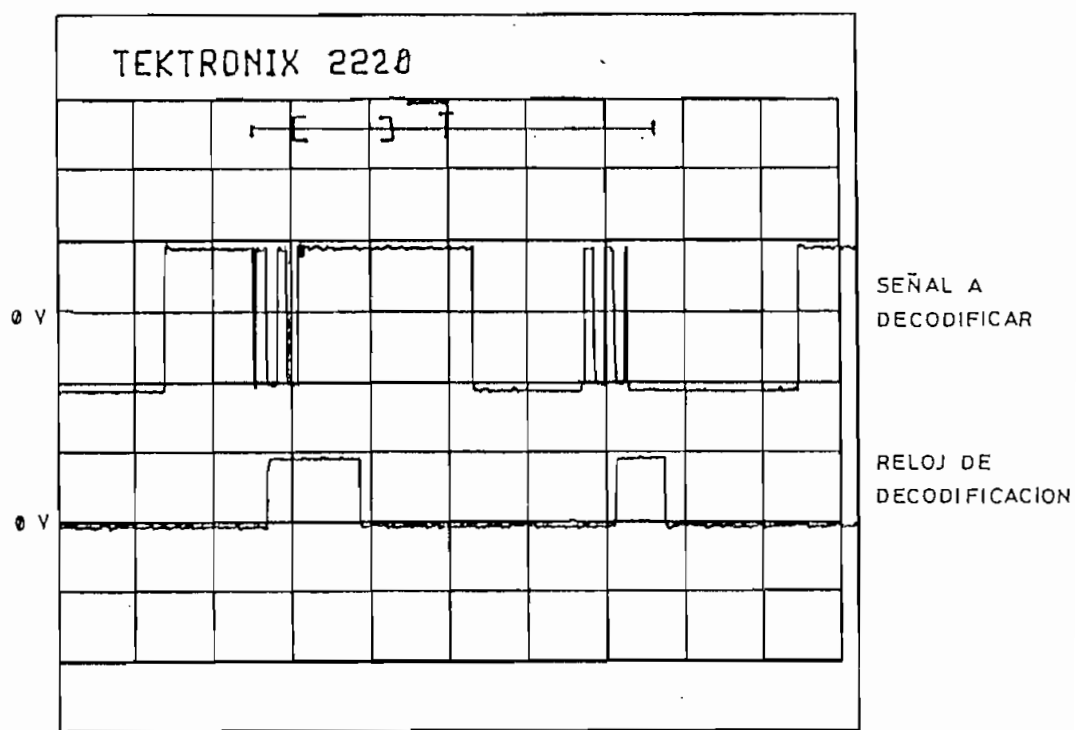
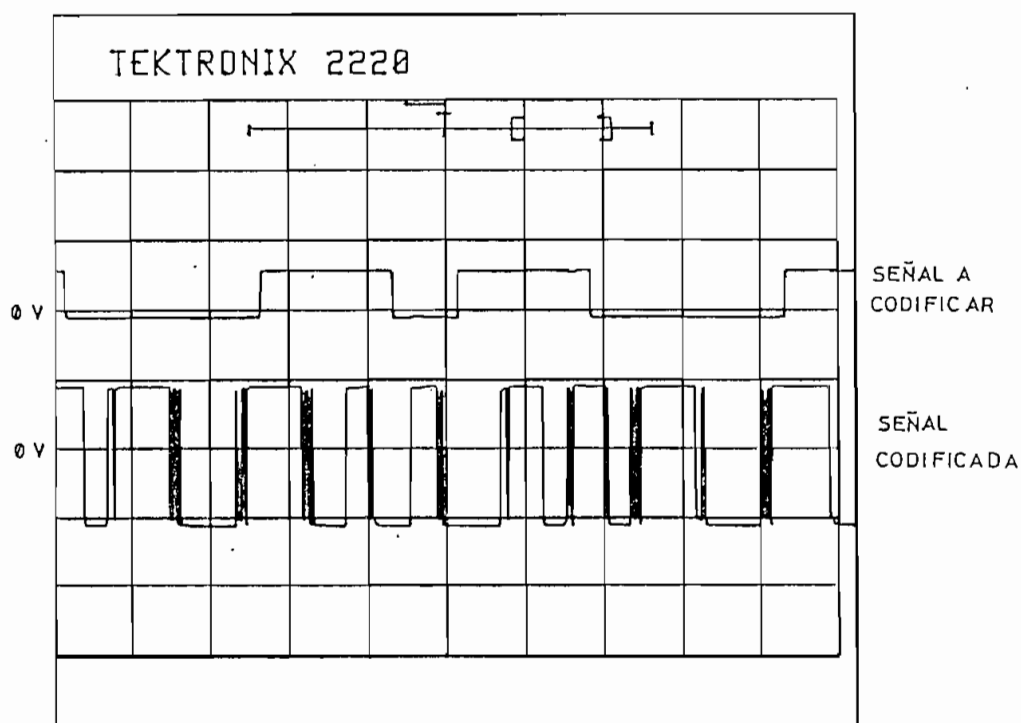


Fig.3.19. Decodificación Manchester diferencial



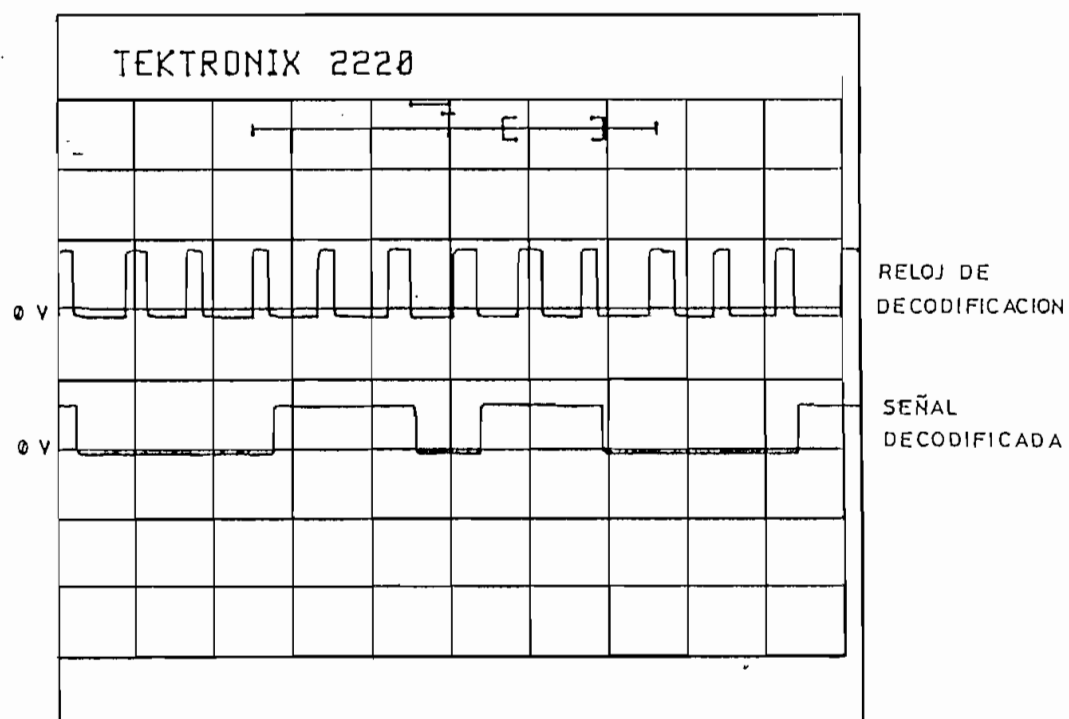
(V: 5 V/div, t: 0.2 ms/div)

Fig. 3.20. Reloj de decodificación (Manchester diferencial)



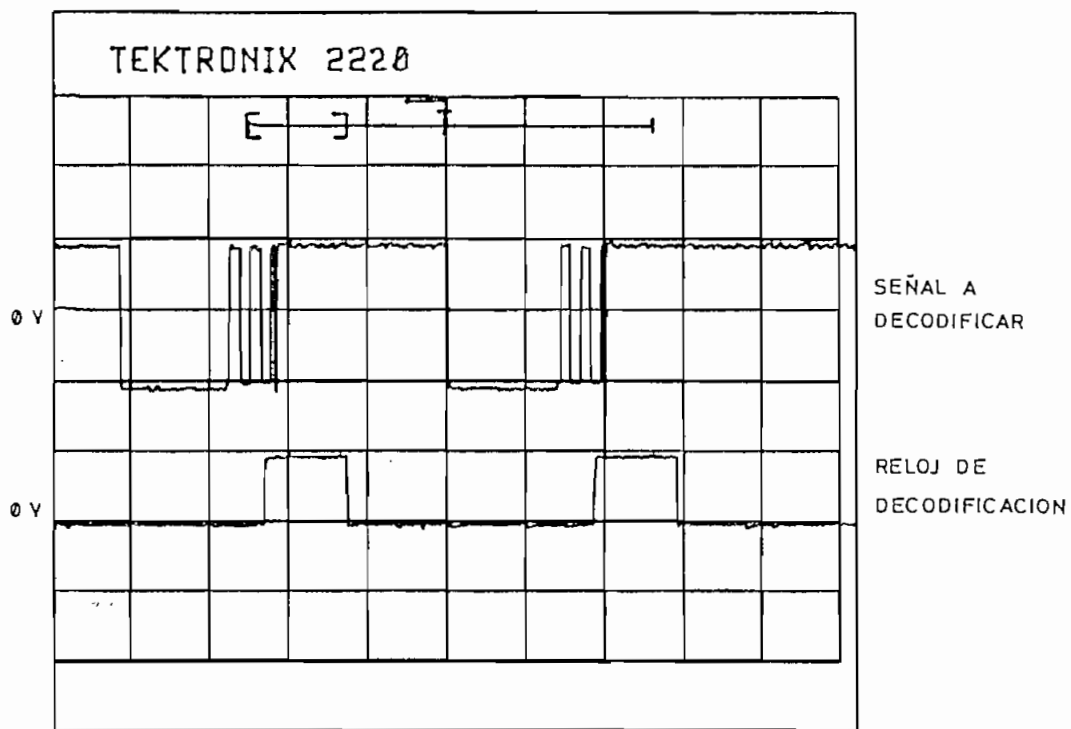
(V: 5 V/div, t: 1 ms/div)

Fig. 3.21. Codificación bifase-M



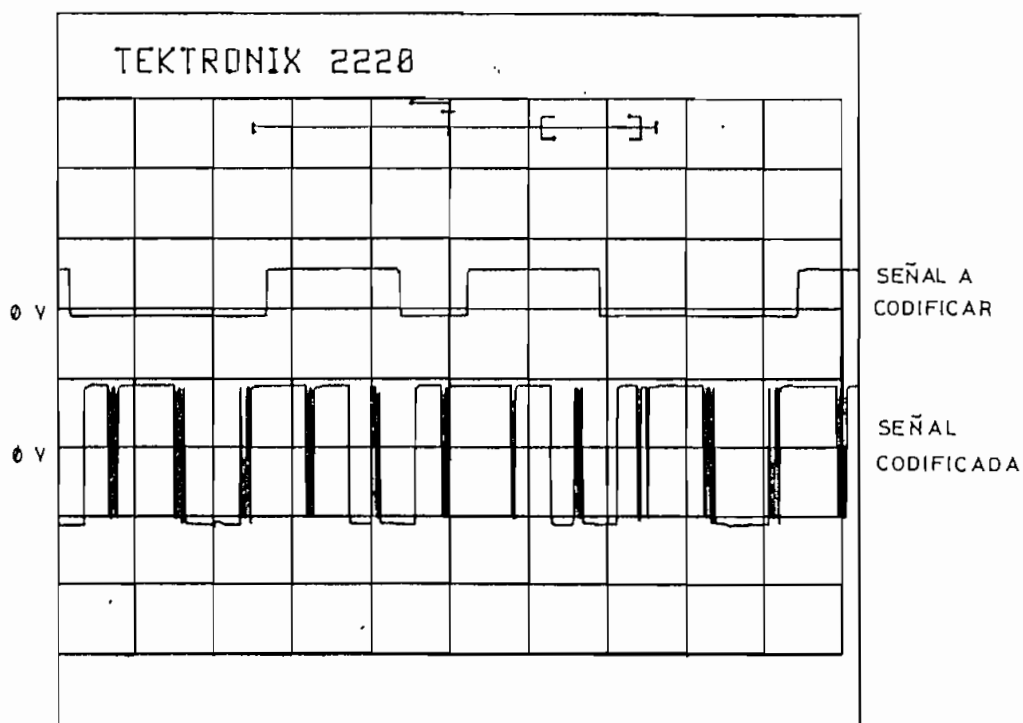
(V: 5 V/div, t: 1 ms/div)

Fig. 3.22. Decodificación bifase-M



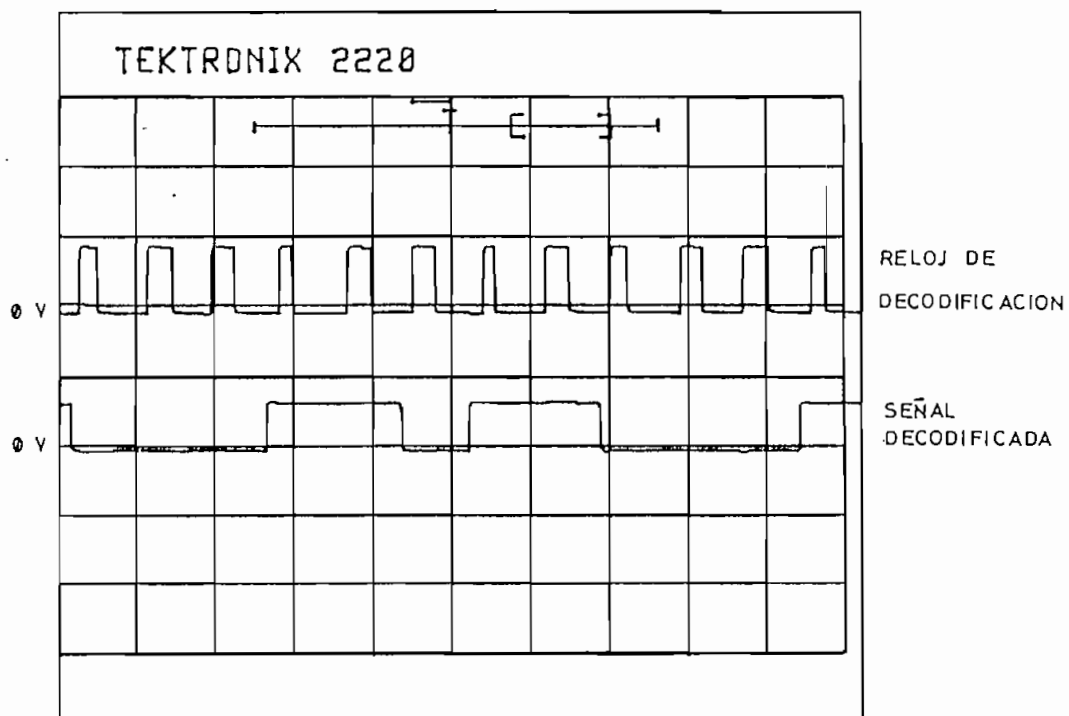
(V: 5 V/div, t: 0.2 ms/div)

Fig. 3.23. Reloj de decodificación (bifase-M)



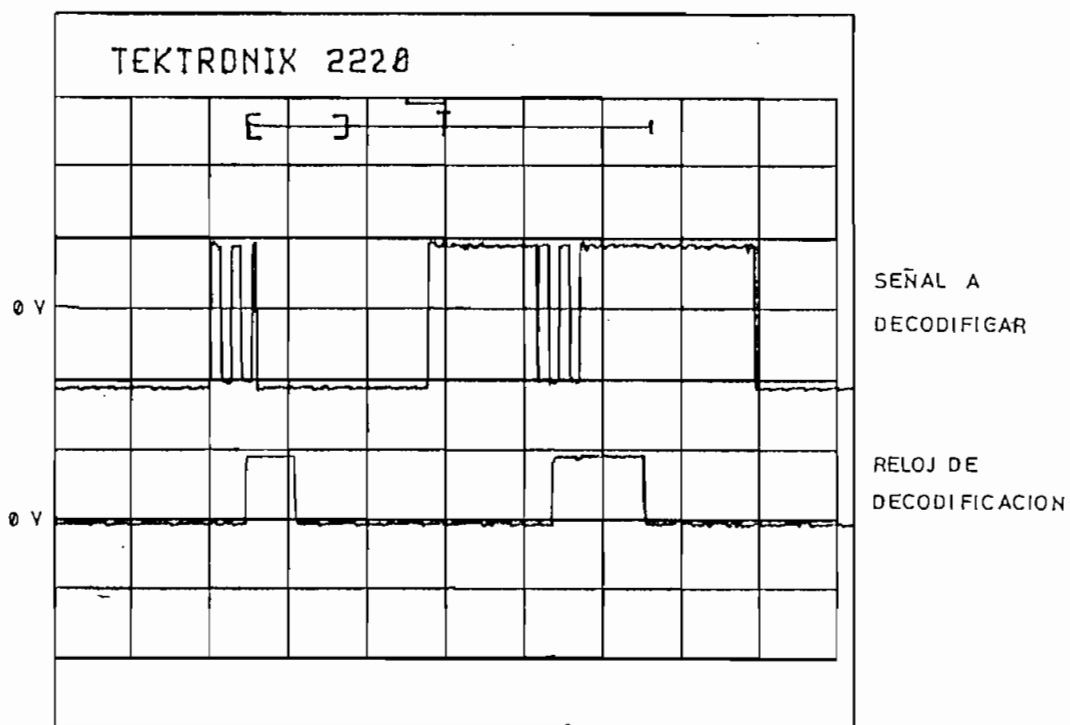
(V: 5 V/div, t: 1 ms/div)

Fig. 3.24. Codificación Miller



(V: 5 V/div, t: 1 ms/div)

Fig. 3.25. Decodificación Miller



(V: 5 V/div, t: 0.2 ms/div)

Fig. 3.26. Reloj de decodificación (Miller)

Para la verificación del formato de codificación de los códigos con disminución de velocidad (4B-3T y MS43), es necesario tomar en cuenta que la codificación toma cuatro dígitos binarios para codificarlos como tres dígitos ternarios. La secuencia binaria que se utiliza en la prueba de formato (01101100), puede ser codificada de cuatro maneras diferentes según la forma de agrupar los arreglos de cuatro bits (0110 1100, 1101 1000, 1011 0001, 0110 0011) y considerando que los primeros bits a codificar son los menos significativos.

En la figura 3.27 se tiene una posibilidad de codificación 4B-3T, que de acuerdo a la tabla 1.2 (capítulo I) corresponde al siguiente formato:

Entrada binaria	0011	0110	0011	0110	0011
Salida ternaria	+++	-00	+++	-00	+++
Disparidad actual	+1	-1	+1	-1	+1

Es importante aclarar que de cada grupo de tres elementos ternarios de la tabla 1.2, se emite primero el de la derecha (haciendo alusión a un dígito ternario menos significativo), por lo que en la figura 3.27 se observa un flujo ternario correspondiente a: -00+-+00+-+. Se puede advertir además que existe un retardo de algo más de 4 períodos de bit entre la presencia de los dígitos binarios y el aparecimiento de su correspondientes palabras código; esto se debe a que se necesita $3\frac{1}{2}$ períodos de bit para ingresar los cuatro dígitos binarios a codificar, además del tiempo que el microcontrolador requiere para ejecutar el algoritmo de codificación.

En la figura 3.28 se tiene la señal decodificada con su correspondiente reloj de decodificación. La figura 3.29 muestra la señal a decodificar con su correspondiente reloj de decodificación en donde se observa que el muestreo

(por flanco negativo) ocurre a la mitad de la duración del bit. Una relación entre los relojes de codificación y decodificación (también válida para el código MS43) se representa en la figura 3.30.

Para el código de línea MS43, la codificación se la realiza de manera similar según la tabla 1.3, con su correspondiente diagrama de estados y con las consideraciones realizadas anteriormente. Con estas apreciaciones, una de las posibilidades de codificación de la secuencia de prueba es:

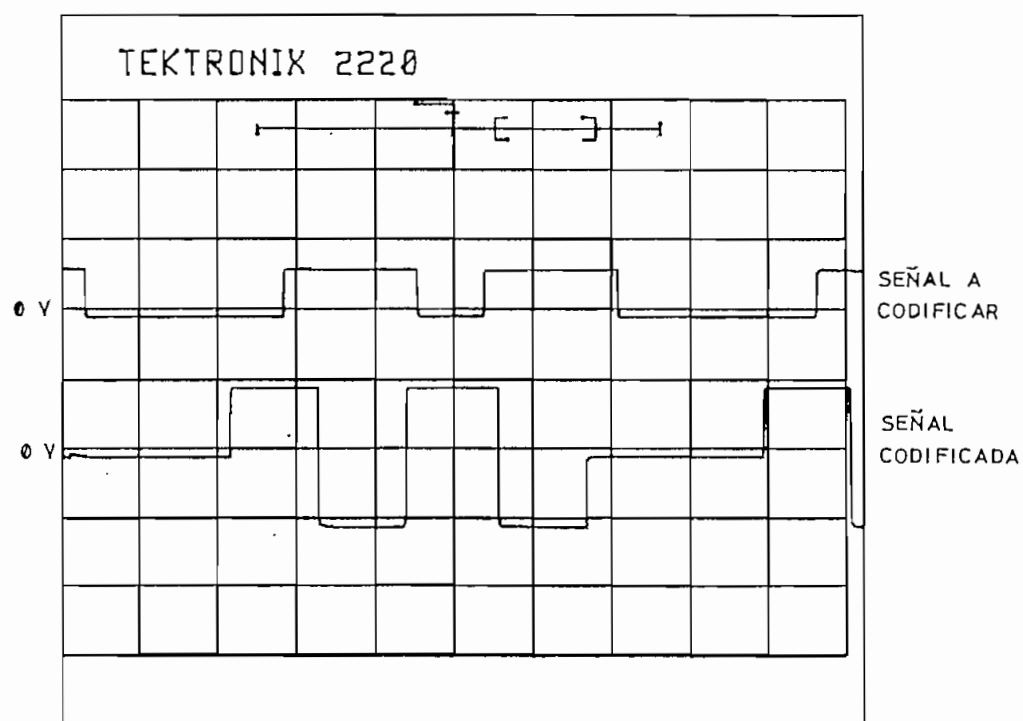
Entrada binaria	1100	0110	1100	0110	1100
Salida ternaria	00+	0+-	--0	0+-	00+
Disparidad actual	+2	+2	-1	-1	+1

De donde, la secuencia ternaria que se observa en la figura 3.31 se presenta como: 00+0+---00+-00+. El retardo en la codificación es algo menor que el establecido para el código 4B-3T debido a que el algoritmo de codificación es más directo.

La secuencia que se indica, al ser decodificada permite la recuperación del flujo original de bits tal como se muestra en la figura 3.32. El muestreo de la señal codificada se ilustra en la figura 3.33.

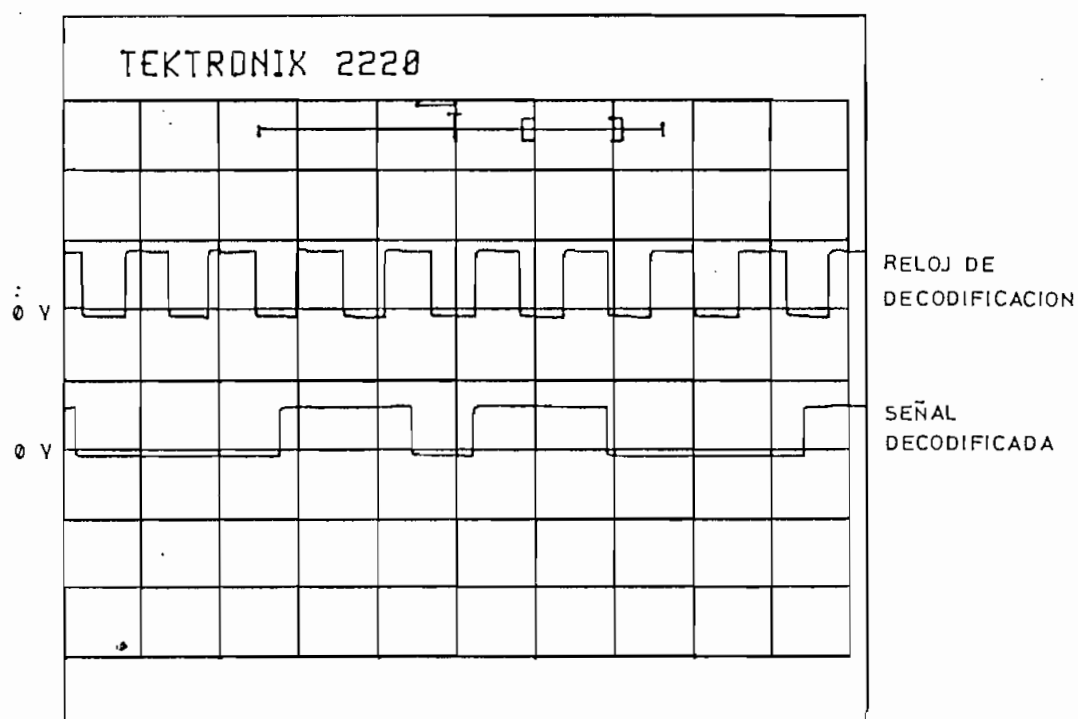
Para el código B3ZS (figura 3.34), se observa el reemplazo de tres ceros consecutivos por la secuencia +0+, lo que implica que el pulso precedente fue negativo y que el número de 1, desde la última sustitución es par. La decodificación de esta secuencia se presenta en la figura 3.35 y un detalle del muestreo de la señal codificada en la figura 3.36.

Para el código HDB3 la secuencia binaria de prueba es 10100000. Mediante esta secuencia, en la figura 3.37 se observa la sustitución de cuatro ceros consecutivos por los niveles -00- debido a que el pulso precedente es positivo y que el número de 1_x desde la última sustitución es par. Para complementar la presentación de este código, en la figura 3.38 se observa la decodificación HDB3 y en la figura 3.39 el reloj de decodificación sincronizado con los datos entrantes al CODEC.



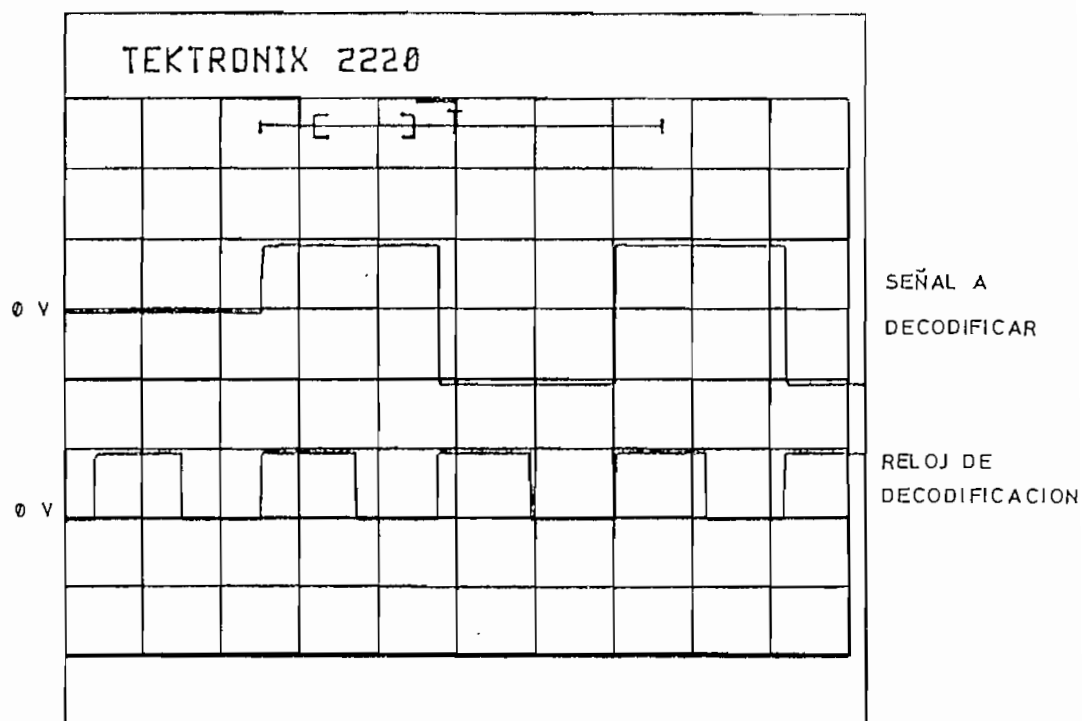
(V: 5 V/div, t: 1 ms/div)

Fig. 3.27. Codificación 4B-3T



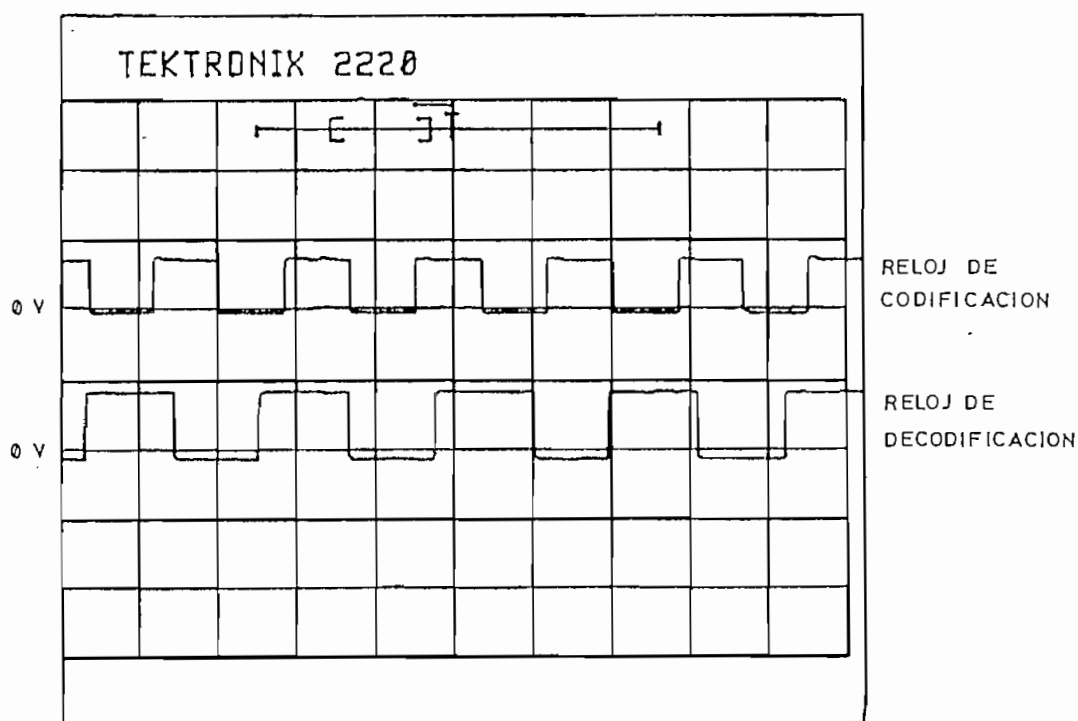
(V: 5 V/div, t: 1 ms/div)

Fig. 3.28. Decodificación 4B-3T



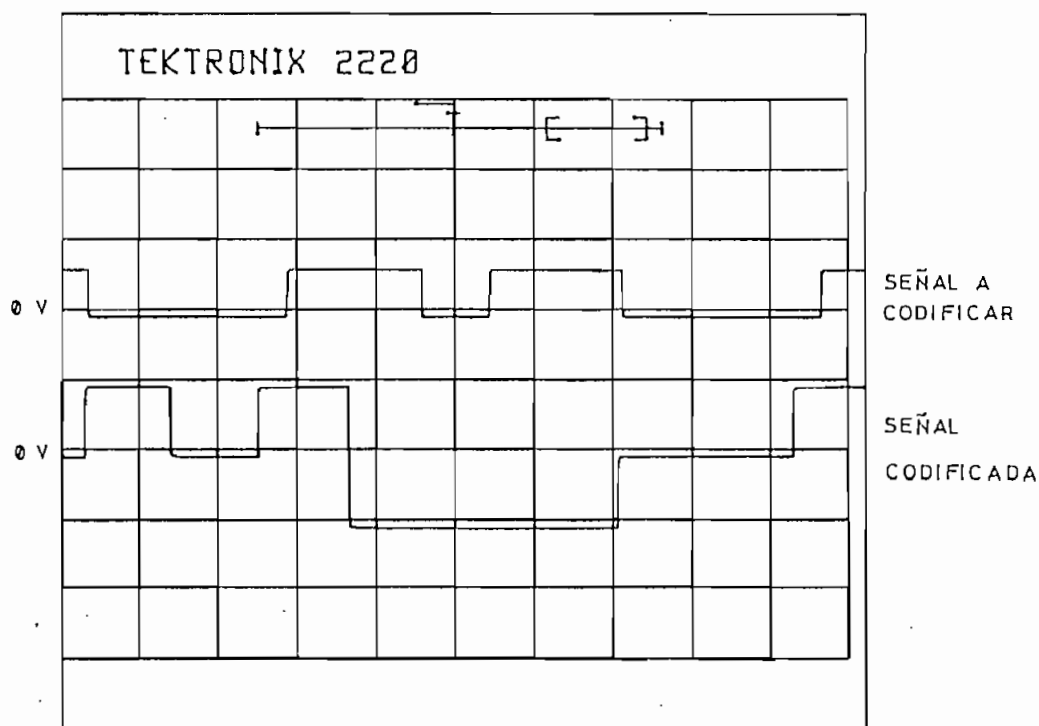
(V: 5 V/div, t: 0.5 ms/div)

Fig. 3.29. Reloj de decodificación (4B-3T)



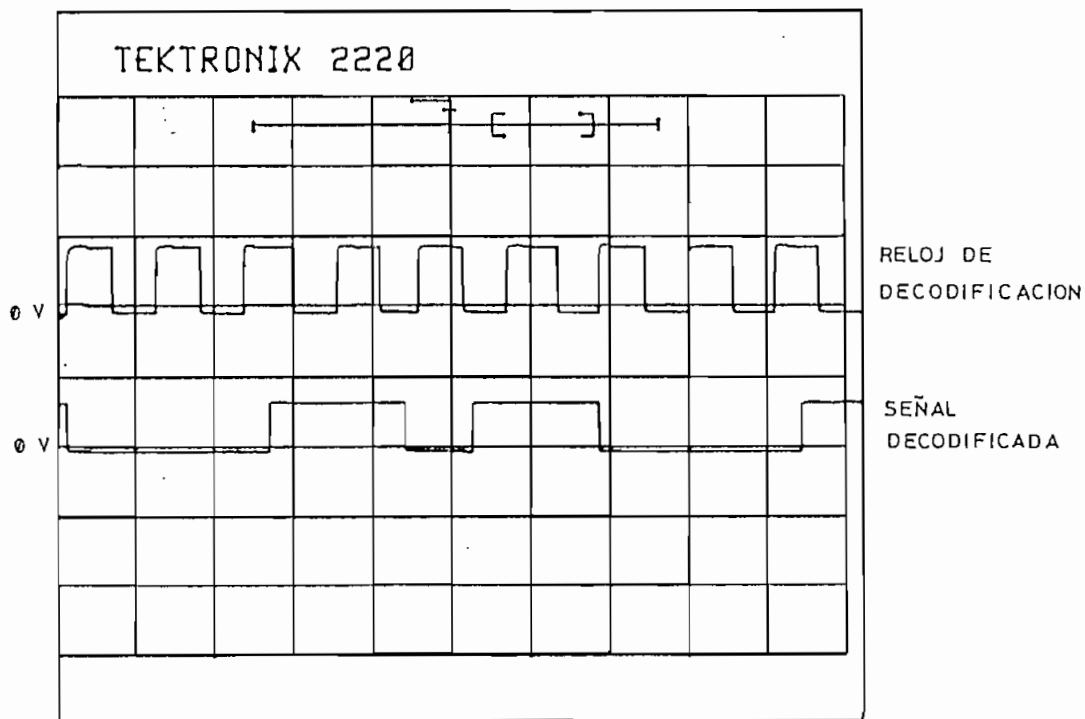
(V: 5 V/div, t: 0.5 ms/div)

Fig.3.30. Relojes de codificación y decodificación



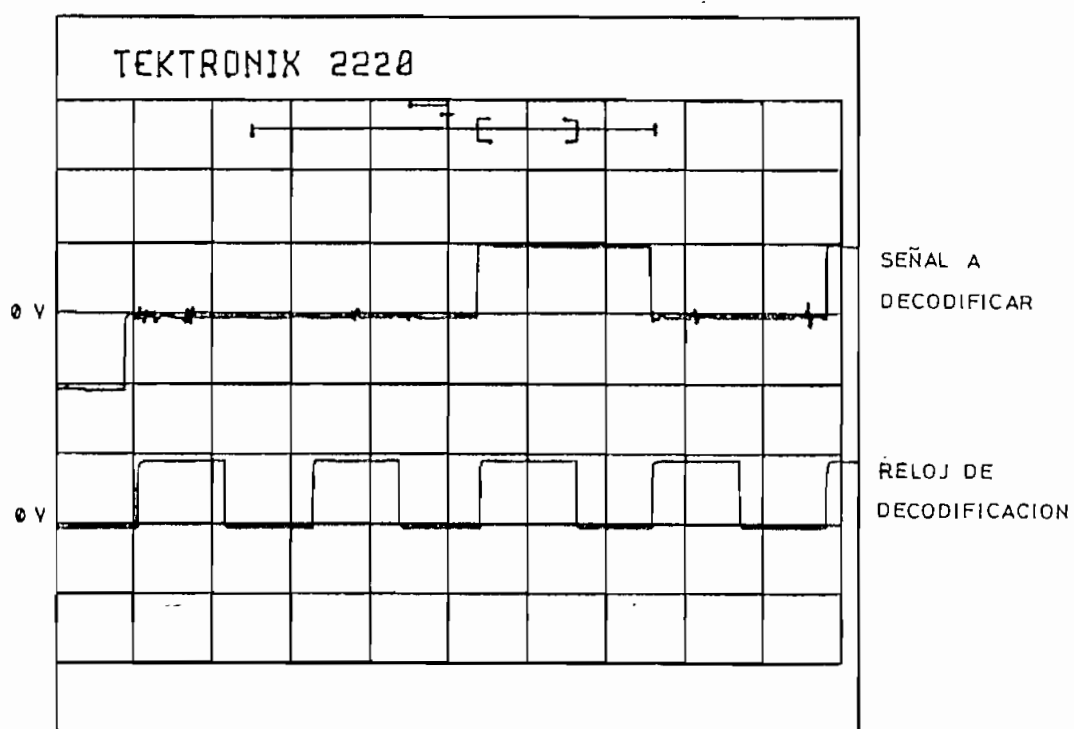
(V: 5 V/div, t: 1 ms/div)

Fig. 3.31. Codificación MS43



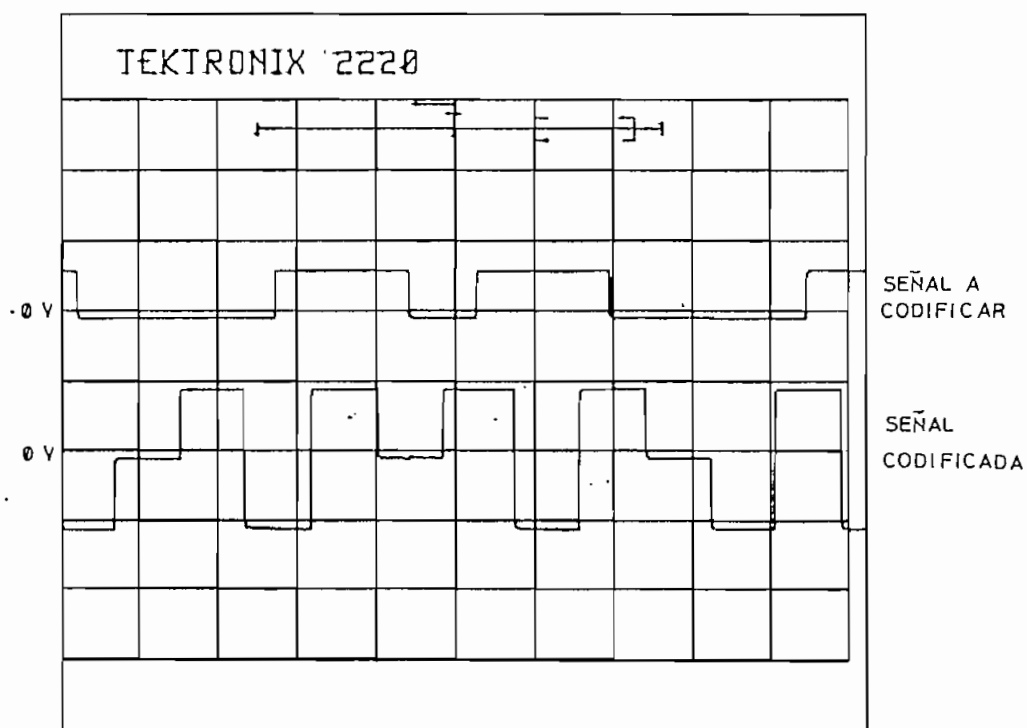
(V: 5 V/div, t: 1 ms/div)

Fig. 3.32. Decodificación MS43

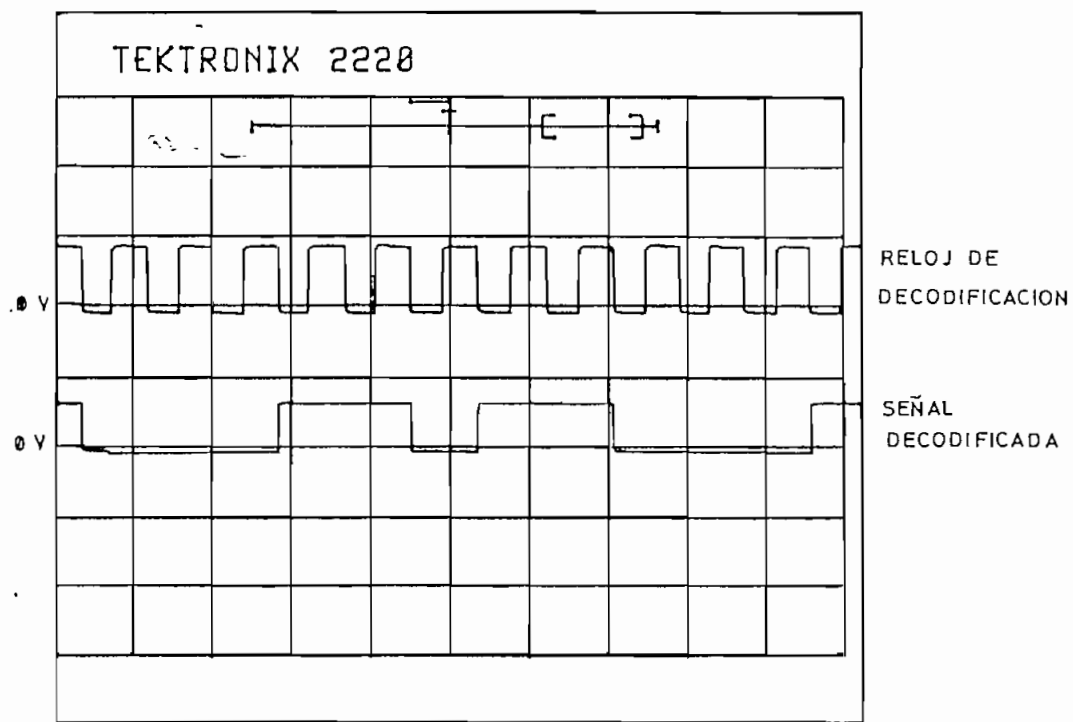


(V: 5 V/div, t: 0.5 ms/div)

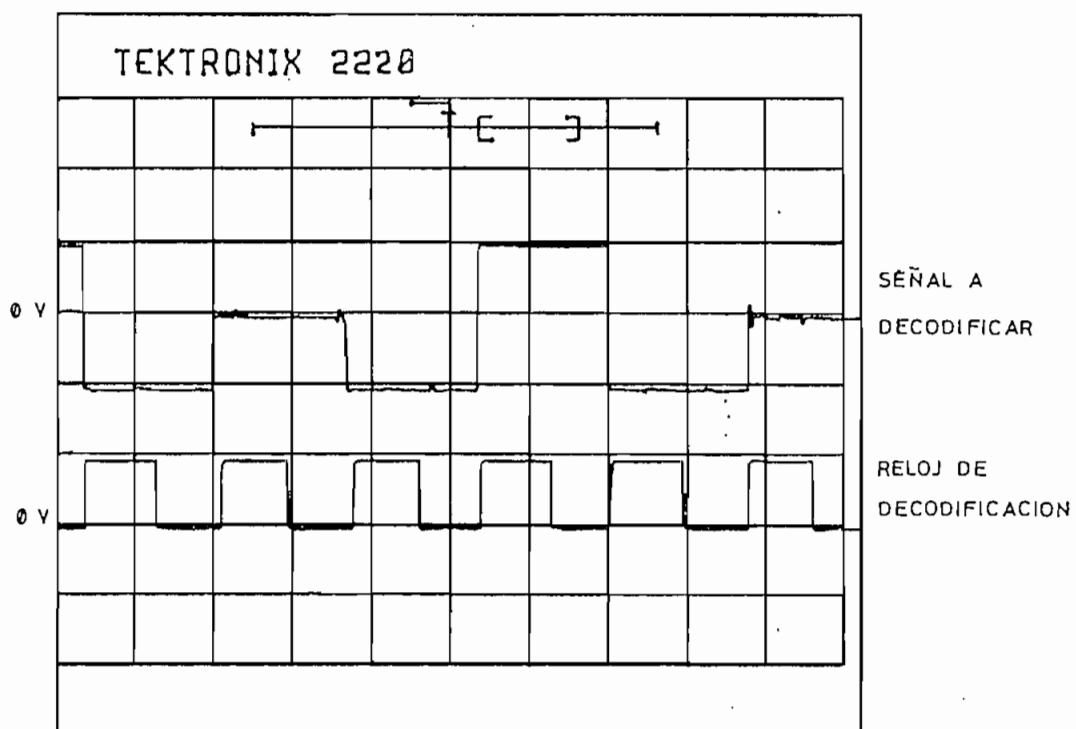
Fig. 3.33. Reloj de decodificación (MS43)



(V: 5 V/div, t: 1 ms/div) .
Fig. 3.34. Codificación B3ZS

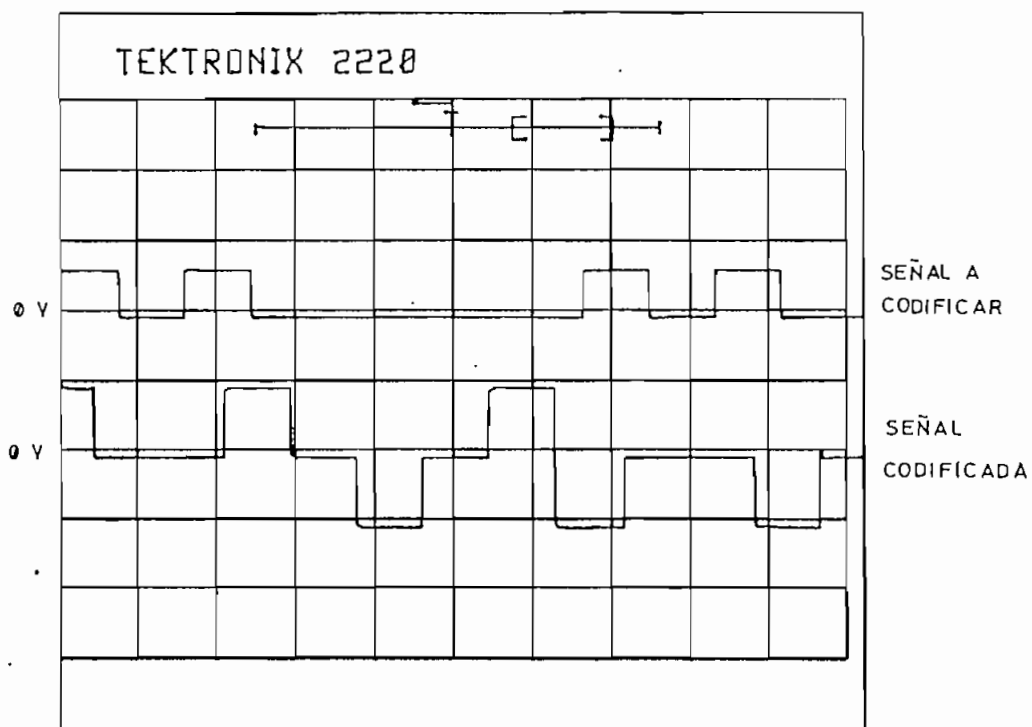


(V: 5 V/div, t: 1 ms/div)
Fig. 3.35. Decodificación B3ZS



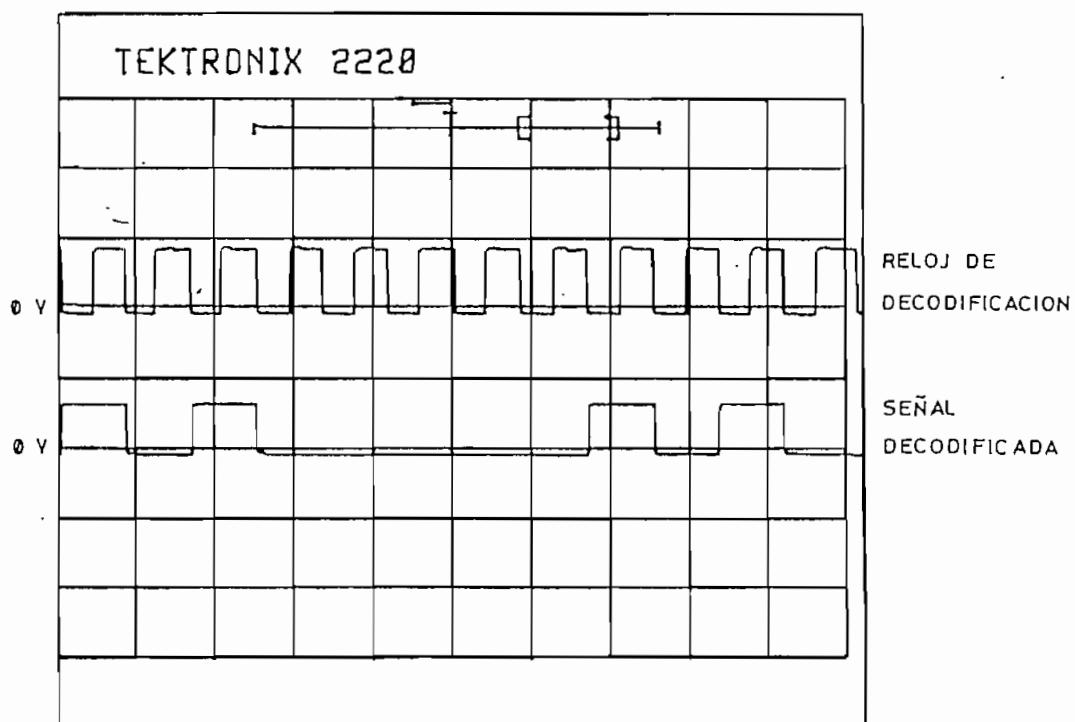
(V: 5 V/div, t: 0.5 ms/div)

Fig. 3.36. Reloj de decodificación (B3ZS)



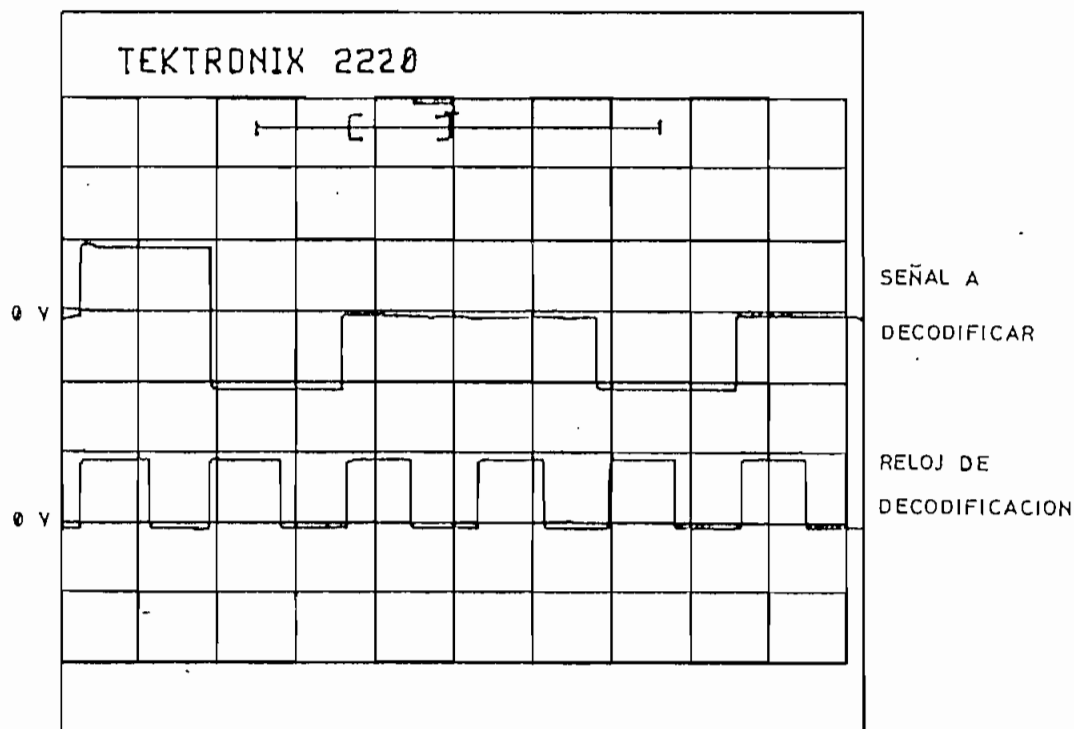
(V: 5 V/div, t: 1 ms/div)

Fig. 3.37. Codificación HDB3



(V: 5 V/div, t: 1 ms/div)

Fig. 3.38. Decodificación HDB3



(V: 5 V/div, t: 0.5 ms/div)

Fig. 3.39. Reloj de decodificación (HDB3)

3.3. VELOCIDAD DE TRANSMISION.

La velocidad de transmisión que soporta el equipo difiere según los códigos de línea y según el tipo de transmisión que se esté usando (*half* o *full duplex*). Si el algoritmo de codificación/decodificación, o la recuperación de la señal de reloj son complicados, esto influirá en una disminución de la máxima velocidad que tolera el equipo.

Cuando se realiza una transmisión *half duplex*, el microcontrolador sólo realiza una tarea de manera continua; pero en el caso de una transmisión *full duplex*, el microcontrolador debe realizar tanto la codificación como la decodificación, por lo que la capacidad de procesamiento para cada ocupación se reduce.

En la tabla 3.1 se presenta la velocidad máxima para cada uno de los códigos, resultados que se han obtenido mediante la transferencia de un bloque de datos, entre dos terminales de comunicaciones QVT-101, cada uno conectado a un CODEC.

De esta tabla se observa que para los códigos de línea más simples de implementar como son el NRZ polar y el AMI se alcanza una velocidad de transmisión binaria de 19200 bit/s, en tanto que para los esquemas más complejos como son el 4B-3T y el MS43, sólo se llega a 1200 bit/s. Esto se debe básicamente a que el procesamiento que debe hacer el microcontrolador es muy sencillo y rápido en el caso de los dos primeros códigos, en tanto que para los esquemas complejos el algoritmo es demasiado extenso requiriendo el microcontrolador un tiempo relativamente grande para el procesamiento de cada bit o elemento de señal.

CODIGO DE LINEA	Transmisión <i>Half Duplex</i>	Transmisión <i>Full Duplex</i>
NRZ POLAR	19200 bit/s	4800 bit/s
AMI	19200 bit/s	4800 bit/s
RZ POLAR	4800 bit/s	1200 bit/s
MANCHESTER DIFERENCIAL	2400 bit/s	1200 bit/s
BIFASE - M	2400 bit/s	2400 bit/s
MILLER	2400 bit/s	1200 bit/s
4B - 3T	1200 bit/s	300 bit/s
MS43	1200 bit/s	300 bit/s
B3ZS	4800 bit/s	1200 bit/s
HDB3	4800 bit/s	1200 bit/s

Tabla 3.1. Máximas velocidades permitidas por el CODEC

Con relación al tiempo de procesamiento que requiere el microcontrolador para efectuar la codificación de un bit o la decodificación de un elemento de señal, en la tabla 3.2 se presentan los resultados promedio obtenidos de mediciones realizadas en el equipo.

CODIGO DE LINEA	TIEMPO DE PROCESAMIENTO	
	CODIFICACION	DECODIFICACION
NRZ POLAR	15 μ s	14 μ s
AMI	16 μ s	16 μ s
RZ POLAR	16 μ s	20 μ s
MANCHESTER DIFERENCIAL	20 μ s	26 μ s
BIFASE - M	20 μ s	40 μ s
MILLER	22 μ s	220 μ s
4B - 3T	250 μ s	460 μ s
MS43	180 μ s	460 μ s
B3ZS	30 μ s	30 μ s
HDB3	30 μ s	35 μ s

Tabla 3.2. Tiempos de procesamiento

Los tiempos promedio de procesamiento se han obtenido de una manera aproximada a partir del instante en que ingresa el último bit o elemento de señal necesario para que se realice la codificación/decodificación. Estos tiempos permiten observar el por qué se reduce la velocidad de transmisión según el esquema de codificación y según se realice una o dos tareas por parte del microcontrolador. Para el caso particular de los códigos bifase, el limitante no es el tiempo de procesamiento sino la dificultad de obtener una señal de reloj de decodificación que sea estable a frecuencias elevadas.

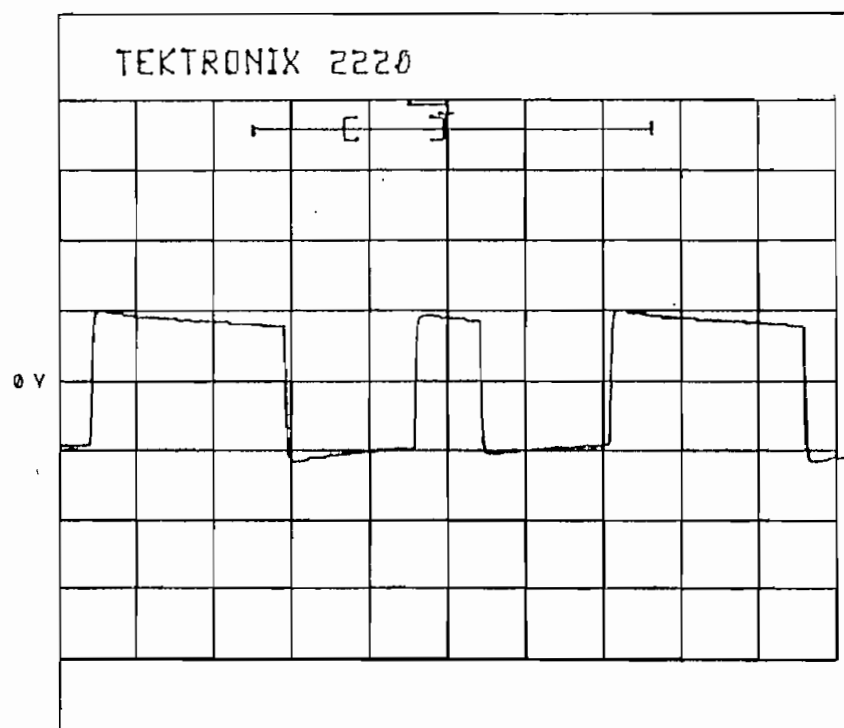
3.4. COMPARACION DE LA ELIMINACION DE LA COMPONENTE CONTINUA.

Como se mencionó en el capítulo 1, una de las ventajas en la utilización de los códigos de línea, es la tendencia a eliminar la componente continua. Como una forma de establecer una comparación entre los diferentes esquemas de codificación, la prueba que se realizará en este punto es la utilización de un transformador de audio, por el que se transmitirá la señal en banda base, cuya forma de onda en el secundario estará libre de la componente continua que el código provee. La observación de esta señal permitirá establecer la bondad del código en este aspecto.

La prueba se la realiza con una velocidad de transmisión de 1200 bit/s, de modo que el ancho de banda del transformador de audio (aproximadamente 15 kHz), permita el paso de la señal de banda base. Las secuencias de bits corresponden a las mismas que se utilizaron en las pruebas de formato. El circuito empleado para la realización de esta prueba se muestra en la figura 3.40.

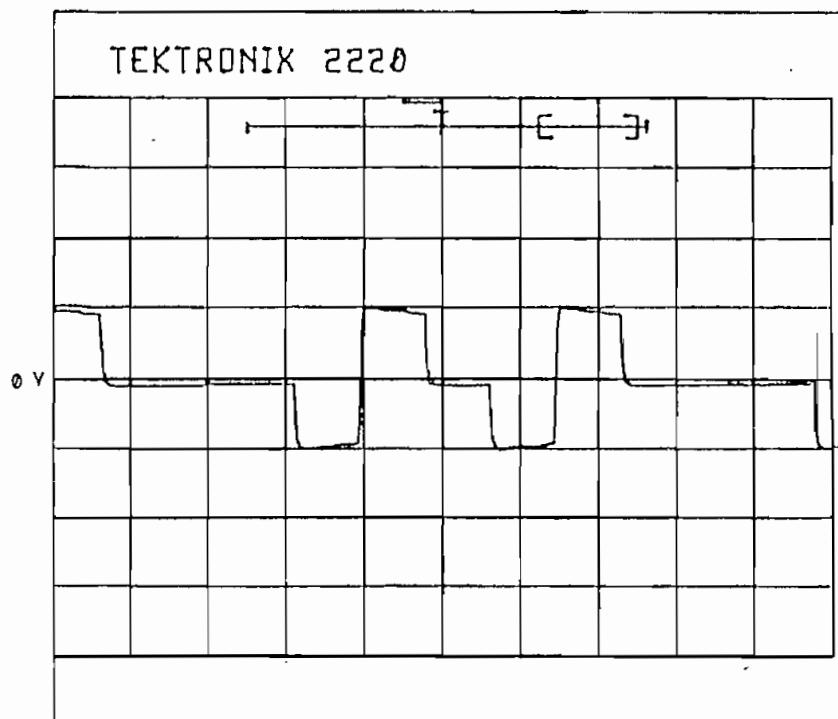
Las señales que se emplean son los controles de codificación A y B (explicados en el numeral 2.2 y

En el caso de transmitirse esta señal, en recepción



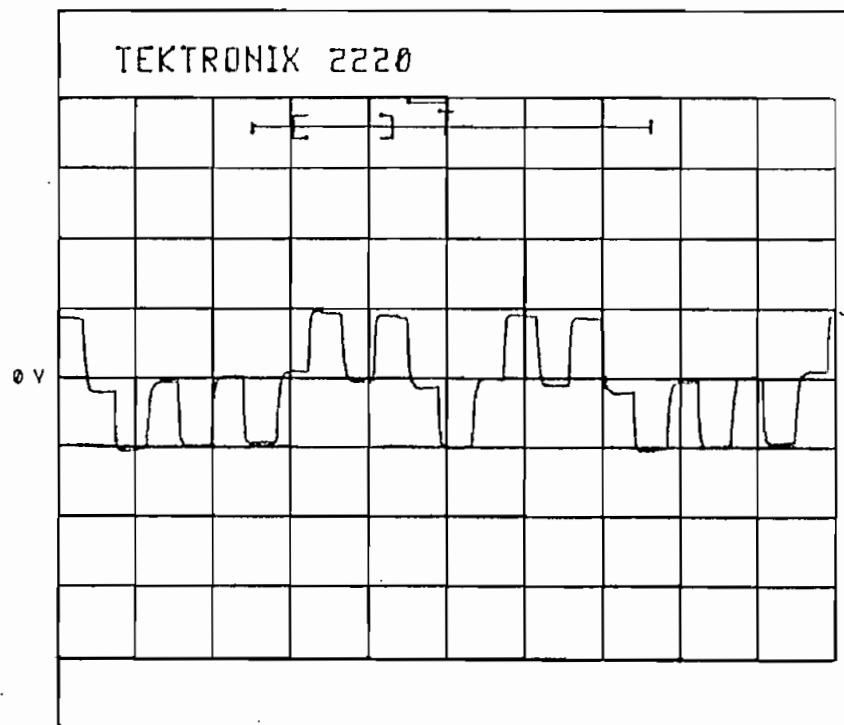
(V: 5 V/div, t: 1 ms/div)

Fig. 3.41. Código NRZ polar



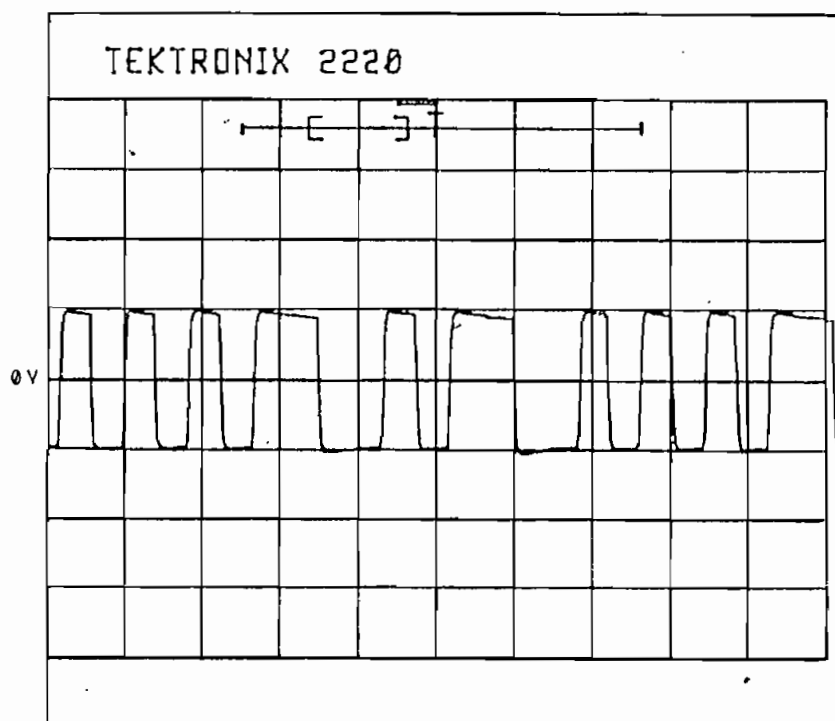
(V: 5 V/div, t: 1 ms/div)

Fig. 3.42. Código AMI



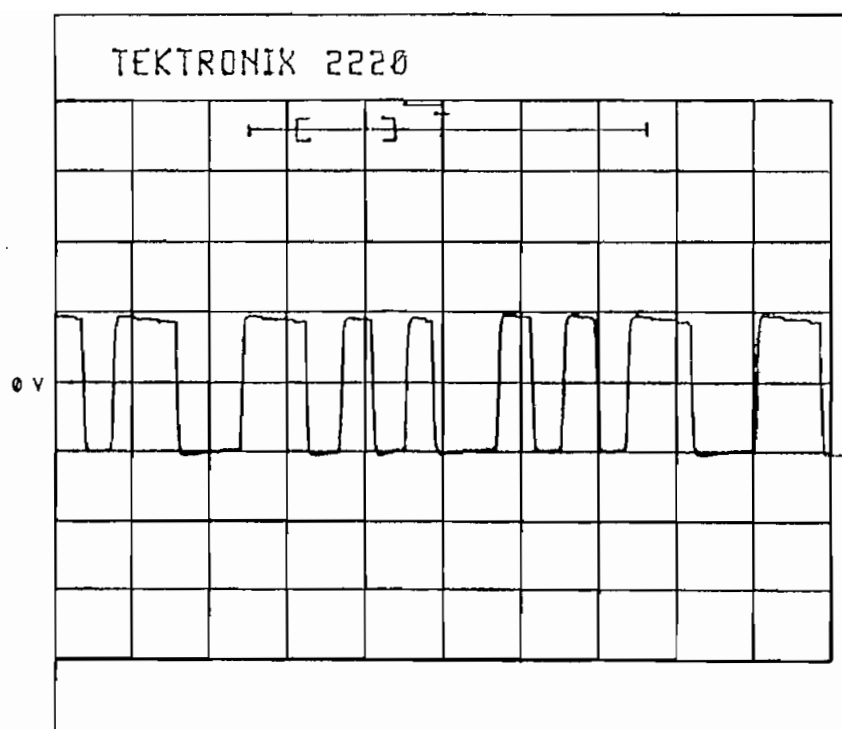
(V: 5 V/div, t: 1 ms/div)

Fig. 3.43. Código RZ polar



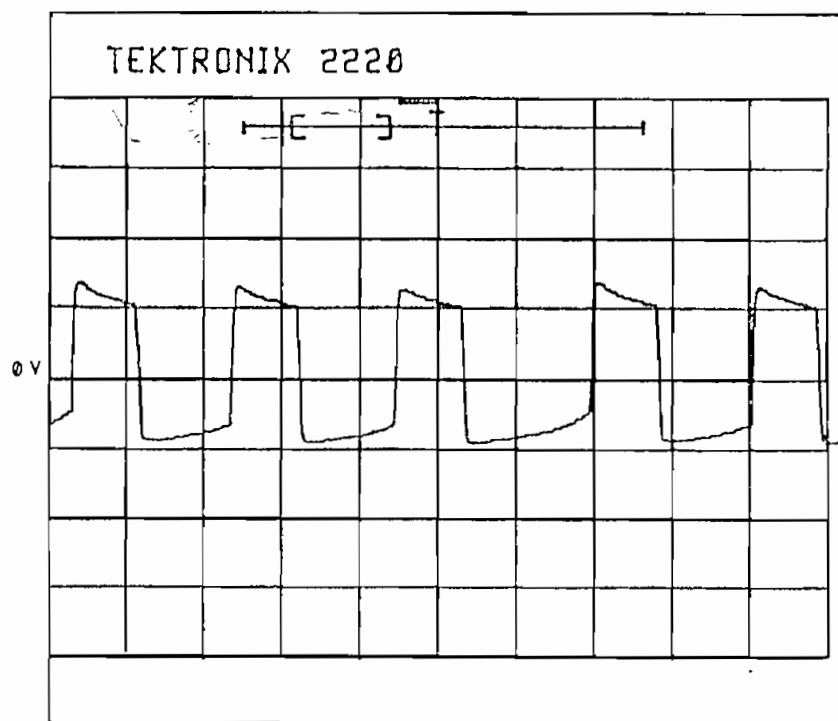
(V: 5 V/div, t: 1 ms/div)

Fig. 3.44. Código Manchester diferencial -217-



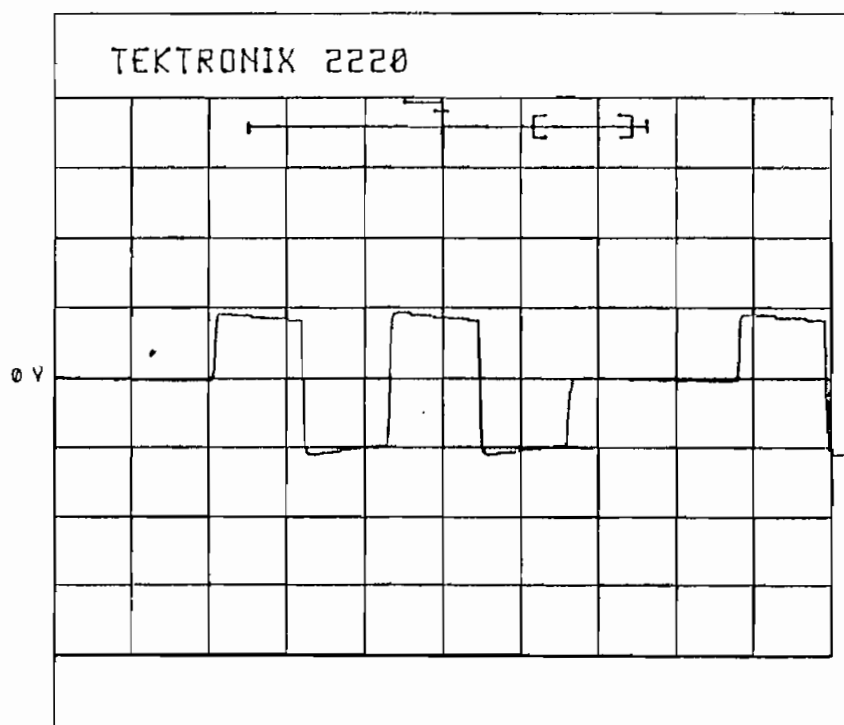
(V: 5 V/div, t: 1 ms/div)

Fig. 3. 45. Código bifase - M



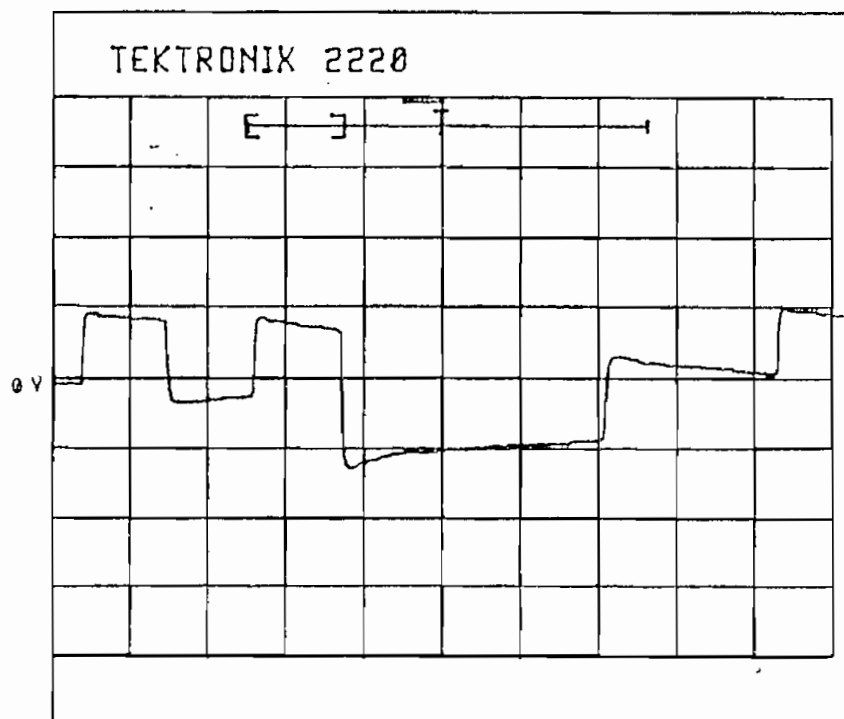
(V: 5 V/div, t: 1 ms/div)

Fig. 3.46. Código Miller



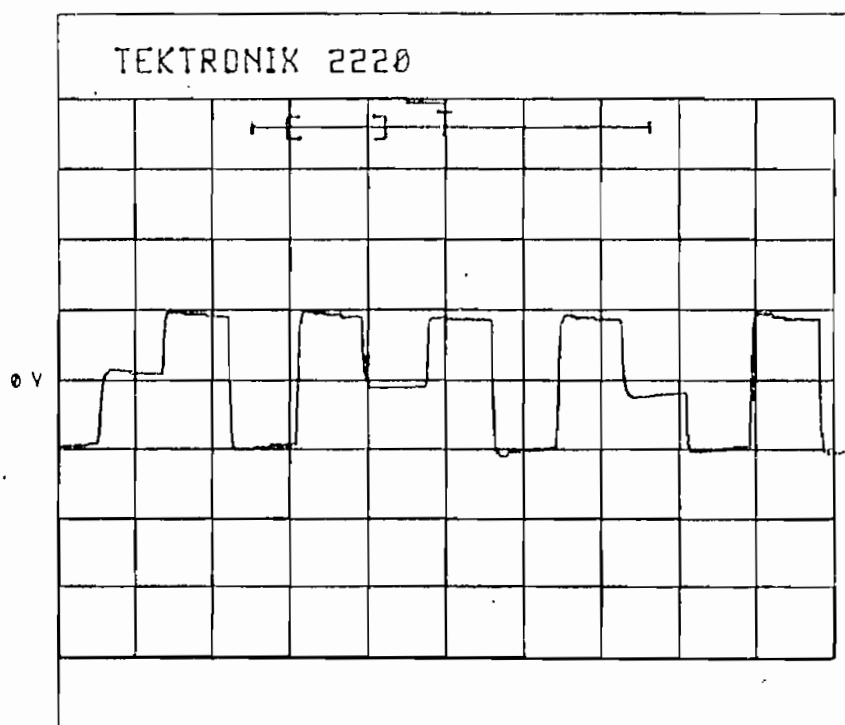
(V: 5 V/div, t: 1 ms/div)

Fig. 3.47. Código 4B-3T



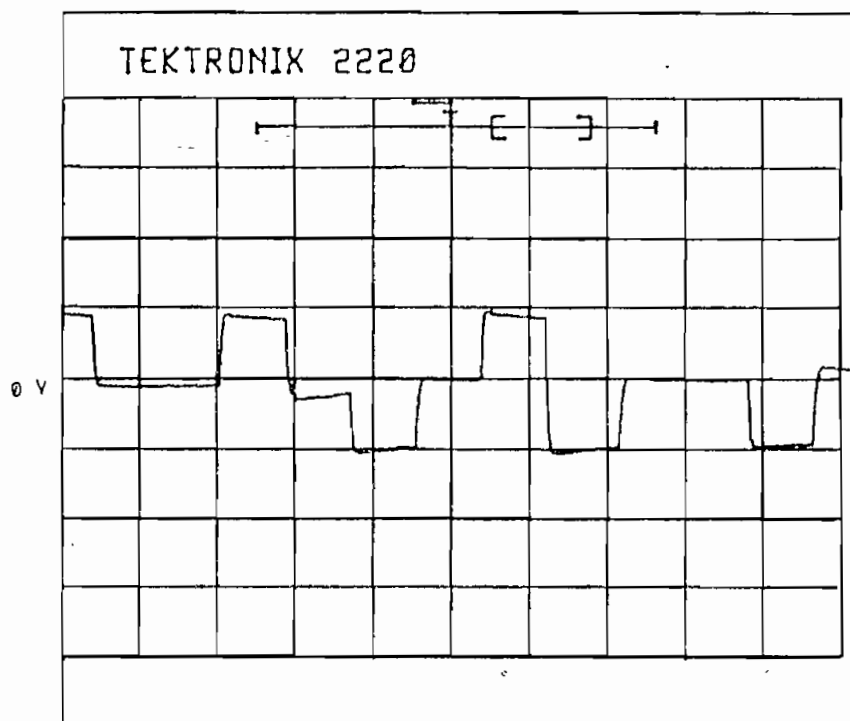
(V: 5 V/div, t: 1 ms/div)

Fig. 3.48. Código MS43



(V: 5 V/div, t: 1 ms/div)

Fig. 3.49. Código B3ZS



(V: 5 V/div, t: 1 ms/div)

Fig. 3.50. Código HDB3

El análisis de los gráficos anteriores indica que la presencia de una componente continua en la señal codificada, ocasiona que ésta tienda a atenuarse con el transcurso del tiempo; este efecto se minimiza si existen transiciones positivas y negativas con la suficiente frecuencia, de modo que el promedio de la señal tienda a cero (eliminación de la componente continua). Por ello se puede afirmar que los mejores esquemas de codificación, respecto a la eliminación de la componente continua, son el bifase tipo M, el Manchester diferencial y el AMI.

Por el contrario, los esquemas de codificación que son más afectados por la atenuación de la componente continua son el NRZ polar y el código Miller. El código RZ polar presenta una razón de trabajo inferior al 100% con lo cual se trata de hacer menos duradera la presencia de una polaridad, pero al no compensarla con la polaridad opuesta el método no resulta muy eficaz.

Los códigos de línea B3ZS y HDB3 no presentan una mejoría considerable en la disminución de la componente continua, pero tienen un comportamiento superior al observado para los códigos 4B-3T y MS43.

3.5. LIMITACIONES DEL EQUIPO.

En este punto, luego de haber realizado la evaluación experimental del codec didáctico en banda base (CBB), conviene establecer las limitaciones que se presentan.

En primer lugar, se ha observado que la velocidad de transmisión alcanzada por los diferentes códigos no es uniforme sino que varía de acuerdo al formato que se use.

Por otra parte, para las velocidades de transmisión máximas, se nota que hay más facilidad para mantener el

sincronismo en el caso de una transferencia continua de datos (por ejemplo en la transferencia de un archivo), antes que para la transferencia de caracteres aislados. En este último caso pueden presentarse errores ocasionales que ocurren cuando el reloj de decodificación pierde momentáneamente el sincronismo, pero cuando éste es recuperado se vuelve a la transmisión normal.

Un aspecto que se debe tener presente es que la sincronización de los códigos 4B-3T y MS43 demanda del reconocimiento de un bloque de tres dígitos ternarios a partir de los cuales quedarán sincronizados el transmisor y el receptor. Esta operación puede no ser exitosa al primer intento por errores esporádicos en el reloj de decodificación, por lo que se puede requerir de un nuevo ciclo de inicialización, especialmente cuando se trabaja en los límites extremos de velocidad que soportan estos códigos.

Sin embargo de todo ello, el Codec en Banda Base (CBB) ha sido diseñado como un elemento didáctico que permita el monitoreo de las señales involucradas en el proceso de codificación/decodificación, y es así que el dispositivo cumple este objetivo de manera óptima, sobre todo si se mantiene una secuencia binaria estable, que puede provenir de un generador de secuencias o del flujo repetitivo de un caracter ASCII.

El equipo no se ha concebido para cumplir con un requerimiento particular en cuanto a un tipo de código y a una alta velocidad de transmisión sino que se lo ha diseñado para una diversidad de esquemas de codificación con las consiguientes limitaciones de velocidad.

Para cumplir con requerimientos puntuales, la industria ha desarrollado circuitos integrados específicos, tales como el TMC2201, el TMC2202/12 y el TM2303/4 de Texas

Instruments¹, que ejecutan los códigos AMI y HDB3; los C.I. de National Semiconductor² TP3401, TP3402, TP340, TP3420 y TP342 que siendo circuitos integrados de aplicación en ISDN (red digital de servicios integrados), emplean el esquema AMI como código de línea. Están además los C.I. DP8340 y DP8341 de National Semiconductors³ que permiten una transmisión de la información con un formato bifase y con un protocolo adicional.

Estos circuitos integrados, gracias a la alta tecnología de integración, cumplen con requerimientos particulares, pudiendo alcanzar velocidades en el orden de las decenas o centenas de kbits/s, por ello son utilizados comercialmente. En aplicaciones puntuales como una red informática, las tarjetas que permiten el enlace entre computadores están provistas de los circuitos integrados que realizan la codificación y decodificación de las secuencias de bits, con un solo esquema de codificación, pero a una gran velocidad.

A pesar de todo ello, no resulta práctica la utilización de estos circuitos integrados en el desarrollo del presente dispositivo ya que la diversidad de códigos de línea, su velocidad y el procesamiento bit a bit no pueden ser conseguidos con circuitos individuales sin conformar un esquema demasiado complejo debido a que la aplicación de cada circuito integrado es específica y difiere en velocidad máxima, tipo de protocolo, manejo de la línea de transmisión, etc.

¹ TEXAS INSTRUMENTS, Telecom Circuits Selection Guide (VLSI/LSI for Telecom Applications), U.S.A., 1983, p. 31-38.

² NATIONAL SEMICONDUCTOR, Telecommunications Databook, Santa Clara, CA, 1990, p. 2-3 a 2-128.

³ NATIONAL SEMICONDUCTOR, Interface, bipolar LSI, bipolar memory, programmable logic DATABOOK, Santa Clara, CA, 1983, p. 9-12 a 9-22.

CAPITULO IV

CONCLUSIONES

Si bien a lo largo de los años (desde la década de 1960) se han propuesto varias formas de expresar los datos binarios para ser transmitidos, muchas de éstas han sido abandonadas o han servido sólo para el desarrollo posterior de un mejor método. Sin embargo, algunos de los primeros esquemas propuestos siguen todavía utilizándose gracias a que sus características globales son ventajosas.

El código NRZ polar está presente siempre que la transmisión serial de datos involucre la norma EIA RS-232 (V.24 del CCITT) para distancias máximas de 15 metros; se trata tan solamente de una equivalencia de los dígitos binarios a dos niveles de voltaje y por tanto su utilización no ofrece mayores ventajas. El código AMI por otro lado, a pesar de haber sido uno de los primeros en ser propuesto, es de gran aceptación gracias a su capacidad de disminuir los niveles de componente continua, a la buena cantidad de transiciones que presenta y a la facilidad de su implementación; por estas razones permite alcanzar grandes velocidades de transmisión como se evidencia en la exposición de resultados en el capítulo III.

Por otro lado, se ha observado en la práctica que los códigos con retorno a cero (RZ), no resultan convenientes ya que aumentan la velocidad de transmisión codificada, requieren por tanto un canal de mayor ancho de banda y su transición hasta cero no es garantía de una mejor sincronización. La ventaja que presentan es la disminución en el gasto de potencia y por ello se suele utilizar cuando existe necesidad de reducir el consumo de energía.

Los esquemas de codificación bifase, presentan muchas transiciones que ayudan en la recuperación de la señal de reloj, pero esto debe ser adecuadamente aprovechado mediante la utilización de circuitos de altísima estabilidad que garanticen una invariabilidad en la fase del reloj de muestreo. Debido a que el cumplimiento de esta premisa involucra un diseño complejo y costoso en el decodificador, una solución práctica consiste en la utilización de un protocolo mediante el cual la información se envíe en tramas (arreglos de datos), las mismas que deben tener periódicamente pulsos y caracteres de sincronismo. Los pulsos servirán para controlar el reloj de muestreo, en tanto que los caracteres permitirán establecer el inicio y el fin de la información propiamente dicha.

Este procedimiento de sincronismo demanda que el controlador de la comunicación pueda 'dialogar' con el equipo terminal de datos (DTE), para que éste envíe la información cuando se haya transmitido una trama o que espere cuando todavía no se lo ha hecho. En el caso del presente trabajo no se ha implementado un protocolo debido a que el equipo está concebido para trabajar con cualquier flujo de datos entrante que no necesariamente será provisto por un terminal inteligente que pueda entender el protocolo.

Además del protocolo de comunicación, se puede contribuir al mejoramiento de estos sistemas de codificación si se los combina con un esquema diferencial, de modo que la decodificación de un elemento de señal dependa no sólo de la cuantificación en magnitud y tiempo sino también del valor del elemento que lo precedió; un ejemplo de ello es el código Manchester diferencial.

Cuando se trata de realizar transmisiones a altas velocidades y sobre grandes distancias, un esquema con

disminución de la velocidad codificada será el adecuado. En este caso se debe realizar además de la sincronización a nivel de bits, una sincronización a nivel de palabras ternarias, ya que de lo contrario se decodificará una secuencia totalmente diferente de la que se envió originalmente. Con este objetivo, en el CODEC diseñado, se envía una secuencia inicial de sincronismo; sin embargo sería necesario establecer además un protocolo que permita el envío de la secuencia de sincronismo de una manera periódica ya que seguramente se perderá el sincronismo con el transcurso del tiempo y será necesario recuperarlo mediante el reconocimiento de la palabra de sincronismo. Estos códigos resultan, por lo tanto, muy complejos de implementar y la decisión de su utilización deberá ser respaldada por una gran ventaja en la disminución del ancho de banda requerido.

Como un compromiso entre la complejidad de la implementación de los códigos y sus ventajas, se presentan los códigos que evitan la presencia de un excesivo número de ceros (ausencia de pulsos). Por ser los más difundidos se han implementado el HDB3 y el B3ZS que no es más que el HDB2.

Como una característica general se debe mencionar que sin importar el esquema de codificación que se elija y por más complicado que sea su algoritmo, se encontrará que el proceso de decodificación demanda siempre mayor cuidado que el de codificación. Esto se debe básicamente a que los elementos de señal que llegan al receptor, luego de atravesar la línea de transmisión, están atenuados y dispersos en el tiempo (con lo cual se produce la interferencia intersímbolo). Esto ocasiona que la decodificación ocasione errores debido a defasamientos en el muestreo de los elementos de señal, lo que se conoce como *jitter*.

Con la finalidad de evitar estos efectos, se realiza la ecualización que no es más que la adecuación de la señal a la línea de transmisión y la formación apropiada de los pulsos; este procedimiento junto con la sincronización de los elementos de señal y el reloj de muestreo son los medios que permiten realizar una transmisión digital confiable.

Respecto a la sincronización, se debe manifestar que ésta se la ha realizado tradicionalmente mediante la alimentación de la señal codificada y rectificada a un circuito resonante que oscilará a la frecuencia del reloj de muestreo. Para ello es necesario extraer el tono de esta frecuencia, procedimiento que requiere el conocimiento de la densidad espectral de potencia de la señal codificada para ubicar la línea espectral requerida en el punto de máxima densidad de energía. He aquí la importancia del programa que permite visualizar la densidad espectral de potencia de los códigos de línea. Del análisis realizado en este trabajo se encuentra que el máximo de potencia no está siempre a la frecuencia requerida o un múltiplo de la misma y por ello se tiene dificultad en la extracción del reloj de muestreo para la decodificación.

El método anteriormente descrito es adecuado para obtener el reloj de decodificación de señales que se transmiten a velocidades altas (cientos o miles de kbit/s) en donde los elementos utilizados son fácilmente realizables. Cuando las velocidades de transmisión son pequeñas existe el problema de requerir elementos demasiado grandes y costosos.

La utilización de circuitos digitales que faciliten este trabajo es una alternativa válida, sobre todo a bajas velocidades y por ello el CODEC diseñado emplea este método de sincronización. Por otro lado, la utilización de un circuito resonante implica que la transmisión se la realiza

a una sola velocidad establecida, cosa que no sucede en el diseño del dispositivo en cuestión, en donde las velocidades son diversas y pueden ser seleccionadas digitalmente. Con el avance tecnológico, circuitos digitales completos son integrados en pequeñas pastillas de silicio que además de disminuir el consumo de potencia, permiten aumentar grandemente la velocidad de transmisión de los datos; en este sentido, el uso de estos circuitos especializados aumentará en los casos prácticos la calidad de la transmisión digital.

Si bien el CODEC diseñado está limitado para transmisión a bajas velocidades, cumple cabalmente su objetivo de ser un elemento didáctico que permite la visualización de la transmisión digital en banda base. Su característica más importante está en la versatilidad, ya que permite escoger diez esquemas diferentes de codificación con varias velocidades de transmisión *half-duplex* o *full-duplex*, además de aceptar y entregar niveles RS-232 o TTL en el proceso de codificación/decodificación. Puede funcionar por tanto con cualquier fuente de datos como computadores personales, terminales de comunicaciones no inteligentes, generadores de secuencias, conversores analógico digitales, etc.

Por lo dicho anteriormente se concluye que será suficiente con poseer cualquier fuente binaria de datos para que el equipo los codifique, los transmita y los decodifique, de modo que mediante la observación y medición de las señales generadas se pueda tener una percepción completa del proceso en banda base. Al ser un equipo didáctico, los usuarios podrán corroborar en la práctica lo que enseña la teoría de codificación en banda base.

Una característica que aumenta la versatilidad del equipo es el hecho de disponer de dos módulos, que permitirán demostrar el principio de la repetición

regenerativa de las señales en banda base. Además, usando sólo uno de ellos, mediante un lazo local y empleando el modo *full duplex* se podrá analizar el principio de codificación y decodificación en el mismo dispositivo.

El programa para computador personal ("DEP") que se ha diseñado para el estudio de la densidad espectral de potencia de los códigos de línea, cumple una función complementaria en el tratamiento del tema ya que muestra la manera como se distribuye la potencia de la señal codificada en el espectro de frecuencias. Este análisis permite llegar a consideraciones prácticas respecto a un compromiso entre ancho de banda, consumo de potencia y facilidad en la recuperación de la señal de reloj. En todo caso, un examen del gráfico de la d.e.p. será una de gran ayuda al momento de analizar los códigos usados para transmisión en banda base y se constituye por lo tanto en un elemento didáctico de gran importancia dentro del tratamiento del tema.

El método expuesto en el desarrollo del CODEC es la base en la operación de un repetidor regenerativo, el cual tendrá como función el recibir una señal en banda base, reconocerla y volver a transmitirla con sus niveles bien conformados. La distancia que separa dos repetidores es variable, desde decenas de metros hasta varios kilómetros; esto depende de la velocidad de transmisión, la calidad del repetidor, del tipo de línea de transmisión, los requerimientos de la tasa de error (BER) y los niveles de potencia de transmisión. En el caso del CODEC diseñado, éste no es un requerimiento serio ya que no se lo ha concebido como un dispositivo que vaya a ocupar un lugar específico dentro de un sistema de comunicación, además de que la amplitud de sus niveles de señal es relativamente baja (5 voltios). En la práctica, los requerimientos del sistema de transmisión en donde va a trabajar el repetidor serán los que determinen el tipo de código a usarse, los

niveles de potencia, el método de sincronización, etc.

Finalmente se debe manifestar que, como un complemento al presente trabajo, sería recomendable la realización de un estudio más profundo sobre la recuperación de la señal de reloj y los métodos de sincronización que se pueden aplicar en recepción, ya que además del método digital empleado se podría probar el uso del lazo asegurador de fase (PLL) de varios órdenes, el método analógico del circuito resonante y algunos otros que existen con este fin, además de mostrar las características de cada uno en cuanto a la tasa de bits erróneos (BER), la relación señal a ruido que se requiere y la medición del *jitter* que existe en cada caso.

Para realizar estas pruebas se requiere la construcción de un generador de secuencias pseudoaleatorias, además de medidores apropiados para el BER y el *jitter*, cuestiones que son fundamentales para el equipamiento de un laboratorio en donde se estudie los fundamentos de la transmisión de datos.

BIBLIOGRAFIA

- BELLAMY J., Digital telephony, John Wiley & Sons, New York, 1982.
- BLACK U., Redes de computadoras, Macrobit, México, 1990.
- BORLAND, Turbo pascal 6.0, programmer's guide, U.S.A., 1990.
- BYLANSKY P. e INGRAM D., Digital transmission systems, IEE Telecommunications Series 4, England, 1979.
- DAVENPORT W., Comunicación moderna de datos, GLEM, Buenos Aires, 1975.
- FEHER K., Digital communications, Prentice Hall, Englewood Cliffs, 1981.
- FREEMAN R., Sistemas de telecomunicaciones, LIMUSA, México, 1991.
- IEEE Communications magazine, Digital signaling techniques, STALLINGS W., Vol. 22 (Nº 12, 1984).
- IEEE Trans. on communications, Digital PCM bit synchronizer and detector, ALI EL MOGAZY y OTROS, Vol. COM-28 (Nº 8, 1980).
- INTEL, Embedded microcontrollers and processors, U.S.A., vol. I, 1993.
- KUSTRA R. y TUJSNAIDER O., Principios de comunicaciones digitales, vol. II, Colección Técnica AHCIET-ICI, FICUM S. A., 1978.
- MACCHI C. y GUILBERT J., Tèlèinformatique, DUNOD, Paris, 1979.
- MAXIM, New releases databook, U. K., vol. I, 1992.
- NATIONAL SEMICONDUCTOR, CMOS databook, 1981.

NATIONAL SEMICONDUCTOR,	<u>Interface, bipolar LSI, bipolar memory, programmable logic databook</u> , Santa Clara, CA, 1983.
NATIONAL SEMICONDUCTOR,	<u>Linear databook</u> , 1981.
NATIONAL SEMICONDUCTOR,	<u>Telecommunications databook</u> , Santa Clara, CA, 1990.
PHILIPS,	<u>Master product catalog</u> , U.S.A., 1992.
STREMLER F.,	<u>Sistemas de comunicación</u> , Fondo Educativo Interamericano, México, 1991.
TANENBAUM A.,	<u>Redes de ordenadores</u> , Prentice Hall, 2 ^{da} ed., México, 1991.
TEXAS INSTRUMENTS,	<u>Telecom circuits selection guide</u> , U.S.A., 1983.
TEXAS INSTRUMENTS,	<u>The TTL databook</u> , U.S.A., vols. I,II, 1985.
VIDALLER J. y OTROS,	<u>Transmisión de datos</u> , E.T.S. Ingenieros Telecomunicaciones-Universidad Politécnica de Madrid, Madrid, 2 ^{da} ed., 1979.

GUIA DE OPERACION DEL EQUIPO

La utilización del CODEC Didáctico en Banda Base demanda de un procedimiento interactivo simple que se resume en los siguientes términos.

Luego de encender el CODEC mediante el interruptor colocado a la derecha del equipo, se encenderá el LED de *POWER* y se mostrarán en el display los mensajes de presentación en el siguiente orden:

a. E.P.N. - F.I.E.
 - 1994 -

b. CODEC DIDACTICO
 EN BANDA BASE

c. INICIALIZACION
 DEL SISTEMA

d. ** CODIGO **
 NRZ POLAR

Después de mostrarse el último mensaje, se debe utilizar las teclas para escoger el esquema de codificación que se desea estudiar. Existen para el efecto tres teclas, con la tecla de la derecha se avanza en el menú, con la de la izquierda se retrocede y con la tecla central se acepta la opción mostrada. Para asegurar que la elección es correcta se pide confirmación, a lo cual se debe responder presionando la tecla central si la selección es correcta, o cualquiera de las teclas laterales si no lo es. En este último caso se regresa a la opción anterior y se puede seguir el proceso de selección.

Después de haber elegido el esquema de codificación, se elige el tipo de transmisión, pudiendo ser

ésta: *full duplex* o *half duplex*. Si la transmisión es *full duplex* se pasará al siguiente parámetro, pero en caso de ser *half duplex* se debe establecer primero si el equipo actuará como codificador o como decodificador.

El siguiente paso es establecer la velocidad de transmisión, que está comprendida entre 150 bit/s y la máxima velocidad de transmisión permisible para la configuración particular (tabla 3.1).

Si el equipo funciona en modo *full duplex* o en modo *half duplex* pero como codificador, se deberá elegir si los niveles de entrada son RS-232 o TTL, con este último paso se habrá concluido la inicialización. En caso de que el dispositivo actúe sólo como decodificador se omitirá esta última selección y se terminará el proceso de inicialización.

Cuando se utilizan los esquemas 4B-3T o MS43, antes de finalizar se debe enviar una secuencia de sincronismo que permitirá una adecuada decodificación de los elementos de señal. Para ello es necesario que en los dos equipos se haya escogido los parámetros adecuados y que se esté esperando el envío de la secuencia de sincronismo. En el caso *half duplex* se debe inicializar primero el receptor para que éste se encuentre a la espera de la palabra de sincronismo.

Luego de que se ha terminado este procedimiento, en el display se muestra el tipo de código de línea y la velocidad de transmisión, mientras que en el panel frontal se encenderán los LED's correspondientes a la configuración que se acaba de 'setear'. El equipo entonces entra en una tarea continua de codificación y/o decodificación, de la que sólo saldrá presionando cualquiera de las tres teclas.

La señal de datos a codificar (en niveles RS-232 o

TTL) es ingresada mediante el pin de recepción (pin 3) de un conector DB25 marcado como **DTE** (equipo terminal de datos) y se entrega decodificada en el pin de transmisión de este mismo conector (pin 2) en niveles RS-232. Además, la señal mostrada en los puntos de prueba correspondientes y que se encuentran en el panel frontal, permitirán obtener la señal decodificada en niveles RS-232 y TTL.

La señal codificada que se transmite a otro CODEC remoto, o que se recibe desde el mismo para ser decodificada pasa por un conector DB9 que corresponde a la línea de transmisión (L. de Tx.), a través del pin 2 para recepción y a través del pin 3 para transmisión.

Cuando se está realizando la decodificación a una velocidad de 19200 bit/s, se da el caso que el microcontrolador demora mucho tiempo en atender a la interrupción de una tecla debido a la continuidad con que debe atender a la interrupción de decodificación y por tanto el sistema no se reinicializa inmediatamente. En este caso se recomienda usar el **reset** del sistema que se encuentra en la parte lateral derecha y que volverá a inicializar el equipo.

UTILIZACION DEL PROGRAMA PARA EL ANALISIS DE LA D.E.P.

El programa se ejecuta escribiendo DEP.EXE y automáticamente se detectará el tipo de adaptador gráfico que se posee, usándose el manejador correspondiente. Para esto es necesario que en el mismo directorio en donde está el programa ejecutable se carguen los manejadores gráficos (*.BGI) y los archivos de *fonts* (*.CHR). Para el análisis de la densidad espectral de potencia (d.e.p.) de los códigos de línea existen dos posibilidades que se contemplan en el menú principal:

- a. 'Según los Códigos'
- b. 'Según las Probabilidades'
- c. 'Salir al D. O. S.'

Un cursor se desplaza verticalmente para posicionarse en cada una de las opciones; para aceptar la opción se debe presionar la tecla 'ENTER'.

Si se ha elegido la primera opción del menú principal, se entrará a la comparación de la d.e.p. de los códigos que se seleccionen. Para realizar esta comparación se deberá escoger los códigos a compararse y la probabilidad *p*. Con esta finalidad, las opciones que se presentan en pantalla son:

- a. 'Seleccionar los parámetros'
- b. 'Ver espectros de potencia'
- c. 'Regresar al menú principal'

La opción 'Seleccionar parámetros' permite escoger los códigos cuya d.e.p. se va a comparar y el valor de la probabilidad *p* que será un parámetro común en la comparación. Para ello, en la pantalla se muestra un

listado de los códigos disponibles, de modo que se pueda navegar a través de esta lista mediante las teclas de desplazamiento vertical ($\uparrow$ y $\downarrow$). Para escoger uno de los esquemas de codificación se debe llevar el cursor hasta él y presionar 'ENTER' con lo cual se activa una marca al lado izquierdo del nombre. Para eliminar un código del conjunto seleccionado, bastará con presionar nuevamente 'ENTER'; de este modo se puede seleccionar todos los códigos de línea que se desee comparar simultáneamente.

Existe además una opción para escoger el valor de la probabilidad de entrada de los 1_L al codificador; este valor es por defecto $p = 0.5$; para cambiar este valor, se deberá presionar 'ENTER' en esta opción e ingresar un valor que deberá estar comprendido entre 0 y 1, si no se cumple con este limitante, se volverá a pedir que se ingrese un valor adecuado. La tercera opción de este menú permite regresar al menú anterior.

La opción 'Ver espectros de potencia' permite visualizar los gráficos de la d.e.p. para los códigos escogidos. Si no se ha seleccionado ningún esquema de codificación, se mostrará un mensaje indicando este hecho y cualquier tecla que se presione llevará el cursor hasta la opción 'Seleccionar los parámetros'. Si el monitor es de color, el gráfico de cada código aparecerá con diferente color y al lado derecho se enlistará, con el mismo color los nombres de los esquemas de codificación graficados. Un número ubicado en cada gráfico permitirá realizar la correspondencia entre la curva y el nombre del respectivo código de línea. Finalmente, la opción 'Volver al menú principal' permitirá regresar a elegir la otra posibilidad de análisis o salir al sistema operativo.

Cuando se requiera analizar la d.e.p. 'Según las Probabilidades', se debe escoger un código del cual se observará la d.e.p. para tres valores del parámetro p .

Puesto que el caso en que tanto el 1_L como el 0_L tienen la misma probabilidad de ocurrir ($p = 0.5$) es el más común, la comparación se la realizará siempre incluyendo este caso, por lo que el usuario tendrá la posibilidad de ingresar dos valores de probabilidad como parámetros. La pantalla de presentación muestra cuatro opciones, las mismas que son:

- a. 'Seleccionar código'
- b. 'Seleccionar probabilidades'
- c. 'Ver espectros de potencia'
- d. 'Regresar al menú principal'

Luego de seleccionar el código y los valores de probabilidad se puede ver los espectros de potencia correspondientes, opción a la que no se podrá acceder si no se ha elegido un código de línea. En caso de no seleccionar otros valores de probabilidad, el gráfico se ejecutará solamente para $p = 0.5$.

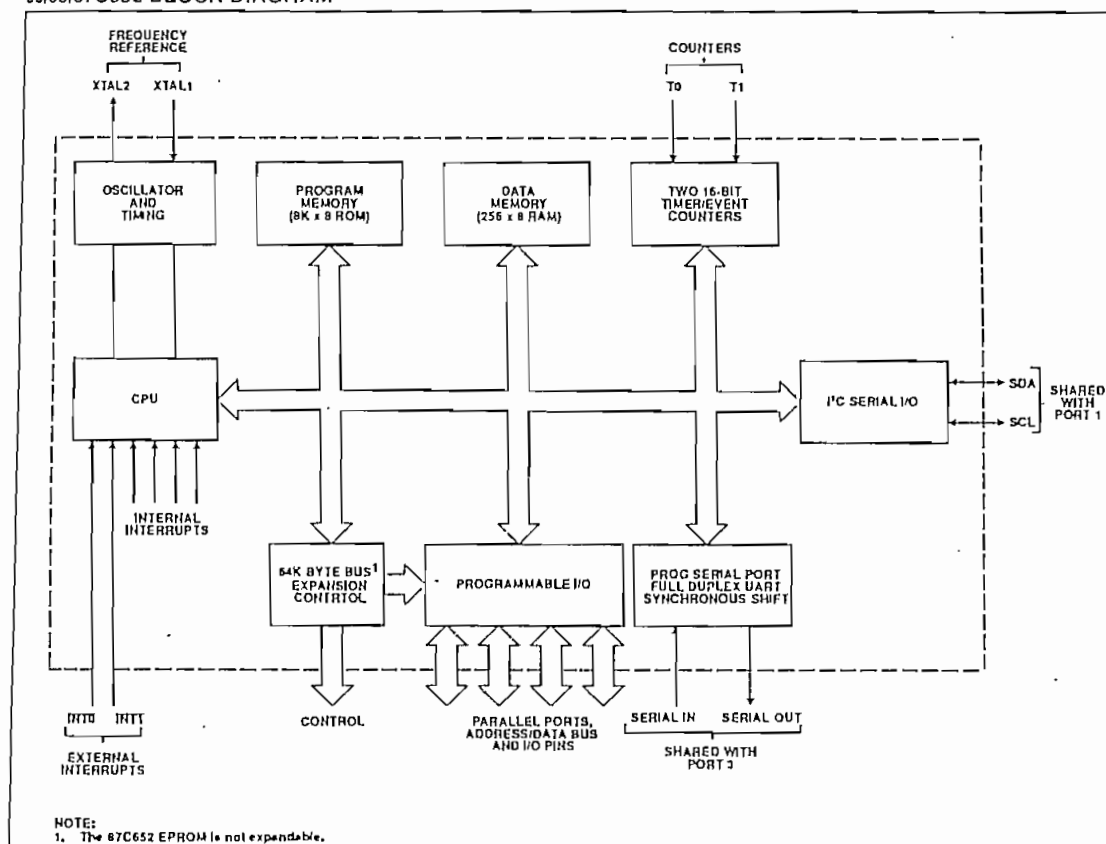
Product Spotlights

80/83/87C652 – CMOS Single-Chip, 8-Bit Microcontroller

FEATURES

- 80C51 central processing unit
- 8K x 8 ROM expandable externally to 64K bytes (87C652 EPROM is not expandable)
- 256 x 8 RAM, expandable externally to 64K bytes
- Two standard 16-bit timer/counters
- Four 8-bit I/O ports
- I²C-bus serial I/O port with byte oriented master and slave functions
- Full-duplex UART facilities
- Power control modes
 - Idle mode
 - Power-down mode
- Five package styles
- Extended temperature ranges
- OTP package available

80/83/87C652 BLOCK DIAGRAM



02-9205472

MCS®-51 FAMILY OF MICROCONTROLLERS ARCHITECTURAL OVERVIEW

CONTENTS	PAGE
INTRODUCTION	5-3
CHMOS Devices	5-5
MEMORY ORGANIZATION IN MCS®-51 DEVICES	5-5
Logical Separation of Program and Data Memory	5-5
Program Memory	5-6
Data Memory	5-7
THE MCS®-51 INSTRUCTION SET	5-8
Program Status Word	5-8
Addressing Modes	5-9
Arithmetic Instructions	5-9
Logical Instructions	5-11
Data Transfers	5-11
Boolean Instructions	5-13
Jump Instructions	5-15
CPU TIMING	5-16
Machine Cycles	5-17
Interrupt Structure	5-19
ADDITIONAL REFERENCES	5-21

INTRODUCTION

The 8051 is the original member of the MCS®-51 family, and is the core for all MCS-51 devices. The features of the 8051 core are:

- 8-bit CPU optimized for control applications
- Extensive Boolean processing (single-bit logic) capabilities
- 64K Program Memory address space
- 64K Data Memory address space
- 4K bytes of on-chip Program Memory
- 128 bytes of on-chip Data RAM
- 32 bidirectional and individually addressable I/O lines
- Two 16-bit timer/counters
- Full duplex UART
- 6-source/5-vector interrupt structure with two priority levels
- On-chip clock oscillator

The basic architectural structure of this 8051 core is shown in Figure 1.

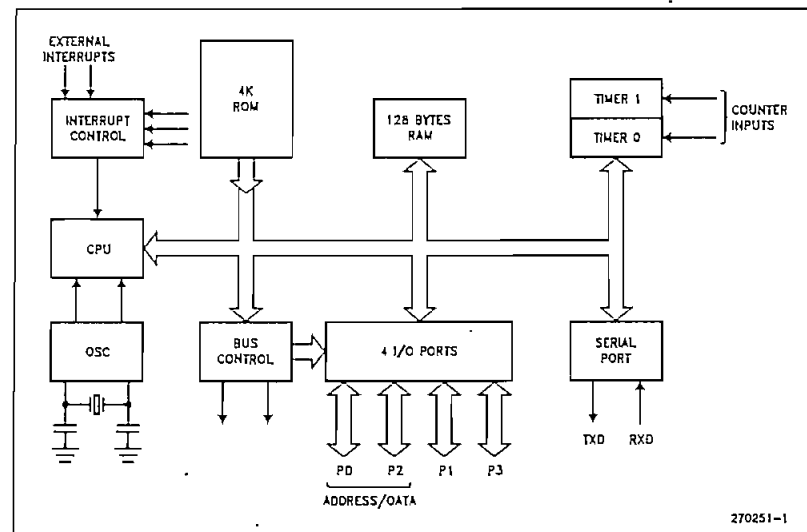


Figure 1. Block Diagram of the 8051 Core

Each device on the MCS-51 family consists of all the core features plus some additional features. A feature comparison of all the MCS-51 devices is shown in Table 1.

Table 1. The MCS-51 Family of Microcontrollers

Device	ROMless Version	EPROM Version	ROM Bytes	RAM Bytes	8-Bit I/O Ports	16-Bit Timer/Counters	Programmable Counter Array (PCA)	UART	Serial Expansion Port (SEP)	Global Serial Channel (GSC)	DMA Channels	A/D Channels	Interrupt Sources/Vectors	Power Down and Idle Modes
8051	8031	—	4K	128	4	2	—	1	—	—	—	—	6/5	—
8051AH	8031AH	8751H 8751BH	4K	128	4	2	—	1	—	—	—	—	6/5	—
8052AH	8032AH	8752BH	8K	256	4	3	—	1	—	—	—	—	8/6	—
80C51BH	80C51BH	87C51	4K	128	4	2	—	1	—	—	—	—	6/5	—
80C52	80C32	—	8K	256	4	3	—	1	—	—	—	—	8/6	—
83C51FA	80C51FA	87C51FA	8K	256	4	3	1	1	—	—	—	—	14/7	—
83C51FB	80C51FA	87C51FB	16K	256	4	3	1	1	—	—	—	—	14/7	—
83C152JA	80C152JA	—	8K	256	5	2	—	1	—	—	2	—	19/11	—
—	80C152JB	—	—	256	7	2	—	1	—	—	2	—	19/11	—
83C152JC	80C152JC	—	8K	256	5	2	—	1	—	—	2	—	19/11	—
—	80C152JD	—	—	256	7	2	—	1	—	—	2	—	19/11	—
83C452	80C452	87C452P	8K	256	5	2	—	1	—	—	—	—	9/8	—

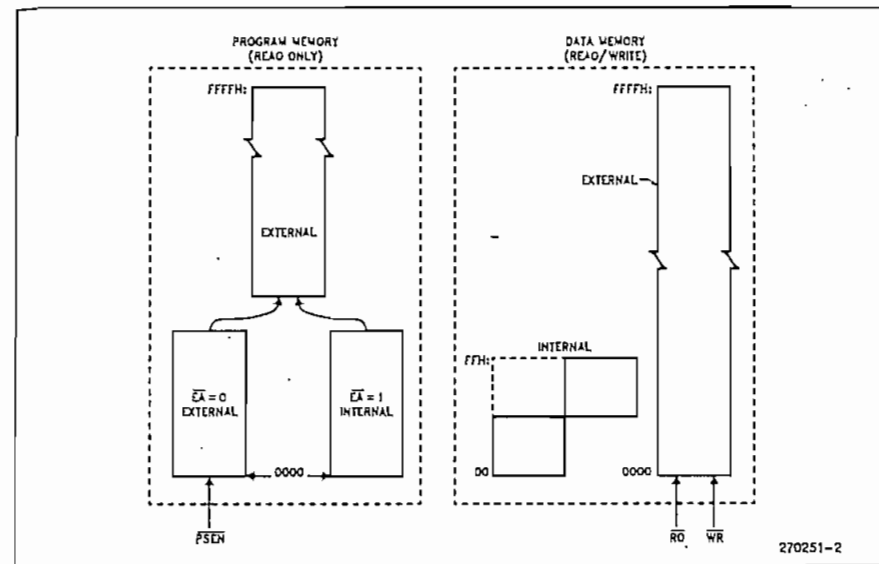


Figure 2. MCS-51 Memory Structure

CHMOS Devices

Functionally, the CHMOS devices (designated with "C" in the middle of the device name) are all fully compatible with the 8051, but being CMOS, draw less current than an HMOS counterpart. To further exploit the power savings available in CMOS circuitry, two reduced power modes are added:

- Software-invoked Idle Mode, during which the CPU is turned off while the RAM and other on-chip peripherals continue operating. In this mode, current draw is reduced to about 15% of the current drawn when the device is fully active.
- Software-invoked Power Down Mode, during which all on-chip activities are suspended. The on-chip RAM continues to hold its data. In this mode the device typically draws less than 10 μ A.

Although the 80C51BH is functionally compatible with its HMOS counterpart, specific differences between the two types of devices must be considered in the design of an application circuit if one wishes to ensure complete interchangeability between the HMOS and CHMOS devices. These considerations are discussed in the Application Note AP-252, "Designing with the 80C51BH".

For more information on the individual devices and features listed in Table 1, refer to the Hardware Descriptions and Data Sheets of the specific device.

MEMORY ORGANIZATION IN MCS-51 DEVICES

Logical Separation of Program and Data Memory

All MCS-51 devices have separate address spaces for Program and Data Memory, as shown in Figure 2. The logical separation of Program and Data Memory allows the Data Memory to be accessed by 8-bit addresses, which can be more quickly stored and manipulated by an 8-bit CPU. Nevertheless, 16-bit Data Memory addresses can also be generated through the DPTR register.

Program Memory can only be read, not written. There can be up to 64K bytes of Program Memory. In the ROM and EPROM versions of these devices the lowest 4K, 8K or 16K bytes of Program Memory are provided on-chip. Refer to Table 1 for the amount of on-chip ROM (or EPROM) on each device. In the ROMless versions all Program Memory is external. The read strobe for external Program Memory is the signal PSEN (Program Store Enable).

Data Memory occupies a separate address space from Program Memory. Up to 64K bytes of external RAM can be addressed in the external Data Memory space. The CPU generates read and write signals, RD and WR, as needed during external Data Memory accesses.

External Program Memory and external Data Memory may be combined if desired by applying the RD and PSEN signals to the inputs of an AND gate and using the output of the gate as the read strobe to the external Program/Data memory.

Program Memory

Figure 3 shows a map of the lower part of the Program Memory. After reset, the CPU begins execution from location 0000H.

As shown in Figure 3, each interrupt is assigned a fixed location in Program Memory. The interrupt causes the CPU to jump to that location, where it commences execution of the service routine. External Interrupt 0, for example, is assigned to location 0003H. If External Interrupt 0 is going to be used, its service routine must begin at location 0003H. If the interrupt is not going to be used, its service location is available as general purpose Program Memory.

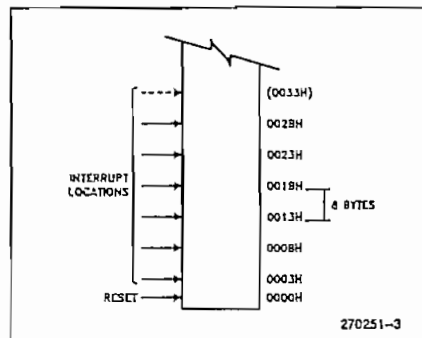


Figure 3. MCS[®]-51 Program Memory

The interrupt service locations are spaced at 8-byte intervals: 0003H for External Interrupt 0, 000BH for Timer 0, 0013H for External Interrupt 1, 001BH for Timer 1, etc. If an interrupt service routine is short enough (as is often the case in control applications), it can reside entirely within that 8-byte interval. Longer service routines can use a jump instruction to skip over subsequent interrupt locations, if other interrupts are in use.

The lowest 4K (or 8K or 16K) bytes of Program Memory can be either in the on-chip ROM or in an external ROM. This selection is made by strapping the EA (External Access) pin to either VCC or VSS.

In the 4K byte ROM devices, if the EA pin is strapped to VCC, then program fetches to addresses 0000H through 0FFFH are directed to the internal ROM. Program fetches to addresses 1000H through FFFFH are directed to external ROM.

In the 8K byte ROM devices, EA = VCC selects addresses 0000H through 1FFFH to be internal, and addresses 2000H through FFFFH to be external.

In the 16K byte ROM devices, EA = VCC selects addresses 0000H through 3FFFH to be internal, and addresses 4000H through FFFFH to be external.

If the EA pin is strapped to VSS, then all program fetches are directed to external ROM. The ROMless parts must have this pin externally strapped to VSS to enable them to execute properly.

The read strobe to external ROM, PSEN, is used for all external program fetches. PSEN is not activated for internal program fetches.

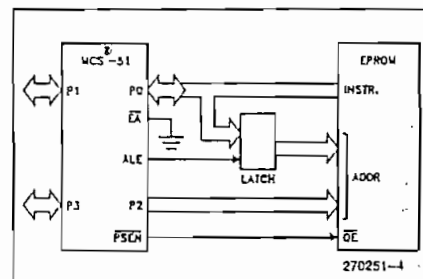


Figure 4. Executing from External Program Memory

The hardware configuration for external program execution is shown in Figure 4. Note that 16 I/O lines (Ports 0 and 2) are dedicated to bus functions during external Program Memory fetches. Port 0 (P0 in Figure 4) serves as a multiplexed address/data bus. It emits the low byte of the Program Counter (PCL) as an address, and then goes into a float state awaiting the arrival of the code byte from the Program Memory. During the time that the low byte of the Program Counter is valid on P0, the signal ALE (Address Latch Enable) clocks this byte into an address latch. Meanwhile, Port 2 (P2 in Figure 4) emits the high byte of the Program Counter (PCH). Then PSEN strobes the EPROM and the code byte is read into the microcontroller.

Program Memory addresses are always 16 bits wide, even though the actual amount of Program Memory used may be less than 64K bytes. External program execution sacrifices two of the 8-bit ports, P0 and P2, to the function of addressing the Program Memory.

Data Memory

The right half of Figure 2 shows the internal and external Data Memory spaces available to the MCS-51 user.

Figure 5 shows a hardware configuration for accessing up to 2K bytes of external RAM. The CPU in this case is executing from internal ROM. Port 0 serves as a multiplexed address/data bus to the RAM, and 3 lines of Port 2 are being used to page the RAM. The CPU generates RD and WR signals as needed during external RAM accesses.

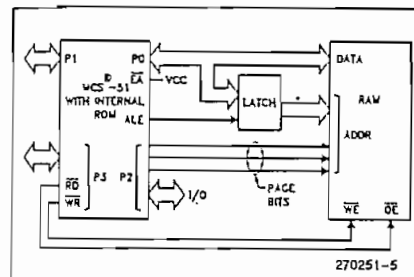


Figure 5. Accessing External Data Memory. If the Program Memory is Internal, the Other Bits of P2 are Available as I/O.

There can be up to 64K bytes of external Data Memory. External Data Memory addresses can be either 1 or 2 bytes wide. One-byte addresses are often used in conjunction with one or more other I/O lines to page the RAM, as shown in Figure 5. Two-byte addresses can also be used, in which case the high address byte is emitted at Port 2.

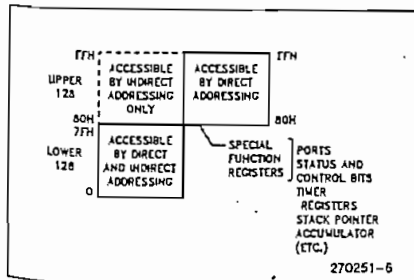


Figure 6. Internal Data Memory

Internal Data Memory is mapped in Figure 6. The memory space is shown divided into three blocks, which are generally referred to as the Lower 128, the Upper 128, and SFR space.

Internal Data Memory addresses are always one byte wide, which implies an address space of only 256 bytes. However, the addressing modes for internal RAM can in fact accommodate 384 bytes, using a simple trick. Direct addresses higher than 7FH access one memory space, and indirect addresses higher than 7FH access a different memory space. Thus Figure 6 shows the Upper 128 and SFR space occupying the same block of addresses, 80H through FFH, although they are physically separate entities.

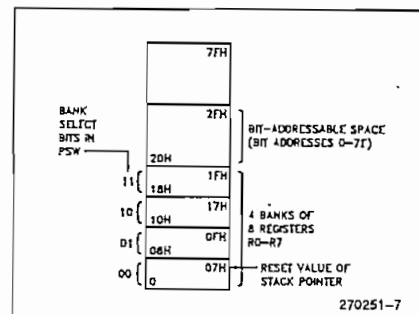


Figure 7. The Lower 128 Bytes of Internal RAM

The Lower 128 bytes of RAM are present in all MCS-51 devices as mapped in Figure 7. The lowest 32 bytes are grouped into 4 banks of 8 registers. Program instructions call out these registers as R0 through R7. Two bits in the Program Status Word (PSW) select which register bank is in use. This allows more efficient use of code space, since register instructions are shorter than instructions that use direct addressing.

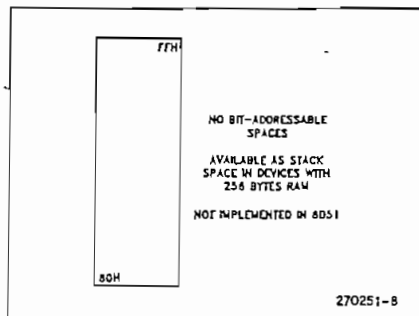


Figure 8. The Upper 128 Bytes of Internal RAM

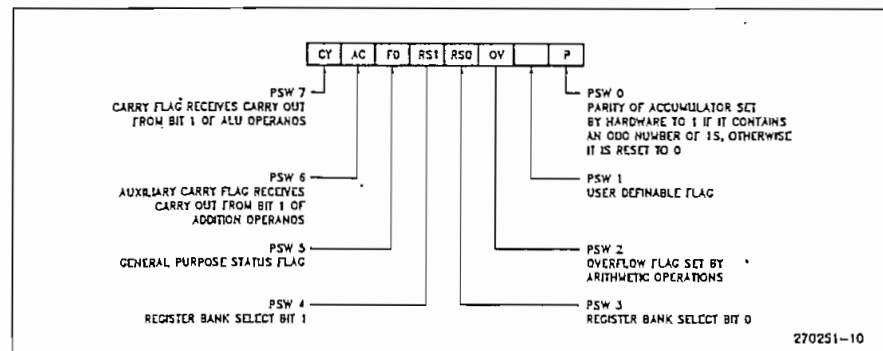


Figure 10. PSW (Program Status Word) Register in MCS-51 Devices

The next 16 bytes above the register banks form a block of bit-addressable memory space. The MCS-51 instruction set includes a wide selection of single-bit instructions, and the 128 bits in this area can be directly addressed by these instructions. The bit addresses in this area are 00H through 7FH.

All of the bytes in the Lower 128 can be accessed by either direct or indirect addressing. The Upper 128 (Figure 8) can only be accessed by indirect addressing. The Upper 128 bytes of RAM are not implemented in the 8051, but are in the devices with 256 bytes of RAM. (See Table 1).

Figure 9 gives a brief look at the Special Function Register (SFR) space. SFRs include the Port latches, timers, peripheral controls, etc. These registers can only be accessed by direct addressing. In general, all MCS-51 microcontrollers have the same SFRs as the 8051, and at the same addresses in SFR space. However, enhancements to the 8051 have additional SFRs that are not present in the 8051, nor pernapns in other proliferations of the family.

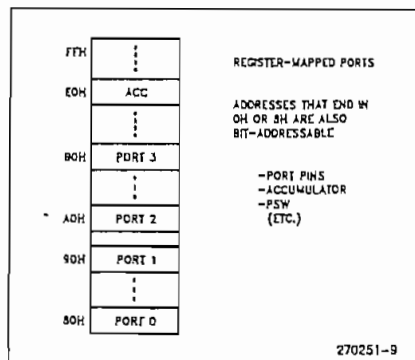


Figure 9. SFR Space

Sixteen addresses in SFR space are both byte- and bit-addressable. The bit-addressable SFRs are those whose address ends in 00BH. The bit addresses in this area are 80H through FFH.

THE MCS-51 INSTRUCTION SET

All members of the MCS-51 family execute the same instruction set. The MCS-51 instruction set is optimized for 8-bit control applications. It provides a variety of fast addressing modes for accessing the internal RAM to facilitate byte operations on small data structures. The instruction set provides extensive support for one-bit variables as a separate data type, allowing direct bit manipulation in control and logic systems that require Boolean processing.

An overview of the MCS-51 instruction set is presented below, with a brief description of how certain instructions might be used. References to "the assembler" in this discussion are to Intel's MCS-51 Macro Assembler, ASM51. More detailed information on the instruction set can be found in the MCS-51 Macro Assembler User's Guide (Order No. 9800937 for ISIS Systems, Order No. 122752 for DOS Systems).

Program Status Word

The Program Status Word (PSW) contains several status bits that reflect the current state of the CPU. The PSW, shown in Figure 10, resides in SFR space. It contains the Carry bit, the Auxiliary Carry (for BCD operations), the two register bank select bits, the Overflow flag, a Parity bit, and two user-definable status flags.

The Carry bit, other than serving the functions of a Carry bit in arithmetic operations, also serves as the "Accumulator" for a number of Boolean operations.

The bits RS0 and RS1 are used to select one of the four register banks shown in Figure 7. A number of instructions refer to these RAM locations as R0 through R7. The selection of which of the four banks is being referred to is made on the basis of the bits RS0 and RS1 at execution time.

The Parity bit reflects the number of 1s in the Accumulator: P = 1 if the Accumulator contains an odd number of 1s, and P = 0 if the Accumulator contains an even number of 1s. Thus the number of 1s in the Accumulator plus P is always even.

Two bits in the PSW are uncommitted and may be used as general purpose status flags.

Addressing Modes

The addressing modes in the MCS-51 instruction set are as follows:

DIRECT ADDRESSING

In direct addressing the operand is specified by an 8-bit address field in the instruction. Only internal Data RAM and SFRs can be directly addressed.

INDIRECT ADDRESSING

In indirect addressing the instruction specifies a register which contains the address of the operand. Both internal and external RAM can be indirectly addressed.

The address register for 8-bit addresses can be R0 or R1 of the selected register bank, or the Stack Pointer. The address register for 16-bit addresses can only be the 16-bit "data pointer" register, DPTR.

REGISTER INSTRUCTIONS

The register banks, containing registers R0 through R7, can be accessed by certain instructions which carry a 3-bit register specification within the opcode of the instruction. Instructions that access the registers this way are code efficient, since this mode eliminates an address byte. When the instruction is executed, one of the eight registers in the selected bank is accessed. One of four banks is selected at execution time by the two bank select bits in the PSW.

REGISTER-SPECIFIC INSTRUCTIONS

Some instructions are specific to a certain register. For example, some instructions always operate on the Accumulator, or Data Pointer, etc., so no address byte is needed to point to it. The opcode itself does that. Instructions that refer to the Accumulator as A assemble as accumulator-specific opcodes.

IMMEDIATE CONSTANTS

The value of a constant can follow the opcode in Program Memory. For example,

```
MOV A, #100
```

loads the Accumulator with the decimal number 100. The same number could be specified in hex digits as 64H.

INDEXED ADDRESSING

Only Program Memory can be accessed with indexed addressing, and it can only be read. This addressing mode is intended for reading look-up tables in Program Memory. A 16-bit base register (either DPTR or the Program Counter) points to the base of the table, and the Accumulator is set up with the table entry number. The address of the table entry in Program Memory is formed by adding the Accumulator data to the base pointer.

Another type of indexed addressing is used in the "case jump" instruction. In this case the destination address of a jump instruction is computed as the sum of the base pointer and the Accumulator data.

Arithmetic Instructions

The menu of arithmetic instructions is listed in Table 2. The table indicates the addressing modes that can be used with each instruction to access the <byte> operand. For example, the ADD A, <byte> instruction can be written as:

```
ADD A, 7FH    (direct addressing)
ADD A, @R0    (indirect addressing)
ADD A, R7     (register addressing)
ADD A, #127   (immediate constant)
```

The execution times listed in Table 2 assume a 12 MHz clock frequency. All of the arithmetic instructions execute in 1 μ s except the INC DPTR instruction, which takes 2 μ s, and the Multiply and Divide instructions, which take 4 μ s.

Note that any byte in the internal Data Memory space can be incremented or decremented without going through the Accumulator.

One of the INC instructions operates on the 16-bit Data Pointer. The Data Pointer is used to generate 16-bit addresses for external memory, so being able to increment it in one 16-bit operation is a useful feature.

The MUL AB instruction multiplies the Accumulator by the data in the B register and puts the 16-bit product into the concatenated B and Accumulator registers.

Table 2. A List of the MCS-51 Arithmetic Instructions

Mnemonic	Operation	Addressing Modes				Execution Time (μ s)
		Dlr	Ind	Reg	Imm	
ADD A, <byte>	A = A + <byte>	X	X	X	X	1
ADDC A, <byte>	A = A + <byte> + C	X	X	X	X	1
SUBB A, <byte>	A = A - <byte> - C	X	X	X	X	1
INC A	A = A + 1	Accumulator only				1
INC <byte>	<byte> = <byte> + 1	X	X	X		1
INC DPTR	DPTR = DPTR + 1	Data Pointer only				2
DEC A	A = A - 1	Accumulator only				1
DEC <byte>	<byte> = <byte> - 1	X	X	X		1
MUL AB	B:A = B x A	ACC and B only				4
DIV AB	A = Int [A/B] B = Mod [A/B]	ACC and B only				4
DA A	Decimal Adjust	Accumulator only				1

The DIV AB instruction divides the Accumulator by the data in the B register and leaves the 8-bit quotient in the Accumulator, and the 8-bit remainder in the B register.

Oddly enough, DIV AB finds less use in arithmetic "divide" routines than in radix conversions and programmable shift operations. An example of the use of DIV AB in a radix conversion will be given later. In shift operations, dividing a number by 2ⁿ shifts its n bits to the right. Using DIV AB to perform the division

completes the shift in 4 μ s and leaves the B register holding the bits that were shifted out.

The DA A instruction is for BCD arithmetic operations. In BCD arithmetic, ADD and ADDC instructions should always be followed by a DA A operation, to ensure that the result is also in BCD. Note that DA A will not convert a binary number to BCD. The DA A operation produces a meaningful result only as the second step in the addition of two BCD bytes.

Table 3. A List of the MCS-51 Logical Instructions

Mnemonic	Operation	Addressing Modes				Execution Time (μ s)
		Dlr	Ind	Reg	Imm	
ANL A, <byte>	A = A .AND. <byte>	X	X	X	X	1
ANL <byte>, A	<byte> = <byte> .AND. A	X				1
ANL <byte>, #data	<byte> = <byte> .AND. #data	X				2
ORL A, <byte>	A = A .OR. <byte>	X	X	X	X	1
ORL <byte>, A	<byte> = <byte> .OR. A	X				1
ORL <byte>, #data	<byte> = <byte> .OR. #data	X				2
XRL A, <byte>	A = A .XOR. <byte>	X	X	X	X	1
XRL <byte>, A	<byte> = <byte> .XOR. A	X				1
XRL <byte>, #data	<byte> = <byte> .XOR. #data	X				2
CPL A	A = 00H	Accumulator only				1
CPL A	A = .NOT. A	Accumulator only				1
RL A	Rotate ACC Left 1 bit	Accumulator only				1
RLC A	Rotate Left through Carry	Accumulator only				1
RR A	Rotate ACC Right 1 bit	Accumulator only				1
RRC A	Rotate Right through Carry	Accumulator only				1
SWAP A	Swap Nibbles in A	Accumulator only				1

Logical Instructions

Table 3 shows the list of MCS-51 logical instructions. The instructions that perform Boolean operations (AND, OR, Exclusive OR, NOT) on bytes perform the operation on a bit-by-bit basis. That is, if the Accumulator contains 0010101B and <byte> contains 01010011B, then

```
ANL A, <byte>
```

will leave the Accumulator holding 00010001B.

The addressing modes that can be used to access the <byte> operand are listed in Table 3. Thus, the ANL A, <byte> instruction may take any of the forms

```
ANL A, 7FH      (direct addressing)
ANL A, @R1      (indirect addressing)
ANL A, R6        (register addressing)
ANL A, #53H     (immediate constant)
```

All of the logical instructions that are Accumulator-specific execute in 1 μ s (using a 12 MHz clock). The others take 2 μ s.

Note that Boolean operations can be performed on any byte in the lower 128 internal Data Memory space or the SFR space using direct addressing, without having to use the Accumulator. The XRL <byte>, #data instruction, for example, offers a quick and easy way to invert port bits, as in

```
XRL P1, #0FFH
```

If the operation is in response to an interrupt, not using the Accumulator saves the time and effort to stack it in the service routine.

The Rotate instructions (RL A, RLC A, etc.) shift the Accumulator 1 bit to the left or right. For a left rotation, the MSB rolls into the LSB position. For a right rotation, the LSB rolls into the MSB position.

Table 4. A List of the MCS-51 Data Transfer Instructions that Access Internal Data Memory Space

Mnemonic	Operation	Addressing Modes				Execution Time (μ s)
		Dlr	Ind	Reg	Imm	
MOV A, <src>	A = <src>	X	X	X	X	1
MOV <dest>, A	<dest> = A	X	X	X		1
MOV <dest>, <src>	<dest> = <src>	X	X	X	X	2
MOV DPTR, #data16	DPTR = 16-bit immediate constant.				X	2
PUSH <src>	INC SP; MOV "@SP", <src>	X				2
POP <dest>	MOV <dest>, "@SP"; DEC SP	X				2
XCH A, <byte>	ACC and <byte> exchange data	X	X	X		1
XCHD A, @R1	ACC and @R1 exchange low nibbles		X			1

The SWAP A instruction interchanges the high and low nibbles within the Accumulator. This is a useful operation in BCD manipulations. For example, if the Accumulator contains a binary number which is known to be less than 100, it can be quickly converted to BC by the following code:

```
MOV B, #10
DIV AB
SWAP A
ADD A, B
```

Dividing the number by 10 leaves the tens digit in the low nibble of the Accumulator, and the ones digit in the B register. The SWAP and ADD instructions move the tens digit to the high nibble of the Accumulator, and the ones digit to the low nibble.

Data Transfers

INTERNAL RAM

Table 4 shows the menu of instructions that are available for moving data around within the internal memory spaces, and the addressing modes that can be used with each one. With a 12 MHz clock, all of these instructions execute in either 1 or 2 μ s.

The MOV <dest>, <src> instruction allows data to be transferred between any two internal RAM or SFR locations without going through the Accumulator. Remember the Upper 128 bytes of data RAM can be accessed only by indirect addressing, and SFR space only by direct addressing.

Note that in all MCS-51 devices, the stack resides in on-chip RAM, and grows upwards. The PUSH instruction first increments the Stack Pointer (SP), then copies the byte into the stack. PUSH and POP use only direct addressing to identify the byte being saved or restored.

but the stack itself is accessed by indirect addressing using the SP register. This means the stack can go into the Upper 128, if they are implemented, but not into SFR space.

In devices that do not implement the Upper 128, if the SP points to the Upper 128, PUSHed bytes are lost, and POPped bytes are indeterminate.

The Data Transfer instructions include a 16-bit MOV that can be used to initialize the Data Pointer (DPTR) for look-up tables in Program Memory, or for 16-bit external Data Memory accesses.

The XCH A, <byte> instruction causes the Accumulator and addressed byte to exchange data. The XCHD A, @R1 instruction is similar, but only the low nibbles are involved in the exchange.

To see how XCH and XCHD can be used to facilitate data manipulations, consider first the problem of shifting an 8-digit BCD number two digits to the right. Figure 11 shows how this can be done using direct MOVs, and for comparison how it can be done using XCH instructions. To aid in understanding how the code works, the contents of the registers that are holding the BCD number and the content of the Accumulator are shown alongside each instruction to indicate their status after the instruction has been executed.

	2A	2B	2C	2D	2E	ACC
MOV A, 2EH	00	12	34	56	78	78
MOV 2EH, 2DH	00	12	34	56	56	78
MOV 2DH, 2CH	00	12	34	34	56	78
MOV 2CH, 2BH	00	12	12	34	56	78
MOV 2BH, #0	00	00	12	34	56	78
(a) Using direct MOVs: 14 bytes, 9 μ s						
	2A	2B	2C	2D	2E	ACC
CLR A	00	12	34	56	78	00
XCH A, 2BH	00	00	34	56	78	12
XCH A, 2CH	00	00	12	56	78	34
XCH A, 2DH	00	00	12	34	78	56
XCH A, 2EH	00	00	12	34	56	78
(b) Using XCHs: 9 bytes, 5 μ s						

Figure 11. Shifting a BCD Number Two Digits to the Right

After the routine has been executed, the Accumulator contains the two digits that were shifted out on the right. Doing the routine with direct MOVs uses 14 code bytes and 9 μ s of execution time (assuming a 12 MHz clock). The same operation with XCHs uses less code and executes almost twice as fast.

To right-shift by an odd number of digits, a one-digit shift must be executed. Figure 12 shows a sample of code that will right-shift a BCD number one digit, using the XCHD instruction. Again, the contents of the registers holding the number and of the Accumulator are shown alongside each instruction.

	2A	2B	2C	2D	2E	ACC
MOV R1, #2EH	00	12	34	56	78	XX
MOV R0, #2DH	00	12	34	56	78	XX
loop for R1 = 2EH:						
LOOP: MOV A, @R1	00	12	34	56	78	78
XCHD A, @R0	00	12	34	58	78	76
SWAP A	00	12	34	58	78	67
MOV @R1, A	00	12	34	58	67	67
DEC R1	00	12	34	58	67	67
DEC R0	00	12	34	58	67	67
CJNE R1, #2AH, LOOP	00	12	34	58	67	67
loop for R1 = 2DH:						
loop for R1 = 2CH:	00	18	23	45	67	23
loop for R1 = 2BH:	08	01	23	45	67	01
CLR A	08	01	23	45	67	00
XCH A, 2AH	00	01	23	45	67	08

Figure 12. Shifting a BCD Number One Digit to the Right

First, pointers R1 and R0 are set up to point to the two bytes containing the last four BCD digits. Then a loop is executed which leaves the last byte, location 2EH, holding the last two digits of the shifted number. The pointers are decremented, and the loop is repeated for location 2DH. The CJNE instruction (Compare and Jump if Not Equal) is a loop control that will be described later.

The loop is executed from LOOP to CJNE for R1 = 2EH, 2DH, 2CH and 2BH. At that point the digit that was originally shifted out on the right has propagated to location 2AH. Since that location should be left with 0s, the lost digit is moved to the Accumulator.

EXTERNAL RAM

Table 5 shows a list of the Data Transfer instructions that access external Data Memory. Only indirect addressing can be used. The choice is whether to use a one-byte address, @Ri, where Ri can be either R0 or R1 of the selected register bank, or a two-byte address, @DPTR. The disadvantage to using 16-bit addresses if only a few K bytes of external RAM are involved is that 16-bit addresses use all 8 bits of Port 2 as address bus. On the other hand, 8-bit addresses allow one to address a few K bytes of RAM, as shown in Figure 5, without having to sacrifice all of Port 2.

All of these instructions execute in 2 μ s, with a 12 MHz clock.

Table 5. A List of the MCS-51 Data Transfer Instructions that Access External Data Memory Space

Address Width	Mnemonic	Operation	Execution Time (μ s)
8 bits	MOVX A, @Ri	Read external RAM @Ri	2
8 bits	MOVX @Ri, A	Write external RAM @Ri	2
16 bits	MOVX A, @DPTR	Read external RAM @DPTR	2
16 bits	MOVX @DPTR, A	Write external RAM @DPTR	2

Note that in all external Data RAM accesses, the Accumulator is always either the destination or source of the data.

The read and write strobes to external RAM are activated only during the execution of a MOVX instruction. Normally these signals are inactive, and in fact if they're not going to be used at all, their pins are available as extra I/O lines. More about that later.

LOOKUP TABLES

Table 6 shows the two instructions that are available for reading lookup tables in Program Memory. Since these instructions access only Program Memory, the lookup tables can only be read, not updated. The mnemonic is MOVC for "move constant".

If the table access is to external Program Memory, then the read strobe is PSEN.

Table 6. The MCS-51 Lookup Table Read Instructions

Mnemonic	Operation	Execution Time (μ s)
MOVC A, @A+DPTR	Read Pgm Memory at (A+DPTR)	2
MOVC A, @A+PC	Read Pgm Memory at (A+PC)	2

The first MOVC instruction in Table 6 can accommodate a table of up to 256 entries, numbered 0 through 255. The number of the desired entry is loaded into the Accumulator, and the Data Pointer is set up to point to beginning of the table. Then

MOVC A, @A+DPTR

copies the desired table entry into the Accumulator.

The other MOVC instruction works the same way, except the Program Counter (PC) is used as the table base, and the table is accessed through a subroutine. First the number of the desired entry is loaded into the Accumulator, and the subroutine is called:

MOV A, ENTRY_NUMBER
CALL TABLE

The subroutine "TABLE" would look like this:

TABLE: MOVC A, @A+PC
RET

The table itself immediately follows the RET (return) instruction in Program Memory. This type of table can have up to 255 entries, numbered 1 through 255. Number 0 can not be used, because at the time the MOVC instruction is executed, the PC contains the address of the RET instruction. An entry numbered 0 would be the RET opcode itself.

Boolean Instructions

MCS-51 devices contain a complete Boolean (single-bit) processor. The internal RAM contains 128 addressable bits, and the SFR space can support up to 128 other addressable bits. All of the port lines are bit-addressable, and each one can be treated as a separate single-bit port. The instructions that access these bits are not just conditional branches, but a complete menu of move, set, clear, complement, OR, and AND instructions. These kinds of bit operations are not easily obtained in other architectures with any amount of byte-oriented software.

Table 7. A List of the MCS®-51 Boolean Instructions

Mnemonic	Operation	Execution Time (μs)
ANL C,bit	C = C.AND. bit	2
ANL C,/bit	C = C.AND. .NOT. bit	2
ORL C,bit	C = C.OR. bit	2
ORL C,/bit	C = C.OR. .NOT. bit	2
MOV C,bit	C = bit	1
MOV bit,C	bit = C	2
CLR C	C = 0	1
CLR bit	bit = 0	1
SETB C	C = 1	1
SETB bit	bit = 1	1
CPL C	C = .NOT. C	1
CPL bit	bit = .NOT. bit	1
JC rel	Jump if C = 1	2
JNC rel	Jump if C = 0	2
JB bit,rel	Jump if bit = 1	2
JNB bit,rel	Jump if bit = 0	2
JBC bit,rel	Jump if bit = 1; CLR bit	2

The instruction set for the Boolean processor is shown in Table 7. All bit accesses are by direct addressing. Bit addresses 00H through 7FH are in the Lower 128, and bit addresses 80H through FFH are in SFR space.

Note how easily an internal flag can be moved to a port pin:

```
MOV C,FLAG
MOV P1.0,C
```

In this example, FLAG is the name of any addressable bit in the Lower 128 or SFR space. An I/O line (the LSB of Port 1, in this case) is set or cleared depending on whether the flag bit is 1 or 0.

The Carry bit in the PSW is used as the single-bit Accumulator of the Boolean processor. Bit instructions that refer to the Carry bit as C assemble as Carry-specific instructions (CLR C, etc). The Carry bit also has a direct address, since it resides in the PSW register, which is bit-addressable.

Note that the Boolean instruction set includes ANL and ORL operations, but not the XRL (Exclusive OR) operation. An XRL operation is simple to implement in software. Suppose, for example, it is required to form the Exclusive OR of two bits:

```
C = bit1.XRL. bit2
```

The software to do that could be as follows:

```
MOV C,bit1
JNB bit2,OVER
CPL C
OVER: (continue)
```

First, bit1 is moved to the Carry. If bit2 = 0, then C now contains the correct result. That is, bit1.XRL. bit2 = bit1 if bit2 = 0. On the other hand, if bit2 = 1 C now contains the complement of the correct result. It need only be inverted (CPL C) to complete the operation.

This code uses the JNB instruction, one of a series of bit-test instructions which execute a jump if the addressed bit is set (JC, JB, JBC) or if the addressed bit is not set (JNC, JNB). In the above case, bit2 is being tested, and if bit2 = 0 the CPL C instruction is jumped over.

JBC executes the jump if the addressed bit is set, and also clears the bit. Thus a flag can be tested and cleared in one operation.

All the PSW bits are directly addressable, so the Parity bit, or the general purpose flags, for example, are also available to the bit-test instructions.

RELATIVE OFFSET

The destination address for these jumps is specified to the assembler by a label or by an actual address in Program Memory. However, the destination address assembles to a relative offset byte. This is a signed (two's complement) offset byte which is added to the PC in two's complement arithmetic if the jump is executed.

The range of the jump is therefore -128 to +127 Program Memory bytes relative to the first byte following the instruction.

Jump Instructions

Table 8 shows the list of unconditional jumps.

Table 8. Unconditional Jumps in MCS®-51 Devices

Mnemonic	Operation	Execution Time (μs)
JMP addr	Jump to addr	2
JMP @A+DPTR	Jump to A+DPTR	2
CALL addr	Call subroutine at addr	2
RET	Return from subroutine	2
RETI	Return from interrupt	2
NOP	No operation	1

The Table lists a single "JMP addr" instruction, but in fact there are three—SJMP, LJMP and AJMP—which differ in the format of the destination address. JMP is a generic mnemonic which can be used if the programmer does not care which way the jump is encoded.

The SJMP instruction encodes the destination address as a relative offset, as described above. The instruction is 2 bytes long, consisting of the opcode and the relative offset byte. The jump distance is limited to a range of -128 to +127 bytes relative to the instruction following the SJMP.

The LJMP instruction encodes the destination address as a 16-bit constant. The instruction is 3 bytes long, consisting of the opcode and two address bytes. The destination address can be anywhere in the 64K Program Memory space.

The AJMP instruction encodes the destination address as an 11-bit constant. The instruction is 2 bytes long, consisting of the opcode, which itself contains 3 of the 11 address bits, followed by another byte containing the low 8 bits of the destination address. When the instruction is executed, these 11 bits are simply substituted for the low 11 bits in the PC. The high 5 bits stay the same. Hence the destination has to be within the same 2K block as the instruction following the AJMP.

In all cases the programmer specifies the destination address to the assembler in the same way: as a label or as a 16-bit constant. The assembler will put the destination address into the correct format for the given instruction. If the format required by the instruction will not support the distance to the specified destination address, a "Destination out of range" message is written into the List file.

The JMP @A+DPTR instruction supports case jumps. The destination address is computed at execution time as the sum of the 16-bit DPTR register and

the Accumulator. Typically, DPTR is set up with the address of a jump table, and the Accumulator is given an index to the table. In a 5-way branch, for example, an integer 0 through 4 is loaded into the Accumulator. The code to be executed might be as follows:

```
MOV DPTR,#JUMP_TABLE
MOV A,INDEX_NUMBER
RL A
JMP @A+DPTR
```

The RL A instruction converts the index number (0 through 4) to an even number on the range 0 through 8, because each entry in the jump table is 2 bytes long:

```
JUMP_TABLE:
AJMP CASE_0
AJMP CASE_1
AJMP CASE_2
AJMP CASE_3
AJMP CASE_4
```

Table 8 shows a single "CALL addr" instruction, but there are two of them—LCALL and ACALL—which differ in the format in which the subroutine address is given to the CPU. CALL is a generic mnemonic which can be used if the programmer does not care which way the address is encoded.

The LCALL instruction uses the 16-bit address format, and the subroutine can be anywhere in the 64K Program Memory space. The ACALL instruction uses the 11-bit format, and the subroutine must be in the same 2K block as the instruction following the ACALL.

In any case the programmer specifies the subroutine address to the assembler in the same way: as a label or as a 16-bit constant. The assembler will put the address into the correct format for the given instructions.

Subroutines should end with a RET instruction, which returns execution to the instruction following the CALL.

RETI is used to return from an interrupt service routine. The only difference between RET and RETI is that RETI tells the interrupt control system that the interrupt in progress is done. If there is no interrupt in progress at the time RETI is executed, then the RETI is functionally identical to RET.

Table 9 shows the list of conditional jumps available to the MCS-51 user. All of these jumps specify the destination address by the relative offset method; and so are limited to a jump distance of -128 to +127 bytes from the instruction following the conditional jump instruction. Important to note, however, the user specifies to the assembler the actual destination address the same way as the other jumps: as a label or a 16-bit constant.

states and phases for various kinds of instructions. Normally two program fetches are generated during each machine cycle, even if the instruction being executed doesn't require it. If the instruction being executed doesn't need more code bytes, the CPU simply ignores the data fetch, and the Program Counter is not incremented.

Execution of a one-cycle instruction (Figure 15A and B) begins during State 1 of the machine cycle, when the opcode is latched into the Instruction Register. A second fetch occurs during S4 of the same machine cycle. Execution is complete at the end of State 6 of this machine cycle.

The MOVX instructions take two machine cycles to execute. No program fetch is generated during the second cycle of a MOVX instruction. This is the only time program fetches are skipped. The fetch/execute sequence for MOVX instructions is shown in Figure 15B.

The fetch/execute sequences are the same whether the Program Memory is internal or external to the chip. Execution times do not depend on whether the Program Memory is internal or external.

Figure 16 shows the signals and timing involved in program fetches when the Program Memory is external. If Program Memory is external, then the Program Memory read strobe PSEN is normally activated twice per machine cycle, as shown in Figure 16(A).

If an access to external Data Memory occurs, as shown in Figure 16(B), two PSENs are skipped, because the address and data bus are being used for the Data Memory access.

Note that a Data Memory bus cycle takes twice as much time as a Program Memory bus cycle. Figure 16 shows the relative timing of the addresses being emitted at Ports 0 and 2, and of ALE and PSEN. ALE is used to latch the low address byte from P0 into the address latch.

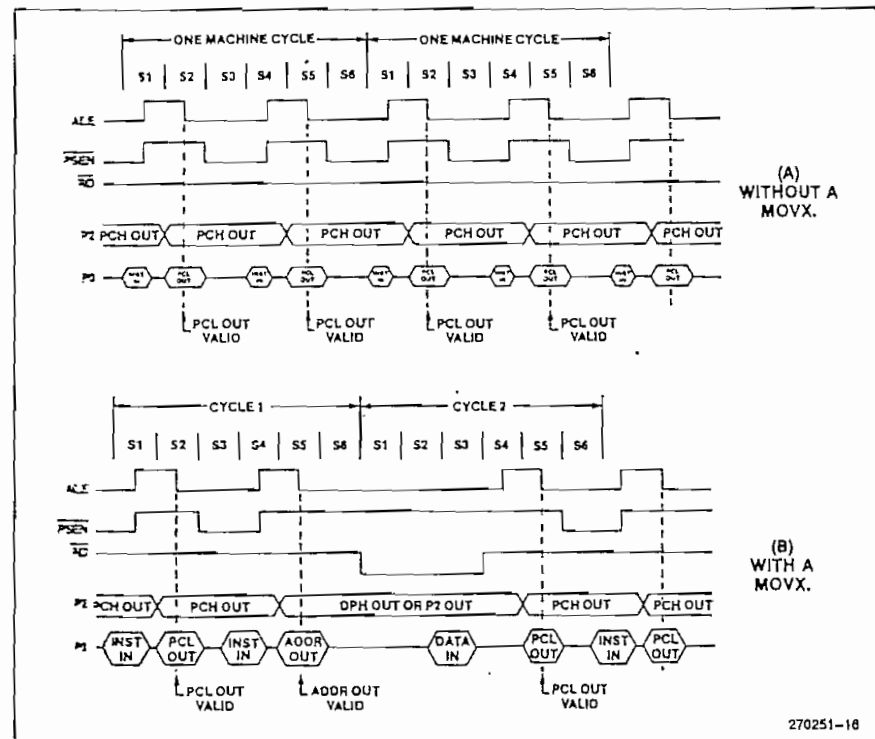


Figure 16. Bus Cycles in MCS-51 Devices Executing from External Program Memory

When the CPU is executing from internal Program Memory, PSEN is not activated, and program addresses are not emitted. However, ALE continues to be activated twice per machine cycle and so is available as a clock output signal. Note, however, that one ALE is skipped during the execution of the MOVX instruction.

Interrupt Structure

The 8051 core provides 5 interrupt sources: 2 external interrupts, 2 timer interrupts, and the serial port interrupt. What follows is an overview of the interrupt structure for the 8051. Other MCS-51 devices have additional interrupt sources and vectors as shown in Table 1. Refer to the appropriate chapters on other devices for further information on their interrupts.

INTERRUPT ENABLES

Each of the interrupt sources can be individually enabled or disabled by setting or clearing a bit in the SFR

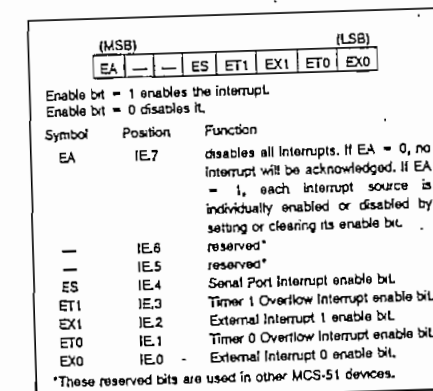


Figure 17. IE (Interrupt Enable) Register in the 8051

named IE (Interrupt Enable). This register also contains a global disable bit, which can be cleared to disable all interrupts at once. Figure 17 shows the IE register for the 8051.

INTERRUPT PRIORITIES

Each interrupt source can also be individually programmed to one of two priority levels by setting or clearing a bit in the SFR named IP (Interrupt Priority). Figure 18 shows the IP register in the 8051.

A low-priority interrupt can be interrupted by a high-priority interrupt, but not by another low-priority interrupt. A high-priority interrupt can't be interrupted by any other interrupt source.

If two interrupt requests of different priority levels are received simultaneously, the request of higher priority level is serviced. If interrupt requests of the same priority level are received simultaneously, an internal polling sequence determines which request is serviced. Thus within each priority level there is a second priority structure determined by the polling sequence.

Figure 19 shows, for the 8051, how the IE and IP registers and the polling sequence work to determine which if any interrupt will be serviced.

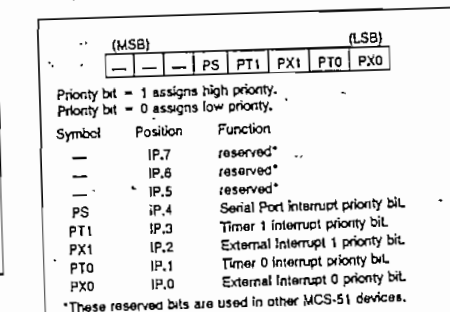


Figure 18. IP (Interrupt Priority) Register in the 8051

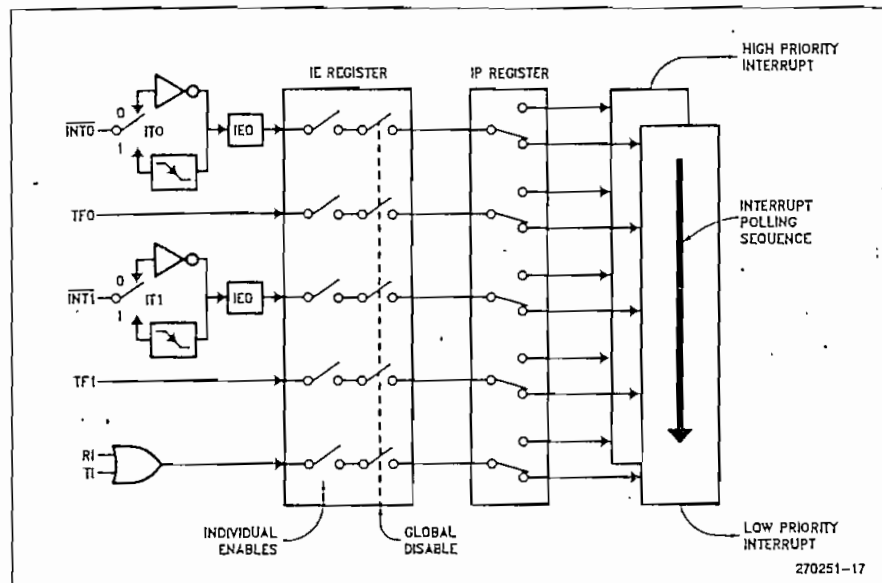


Figure 19. 8051 Interrupt Control System

In operation, all the interrupt flags are latched into the interrupt control system during State 5 of every machine cycle. The samples are polled during the following machine cycle. If the flag for an enabled interrupt is found to be set (1), the interrupt system generates an LCALL to the appropriate location in Program Memory, unless some other condition blocks the interrupt. Several conditions can block an interrupt, among them that an interrupt of equal or higher priority level is already in progress.

The hardware-generated LCALL causes the contents of the Program Counter to be pushed onto the stack, and reloads the PC with the beginning address of the service routine. As previously noted (Figure 3), the service routine for each interrupt begins at a fixed location.

Only the Program Counter is automatically pushed onto the stack, not the PSW or any other register. Having only the PC be automatically saved allows the programmer to decide how much time to spend saving which other registers. This enhances the interrupt response time, albeit at the expense of increasing the programmer's burden of responsibility. As a result, many interrupt functions that are typical in control applications—toggling a port pin, for example, or reloading a timer, or unloading a serial buffer—can often be com-

pleted in less time than it takes other architectures to commence them.

SIMULATING A THIRD PRIORITY LEVEL IN SOFTWARE

Some applications require more than the two priority levels that are provided by on-chip hardware in MCS-51 devices. In these cases, relatively simple software can be written to produce the same effect as a third priority level.

First, interrupts that are to have higher priority than 1 are assigned to priority 1 in the IP (Interrupt Priority) register. The service routines for priority 1 interrupts that are supposed to be interruptible by "priority 2" interrupts are written to include the following code:

```

PUSH    IE
MOV     IE,#MASK
CALL    LABEL
.....
(execute service routine)
.....
POP     IE
RET
LABEL: RETI

```

As soon as any priority 1 interrupt is acknowledged, the IE (Interrupt Enable) register is re-defined so as to disable all but "priority 2" interrupts. Then, a CALL to LABEL executes the RETI instruction, which clears the priority 1 interrupt-in-progress flip-flop. At this point any priority 1 interrupt that is enabled can be serviced, but only "priority 2" interrupts are enabled.

POPPing IE restores the original enable byte. Then a normal RET (rather than another RETI) is used to terminate the service routine. The additional software adds 10 μ s (at 12 MHz) to priority 1 interrupts.

ADDITIONAL REFERENCES

The following application notes are found in the *Embedded Control Applications* handbook. (Order Number: 270648)

1. AP-69 "An Introduction to the Intel MCS-51 Single-Chip Microcomputer Family"
2. AP-70 "Using the Intel MCS-51 Boolean Processing Capabilities"

MCS®-51 INSTRUCTION SET

Table 10. 8051 Instruction Set Summary

Interrupt Response Time: Refer to Hardware Description Chapter.

Instructions that Affect Flag Settings⁽¹⁾

Instruction	Flag	Instruction	Flag
	C OV AC		C OV AC
ADD	X X X	CLRC	C
ADDC	X X X	CPLC	X
SUBB	X X X	ANL C,bit	X
MUL	O X	ANL C,bit	X
DIV	O X	ORL C,bit	X
DA	X	ORL C,bit	X
RRC	X	MOVC,bit	X
RLC	X	CJNE	X
SETB C	1		

⁽¹⁾Note that operations on SFR byte address 208 or bit addresses 209-215 (i.e., the PSW or bits in the PSW) will also affect flag settings.

Note on instruction set and addressing modes:

- Rn — Register R7-R0 of the currently selected Register Bank.
- direct — 8-bit internal data location's address. This could be an Internal Data RAM location (0-127) or a SFR (i.e., I/O port, control register, status register, etc. (128-255)).
- @Ri — 8-bit internal data RAM location (0-255) addressed indirectly through register Ri or R0.
- #data — 8-bit constant included in instruction.
- #data 16 — 16-bit constant included in instruction.
- addr 16 — 16-bit destination address. Used by LCALL & LJMP. A branch can be anywhere within the 64K-byte Program Memory address space.
- addr 11 — 11-bit destination address. Used by ACALL & AJMP. The branch will be within the same 2K-byte page of program memory as the first byte of the following instruction.
- rel — Signed (two's complement) 8-bit offset byte. Used by SJMP and all conditional jumps. Range is -128 to +127 bytes relative to first byte of the following instruction.
- bit — Direct Addressed bit in Internal Data RAM or Special Function Register.

Mnemonic	Description	Byte	Oscillator Period
ARITHMETIC OPERATIONS			
ADD A,Rn	Add register to Accumulator	1	12
ADD A,direct	Add direct byte to Accumulator	2	12
ADD A,@Ri	Add indirect RAM to Accumulator	1	12
ADD A,#data	Add immediate data to Accumulator	2	12
ADDC A,Rn	Add register to Accumulator with Carry	1	12
ADDC A,direct	Add direct byte to Accumulator with Carry	2	12
ADDC A,@Ri	Add indirect RAM to Accumulator with Carry	1	12
ADDC A,#data	Add immediate data to Acc with Carry	2	12
SUBB A,Rn	Subtract Register from Acc with borrow	1	12
SUBB A,direct	Subtract direct byte from Acc with borrow	2	12
SUBB A,@Ri	Subtract indirect RAM from ACC with borrow	1	12
SUBB A,#data	Subtract immediate data from Acc with borrow	2	12
INC A	Increment Accumulator	1	12
INC Rn	Increment register	1	12
INC direct	Increment direct byte	2	12
INC @Ri	Increment direct RAM	1	12
DEC A	Decrement Accumulator	1	12
DEC Rn	Decrement Register	1	12
DEC direct	Decrement direct byte	2	12
DEC @Ri	Decrement Indirect RAM	1	12

All mnemonics copyrighted ©Intel Corporation 1980

Table 10. 8051 Instruction Set Summary (Continued)

Mnemonic	Description	Byte	Oscillator Period
ARITHMETIC OPERATIONS (Continued)			
INC DPTR	Increment Data Pointer	1	24
MUL AB	Multiply A & B	1	48
DIV AB	Divide A by B	1	48
DA A	Decimal Adjust Accumulator	1	12
LOGICAL OPERATIONS			
ANL A,Rn	AND Register to Accumulator	1	12
ANL A,direct	AND direct byte to Accumulator	2	12
ANL A,@Ri	AND indirect RAM to Accumulator	1	12
ANL A,#data	AND immediate data to Accumulator	2	12
ANL direct,A	AND Accumulator to direct byte	2	12
ANL direct,#data	AND immediate data to direct byte	3	24
ORL A,Rn	OR register to Accumulator	1	12
ORL A,direct	OR direct byte to Accumulator	2	12
ORL A,@Ri	OR indirect RAM to Accumulator	1	12
ORL A,#data	OR immediate data to Accumulator	2	12
ORL direct,A	OR Accumulator to direct byte	2	12
ORL direct,#data	OR immediate data to direct byte	3	24
XRL A,Rn	Exclusive-OR register to Accumulator	1	12
XRL A,direct	Exclusive-OR direct byte to Accumulator	2	12
XRL A,@Ri	Exclusive-OR indirect RAM to Accumulator	1	12
XRL A,#data	Exclusive-OR immediate data to Accumulator	2	12
XRL direct,A	Exclusive-OR Accumulator to direct byte	2	12
XRL direct,#data	Exclusive-OR immediate data to direct byte	3	24
CLR A	Accumulator Clear	1	12
CPL A	Accumulator Complement	1	12
LOGICAL OPERATIONS (Continued)			
RL A	Rotate Accumulator Left	1	12
RLC A	Rotate Accumulator Left through the Carry	1	12
RR A	Rotate Accumulator Right	1	12
RRC A	Rotate Accumulator Right through the Carry	1	12
SWAP A	Swap nibbles within the Accumulator	1	12
DATA TRANSFER			
MOV A,Rn	Move register to Accumulator	1	12
MOV A,direct	Move direct byte to Accumulator	2	12
MOV A,@Ri	Move indirect RAM to Accumulator	1	12
MOV A,#data	Move immediate data to Accumulator	2	12
MOV Rn,A	Move Accumulator to register	1	12
MOV Rn,direct	Move direct byte to register	2	24
MOV Rn,#data	Move immediate data to register	2	12
MOV direct,A	Move Accumulator to direct byte	2	12
MOV direct,Rn	Move register to direct byte	2	24
MOV direct,direct	Move direct byte to direct byte	3	24
MOV direct,@Ri	Move indirect RAM to direct byte	2	24
MOV direct,#data	Move immediate data to direct byte	3	24
MOV @Ri,A	Move Accumulator to Indirect RAM	1	12

All mnemonics copyrighted ©Intel Corporation 1980

Table 10. 8051 Instruction Set Summary (Continued)

Mnemonic	Description	Byte	Oscillator Period
DATA TRANSFER (Continued)			
MOV @Ri,direct	Move direct byte to indirect RAM	2	24
MOV @Ri,#data	Move immediate data to indirect RAM	2	12
MOV DPTR,#data16	Load Data Pointer with a 16-bit constant	3	24
MOVC A,@A+DPTR	Move Code byte relative to DPTR to Acc	1	24
MOVC A,@A+PC	Move Code byte relative to PC to Acc	1	24
MOVX A,@RI	Move External RAM (8-bit addr) to Acc	1	24
MOVX A,@DPTR	Move External RAM (16-bit addr) to Acc	1	24
MOVX @RI,A	Move Acc to External RAM (8-bit addr)	1	24
MOVX @DPTR,A	Move Acc to External RAM (16-bit addr)	1	24
PUSH direct	Push direct byte onto stack	2	24
POP direct	Pop direct byte from stack	2	24
XCH A,Rn	Exchange register with Accumulator	1	12
XCH A,direct	Exchange direct byte with Accumulator	2	12
XCH A,@RI	Exchange Indirect RAM with Accumulator	1	12
XCHD A,@RI	Exchange low-order Digit Indirect RAM with Acc	1	12

Mnemonic	Description	Byte	Oscillator Period
BOOLEAN VARIABLE MANIPULATION			
CLR C	Clear Carry	1	12
CLR bit	Clear direct bit	2	12
SETB C	Set Carry	1	12
SETB bit	Set direct bit	2	12
CPL C	Complement Carry	1	12
CPL bit	Complement direct bit	2	12
ANL C,bit	AND direct bit to CARRY	2	24
ANL C,/bit	AND complement of direct bit to Carry	2	24
ORL C,bit	OR direct bit to Carry	2	24
ORL C,/bit	OR complement of direct bit to Carry	2	24
MOV C,bit	Move direct bit to Carry	2	12
MOV bit,C	Move Carry to direct bit	2	24
JC rel	Jump if Carry is set	2	24
JNC rel	Jump if Carry not set	2	24
JB bit,rel	Jump if direct Bit is set	3	24
JNB bit,rel	Jump if direct Bit is Not set	3	24
JBC bit,rel	Jump if direct Bit is set & clear bit	3	24
PROGRAM BRANCHING			
ACALL addr11	Absolute Subroutine Call	2	24
LCALL addr16	Long Subroutine Call	3	24
RET	Return from Subroutine	1	24
RETI	Return from Interrupt	1	24
AJMP addr11	Absolute Jump	2	24
LJMP addr16	Long Jump	3	24
SJMP rel	Short Jump (relative addr)	2	24

All mnemonics copyrighted ©Intel Corporation 1980

Table 10. 8051 Instruction Set Summary (Continued)

Mnemonic	Description	Byte	Oscillator Period
PROGRAM BRANCHING (Continued)			
JMP @A+DPTR	Jump indirect relative to the DPTR	1	24
JZ rel	Jump if Accumulator is Zero	2	24
JNZ rel	Jump if Accumulator is Not Zero	2	24
CJNE A,direct,rel	Compare direct byte to Acc and Jump if Not Equal	3	24
CJNE A,#data,rel	Compare immediate to Acc and Jump if Not Equal	3	24

Mnemonic	Description	Byte	Oscillator Period
PROGRAM BRANCHING (Continued)			
CJNE Rn,#data,rel	Compare immediate to register and Jump if Not Equal	3	24
CJNE @RI,#data,rel	Compare immediate to indirect and Jump if Not Equal	3	24
DJNZ Rn,rel	Decrement register and Jump if Not Zero	2	24
DJNZ direct,rel	Decrement direct byte and Jump if Not Zero	3	24
NOP	No Operation	1	12

All mnemonics copyrighted ©Intel Corporation 1980

Features

- 200 ns Access Times at 0 to 70°C
- Programmed Using Intelligent Algorithm
 - Typically 5 ms/byte Programming Time
 - 2 Minutes for 27128 (5143)
 - 1 Minute for 2764 (5133)
- JEDEC Approved Byte-wide Pin Configuration
 - 2764 8K x 8 Organization
 - 27128 16K x 8 Organization
- Low Power Dissipation
 - 100 mA Active Current
 - 30 mA Standby Current
- Extended Temperature Range Available
- Silicon Signature™

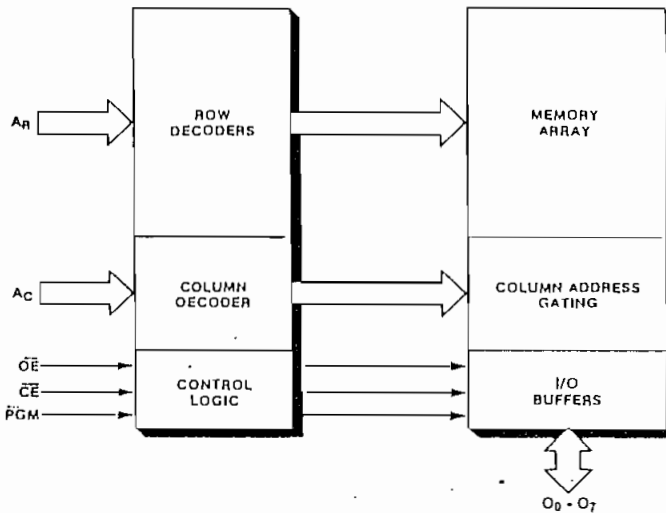
Description

SEEQ's 2764 (5133) and 27128 (5143) are ultraviolet light erasable EPROMs which are organized 8K x 8 and 16K x 8 respectively. They are pin for pin compatible to JEDEC approved 64K and 128K EPROMs in all operational/programming modes. Both devices have access times as fast as 200 ns over the 0 to 70°C temperature and V_{CC} tolerance range. The access time is achieved without sacrificing power since the maximum active and standby currents are 100 mA and 30 mA respectively. The 200 ns allows higher system efficiency by eliminating the need for wait states in today's 8- or 16-bit microcomputers.

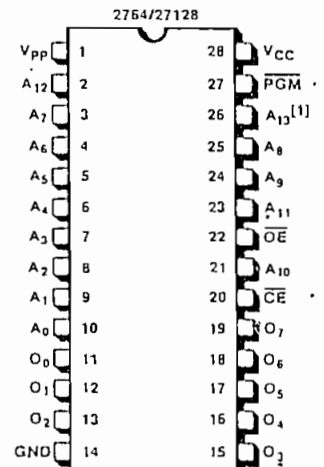
Initially, and after erasure, all bits are in the "1" state. Data is programmed by applying 21 V to V_{PP} and a TTL "0" to pin 27 (program pin). The 2764 (5133) and 27128 (5143) may be programmed with an intelligent

(continued on page 2)

Block Diagram



Pin Configuration



NOTE 1: PIN 26 IS A NO CONNECT ON THE 2764.

Mode Selection

MODE	PINS	$\overline{CE}$ (20)	$\overline{OE}$ (22)	$\overline{PGM}$ (27)	V_{PP} (1)	V_{CC} (28)	Outputs (11-13, 15-19)
Read		V_{IL}	V_{IL}	V_{IH}	V_{CC}	V_{CC}	D_{OUT}
Standby		V_{IH}	X	X	V_{CC}	V_{CC}	High Z
Program		V_{IL}	V_{IH}	V_{IL}	V_{PP}	V_{CC}	D_{IN}
Program Verify		V_{IL}	V_{IL}	V_{IH}	V_{PP}	V_{CC}	D_{OUT}
Program Inhibit		V_{IH}	X	X	V_{PP}	V_{CC}	High Z
Silicon Signature™		V_{IL}	V_{IL}	V_{IH}	V_{CC}	V_{CC}	Encoded Data

X can be either V_{IL} or V_{IH}

For Silicon Signature™: A_0 - A_3 are toggled. $A_4 = V_{IL}$, $A_9 = 12V$, all other addresses are at any TTL level.

Pin Names

A_C	ADDRESSES — COLUMN (LSB)
A_R	ADDRESSES — ROW
$\overline{CE}$	CHIP ENABLE
$\overline{OE}$	OUTPUT ENABLE
$O_0 - O_7$	OUTPUTS
$\overline{PGM}$	PROGRAM

2764 (5133)
27128 (5143)
 PRODUCT DESCRIPTION

algorithm that is now available on commercial programmers. The programming time is typically 5 ms/byte or 2 minutes for all 16K bytes of the 27128. The 2764 requires only half of this time, about a minute for 8K bytes. This faster time improves manufacturing throughput time by hours over conventional 50 ms algorithms. Commercial programmers (e.g. Data I/O, Pro-log, Digelec, Kontron, and Stag) have implemented this fast algorithm for SEEQ's EPROMs. If desired, both EPROMs may be

programmed using the conventional 50 ms programming specification of older generation EPROMs.

Incorporated on SEEQ's EPROMs is Silicon Signature™. Silicon Signature contains encoded data which identifies SEEQ as the EPROM manufacturer, the product's fab location, and programming information. This data is encoded in ROM to prevent erasure by ultraviolet light.

Absolute Maximum Stress Ratings

Temperature

Storage -65° C to +150° C

Under Bias -10° C to +80° C

All Inputs or Outputs with

Respect to Ground +7V to -0.6V

V_{PP} During Programming with

Respect to Ground +22V to -0.6V

Voltage on A₉ with

Respect to Ground +15.5V to -0.6V

*COMMENT: Stresses above those listed under "Absolute Maximum Ratings" may cause permanent damage to the device. This is a stress rating only and functional operation of the device at these or any other conditions above those indicated in the operational sections of this specification is not implied. Exposure to absolute maximum rating conditions for extended periods may affect device reliability.

Recommended Operating Conditions (27XX = 2764 and 27128, all)

	27XX-200, 27XX-250 27XX-300, 27XX-450	27XX-2, 27XX-3, 27XX-4
V _{CC} Supply Voltage ¹	5 V ± 10%	5 V ± 5%
Temperature Range (Read Mode)	0 to 70° C	0 to 70° C
V _{PP} During Programming	21 ± 0.5 V	21 ± 0.5 V

DC Operating Characteristics During Read or Programming

Symbol	Parameter	Limits		Unit	Test Conditions
		Min.	Max.		
I _{IN}	Input Leakage Current		10	μA	V _{IN} = V _{CC} Max.
I _O	Output Leakage Current		10	μA	V _{OUT} = V _{CC} Max.
I _{PP} ¹	V _{PP} Current Read Mode		5	mA	V _{PP} = V _{CC} Max.
	Prog. Mode		30	mA	V _{PP} = 21.5V
I _{CC1} ¹	V _{CC} Standby Current		30	mA	$\overline{CE} = V_{IH}$
I _{CC2} ¹	V _{CC} Active Current		100	mA	$\overline{CE} = \overline{OE} = V_{IL}$
V _{IL}	Input Low Voltage	-0.1	0.8	V	
V _{IH}	Input High Voltage	2	V _{CC} + 1	V	
V _{OL}	Output Low Voltage		0.45	V	I _{OL} = 2.1 mA
V _{OH}	Output High Voltage	2.4		V	I _{OH} = -400 μA

NOTES:

- The 5133 and 5143 have the same dash numbers and operate with the same operating conditions as the 2764 and 27128 respectively. The specifications are exactly the same.
- V_{CC} must be applied simultaneously or before V_{PP} and removed simultaneously or after V_{PP}.

AC Operating Characteristics During Read

Symbol	Parameter	Limits (nsec)								Test Conditions
		27XX-2 27XX-200		27XX-250		27XX-3 27XX-300		27XX-4 27XX-450		
		Min.	Max.	Min.	Max.	Min.	Max.	Min.	Max.	
t _{ACC}	Address to Data Valid		200		250		300		450	$\overline{CE} = \overline{OE} = V_{IL}$
t _{CE}	Chip Enable to Data Valid		200		250		300		450	$\overline{OE} = V_{IL}$
t _{OE}	Output Enable to Data Valid		75		100		120		150	$\overline{CE} = V_{IL}$
t _{DF}	Output Enable to Output Float	0	60	0	60	0	105	0	130	$\overline{CE} = V_{IL}$
t _{OH}	Output Hold from Chip Enable, Addresses, or Output Enable whichever occurred first	0		0		0		0		$\overline{CE} = \overline{OE} = V_{IL}$

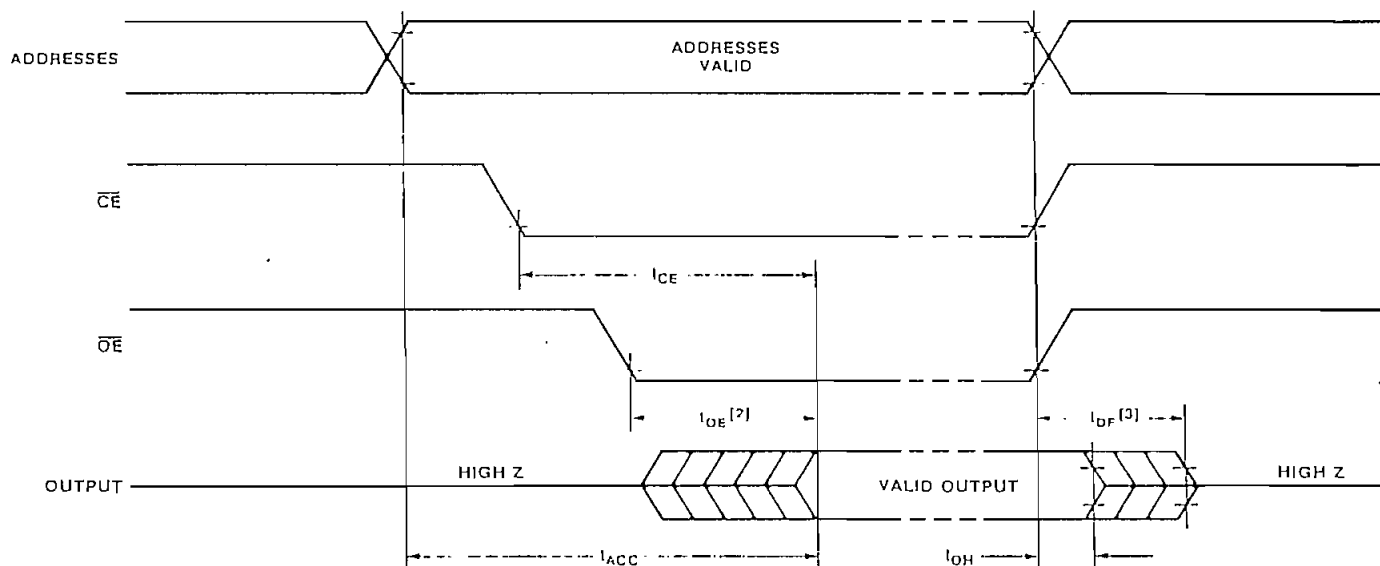
Capacitance^[1]

Symbol	Parameter	Typ.	Max.	Unit	Conditions
C_{IN}	Input Capacitance	4	6	pF	$V_{IN} = 0V$
C_{OUT}	Output Capacitance	8	12	pF	$V_{OUT} = 0V$

A.C. Test Conditions

Output Load: 1 TTL gate and $C_L = 100$ pF
 Input Rise and Fall Times: $\leq 20ns$
 Input Pulse Levels: 0.45V to 2.4V
 Timing Measurement Reference Level:
 Inputs 1V and 2V
 Outputs 0.8V and 2V

A.C. Waveforms



NOTES:

1. THIS PARAMETER IS SAMPLED AND IS NOT 100% TESTED.
2. $\overline{OE}$ MAY BE DELAYED UP TO $t_{ACC} - t_{OE}$ AFTER THE FALLING EDGE OF $\overline{CE}$ WITHOUT IMPACT ON t_{ACC} .
3. t_{DF} IS SPECIFIED FROM $\overline{OE}$ OR $\overline{CE}$, WHICHEVER OCCURS FIRST.

DMC16207

• Display Format (16 character × 2 line) • Display Fonts (5 × 8 dots) • Driving Method (1/4D)

ABSOLUTE MAXIMUM RATINGS

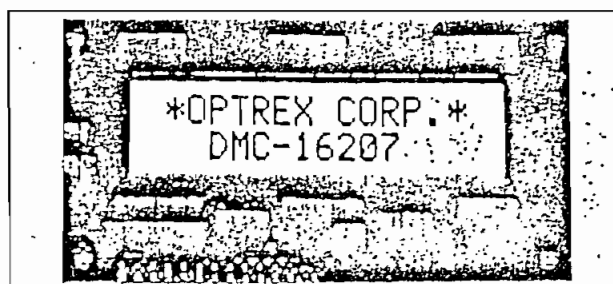
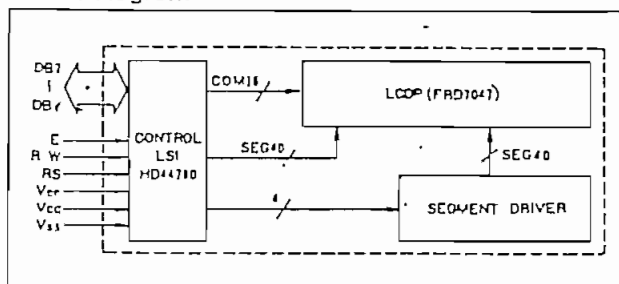
Item	Symbol	Test Condition	Standard Value			Unit
			min.	typ.	max.	
Power Supply Voltage for Logic	V _{CC} -V _{SS}	—	0	—	7	V
Power Supply Voltage for LCD Drive	V _{CC} -V _{EE}	—	0	—	13.5	V
Input Voltage	V _I	—	V _{AS}	—	V _{CC}	V
Operating Temperature	T _a	—	0	—	+50	°C
Storage Temperature	T _{stg}	—	-20	—	+70	°C

ELECTRICAL CHARACTERISTICS

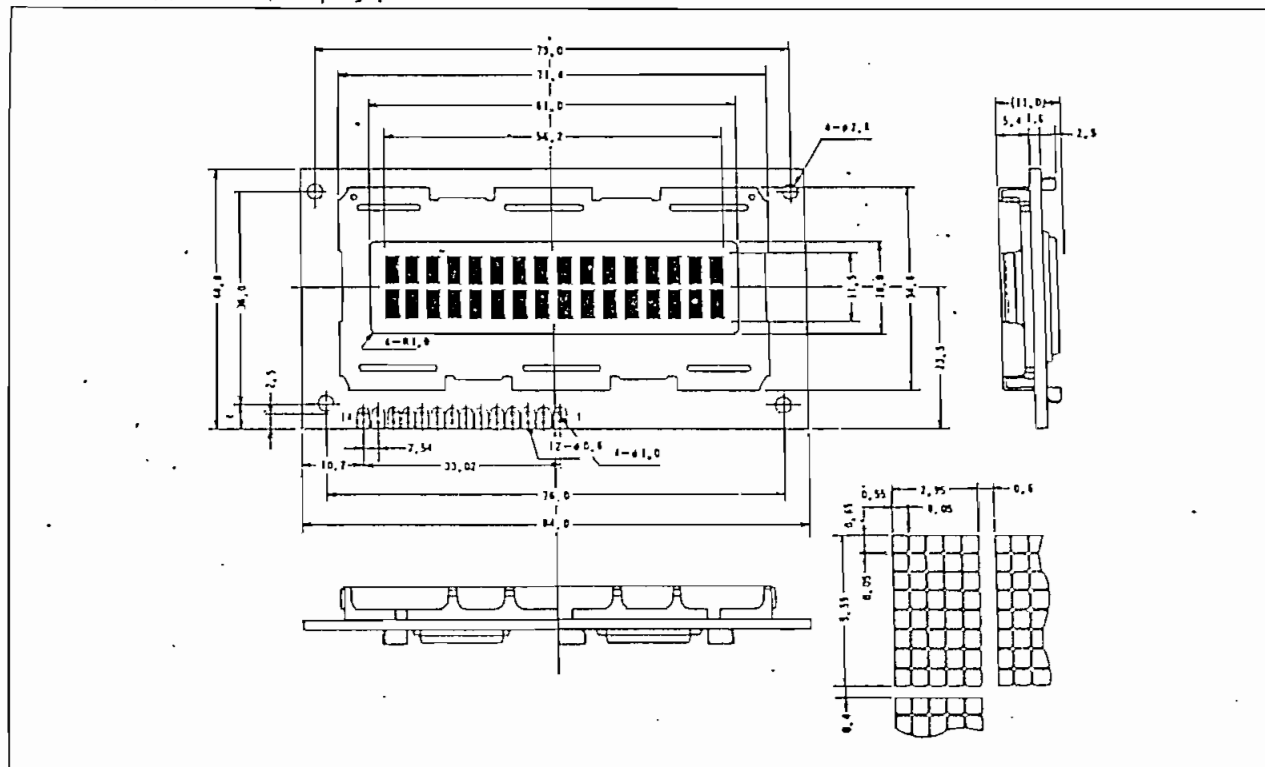
Item	Symbol	Test Condition	Standard Value			Unit
			min.	typ.	max.	
Input "High" Voltage	V _{IH}	—	2.2	—	V _{CC}	V
Input "Low" Voltage	V _{IL}	—	-0.3	—	0.6	V
Output "High" Voltage	V _{OH}	I _{OH} =0.205mA	2.4	—	—	V
Output "Low" Voltage	V _{OL}	I _{OL} =1.2mA	—	—	0.4	V
Power Supply Current	I _{CC}	V _{CC} =5.0V	—	0.5	2.0	mA

* V_{CC} = 5.0V ± 5%, T_a = 25°C

Block diagram

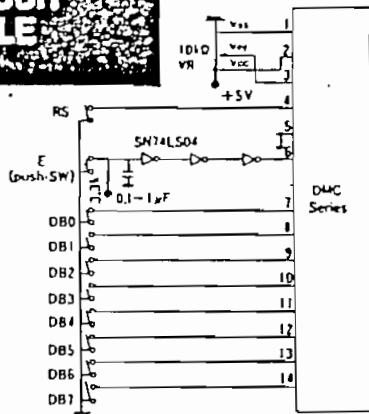


External dimensions / Display pattern



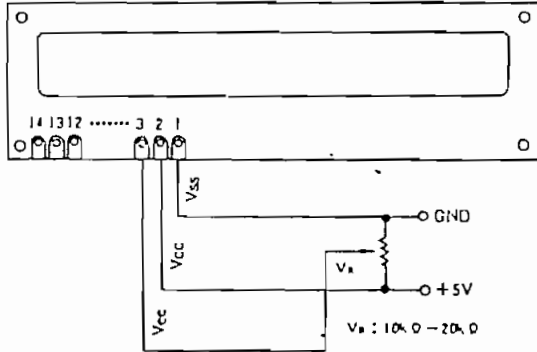
TEST CIRCUIT OF MODULE

SW ON "L" level.
SW OFF "H" level.

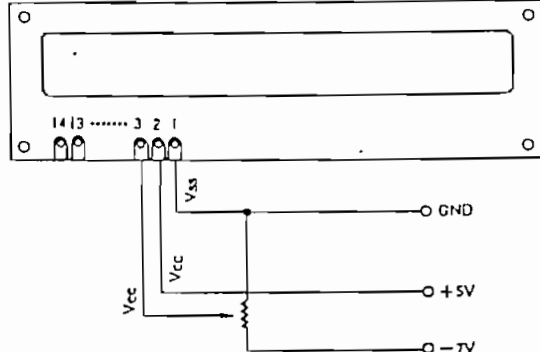


EXAMPLE OF POWER SUPPLY

DMC Module



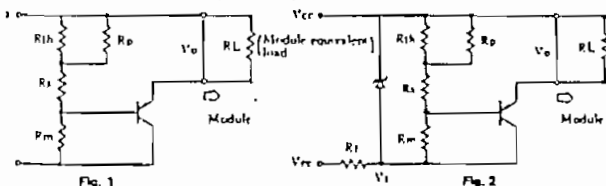
In case of extended temperature version



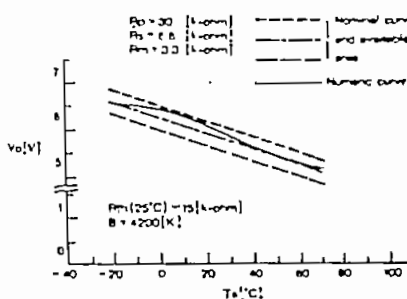
* NOTE: When the voltage of V_{cc} is different from the recommended voltage, the viewing angle may be changed.

• Examples of Temperature Compensation Circuits for Extended Temp Type. (Only for reference)

(A) 1/8 Duty - 1/4 Bias

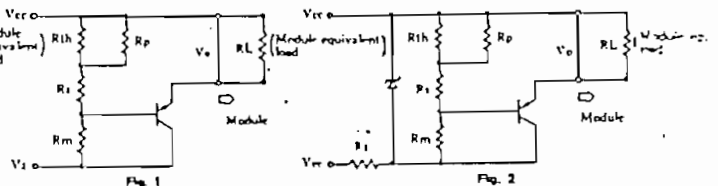


Thermistor $R_{th}(25^{\circ}\text{C}) = 15\text{ k}\Omega$, $B = 4200\text{ K}$
Resistors $R_p = 30\text{ k}\Omega$, $R_s = 1\text{ k}\Omega$, $R_m = 2.2\text{ k}\Omega$
Transistor PNP Type
 $V_{cc} = +5\text{V}$, $V_{be} = 0\text{V}$ (Logic Supply)
 $V_i = 1\text{V}$ to 7.8V to 11.1V
 $V_{ce} < V_i\text{V}$, $R_i = (V_i - V_{ce})/3\text{ k}\Omega$

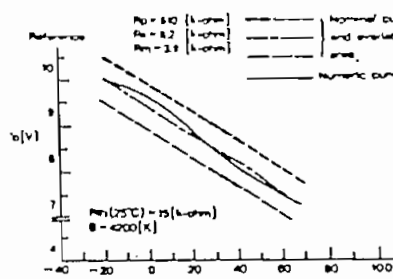


$T_a\text{ [}^{\circ}\text{C]}$	$V_o\text{ [V]}$
-70	8.58
-10	8.50
0	8.40
10	8.28
20	8.09
30	7.86
40	7.67
50	7.47
60	7.29
70	7.15

(B) 1/16 Duty - 1/5 Bias



Thermistor $R_{th}(25^{\circ}\text{C}) = 15\text{ k}\Omega$, $B = 4200\text{ K}$
Resistors $R_p = 110\text{ k}\Omega$, $R_s = 1\text{ k}\Omega$, $R_m = 2.2\text{ k}\Omega$
Transistor PNP Type
 $V_{cc} = +5\text{V}$, $V_{be} = 0\text{V}$ (Logic Supply)
 $V_i = 1\text{V}$ to 10.775V to 11.1V
 $V_{ce} < V_i\text{V}$, $R_i = (V_i - V_{ce})/3\text{ k}\Omega$

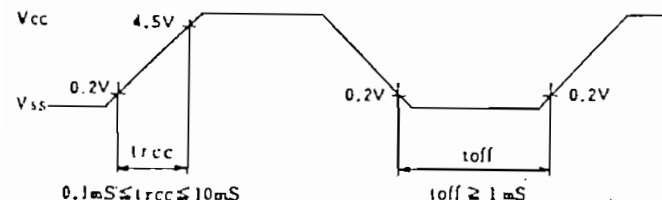


$T_a\text{ [}^{\circ}\text{C]}$	$V_o\text{ [V]}$
-70	10.01
-10	9.84
0	9.60
10	9.28
20	8.89
30	8.49
40	8.11
50	7.78
60	7.53
70	7.30

POWER SUPPLY RESET

The internal reset circuit will not be correctly operated, when the following power supply condition is not satisfied. In this case, please perform initial setting according to the instruction.

Item	Symbol	Measuring Condition	Standard Value			Unit
			min.	typ.	max.	
Power Supply Rise Time	t_{rcc}	—	0.1	—	10	mS
Power Supply Off Time	t_{off}	—	1	—	—	mS



Note: The term t_{off} defines the time when the power supply is off, when the power supply shuts down momentarily or repeats on-off state.

RESET FUNCTION

• Initializing by Internal Reset Circuit

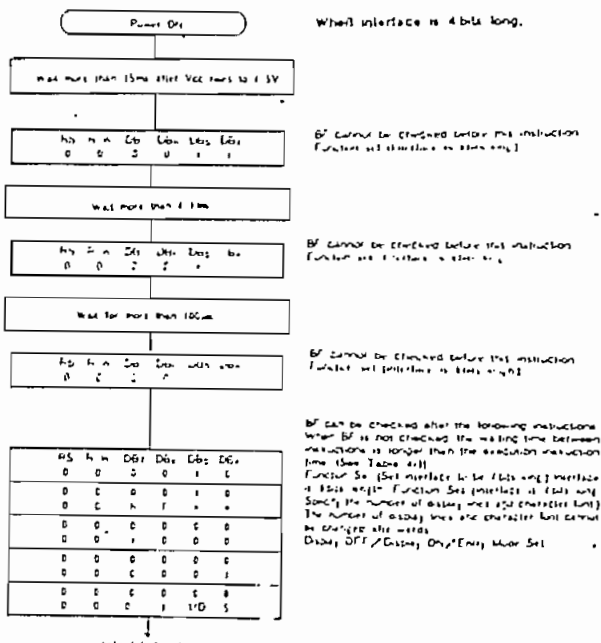
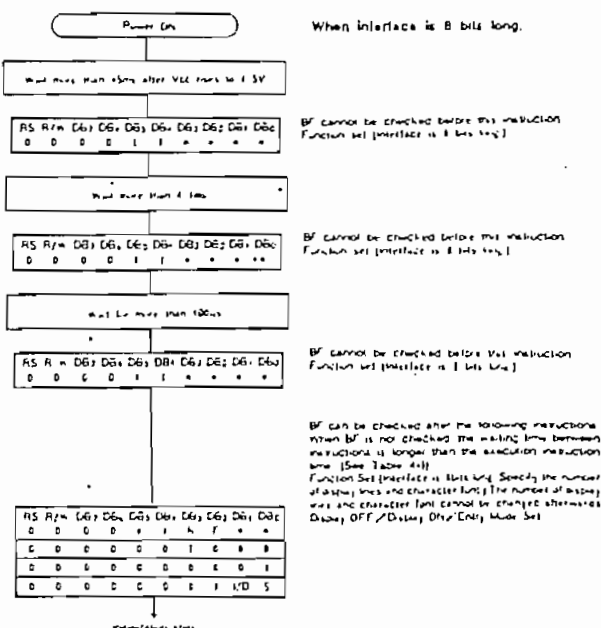
The HD44780 automatically initializes (resets) when power is turned on using the internal reset circuit. The following instructions are executed in initialization. The busy flag (BF) is kept in busy state until initialization ends. (BF=1) The busy state is 10ms after Vcc rises to 4.5V.

- (1) Display clear
- (2) Function set
DL=1: 8bit long interface data
N=0: 1-line display F=0: 5X7dot character font
- (3) Display ON/OFF control
D=0: Display OFF C=0: Cursor OFF B=0: Blink OFF
- (4) Entry mode set
I/D=1: +1(increment) S=0: No shift

Note: When conditions in "Power Supply Conditions Using Internal Reset Circuit" are not met, the internal reset circuit will not operate normally and initialization will not be performed. In this case initialize by MPU according to "initializing by instruction".

• Initializing by Instruction

If the power supply conditions for correctly operating the internal reset circuit are not met, initialization by instruction is required. Use the following procedure for initialization.

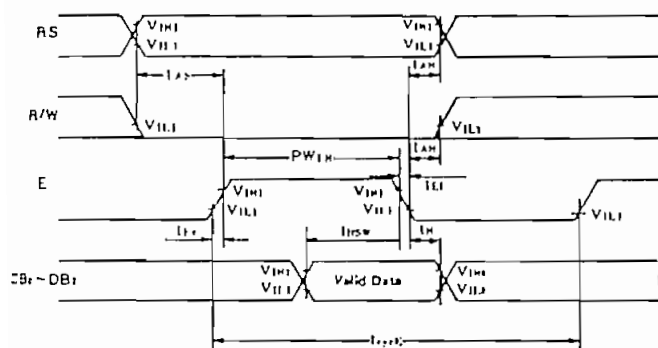


TIMING CHART

Item	Symbol	Measuring Condition	Standard Value			Unit
			min.	typ.	max.	
Enable Cycle Time	T _{CE}	Figs. 1, 2	1000	—	—	nS
Enable Pulse Width, High Level	PW _{EH}	Figs. 1, 2	450	—	—	nS
Enable Rise and Decay Time	t _{ER,LEI}	Figs. 1, 2	—	—	25	nS
Address Setup Time, RS, R/W—E	t _{AS}	Figs. 1, 2	140	—	—	nS
Data Delay Time	t _{DDR}	Fig. 2	—	—	320	nS
Data Setup Time	t _{DSW}	Fig. 1	195	—	—	nS
Data Hold Time	t _H	Fig. 1	10	—	—	nS
Data Hold Time	t _{DH}	Fig. 2	20	—	—	nS
Address Hold Time	t _{AH}	Figs. 1, 2	10	—	—	nS

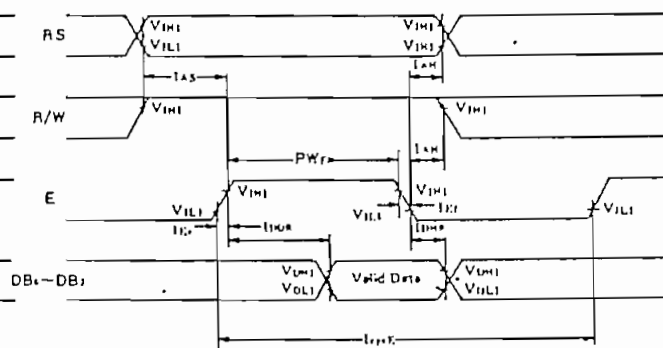
* V_{CC} = 5.0V ± 5%, T_a = 25°C

FIG. 1 WRITE OPERATION



(Write Data from MPU to MODULE)

FIG. 2 READ OPERATION



(Reading Data from MODULE to MPU)

PIN ASSIGNMENT

Pin No.	Symbol	Level	Function	
1	V _{SS}	—	Power Supply	0V (GND)
2	V _{CC}	—		+ 5 V
3	V _{EE}	—		for Liquid Crystal Drive
4	RS	H/L	Register H: Data Input Select L: Instruction Input	
5	R/W	H/L	H: Data Read (Module → MPU) L: Data Write (Module → MPU)	
6	E	H/L—L	Enable Signal	
7	DB0	H/L	Data Bus Line	
8	DB1	H/L		
9	DB2	H/L		
10	DB3	H/L		
11	DB4	H/L		
12	DB5	H/L		
13	DB6	H/L		
14	DB7	H/L		

■ In the data bus line, data transfer is performed two times by the 4-bit or one time by the 8-bit in order to interface with 4-bit or 8-bit MPU.

■ In case interface data length is 4-bit. The data is transferred by using only four buses of DB4—DB7 and the buses of DB0—DB3 are not used. The data transfer to MPU is completed by transferring the data of 4-bits twice. Transfer of upper four bits and low four bits is performed in sequence.

■ In case interface data length is 8-bit. Data transfer is performed by using eight buses of DB0—DB7.

INSTRUCTIONS

Instruction	Code										Description
	RS	R/W	DB 7	DB 6	DB 5	DB 4	DB 3	DB 2	DB 1	DB 0	
Clear Display	0	0	0	0	0	0	0	0	0	1	Clears all display and returns the cursor to the home position (Address 0).
Cursor At Home	0	0	0	0	0	0	0	0	1	*	Returns the cursor to the home position (Address 0). Also returns the display being shifted to the original position. DDRAM contents remain unchanged.
Entry Mode Set	0	0	0	0	0	0	0	1	I/D	S	Sets the cursor move direction and specifies or not to shift the display. These operations are performed during data write and read.
Display On/Off Control	0	0	0	0	0	0	1	D	C	B	Sets ON/OFF of all display (D) cursor ON/OFF (C), and blink of cursor position character (B).
Cursor/Display Shift	0	0	0	0	0	1	S/C	R/L	*	*	Moves the cursor and shifts the display without changing DDRAM contents.
Function Set	0	0	0	0	1	DL	N	F	*	*	Sets interface data length (DL) number of display lines (L) and character font (F).
CGRAM Address Set	0	0	0	1	ACG						Sets the CGRAM address. CGRAM data is sent and received after this setting.
DDRAM Address Set	0	0	1	ADD						Sets the DDRAM address. DDRAM data is sent and received after this setting.	
Busy Flag/Address Read	0	1	BF	AC						Reads Busy flag (BF) indicating internal operation is being performed and reads address counter contents.	
CGRAM/DDRAM Data Write	1	0	WRITE DATA						Writes data into DDRAM or CGRAM.		
CGRAM/DDRAM Data Read	1	1	READ DATA						Reads data from DDRAM or CGRAM.		

Code	Description	Execute Time (max.)
I/D = 1 : Increment I/D = 0 : Decrement S = 1 : With display shift S/C = 1 : Display shift S/C = 0 : Cursor movement R/L = 1 : Shift to the right R/L = 0 : Shift to the left DL = 1 : 8-bit DL = 0 : 4-bit N = 1 : 1/16Duty N = 0 : 1/8Duty, 1/11Duty F = 1 : 5×10dots F = 0 : 5×7dots BF = 1 : Internal operation is being performed BF = 0 : Instruction acceptable	DDRAM : Display Data RAM CGRAM : Character Generator RAM ACG : CGRAM Address ADD : DDRAM Address Corresponds to cursor address. AC : Address Counter, used for both DDRAM and CGRAM * : Invalid	$f_{cp} \text{ or } f_{osc} = 250\text{kHz}$ However, when frequency changes, execution time also changes. Ex When $f_{cp} \text{ or } f_{osc} = 270\text{kHz}$, $40\mu\text{s} \times \frac{250}{270} = 37\mu\text{s}$

FONT TABLE (5×11Dots)

Lower 4-bit	Upper 4-bit	0000	0010	0011	0100	0101	0110	0111	1010	1011	1100	1101	1110	1111
××××0000	CG-RAM (1)		0	a	P	`	P		—	9	z	o	p	
	(2)	!	1	A	Q	a	9	7	7	4	a	q		
××××0010	(3)	"	2	B	R	b	r	"	4	u	x	p	o	
	(4)	#	3	C	S	c	s	1	9	t	e	e	o	
××××0100	(5)	*	4	D	T	d	t	.	I	t	t	p	a	
	(6)	%	5	E	U	e	u	.	*	+	1	e	o	
××××0110	(7)	&	6	F	V	f	v	7	+	2	3	p	z	
	(8)	'	7	G	W	g	w	7	+	7	9	g	π	
××××1000	(1)	(	8	H	X	h	x	4	9	*	U	r	z	
	(2)	)	9	I	Y	i	y	9	7	1	U	1	y	
××××1010	(3)	*	:	J	Z	j	z	z	3	n	v	j	7	
	(4)	+	:	K	L	k	l	*	9	U	o	*	π	
××××1100	(5)	.	<	L	*	1	1	+	9	7	7	o	π	
	(6)	—	=	M	I	m	1	u	z	^	2	1	÷	
××××1110	(7)	.	>	N	^	n	+	3	U	π	1	n		
	(8)	/	7	O	_	o	+	u	y	7	9	o		

*CG RAM : Character pattern area can be rewritten by program.



+5V Powered RS-232 Drivers/Receivers

MAX230-241*

General Description

Maxim's family of line drivers/receivers are intended for all RS-232 and V28/V24 communications interfaces, and in particular, for those applications where $\pm 12V$ is not available. The MAX230, MAX236, MAX240 and MAX241 are particularly useful in battery powered systems since their low power shutdown mode reduces power dissipation to less than $5\mu W$. The MAX233 and MAX235 use no external components and are recommended for applications where printed circuit board space is critical.

All members of the family except the MAX231 and MAX239 need only a single +5V supply for operation. The RS-232 drivers/receivers have on-board charge pump voltage converters which convert the +5V input power to the $\pm 10V$ needed to generate the RS-232 output levels. The MAX231 and MAX239, designed to operate from +5V and +12V, contain a +12V to -12V charge pump voltage converter.

Since nearly all RS-232 applications need both line drivers and receivers, the family includes both receivers and drivers in one package. The wide variety of RS-232 applications require differing numbers of drivers and receivers. Maxim offers a wide selection of RS-232 driver/receiver combinations in order to minimize the package count (see table below).

Both the receivers and the line drivers (transmitters) meet all EIA RS-232C and CCITT V.28 specifications.

Features

- ◆ Operates from Single 5V Power Supply (+5V and +12V — MAX231 and MAX239)
- ◆ Meets All RS-232C and V.28 Specifications
- ◆ Multiple Drivers and Receivers
- ◆ Onboard DC-DC Converters
- ◆ $\pm 9V$ Output Swing with +5V Supply
- ◆ Low Power Shutdown — $<1\mu A$ (typ)
- ◆ 3-State TTL/CMOS Receiver Outputs
- ◆ $\pm 30V$ Receiver Input Levels

Applications

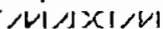
Computers
Peripherals
Modems
Printers
Instruments



Selection Table

Part Number	Power Supply Voltage	No. of RS-232 Drivers	No. of RS-232 Receivers	External Components	Low Power Shutdown /TTL 3-State	No. of Pins
MAX230	+5V	5	0	4 capacitors	Yes/No	20
MAX231	+5V and +7.5V to 13.2V	2	2	2 capacitors	No/No	14
MAX232	+5V	2	2	4 capacitors	No/No	16
MAX233	+5V	2	2	None	No/No	20
MAX234	+5V	4	0	4 capacitors	No/No	16
MAX235	+5V	5	5	None	Yes/Yes	24
MAX236	+5V	4	3	4 capacitors	Yes/Yes	24
MAX237	+5V	5	3	4 capacitors	No/No	24
MAX238	+5V	4	4	4 capacitors	No/No	24
MAX239	+5V and +7.5V to 13.2V	3	5	2 capacitors	No/Yes	24
MAX240	+5V	5	5	4 capacitors	Yes/Yes	44
						(Flatpak)
MAX241	+5V	4	5	4 capacitors	Yes/Yes	28
						(Small Outline)

*Patent Pending



Maxim Integrated Products 2-25

maxim is a registered trademark of Maxim Integrated Products.

+5V Powered RS-232 Drivers/Receivers

ABSOLUTE MAXIMUM RATINGS

V_{CC}	-0.3V to +6V
V^+	($V_{CC} - 0.3V$) to +14V
V^-	+0.3V to -14V
Input Voltages	
T_{IH}	-0.3 to ($V_{CC} + 0.3V$)
R_{IH}	$\pm 30V$
Output Voltages	
T_{OH}	($V^+ + 0.3V$) to ($V^- - 0.3V$)
R_{OH}	-0.3V to ($V_{CC} + 0.3V$)

Short Circuit Duration

T_{OUT}	continuous
Power Dissipation	
CERDIP	675mW
(derate 9.5mW/°C above +70°C)	
Plastic DIP	375mW
(derate 7mW/°C above +70°C)	
Small Outline (SO)	375mW
(derate 7mW/°C above +70°C)	
Lead Temperature (soldering 10 seconds)	+300°C
Storage Temperature	-65°C to +160°C

Stresses above those listed under "Absolute Maximum Ratings" may cause permanent damage to the device. These are stress ratings only, and functional operation of the device at these or any other conditions above those indicated in the operational sections of the specifications is not implied. Exposure to absolute maximum rating conditions for extended periods may affect device reliability.

ELECTRICAL CHARACTERISTICS

(MAX232, 234, 236, 237, 238, 240, 241 $V_{CC} = 5V \pm 10\%$; MAX233, 235 $V_{CC} = 5V \pm 5\%$ C1-C4 = 1.0 μF ; MAX231, 239 $V_{CC} = 5V \pm 10\%$, $V^+ = 7.5V$ to 13.2V; T_A = Operating Temperature Range, Figures 3-14, unless otherwise noted.)

PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
Output Voltage Swing	All Transmitter Outputs loaded with 3k Ω to Ground	± 5	± 9		V
V_{CC} Power Supply Current	No load, $T_A = +25^\circ C$ MAX232-MAX233		5	10	mA
	MAX230, MAX234-238, MAX240-MAX241		7	15	
	MAX231, MAX239		0.4	1	
V^+ Power Supply Current	No load, MAX231 and MAX239 only	MAX231	1.8	5	mA
		MAX239	5	15	
Shutdown Supply Current	Figure 1, $T_A = +25^\circ C$		1	10	μA
Input Logic Threshold Low	T_{IH} , $\overline{EN}$, Shutdown			0.8	V
Input Logic Threshold High	T_{IH}	2.0			V
	$\overline{EN}$, Shutdown	2.4			
Logic Pullup Current	$T_{IH} = 0V$		15	200	μA
RS-232 Input Voltage Operating Range		-30		+30	V
RS-232 Input Threshold Low	$V_{CC} = 5V$, $T_A = +25^\circ C$ (MAX231, 239 $V^+ = 0V$)	0.8	1.2		V
RS-232 Input Threshold High	$V_{CC} = 5V$, $T_A = +25^\circ C$ (MAX231, 239 $V^+ = 12V$)		1.7	2.4	V
RS-232 Input Hysteresis	$V_{CC} = 5V$	0.2	0.5	1.0	V
RS-232 Input Resistance	$T_A = +25^\circ C$, $V_{CC} = 5V$	3	5	7	k Ω
TTL/CMOS Output Voltage Low	$I_{OUT} = 1.6mA$ (MAX231-233, $I_{OUT} = 3.2mA$)			0.4	V
TTL/CMOS Output Voltage High	$I_{OUT} = 1.0mA$	3.5			V
TTL/CMOS Output Leakage Current	$\overline{EN} = V_{CC}$, $0V \leq R_{OUT} \leq V_{CC}$		0.05	± 10	μA
Output Enable Time (Figure 2)	MAX235, MAX236, MAX239, MAX240, MAX241		400		ns
Output Disable Time (Figure 2)	MAX235, MAX236, MAX239, MAX240, MAX241		250		ns
Propagation Delay	RS-232 to TTL		0.5		μs
Instantaneous Slew Rate	$C_L = 10pF$, $R_L = 3-7k\Omega$, $T_A = +25^\circ C$ (Note 1)			30	V/ μs
Transition Region Slew Rate	$R_L = 3k\Omega$, $C_L = 2500pF$, Measured from +3V to -3V or -3V to +3V		3		V/ μs
Output Impedance	$V_{CC} = V^+ = V^- = 0V$, $V_{OUT} = \pm 2V$	300			Ω
RS-232 Output Short Circuit Current			± 10		mA

Note 1: Sample tested

+5V Powered RS-232 Drivers/Receivers

Typical Operating Characteristics

MAX230-241*

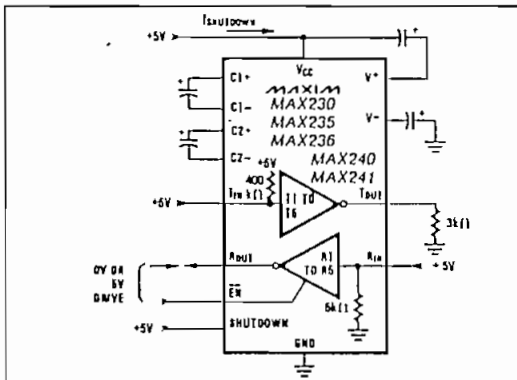
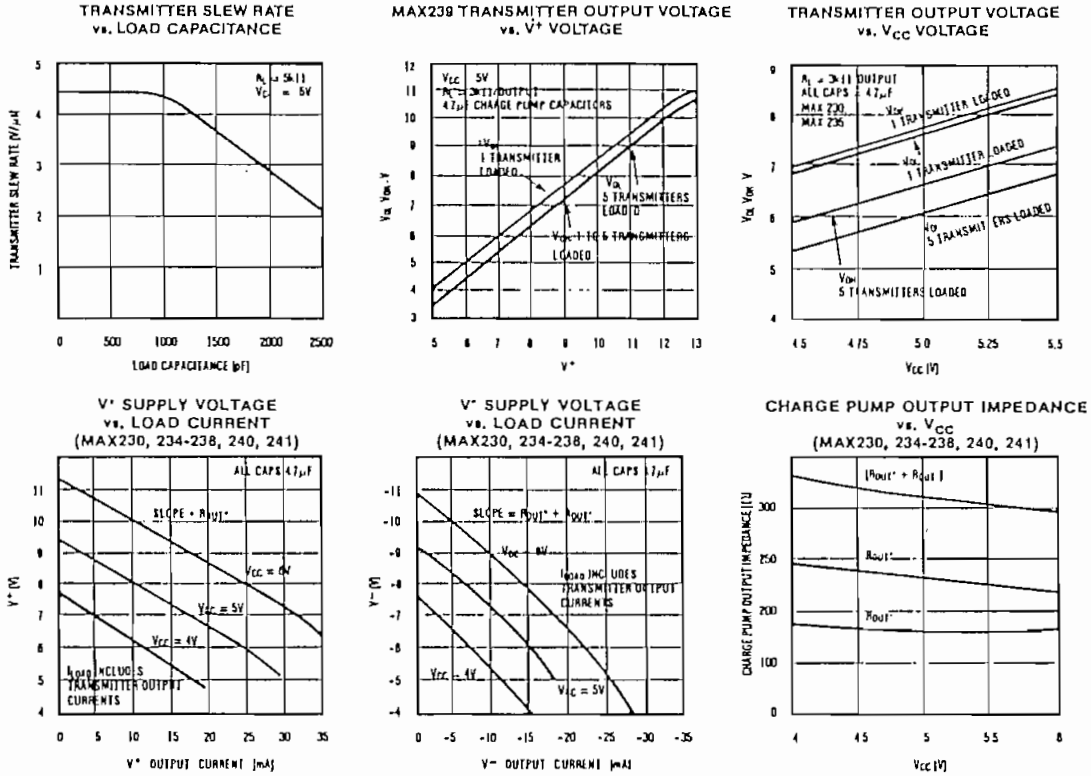


Figure 1. Shutdown Current Test Circuit

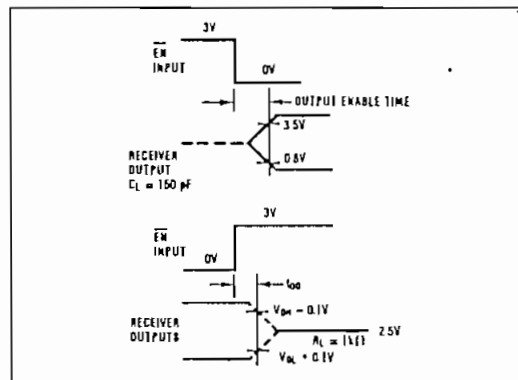
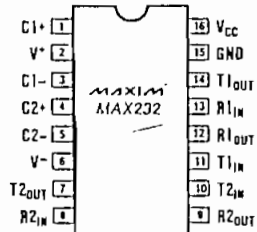


Figure 2. Receiver Output Enable and Disable Timing

+5V Powered RS-232 Drivers/Receivers

MAX230-241*



16 Lead Small Outline
also available.

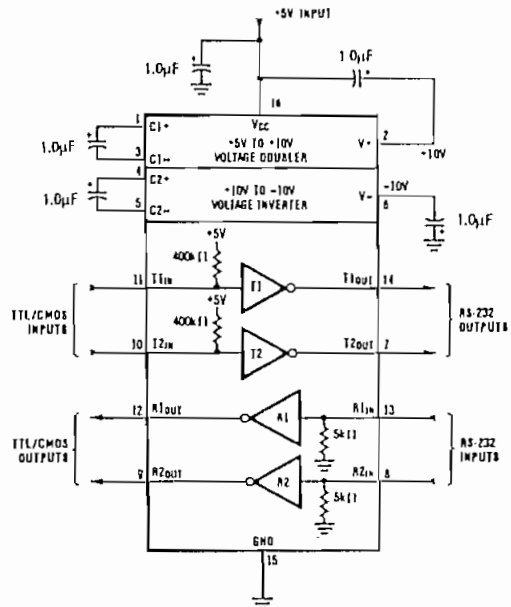
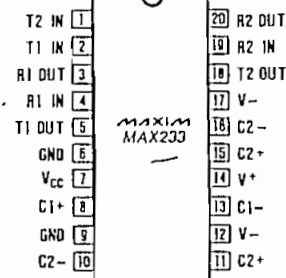


Figure 5. MAX232 Typical Operating Circuit



Small Outline Not Available

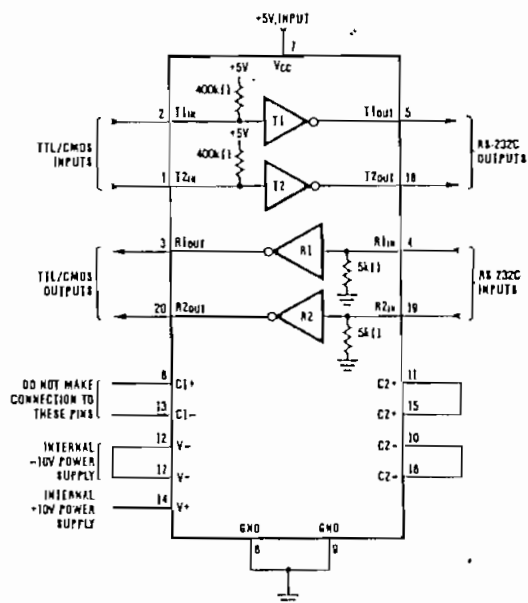


Figure 6. MAX233 Typical Operating Circuit



MM54C922/MM74C922 16-Key Encoder MM54C923/MM74C923 20-Key Encoder

General Description

These CMOS key encoders provide all the necessary logic to fully encode an array of SPST switches. The keyboard scan can be implemented by either an external clock or external capacitor. These encoders also have on-chip pull-up devices which permit switches with up to 50 k Ω on resistance to be used. No diodes in the switch array are needed to eliminate ghost switches. The internal debounce circuit needs only a single external capacitor and can be defeated by omitting the capacitor. A Data Available output goes to a high level when a valid keyboard entry has been made. The Data Available output returns to a low level when the entered key is released, even if another key is depressed. The Data Available will return high to indicate acceptance of the new key after a normal debounce period; this two key roll over is provided between any two switches.

An internal register remembers the last key pressed even after the key is released. The TRI-STATE[®] outputs

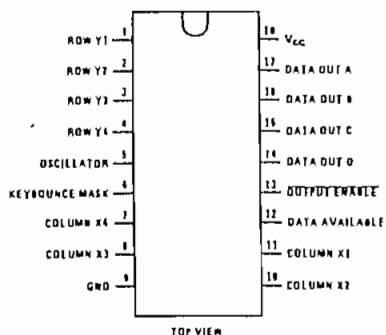
provide for easy expansion and bus operation and are LPTTL compatible.

Features

- 50 k Ω maximum switch on resistance
- On or off chip clock
- On chip row pull-up devices
- 2 key roll-over
- Keybounce elimination with single capacitor
- Last key register at outputs
- TRI-STATE outputs LPTTL compatible
- Wide supply range 3V to 15V
- Low power consumption

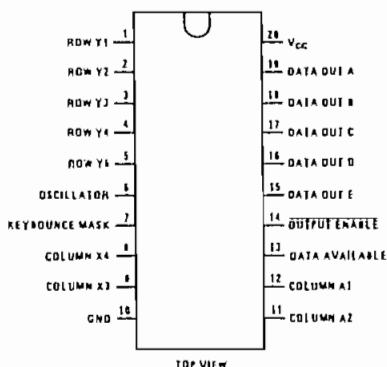
Connection Diagrams

Dual-In-Line Package



Order Number MM54C922N
or MM74C922N
See Package 20

Dual-In-Line Package



Order Number MM54C923N
or MM74C923N
See Package 20A

MM54C922/MM74C922, MM54C923/MM74C923

2

Absolute Maximum Ratings (Note 1)

Voltage at Any Pin	$V_{CC} - 0.3V$ to $V_{CC} + 0.3V$
Operating Temperature Range	
MM54C922, MM54C923	$-55^{\circ}C$ to $+125^{\circ}C$
MM74C922, MM74C923	$-40^{\circ}C$ to $+85^{\circ}C$
Storage Temperature Range	$-65^{\circ}C$ to $+150^{\circ}C$

Package Dissipation
Operating V_{CC} Range
V_{CC}
Lead Temperature (Soldering, 10 seconds)

DC Electrical Characteristics Min/Max limits apply across temperature range unless otherwise specified

PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
CMOS TO CMOS					
V_{T+} Positive-Going Threshold Voltage at Osc and KBM Inputs	$V_{CC} = 5V, I_{IN} \geq 0.7mA$ $V_{CC} = 10V, I_{IN} \geq 1.4mA$ $V_{CC} = 15V, I_{IN} \geq 2.1mA$	3 6 9	3.6 6.8 10	4.3 8.6 12.9	
V_{T-} Negative-Going Threshold Voltage at Osc and KBM Inputs	$V_{CC} = 5V, I_{IN} \geq 0.7mA$ $V_{CC} = 10V, I_{IN} \geq 1.4mA$ $V_{CC} = 15V, I_{IN} \geq 2.1mA$	0.7 1.4 2.1	1.4 3.2 5	2 4 6	
$V_{IN(1)}$ Logical "1" Input Voltage, Except Osc and KBM Inputs	$V_{CC} = 5V,$ $V_{CC} = 10V,$ $V_{CC} = 15V,$	3.5 8 12.5	4.5 9 13.5		
$V_{IN(0)}$ Logical "0" Input Voltage, Except Osc and KBM Inputs	$V_{CC} = 5V,$ $V_{CC} = 10V,$ $V_{CC} = 15V,$		0.5 1 1.5	1.5 2 2.5	
I_{rp} Row Pull-Up Current at Y1, Y2, Y3, Y4 and Y5 Inputs	$V_{CC} = 5V, V_{IN} = 0.1V_{CC}$ $V_{CC} = 10V$ $V_{CC} = 15V$		-2 -10 -22	-5 -20 -45	
$V_{OUT(1)}$ Logical "1" Output Voltage	$V_{CC} = 5V, I_O = -10\mu A$ $V_{CC} = 10V, I_O = -10\mu A$ $V_{CC} = 15V, I_O = -10\mu A$	4.5 9 13.5			
$V_{OUT(0)}$ Logical "0" Output Voltage	$V_{CC} = 5V, I_O = 10\mu A$ $V_{CC} = 10V, I_O = 10\mu A$ $V_{CC} = 15V, I_O = 10\mu A$			0.5 1 1.5	
R_{on} Column "ON" Resistance at X1, X2, X3 and X4 Outputs	$V_{CC} = 5V, V_O = 0.5V$ $V_{CC} = 10V, V_O = 1V$ $V_{CC} = 15V, V_O = 1.5V$		500 300 200	1400 700 500	
I_{CC} Supply Current	$V_{CC} = 5V, \text{Osc at } 0V$ $V_{CC} = 10V$ $V_{CC} = 15V$		0.55 1.1 1.7	1.1 1.9 2.6	
$I_{IN(1)}$ Logical "1" Input Current at Output Enable	$V_{CC} = 15V, V_{IN} = 15V$		0.005	1.0	
$I_{IN(0)}$ Logical "0" Input Current at Output Enable	$V_{CC} = 15V, V_{IN} = 0V$	-1.0	-0.005		
CMOS/LPTTL INTERFACE					
$V_{IN(1)}$ Logical "1" Input Voltage, Except Osc and KBM Inputs	54C, $V_{CC} = 4.5V$ 74C, $V_{CC} = 4.75V$	$V_{CC} - 1.5$ $V_{CC} - 1.5$			
$V_{IN(0)}$ Logical "0" Input Voltage, Except Osc and KBM Inputs	54C, $V_{CC} = 4.5V$ 74C, $V_{CC} = 4.75V$			0.8 0.8	
$V_{OUT(1)}$ Logical "1" Output Voltage	54C, $V_{CC} = 4.5V,$ $I_O = -360\mu A$ 74C, $V_{CC} = 4.75V,$ $I_O = -360\mu A$	2.4 2.4			
$V_{OUT(0)}$ Logical "0" Output Voltage	54C, $V_{CC} = 4.5V,$ $I_O = -360\mu A$ 74C, $V_{CC} = 4.75V,$ $I_O = -360\mu A$			0.4 0.4	

DC Electrical Characteristics (Cont'd.)

PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
OUTPUT DRIVE (See 54C/74C Family Characteristics Data Sheet) (Short Circuit Current)					
ISOURCE Output Source Current (P-Channel)	$V_{CC} = 5V, V_{OUT} = 0V,$ $T_A = 25^\circ C$	-1.75	-3.3		mA
ISOURCE Output Source Current (P-Channel)	$V_{CC} = 10V, V_{OUT} = 0V,$ $T_A = 25^\circ C$	-8	-15		mA
ISINK Output Sink Current (N-Channel)	$V_{CC} = 5V, V_{OUT} = V_{CC},$ $T_A = 25^\circ C$	1.75	3.6		mA
ISINK Output Sink Current (N-Channel)	$V_{CC} = 10V, V_{OUT} = V_{CC},$ $T_A = 25^\circ C$	8	16		mA

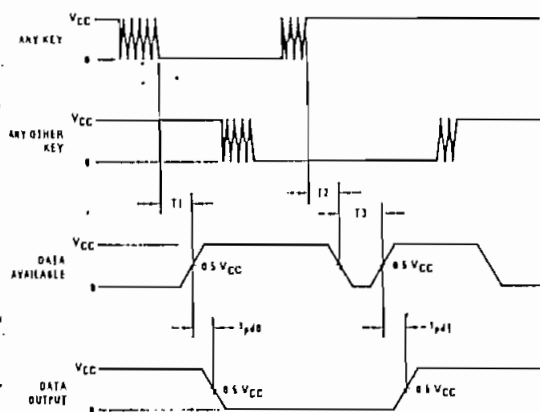
AC Electrical Characteristics $T_A = 25^\circ C, C_L = 50 pF$, unless otherwise noted

PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
t_{pd0}, t_{pd1} Propagation Delay Time to Logical "0" or Logical "1" from D.A.	$C_L = 50 pF$, (Figure 1) $V_{CC} = 5V$ $V_{CC} = 10V$ $V_{CC} = 15V$		60 35 25	150 80 60	ns ns ns
t_{OH}, t_{IH} Propagation Delay Time from Logical "0" or Logical "1" into High Impedance State	$R_L = 10k, C_L = 10pF$ (Figure 2) $V_{CC} = 5V, R_L = 10k$ $V_{CC} = 10V, C_L = 10 pF$ $V_{CC} = 15V$		80 65 50	200 150 110	ns ns ns
t_{HD}, t_{HL} Propagation Delay Time from High Impedance State to a Logical "0" or Logical "1"	$R_L = 10k, C_L = 50 pF$, (Figure 2) $V_{CC} = 5V, R_L = 10k$ $V_{CC} = 10V, C_L = 50 pF$ $V_{CC} = 15V$		100 55 40	250 125 90	ns ns ns
C_{IN} Input Capacitance	Any Input, (Note 2)		5	7.5	pF
C_{OUT} TRI-STATE Output Capacitance	Any Output, (Note 2)		10		pF

Note 1: "Absolute Maximum Ratings" are those values beyond which the safety of the device cannot be guaranteed. Except for "Operating Temperature Range" they are not meant to imply that the devices should be operated at these limits. The table of "Electrical Characteristics" provides conditions for actual device operation.

Note 2: Capacitance is guaranteed by periodic testing.

Switching Time Waveforms



$T1 \approx T2 \approx RC, T3 \approx 0.7 RC$ where $R \approx 10k$ and C is external capacitor at KBM input.

FIGURE 1

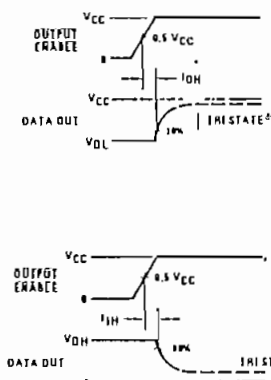
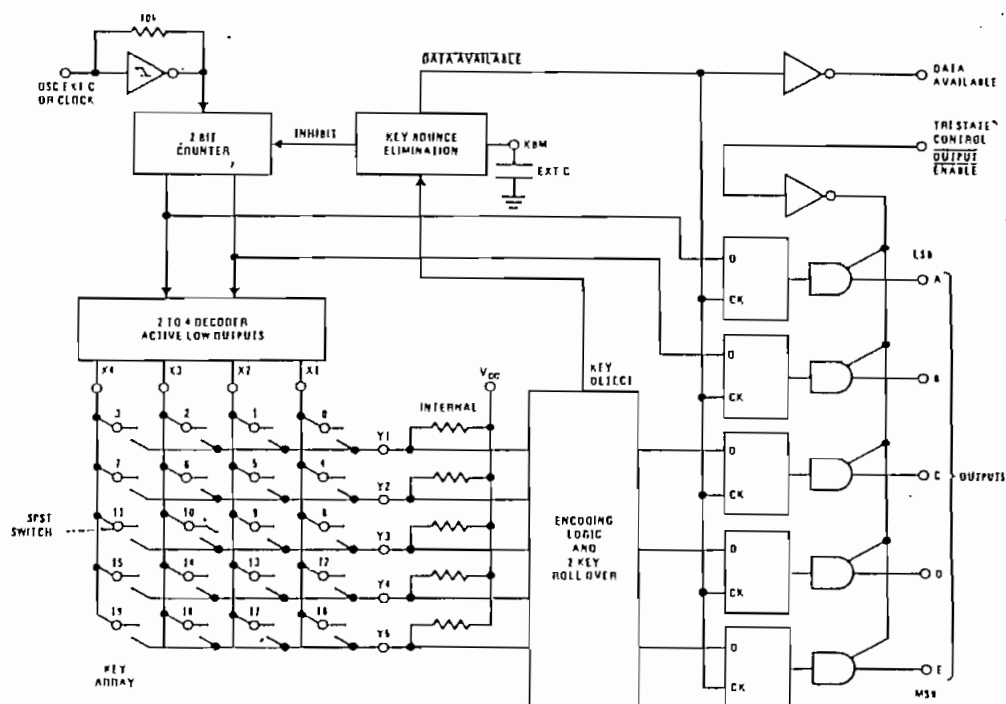


FIGURE 2

Block Diagram

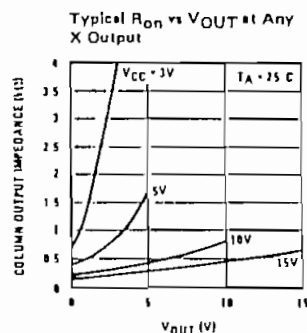
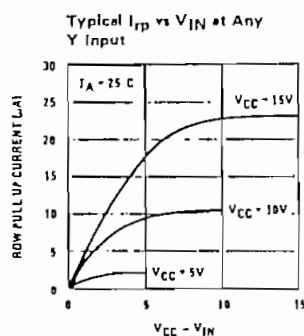


Truth Table

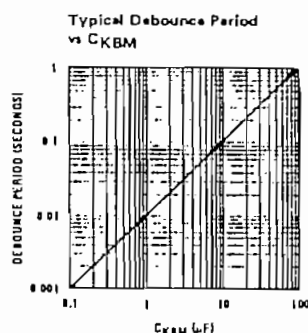
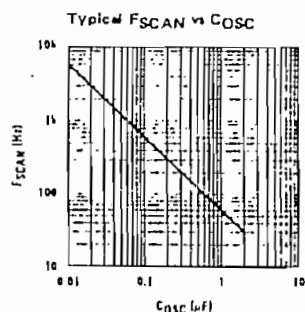
SWITCH POSITION	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	Y1,X1	Y1,X2	Y1,X3	Y1,X4	Y2,X1	Y2,X2	Y2,X3	Y2,X4	Y3,X1	Y3,X2	Y3,X3	Y3,X4	Y4,X1	Y4,X2	Y4,X3	Y4,X4	Y5,X1	Y5,X2	Y5,X3	Y5,X4
A	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
B	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
C	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1	0	0	0	0
D	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1	0	0	0	0
E	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

* Omit for MM54C922/MM74C922

Typical Performance Characteristics

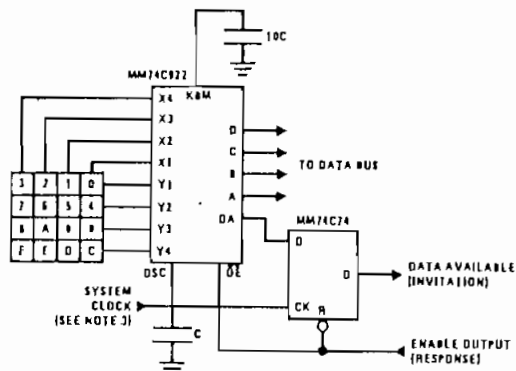


Typical Performance Characteristics (Cont'd.)

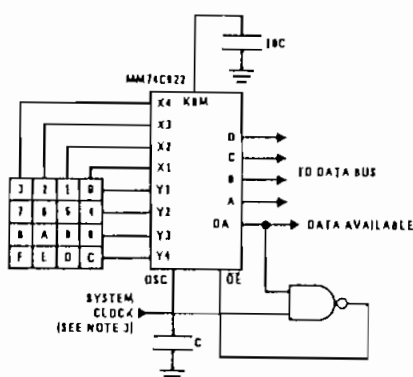


Typical Applications

Synchronous Handshake (MM74C922)

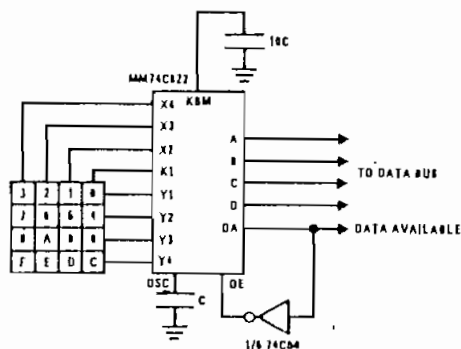


Synchronous Data Entry Onto Bus (MM74C922)



Outputs are enabled when valid entry is made and go into TRI-STATE when key is released.

Asynchronous Data Entry Onto Bus (MM74C922)



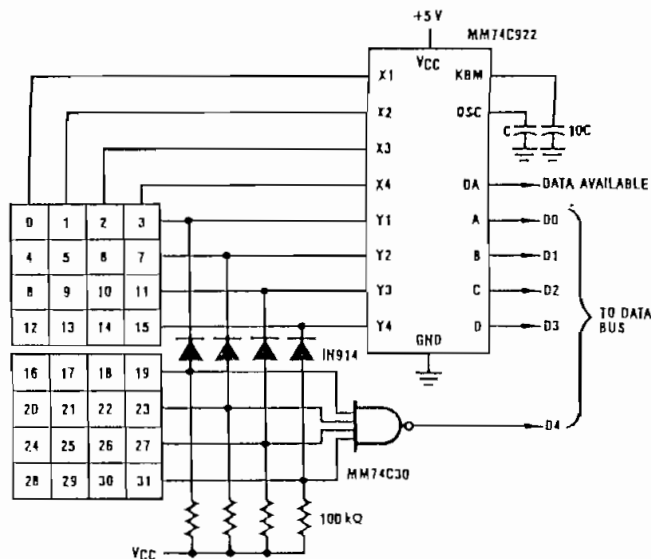
Outputs are in TRI-STATE until key is pressed, then data is placed on bus. When key is released, outputs return to TRI-STATE.

Keyboard Suppliers
Mini Key Series KL
Digitron Company
Pasadena, California
Computronics Engineering
7235 Hollywood Blvd
Hollywood, California 90046

Note 3: The keyboard may be synchronously scanned by omitting the capacitor at osc, and driving osc directly if the system clock rate is lower than 10 kHz.

Typical Application (Cont'd.)

Expansion to 32 Key Encoder (MM74C922)



Theory of Operation

The MM74C922/MM74C923 Keyboard Encoders implement all the logic necessary to interface a 16 or 20 SPST key switch matrix to a digital system. The encoder will convert a key switch closure to a 4 (MM74C922) or 5 (MM74C923) bit nibble. The designer can control both the keyboard scan rate and the key debounce period by altering the oscillator capacitor, C_{OSC} , and the key bounce mask capacitor, C_{MSK} . Thus, the MM74C922/MM74C923's performance can be optimized for many keyboards.

The keyboard encoders connect to a switch matrix that is 4 rows by 4 columns (MM74C922) or 5 rows by 4 columns (MM74C923). When no keys are depressed, the row inputs are pulled high by internal pull-ups and the column outputs sequentially output a logic "0". These outputs are open drain and are therefore low for 25% of the time and otherwise off. The column scan rate is controlled by the oscillator input, which consists of a Schmitt trigger oscillator, a 2-bit counter, and a 2-4-bit decoder.

When a key is depressed, key 0, for example, nothing will happen when the X1 input is off, since Y1 will remain high. When the X1 column is scanned, X1 goes low and Y1 will go low. This disables the counter and keeps X1 low. Y1 going low also initiates the key bounce circuit

timing and locks out the other Y inputs. The key code to be outputted is a combination of the frozen counter value and the decoded Y inputs. Once the key bounce circuit times out, the data is latched, and the Data Available (DAV) output goes high.

If, during the key closure the switch bounces, Y1 will go high again, restarting the scan and resetting the key bounce circuitry. The key may bounce several times, but as soon as the switch stays low for a debounce period, the closure is assumed valid and the data is latched.

A key may also bounce when it is released. To ensure that the encoder does not recognize this bounce as another key closure, the debounce circuit must time out before another closure is recognized.

The two key roll over feature can be illustrated by assuming a key is depressed, and then a second key is depressed. Since all scanning has stopped, and all other Y inputs are disabled, the second key is not recognized until the first key is lifted and the key bounce circuitry has reset.

The output latches feed TRI-STATE*, which are enabled when the Output Enable ($\overline{OE}$) input is taken low.

We carry major manufacturers - call for pricing in quantities over 100.

Computer Products

23 Watt Switching Power Supply

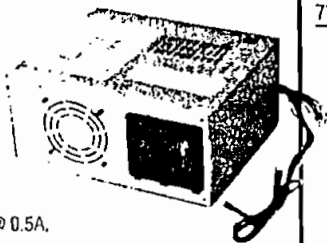


- Input: 120VAC
- Molex style input and output connectors
- Size: 5"L x 3"W x 1.375"H
- Weight: .5 lbs.

Output:
+5V @ 1.8A
+15V @ .450A
-15V @ .450A

Part No. Product No. Price
76611 PS9099 (1300 available) \$12.95

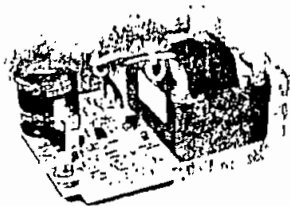
Jameco 28 Watt Switching Power Supply



- Input: 110VAC @ 0.5A, 250VAC @ 0.25A
- Disk drive and fan power connectors and 2 female spade ground connectors
- Size: 7"L x 5.875"W x 3.25"H
- Weight: 3 lbs.

Output:
+5V @ 2A
+12V @ 1.5A

Part No. Product No. Price
77981 PS1439 (900 available) \$14.95



Acme 30 Watt Power Supply

- Input: 100/120/220/230/240 VAC @ 50/60Hz
- On-board terminals
- Size: 4.75"L x 4.75"W x 2.5"H
- Weight: 4 lbs.

Output:
+5VDC @ 6.0A

Part No. Product No. Price
28935 PS413 (300 Available) \$14.95

Jameco 38 Watt Switching Power Supply



- Input: 110V
- 7-pin Molex style output connector and power switch
- Size: 7.6"L x 4.25"W x 1.75"H
- Weight: 1.5 lbs.

Output:
+5V @ 4.0A
+12V @ 1.2A
-12V @ 0.3A

Part No. Product No. Price
96698 DPS401A (250 available) \$17.95

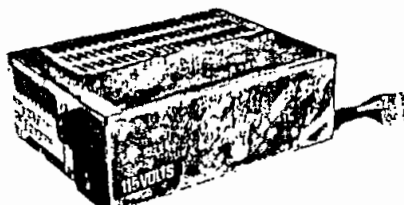
Houston Tracking Systems 40 Watt Switching Power Supply



- Input: 115VAC @ 60Hz
- 2-pin Molex input connector and 4 output connectors
- Size: 5.375"L x 3.5"W x 1.25"H
- Weight: .5 lbs.

Output:
+12V @ .5A
-5V @ .32A
+5V @ 1.5A
-12V @ .075A
+5V @ .4A
+22V @ 1A

Part No. Product No. Price
77972 PS512 (1100 available) \$14.95



BSR 41 Watt Switching Power Supply

- Input: 115VAC @ 50/60Hz
- 7 pin molex style connector for voltage output
- Size: 7"L x 5.25"W x 2.5"H
- Weight: 2.8 lbs.

Output:
+5VDC @ 3.75A
+12VDC @ 1.5A
-12VDC @ 0.4A

Part No. Product No. Price
28927 PS41012 (2000 available) \$17.95



Tectrol 41 Watt Switching Power Supply

- Input: 115VAC @ 50/60Hz
- 12 pin molex style connector for voltage output
- Size: 7"L x 3.375"W x 1.75"H
- Weight: 1 lb.

Output:
+5VDC @ 4.8A
+12VDC @ 1.1A
-5VDC @ .12A
-12VDC @ .34A

Part No. Product No. Price
28898 PS2407 (280 available) \$19.95

Jameco 45 Watt Switching Power Supply

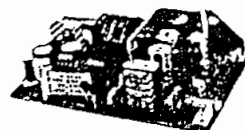


- Input: 110 V/220V @ 60/50 Hz
- 6 pin Molex style header connector
- Size: 5"L x 3"W x 1.6"H
- Weight: .5 lbs.

Output:
+5V @ 3.0A
+12V @ 2.0 A
-12V @ 0.5 A

Part No. Product No. Price
88401 PT4002 (stock item) \$39.95

Zenith 45 Watt Switching Power Supply

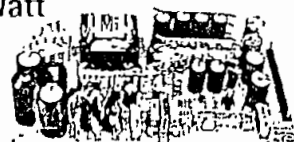


- Input: 110 VAC (Molex style connector) @ 60Hz
- Size: 6.25"L x 4"W x 1.75"H

Output:
+5V @ 3A
+12V @ 2.2A
-12V @ .5A
• Weight: .5lbs.

Part No. Product No. Price
78887 PS204 (300 available) \$19.95

Astec 50 Watt Switching Power Supply

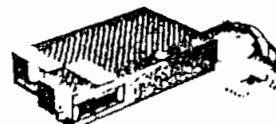


- Input: 115V @ 60Hz
- 8-pin rectangular post male header connector with 2" centers for voltage output
- Size: 7.75"L x 4.25"W x 2"H
- Weight: 1lb.

Output:
+5V @ 6.0A
+12V @ .5A
-5V @ .5A
-12V @ 1A

Part No. Product No. Price
28871 PS9250 (360 available) \$19.95

Teapo 51 Watt Switching Power Supply



- Input: 110V @ 2A
- Size: 8"L x 4.5"W x 1.75"H
- Weight: 2.5 lbs.

Output:
+5VDC @ 2.0-8.0A
+12V @ 0.5A
-5V @ 0.04A
-12VDC @ 0.4A

Part No. Product No. Price
96680 TP433B (400 available) \$19.95



Amperor 53 Watt Switching Power Supply

- Input: 115-220VAC @ 47-63Hz
- 8 pin molex style connector
- External power switch
- Size: 8"L x 5.75"W x 2.25"H
- Weight: 1.5 lbs.

Output:
+5VDC @ 7A
+12VDC @ 1.0A
-12VDC @ 0.5A

Part No. Product No. Price
65912 PS53 (1100 Available) \$19.95

• Order Toll Free 1-800-831-4242 • FAX Toll Free 1-800-237-6948 • BBS 415-637-9025

(a la Recomendación G.702)

Velocidades binarias utilizables disponibles para los servicios

En el caso del acceso a la RDSI para los servicios de banda ancha, en las Recomendaciones de la serie I.200 se especifican las velocidades binarias hasta el primer nivel de jerarquía.

En general, con referencia a las velocidades binarias disponibles para el transporte de las señales de servicio, se aplicarán las siguientes directrices:

A.1 En el caso de las redes que utilizan las jerarquías basadas en la velocidad primaria de 1544 kbit/s, se ha establecido el principio por el que algunos bits de la trama deben reservarse, en particular para el control de extremo a extremo de la calidad de los trayectos digitales cuando hay varias secciones digitales en tándem. Un ejemplo de la aplicación de este principio lo ofrece la velocidad de 1544 kbit/s, en la que se reservan algunos bits tal fin (véase la Recomendación G.704). Dicho principio no implica por necesidad que existe ninguna restricción básica con respecto a la provisión de la jerarquía completa de velocidades binarias. Por ejemplo, a 6312 kbit/s no existe ninguna restricción fundamental respecto de la utilización de la capacidad total del trayecto digital. No obstante, quizá sea preciso tomar en cuenta los principios mencionados.

A.2 En el caso de redes que utilizan la jerarquía basada en 2048 kbit/s, no hay ninguna restricción básica a la utilización de la capacidad total del trayecto digital. Sin embargo, se reconoció que la compatibilidad con las estructuras de trama recomendadas para los diversos niveles de la jerarquía de 2 Mbit/s (por ejemplo, utilización de los mismos esquemas de alineación de trama) podría ser una solución preferida, puesto que ofrece las siguientes ventajas:

- utilización de los mismos dispositivos de codificación para las aplicaciones conmutadas y no conmutadas;
- control de la calidad de extremo a extremo realizada por la red cuando la entidad de mantenimiento que termina el servicio (por ejemplo, el dispositivo de codificación) no pertenece a la red;
- posibilidad de realizar otras funciones necesarias de gestión de red, según las aplicaciones.

Podría reconsiderarse la preferencia por la compatibilidad de las estructuras de trama recomendadas para las aplicaciones en las que puedan identificarse importantes restricciones sobre la utilización eficaz de la capacidad del trayecto digital.

Recomendación G.703**CARACTERÍSTICAS FÍSICAS Y ELÉCTRICAS DE LOS INTERFACES DIGITALES JERÁRQUICOS***(Ginebra, 1972, modificada posteriormente)*

El CCITT

considerando

que se necesitan especificaciones sobre interfaces para poder interconectar los componentes de las redes digitales (secciones digitales, equipo multiplex, centrales) a fin de formar un enlace digital internacional o una conexión digital internacional;

que la Recomendación G.702 define los niveles jerárquicos;

que la Recomendación G.704 trata de las características funcionales de los interfaces asociados con los modos de la red;

que la serie I.430 de Recomendaciones trata de las características de la capa 1 para los interfaces usuario-red de la RDSI;

recomienda

que las características físicas y eléctricas de los interfaces, a las diferentes velocidades binarias jerárquicas estén conformes a la descripción dada en la presente Recomendación.

Observación 1 — Las características de los interfaces a las velocidades binarias no jerárquicas se especifican en las Recomendaciones pertinentes sobre el equipo.

Observación 2 — Las especificaciones de los valores de fluctuación de fase contenidas en los § 6, 7, 8 y 9 están destinadas a su aplicación en los puntos de interconexión internacional.

Observación 3 — Los interfaces descritos en los § 2 a 9 de la presente Recomendación corresponden a los accesos T (acceso de salida) y T' (acceso de entrada) conforme se recomienda para la interconexión en la Recomendación AC/9 del CCIR con referencia al Informe AH/9 de la Comisión de Estudio 9 del CCIR (en dicho Informe se definen los puntos T y T').

1 Interfaz a 64 kbit/s

1.1 Requisitos funcionales

1.1.1 Para el diseño del interfaz se han recomendado los requisitos fundamentales siguientes:

1.1.2 Tres señales atraviesan el interfaz en los dos sentidos, transmisión y recepción, a saber:

- la señal de información a 64 kbit/s;
- la señal de temporización de 64 kHz;
- la señal de temporización de 8 kHz.

Observación 1 — Se debe generar una señal de temporización de 8 kHz, pero no será obligatorio para el equipo en el lado de servicios del interfaz (por ejemplo, señales de datos o señalización) utilizar la señal de temporización de 8 kHz procedente del multiplex MIC o del equipo de acceso a un intervalo de tiempo, ni proporcionar una señal de temporización de 8 kHz al equipo MIC.

Observación 2 — La detección de una avería en un punto situado hacia el origen puede transmitirse a través de un interfaz a 64 kbit/s enviando una señal de indicación de alarma (AIS), interrumpiendo la señal de temporización de 8 kHz en el sentido de recepción, o de ambas formas.

1.1.3 El interfaz debe ser independiente de la secuencia de bits a 64 kbit/s.

Observación 1 — Pueden transmitirse a través del interfaz señales a 64 kbit/s sin ninguna restricción. Sin embargo, esto no implica que puedan realizarse, sobre una base global, trayectos a 64 kbit/s no sujetos a restricción alguna. Esto se debe a que algunas Administraciones se proponen instalar o están instalando vastas redes compuestas de secciones de línea digital cuyas características no permiten la transmisión de largas secuencias de 0. (La Recomendación G.733 prevé equipos multiplex MIC con características apropiadas para estas secciones de línea digital.) En lo que respecta específicamente a fuentes de trenes binarios con temporización de octetos, en redes digitales a 1544 kbit/s se exige que haya, por lo menos, un 1 binario en cada uno de los octetos de una señal digital a 64 kbit/s. En los trenes binarios no sujetos a temporización de octetos, la señal a 64 kbit/s no podrá tener más de 7 ceros consecutivos.

Observación 2 — Aunque el interfaz es independiente de la secuencia de bits, la utilización de la señal AIS (secuencia todos 1) puede dar lugar a la imposición de ciertas limitaciones de menor importancia a la fuente de 64 kbit/s. Por ejemplo, una señal de alineación de trama todos 1 podría ocasionar problemas.

1.1.4 Se han previsto tres tipos de interfaces

1.1.4.1 Interfaz codireccional

El término codireccional se utiliza para describir un interfaz a través del cual la información y las señales de temporización asociadas se transmiten en el mismo sentido (véase la figura 1/G.703).

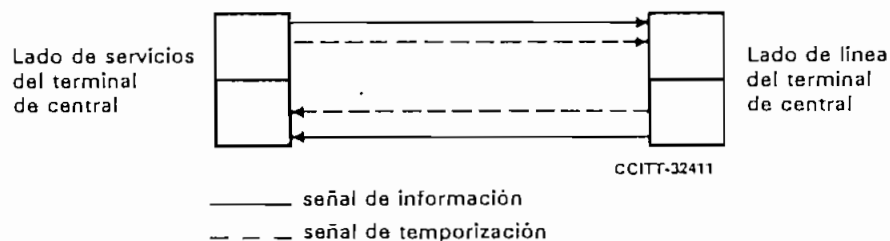


FIGURA 1/G.703
Interfaz codireccional

1.1.4.2 Interfaz de reloj centralizado

El término reloj centralizado se utiliza para describir un interfaz donde, para ambos sentidos de transmisión de la señal de información, las señales de temporización asociadas tanto al terminal de central en el lado de línea como al terminal de central en el lado de servicios se toman de un reloj centralizado que puede derivarse, por ejemplo, de ciertas señales de línea de llegada (véase la figura 2/G.703).

Observación — El interfaz codireccional o el interfaz de reloj centralizado deben utilizarse para redes sincronizadas y para redes plesiócronas cuyos relojes tengan la estabilidad requerida (véase la Recomendación G.811), a fin de asegurar un intervalo adecuado entre los deslizamientos.

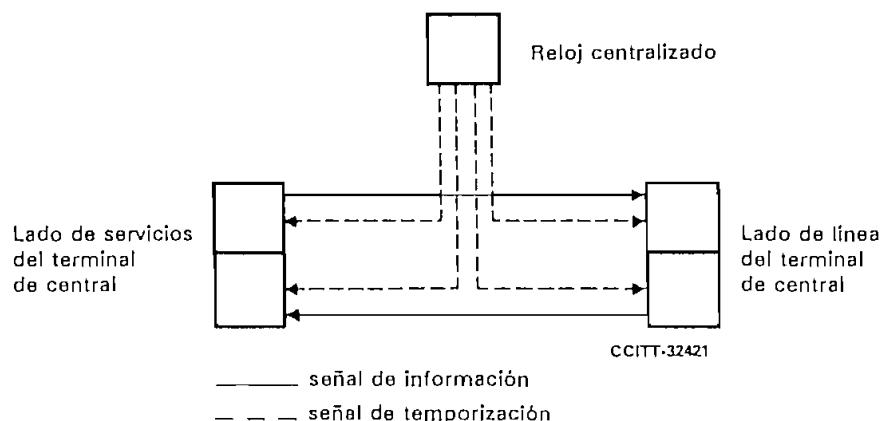


FIGURA 2/G.703
Interfaz de reloj centralizado

1.1.4.3 Interfaz contradireccional

El término contradireccional se utiliza para caracterizar un interfaz a través del cual las señales de temporización asociadas a ambas direcciones de transmisión se dirigen hacia el lado de servicios (por ejemplo, datos o señalización) del interfaz (véase la figura 3/G.703).

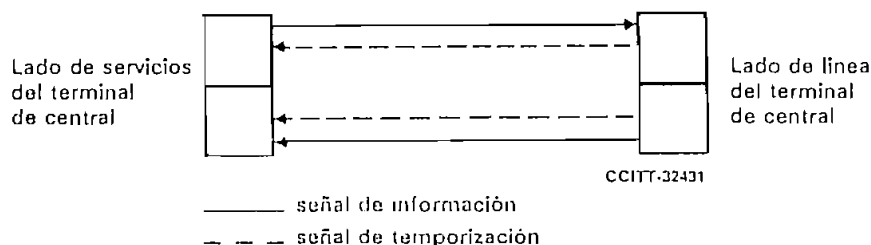


FIGURA 3/G.703
Interfaz contradireccional

1.2 Características eléctricas

1.2.1 Características eléctricas del interfaz codireccional a 64 kbit/s

1.2.1.1 Consideraciones generales

1.2.1.1.1 Velocidad binaria nominal: 64 kbit/s.

1.2.1.1.2 Tolerancia máxima para las señales transmitidas a través del interfaz: ± 100 ppm.

1.2.1.1.3 Las señales de temporización de 64 kHz y 8 kHz se transmitirán codireccionalmente con la señal de información.

1.2.1.1.4 Se utilizará un par simétrico para cada sentido de transmisión; se recomienda la utilización de transformadores.

1.2.1.1.5 Reglas de conversión de código:

Paso 1 — Un periodo de un bit a 64 kbit/s se divide en cuatro intervalos unitarios.

Paso 2 — Un 1 binario se codifica como un bloque constituido por los cuatro bits siguientes:

1 1 0 0

Paso 3 — Un 0 binario se codifica como un bloque constituido por los cuatro bits siguientes:

1 0 1 0

Paso 4 — La señal binaria se convierte en una señal de tres niveles alternando la polaridad de los bloques consecutivos.

Paso 5 — La alternancia de la polaridad de los bloques se viola cada octavo bloque. El bloque con violación indica el último bit de un octeto.

Estas reglas de conversión se ilustran en la figura 4/G.703.

1.2.1.2 Especificaciones en los accesos de salida (véase el cuadro 1/G.703)

1.2.1.3 Especificaciones en los accesos de entrada

La señal digital presentada en los accesos de entrada deberá corresponder a la definición precedente, con las modificaciones que introduzcan las características de los pares de interconexión. La atenuación de estos pares está comprendida entre 0 y 3 dB a la frecuencia de 128 kHz. Esta atenuación tendrá en cuenta posibles pérdidas debidas a la presencia de un repartidor digital entre los equipos.

Observación — Si el par simétrico está blindado, el blindaje se conectará a tierra en el acceso de salida, y se preverá, en caso necesario, su conexión a tierra en el acceso de entrada.

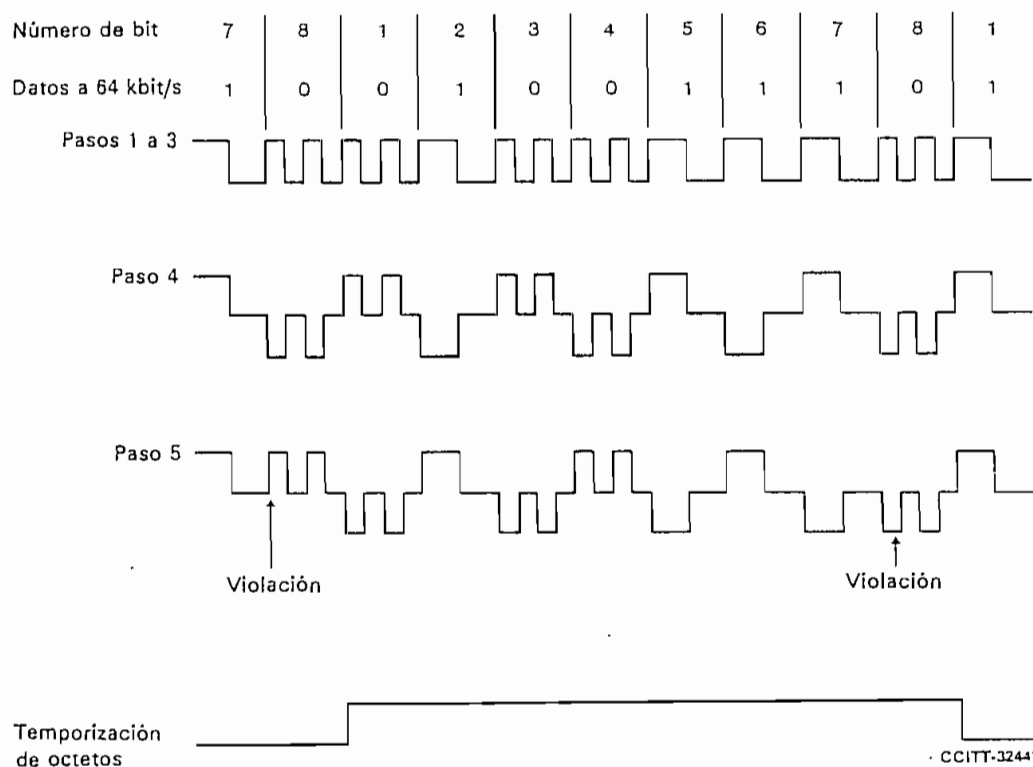
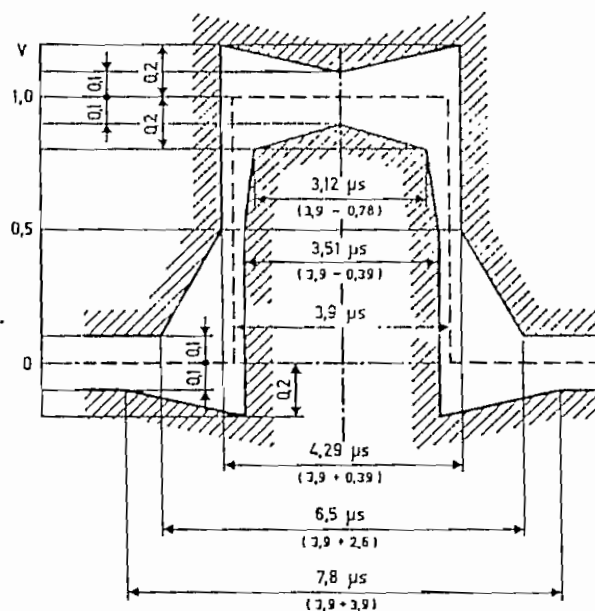
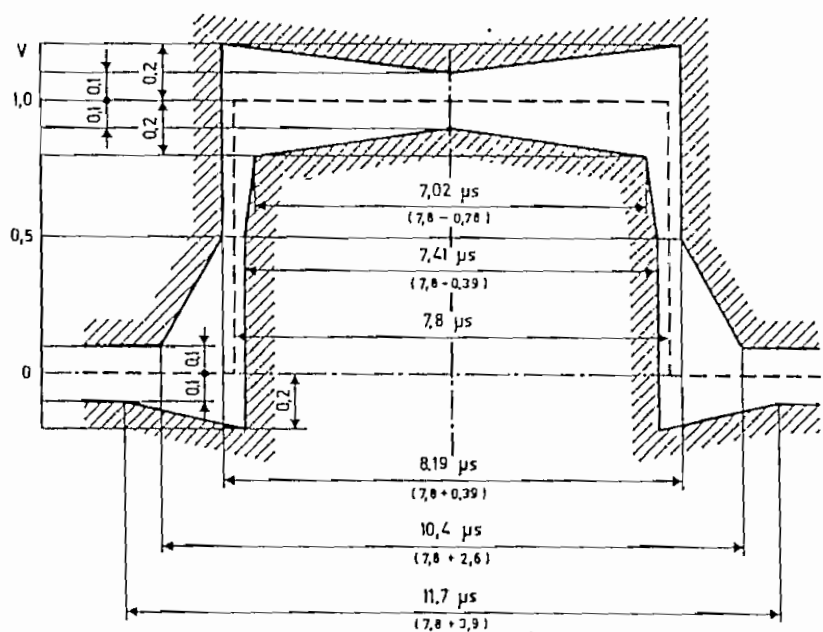


FIGURA 4/G.703



a) Plantilla para un impulso simple



CCITT-16320

b) Plantilla para un impulso doble

Observación — Los límites se aplican a impulsos de cualquier polaridad.

FIGURA 5/G.703

Plantillas para los impulsos en el caso de un interfaz codireccional a 64 kbit/s

CUADRO 1/G.703

Velocidad de símbolos	256 kbaudios
Forma del impulso (forma nominal, rectangular)	Todos los impulsos de una señal válida deben ajustarse a la plantilla de la figura 5/G.703, sea cual fuere la polaridad
Par(es) en cada sentido de transmisión	Un par simétrico
Impedancia de carga de prueba	120 ohmios, resistiva
Tensión de cresta nominal de una «marca» (impulso)	1,0 V
Tensión de cresta de un «espacio» (ausencia de impulso)	0 V $\pm$ 0,10 V
Anchura nominal del impulso	3,9 μ s
Relación entre la amplitud de los impulsos positivos y la de los negativos en el centro del intervalo unitario	De 0,95 a 1,05
Relación entre la anchura de los impulsos positivos y la de los negativos en el punto de semiamplitud nominal	De 0,95 a 1,05

1.2.2 Características eléctricas del interfaz de reloj centralizado a 64 kbit/s

1.2.2.1 Consideraciones generales

1.2.2.1.1 Velocidad binaria nominal: 64 kbit/s. La tolerancia viene determinada por la estabilidad del reloj de la red (véase la Recomendación G.811).

1.2.2.1.2 Para cada sentido de transmisión deberá haber un par simétrico de hilos para la señal de datos. Además, deberá haber pares simétricos de hilos para transportar la señal de temporización compuesta (64 kHz y 8 kHz) de la fuente de reloj central al equipo terminal de central. Se recomienda la utilización de transformadores.

1.2.2.1.3 Reglas de conversión de código

Las señales de datos se codifican en código AMI y los impulsos tienen una relación de trabajo de 100%. Las señales compuestas de temporización transportan la información de temporización de bits a 64 kHz en código AMI con una relación de trabajo de 50 a 70%, y la información sobre la fase del octeto a 8 kHz mediante violaciones a la regla de codificación. La estructura de las señales y sus relaciones de fase nominales se muestran en la figura 6/G.703.

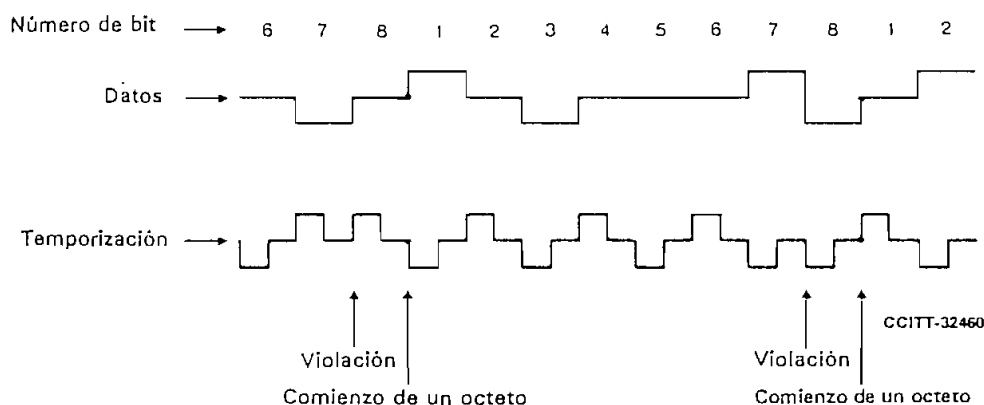


FIGURA 6/G.703

Estructura de las señales en los accesos de salida del terminal de central para el interfaz de reloj centralizado a 64 kbit/s

La corriente de datos en los accesos de salida debe temporizarse por el frente anterior del impulso de temporización, y el instante de detección en los accesos de entrada debe temporizarse por el frente posterior de cada impulso de temporización.

1.2.2.2 Características de los accesos de salida (véase el cuadro 2/G.703).

CUADRO 2/G.703

Parámetros	Datos	Temporización
Forma del impulso	Forma nominal rectangular, con tiempos de establecimiento y caída inferiores a 1 μ s	Forma nominal rectangular, con tiempos de establecimiento y caída inferiores a 1 μ s
Impedancia de carga nominal de prueba	110 ohmios, resistiva	110 ohmios, resistiva
Tensión de cresta de una «marca» (impulso)	a) $1,0 \pm 0,1$ V b) $3,4 \pm 0,5$ V	a) $1,0 \pm 0,1$ V b) $3,0 \pm 0,5$ V
Tensión de cresta de un «espacio» (ausencia de impulso)	a) $0 \pm 0,1$ V b) $0 \pm 0,5$ V	a) $0 \pm 0,1$ V b) $0 \pm 0,5$ V
Anchura nominal del impulso	a) 15,6 μ s b) 15,6 μ s	a) 7,8 μ s b) 9,8 a 10,9 μ s

Observación — La elección entre los juegos de parámetros a) y b) permite tener en cuenta diferentes ambientes de ruido de central y diferentes longitudes máximas de cable entre los tres equipos de central implicados.

1.2.2.3 Características de los accesos de entrada

Las señales digitales presentadas en los accesos de entrada deberán corresponder a la definición precedente, con las modificaciones que introduzcan las características de los pares de interconexión. Los parámetros variables del cuadro 2/G.703 permitirán obtener distancias de interconexión máximas típicas de 350 a 450 m.

1.2.2.4 Características del cable

Las características de transmisión del cable que ha de utilizarse deben seguir estudiándose.

1.2.3 Características eléctricas del interfaz contradiereccional a 64 kbit/s

1.2.3.1 Consideraciones generales

1.2.3.1.1 Velocidad binaria: 64 kbit/s.

1.2.3.1.2 Tolerancia máxima para las señales que se transmitan por el interfaz: ± 100 ppm.

1.2.3.1.3 Para cada sentido de transmisión deberá haber dos pares simétricos: uno para la señal de datos y otro para una señal de temporización compuesta (64 kHz y 8 kHz). Se recomienda la utilización de transformadores.

Observación — Si es necesario, a escala nacional, proporcionar una indicación de alarma separada a través del interfaz, esto puede realizarse interrumpiendo la señal de temporización de 8 kHz en el sentido de transmisión. Se trata, es decir, inhibiendo las violaciones de código introducidas en la señal de temporización compuesta correspondiente (véase más adelante).

1.2.3.1.4 Reglas de conversión de código

Las señales de datos se codifican en código AMI y los impulsos tienen una relación de trabajo del 50%. Las señales compuestas de temporización transportan la información de temporización de bits a 64 kHz mediante el empleo del código AMI con una relación de trabajo del 50%, y la información sobre la fase de la señal de temporización de octetos a 8 kHz, introduciendo violaciones a la regla de codificación. La estructura de las violaciones de fase en los accesos de salida de datos se muestran en la figura 7/G.703.

Los i
retardarán al
impulso de c
anterior del s

1.2.3.1.5 Es

Forma del im
rectangular)

Par(es) en cac

Impedancia d

Tensión de cr
«marca» (im

Tensión de cr
(ausencia de i

Anchura nom

Relación entr
impulsos posi
en el punto d

Relación entr
impulsos posi
en el punto d

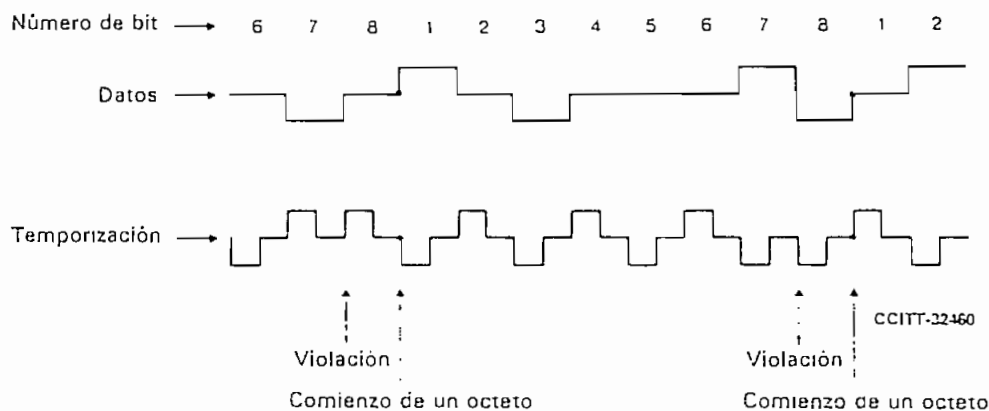


FIGURA 7/G.732

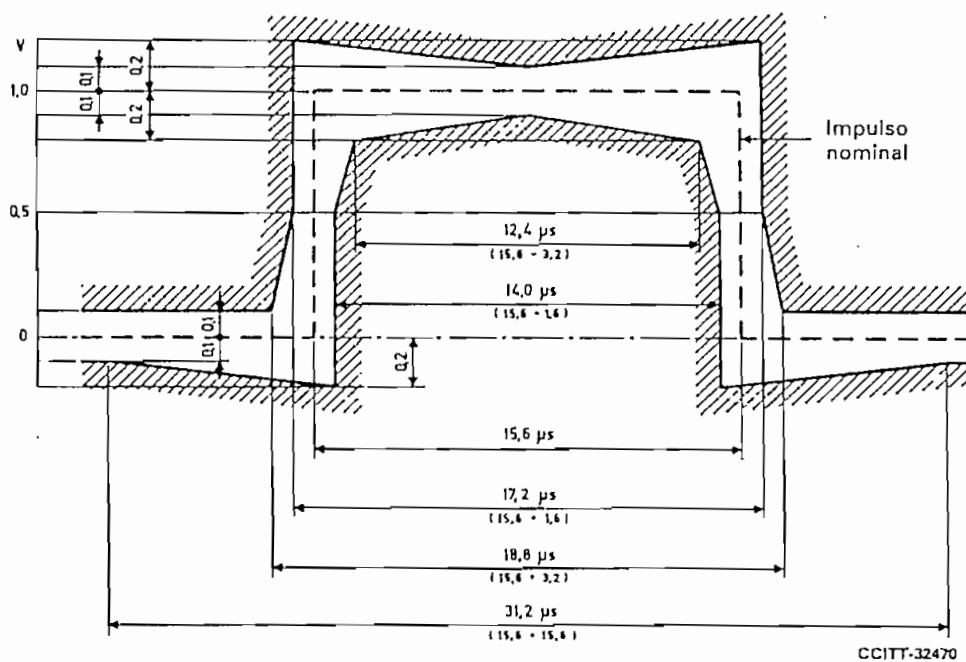
Estructura de las señales en los accesos de salida de datos para el interfaz contradiereccional a 64 kbit/s

Los impulsos de datos recibidos del lado de servicios (por ejemplo: datos o señalización) del interfaz se retardarán algo en relación con los impulsos de temporización correspondientes. El instante de detección de un impulso de datos recibido del lado de línea (por ejemplo: MIC) del interfaz deberá situarse, pues, en el flanco anterior del siguiente impulso de temporización.

1.2.3.1.5 Especificaciones en los accesos de salida (véase el cuadro 3/G.703)

CUADRO 3/G.703

Parámetros	Datos	Temporización
Forma del impulso (forma nominal, rectangular)	Todos los impulsos de una señal válida deben ajustarse a la plantilla de la figura 8/G.703, sea cual fuere la polaridad	Todos los impulsos de una señal válida deben ajustarse a la plantilla de la figura 9/G.703, sea cual fuere la polaridad
Par(es) en cada sentido de transmisión	Un par simétrico	Un par simétrico
Impedancia de carga de prueba	120 ohmios, resistiva	120 ohmios, resistiva
Tensión de cresta nominal de una «marca» (impulso)	1,0 V	1,0 V
Tensión de cresta de un «espacio» (ausencia de impulso)	0 V $\pm$ 0,1 V	0 V $\pm$ 0,1 V
Anchura nominal del impulso	15,6 μ s	7,8 μ s
Relación entre la amplitud de los impulsos positivos y la de los negativos en el centro del intervalo de un impulso	De 0,95 a 1,05	De 0,95 a 1,05
Relación entre la anchura de los impulsos positivos y la de los negativos en el punto de semiamplitud nominal	De 0,95 a 1,05	De 0,95 a 1,05



Observación 1 — Cuando un impulso va inmediatamente seguido de otro de polaridad opuesta, los límites de tiempo para el paso por los puntos de amplitud cero de los impulsos serán $\pm 0,8 \mu s$.

Observación 2 — Los instantes en los que debe producirse la transición de un estado a otro de la señal de datos los determina la señal de temporización. En el lado de servicios (p.e., datos o señalización) del interfaz es esencial que estas transiciones no sean iniciadas antes de los instantes definidos por la señal de temporización recibida.

FIGURA 8/G.703

Plantilla para el impulso de datos en el caso de un interfaz contradireccional a 64 kbit/s

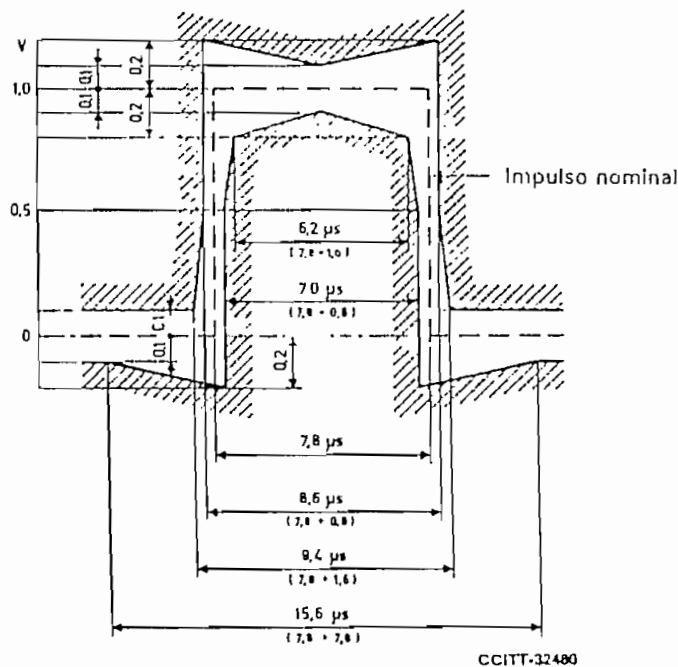


FIGURA 9/G.703

Plantilla para el impulso de temporización en el caso de un interfaz contradireccional a 64 kbit/s

1.2.3.1.6 Especificaciones en los accesos de entrada

Las señales digitales presentadas en los accesos de entrada deberán corresponder a la definición precedente, con las modificaciones que introduzcan las características de los pares de interconexión. La atenuación de estos pares está comprendida entre 0 y 3 dB, a la frecuencia 32 kHz. Esta atenuación tendrá en cuenta posibles pérdidas debidas a la presencia de un repartidor digital entre los equipos.

Observación — Si los pares simétricos están blindados, los blindajes deben conectarse a tierra en el acceso de salida, y se tomarán medidas para, en caso necesario, conectarlos también a tierra en el acceso de entrada.

2 Interfaz a 1544 kbit/s

2.1 La interconexión de señales a 1544 kbit/s a los fines de la transmisión se hace en un repartidor digital.

2.2 La velocidad binaria de la señal debe ser de 1544 kbit/s $\pm$ 50 partes por millón (ppm).

2.3 Se utilizará un par simétrico para cada sentido de transmisión. El jack del repartidor conectado a un par por el que llegan las señales al repartidor se denomina jack de entrada.

El jack del repartidor conectado a un par por el que salen las señales del repartidor se denomina jack de salida.

2.4 La impedancia de carga de prueba será de 100 ohmios, resistiva.

2.5 Se utilizará un código AMI (bipolar) o un código B8ZS. La conexión de sistemas de línea exige un contenido de señal apropiado para garantizar una información de temporización adecuada. Esto puede efectuarse bien mediante codificación B8ZS, mediante pseudoaleatorización, o bien, no permitiendo más de 15 espacios entre marcas sucesivas y asegurando una densidad media de marcas de, por lo menos, 1 de 8.

2.6 La forma de un impulso aislado medido en el jack de salida o en el de entrada deberá estar comprendido dentro de los límites de la plantilla de la figura 10/G.703 y cumplir las demás condiciones indicadas en el cuadro 4/G.703. Para formas de impulso que cumple esta plantilla, la suboscilación de cresta no debe ser superior al 40% del valor de cresta del impulso (marca).

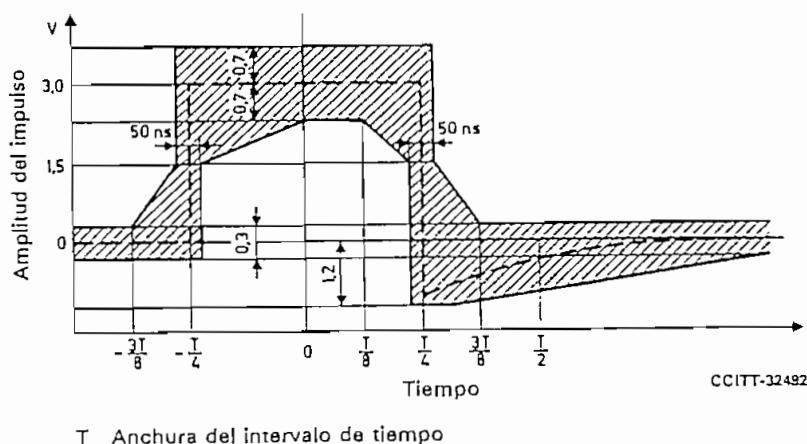


FIGURA 10/G.703

Plantilla para el impulso en el caso de un interfaz a 1544 kbit/s

2.7 La tensión en un intervalo de tiempo que contenga un 0 (espacio) no será superior al mayor de los dos valores siguientes: valor producido en dicho intervalo de tiempo por otros impulsos (marcas) conformes a la plantilla de la figura 10/G.703 o $\pm$ 0,1 de la amplitud de cresta del impulso (marca).

CUADRO 4/G.703
Interfaz digital a 1544 kbit/s^{a)}

Ubicación	Repartidor digital
Velocidad binaria	1544 kbit/s
Par(es) en cada sentido de transmisión	Un par simétrico
Código	AMI ^{b)} o B8ZS ^{c)}
Impedancia de carga de prueba	100 ohmios, resistiva
Forma nominal del impulso	Rectangular
Potencia a 772 kHz	De +12 dBm a +19 dBm
Potencia a 1544 kHz	Por lo menos 25 dB por debajo del nivel de potencia a 772 kHz

La plantilla del impulso para el interfaz digital de primer orden se reproduce en la figura 10/G.703.

ver el § 2.5.

El código B8ZS es un código AMI modificado en el cual se reemplazan ocho ceros consecutivos por 000+-0-+- si el impulso precedente era +, y por 000-+-0+- si era -.

El nivel de la señal es el nivel de potencia medido en una banda de 3 kHz en el jack de entrada para una secuencia «todos 1» transmitida.

- a) En las figuras 10 y 11.
b) El código B6ZS es el código B8ZS con los dos primeros ceros reemplazados por +, y por 0-+- si era +, y por 0+- si era -.
c) El código B8ZS es el código B6ZS con los dos primeros ceros reemplazados por +, y por 0-+- si era +, y por 0+- si era -.

4.4 La impedancia de carga de prueba deberá ser de 100 ohmios.

4.5 Se utilizará un código pseudoternario como se indica en el cuadro 5/G.703.

4.6 La forma de un impulso aislado medido en el jack de salida o en el de entrada deberá quedar dentro de los límites de la plantilla de la figura 11/G.703 o la de la figura 12/G.703, y cumplir las demás condiciones dadas en el cuadro 5/G.703.

4.7 La tensión en un intervalo de tiempo que contenga un 0 (espacio) no será superior al mayor de los dos valores siguientes: valor producido en dicho intervalo por otros impulsos (marcas) conformes a la plantilla de la figura 11/G.703, o $\pm 0,1$ de la amplitud de cresta del impulso (marca).

4.8 Para la interfaz a 32 064 kbit/s la impedancia de carga de prueba será de 100 ohmios.

16 032 kbit/s
32 064 kbit/s

4.9 Impedancia de carga de prueba

Interfaz a 6312 kbit/s

La interconexión de señales a 6312 kbit/s a los fines de la transmisión se hace en el repartidor digital.

La velocidad binaria de la señal debe ser de 6312 kbit/s ± 30 ppm.

Se utilizará un par simétrico con una impedancia característica de 110 ohmios, o un par coaxial con una impedancia característica de 75 ohmios, para cada sentido de transmisión. El jack del repartidor conectado a un par coaxial por el que llegan las señales al repartidor se denomina jack de entrada. El jack del repartidor conectado a un par coaxial por el que salen las señales del repartidor se denomina jack de salida.

La impedancia de carga de prueba será resistiva de 110 o de 75 ohmios según proceda.

Se utilizará un código pseudoternario como se indica en el cuadro 5/G.703.

La forma de un impulso aislado medido en el jack de salida o en el de entrada deberá quedar dentro de los límites de la plantilla de la figura 11/G.703 o la de la figura 12/G.703, y cumplir las demás condiciones dadas en el cuadro 5/G.703.

La tensión en un intervalo de tiempo que contenga un 0 (espacio) no será superior al mayor de los dos valores siguientes: valor producido en dicho intervalo por otros impulsos (marcas) conformes a la plantilla de la figura 11/G.703, o $\pm 0,1$ de la amplitud de cresta del impulso (marca).

Interfaz a 32 064 kbit/s

La interconexión de señales a 32 064 kbit/s para fines de transmisión se efectúa en un repartidor digital.

La señal deberá tener una velocidad binaria de 32 064 kbit/s con una tolerancia de ± 10 ppm.

Se utilizará un par coaxial para cada sentido de transmisión. El jack del repartidor conectado a un par coaxial por el que entran las señales en el repartidor se denomina jack de entrada. El jack del repartidor conectado a un par coaxial por el que salen las señales del repartidor se denomina jack de salida.

CUADRO 3/G.703
Interfaz digital a 6312 kbit/s^{a)}

Ubicación	Repartidor digital		
Velocidad binaria	6312 kbit/s		
Par(es) en cada sentido de transmisión	Un par simétrico	Un par coaxial	
Código	B6ZS ^{b)}	B8ZS ^{c)}	
Impedancia de carga de prueba	110 ohmios, resistiva	75 ohmios, resistiva	
Forma nominal del impulso	Rectangular, determinada por la atenuación del cable (véase la figura 11/G.703)	Rectangular (véase la figura 12/G.703)	
Nivel de la señal	Cuando se transmite una secuencia todos l deben obtenerse los siguientes niveles de potencia, medidos en una banda de 3 kHz: 3156 kHz: de 0,2 a 7,3 dBm 6312 kHz: -20 dBm o menos		3156 kHz: de 6,2 a 13,3 dBm 6312 kHz: -14 dBm o menos

^{a)} En las figuras 11/G.703 y 12/G.703 se reproduce la plantilla del impulso para el interfaz digital de segundo orden.

^{b)} El código B6ZS es un código AMI modificado en el cual seis ceros consecutivos se reemplazan por 0+-0-+ si el impulso anterior era +, y por 0--0+- si era -.

^{c)} El código B8ZS es un código AMI modificado en el cual se reemplazan ocho ceros consecutivos por 000+-0-+ si el impulso precedente era +, y por 000--0+- si era -.

4.4 La impedancia de carga de prueba deberá ser de 75 ohmios $\pm$ 5%, resistiva, y el método de prueba deberá ser directo.

4.5 Se utilizará un código AMI pseudoaleatorizado.

4.6 La forma de un impulso aislado medido en el jack de entrada deberá estar comprendida en la plantilla de la figura 13/G.703.

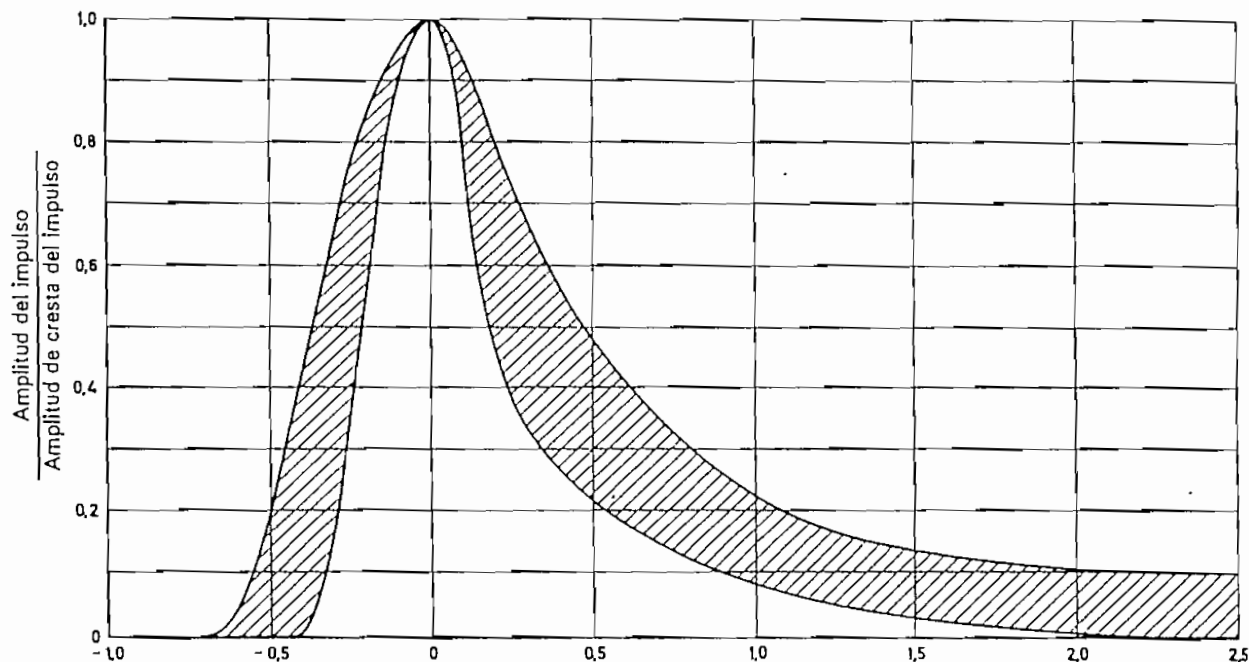
4.7 La tensión en un intervalo de tiempo que contenga un 0 (espacio) no será superior al mayor de los dos valores siguientes: el valor producido en ese intervalo de tiempo por otros impulsos (marcas) comprendidos en la plantilla de la figura 13/G.703, o $\pm$ 0,1 de la amplitud de cresta del impulso (marca).

4.8 Para una secuencia «todos uno» transmitida, la potencia medida en una banda de 3 kHz en el jack de entrada será la siguiente:

16 032 kHz: de +5 dBm a +12 dBm,
32 064 kHz por lo menos 20 dB por debajo del nivel de potencia a 16 032 kHz.

4.9 Impedancia de los conectores y pares coaxiales en el repartidor: 75 ohmios $\pm$ 5%.

	T	Fórmula de la curva
Curva inferior	$T < -0,41$	0
	$-0,41 < T < 0,24$	$0,5 \left[1 + \sin \frac{\pi}{2} \left(1 + \frac{T}{0,205} \right) \right]$
	$0,24 < T$	$0,331 e^{-1,9(T-0,3)}$
Curva superior	$T < -0,72$	0
	$-0,72 < T < 0,2$	$0,5 \left[1 + \sin \frac{\pi}{2} \left(1 + \frac{T}{0,36} \right) \right]$
	$0,2 < T$	$0,1 + 0,72 e^{-2,13(T-0,2)}$

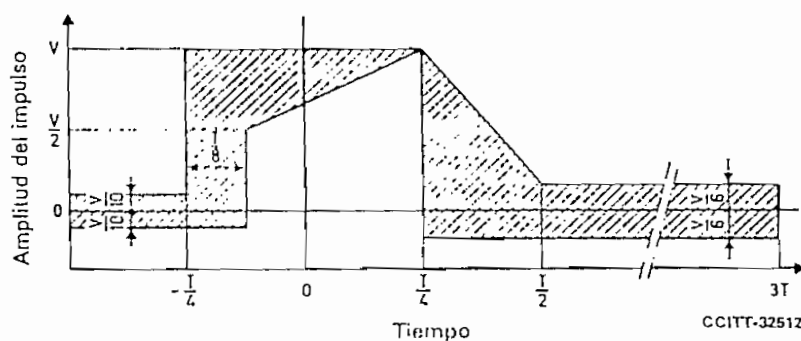


Intervalos de tiempo normalizados con respecto al punto en que se produce la cresta (T)

CCITT-32501

FIGURA 11/G.703

Plantilla del impulso para el interfaz de pares simétricos a 6312 kbit/s



CCITT-32512

T Anchura del intervalo de tiempo

FIGURA 12/G.703

Plantilla del impulso para el interfaz de pares coaxiales a 6312 kbit/s

	T	Fórmula de la curva
Curva inferior	$-0,36 \leq T < -0,30$	$5,76 T + 2,07$
	$-0,30 \leq T < 0$	$0,5 \left[1 + \operatorname{sen} \frac{\pi}{2} \left(1 + \frac{T}{0,25} \right) \right]$
	$0 \leq T < 0,22$	$0,5 \left[1 + \operatorname{sen} \frac{\pi}{2} \left(1 + \frac{T}{0,16} \right) \right]$
	$0,22 \leq T$	$0,11 e^{-3,42 (T - 0,3)}$
Curva superior	$-0,65 \leq T < 0$	$1,05 [1 - e^{-4,8 (T - 0,65)}]$
	$0 \leq T < 0,25$	$0,5 \left[1 + \operatorname{sen} \frac{\pi}{2} \left(1 + \frac{T}{0,28} \right) \right]$
	$0,25 \leq T$	$0,11 + 0,407 e^{-2,1 (T - 0,28)}$

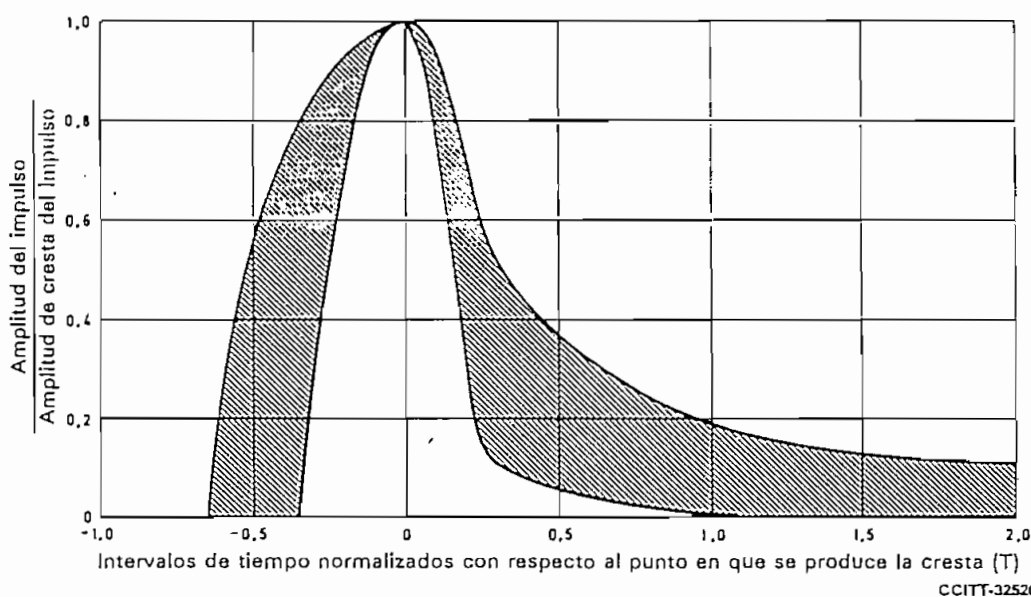


FIGURA 13/G. 703
Plantilla del impulso para el interfaz de pares coaxiales a 32 064 kbit/s

5 Interfaz a 44 736 kbit/s

- 5.1 La interconexión de señales a 44 736 kbit/s para fines de transmisión se hace en un repartidor digital.
- 5.2 La velocidad binaria de la señal debe ser de 44 736 kbit/s $\pm$ 20 ppm.
- 5.3 Se utilizará un par coaxial para cada sentido de transmisión. El jack del repartidor conectado a un par coaxial por el que entran las señales al repartidor se denomina jack de entrada. El jack del repartidor conectado a un par por el que salen las señales del repartidor se denomina jack de salida.
- 5.4 La impedancia de carga de prueba será de 75 ohmios $\pm$ 5%, resistiva, y el método de prueba será directo.
- 5.5 Se utilizará un código bipolar como el especificado en el § 5.5.1.

5.5.1 Código B3ZS

El código B3ZS (*bipolar with three-zero substitution*) es una versión modificada del formato bipolar de impulsos, denominada código bipolar con sustitución de tres ceros. Los bits lógicos 1 tienen un ciclo de trabajo del 50%, y son, generalmente, positivos y negativos alternativamente con respecto al nivel lógico cero. Las excepciones están constituidas por aquellos casos en que aparecen tres ceros lógicos consecutivos en el tren de

bits. En el formato B3ZS, cada bloque de tres ceros consecutivos se sustituye por B0V o 00V, donde B representa un impulso conforme a la regla bipolar y V representa un impulso que viola la regla bipolar. Se elige entre B0V y 00V de tal manera que el número de impulsos B entre impulsos V consecutivos sea impar. Deben insertarse bits de alineación de trama de conformidad con la Recomendación G.752.

5.6 La forma de un impulso aislado medido en el jack de entrada deberá ajustarse a la plantilla de la figura 14/G.703.

5.7 La tensión en un intervalo de tiempo que contenga un cero (espacio) no será superior al mayor de los dos valores siguientes: el valor producido en dicho intervalo de tiempo por otros impulsos (marcas) conformes a la plantilla de la figura 14/G.703 o $\pm 0,05$ de la amplitud de cresta del impulso (marca).

5.8 Cuando se transmita una secuencia todos 1, la potencia medida en una banda de 3 kHz en el jack de entrada deberá ser la siguiente:

22 368 kHz: de $-1,8$ a $+5,7$ dBm,

44 736 kHz: por lo menos 20 dB por debajo del nivel de potencia a 22 368 kHz.

5.9 El repartidor digital para señales a 44 736 kbit/s tendrá las características especificadas en los § 5.9.1 y 5.9.2.

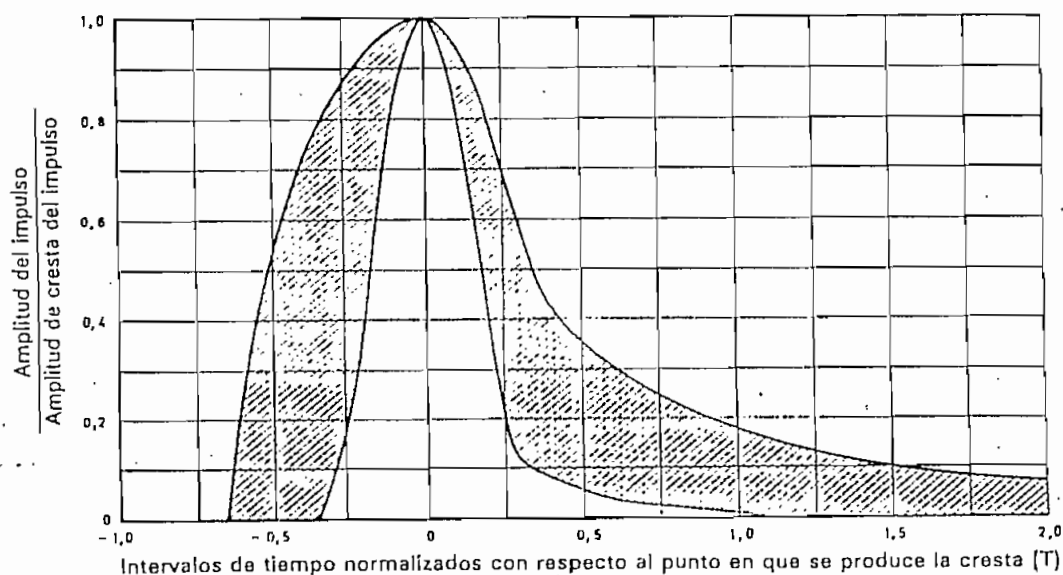
5.9.1 La atenuación entre los jacks de entrada y de salida en el repartidor será la siguiente:

$0,60 \pm 0,55$ dB a 22 368 kHz

(para cualquier combinación de características de atenuación uniforme o conformada).

5.9.2 Impedancia de los conectores y cables coaxiales en el repartidor: 75 ohmios $\pm 5\%$.

	T	Fórmula de la curva
Curva inferior	$T < -0,36$	0
	$-0,36 \leq T \leq 0,28$	$0,5 \left[1 + \sin \frac{\pi}{2} \left(1 + \frac{T}{0,18} \right) \right]$
	$0,28 < T$	$0,11 e^{-3,42 (T - 0,3)}$
Curva superior	$T < -0,65$	0
	$-0,65 < T < 0$	$1,05 [1 - e^{-4,6 (T + 0,65)}]$
	$0 \leq T \leq 0,36$	$0,5 \left[1 + \sin \frac{\pi}{2} \left(1 + \frac{T}{0,34} \right) \right]$
	$0,36 < T$	$0,05 + 0,407 e^{-1,84 (T - 0,36)}$



CCITT-32531

FIGURA 14/G.703

Plantilla del impulso para el interfaz de pares coaxiales a 44 736 kbit/s

6 Interfaz a 2048 kbit/s

6.1 Características generales

Velocidad binaria: 2048 kbit/s $\pm$ 50 ppm

Código: HDB3 (bipolar de alta densidad de orden 3) (la descripción de este código figura en el anexo A)

6.2 Especificaciones en los accesos de salida (véase el cuadro 6/G.703)

CUADRO 6/G.703

Forma del impulso (forma nominal: rectangular)	Todas las marcas de una señal válida deberán ajustarse a la plantilla (figura 15/G.703), independientemente del signo. El valor V corresponde al valor nominal de cresta	
Par(es) en cada sentido de transmisión	Un par coaxial (véase el § 6.4)	Un par simétrico (véase el § 6.4)
Impedancia de carga de prueba	75 ohmios, resistiva	120 ohmios, resistiva
Tensión nominal de cresta de una marca (impulso)	2,37 V	3 V
Tensión de cresta de un espacio (ausencia de impulso)	$0 \pm 0,237$ V	$0 \pm 0,3$ V
Anchura nominal del impulso	244 ns	
Relación entre la amplitud de los impulsos positivos y la de los negativos en el punto medio del intervalo de un impulso	De 0,95 a 1,05	
Relación entre la anchura de los impulsos positivos y la de los negativos en los puntos de semiamplitud nominal	De 0,95 a 1,05	
Fluctuación de fase máxima cresta a cresta en un acceso de salida	Véase el § 2 de la Recomendación G.823	

6.3 Especificaciones en los accesos de entrada

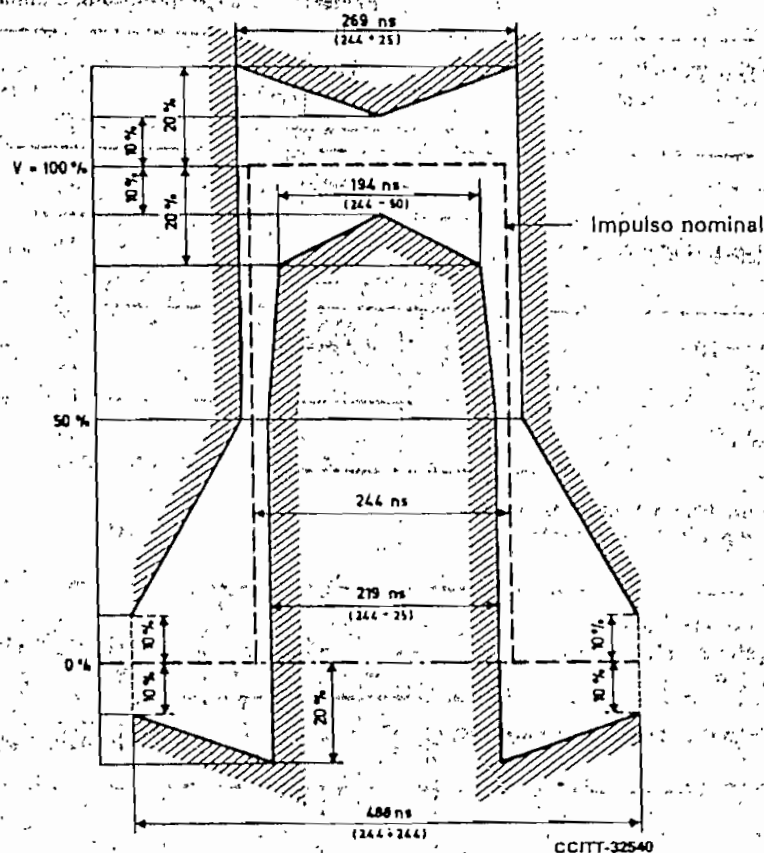
La señal digital presentada en los accesos de entrada deberá corresponder a la definición precedente, con las modificaciones que introduzcan las características de los pares de interconexión. La atenuación de estos pares deberá seguir una ley $\sqrt{f}$ y la atenuación a la frecuencia de 1024 kHz deberá estar comprendida entre 0 y 6 dB. Esta atenuación tendrá en cuenta posibles pérdidas debidas a la presencia de un repartidor digital entre los equipos.

En lo relativo a la fluctuación de fase que ha de tolerarse en los accesos de entrada, véase el § 3 de la Recomendación G.823.

La pérdida de retorno en los accesos de entrada deberá tener los siguientes valores mínimos provisionales:

Frecuencias correspondientes al porcentaje de la velocidad binaria nominal	Pérdida de retorno
2,5 a 5%	12 dB
5 a 100%	18 dB
100 a 150%	14 dB

Nota — La necesidad de incluir en la presente Recomendación un requisito en materia de inmunidad a la interferencia se halla en estudio.



Observación — V corresponde al valor de cresta nominal.

FIGURA 15/G.703
Plantilla para el impulso en el caso de un interfaz a 2048 kbit/s

6.4 Puesta a tierra del conductor exterior o del blindaje

El conductor exterior del par coaxial o el blindaje del par simétrico deberán conectarse a tierra en el acceso de salida; también deberá preverse la conexión a tierra de este conductor exterior o del blindaje en el acceso de entrada necesario.

Interfaz a 8448 kbit/s

7.1 Características generales

Velocidad binaria: 8448 kbit/s $\pm$ 30 ppm

Código: HDB3 (la descripción de este código figura en el anexo A)

7.2 Especificaciones en los accesos de salida (indicadas en el cuadro 7/G.703)

CUADRO 7/G.703

Forma del impulso (forma nominal: rectangular)	Todas las marcas de una señal válida deberán ajustarse a la planilla (figura 16/G.703), independientemente del signo
Par(es) en cada sentido de transmisión	Un par coaxial (véase el § 7.4)
Impedancia de carga de prueba	75 ohmios, resistiva
Tensión nominal de cresta de una marca (impulso)	2,37 V
Tensión de cresta de un espacio (ausencia de impulso)	$0 \pm 0,237$ V
Anchura nominal del impulso	59 ns
Relación entre las anchuras de los impulsos positivos y la de los negativos en el punto medio del intervalo de un impulso	De 0,95 a 1,05
Relación entre las anchuras de los impulsos positivos y los negativos para los puntos de semiamplitud nominal	De 0,95 a 1,05
Fluctuación de fase máxima cresta a cresta en un acceso de salida	Véase el § 2 de la Recomendación G.823

7.3 Especificaciones en los accesos de entrada

La señal digital presentada en los accesos de entrada deberá corresponder a la definición precedente, con las modificaciones que introduzcan las características de los pares de interconexión. La atenuación de estos pares deberá seguir una ley $\sqrt{f}$ y la atenuación a la frecuencia de 4224 kHz deberá estar comprendida entre 0 y 6 dB. Esta atenuación tendrá en cuenta posibles pérdidas debidas a la presencia de un repartidor digital entre los equipos.

En lo relativo a la fluctuación de fase que ha de tolerarse en los accesos de entrada, véase el § 3 de la Recomendación G.823.

La pérdida de retorno en los accesos de entrada deberá tener los siguientes valores mínimos provisionales:

Frecuencias correspondientes al porcentaje de la velocidad binaria nominal	Pérdida de retorno
2,5 a 5%	12 dB
5 a 100%	18 dB
100 a 150%	14 dB

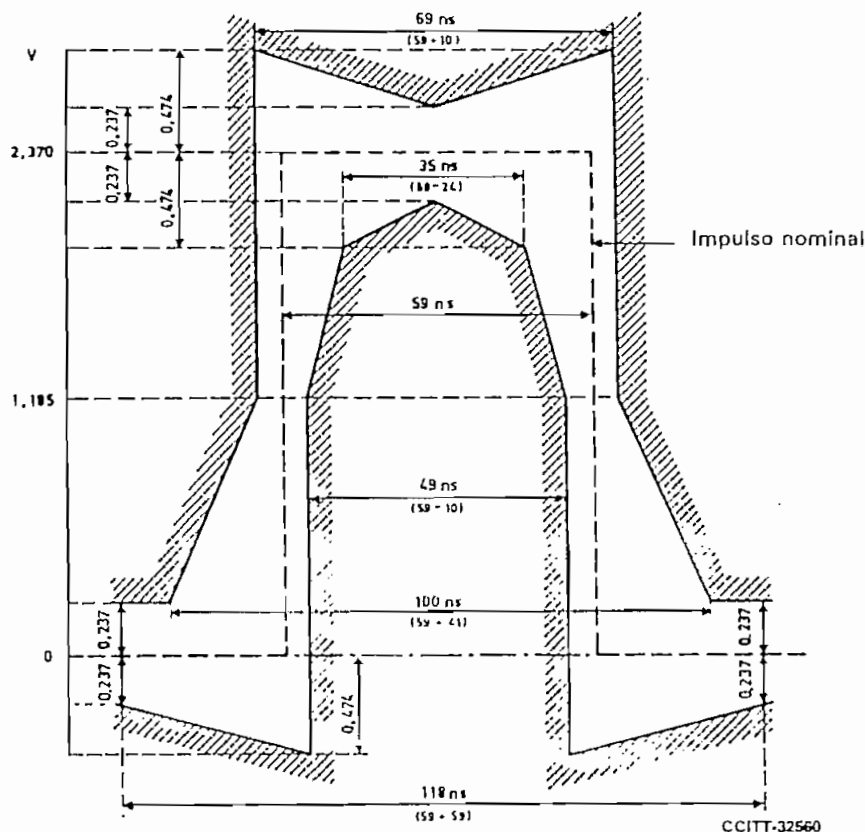


FIGURA 16/G.703

Plantilla para el impulso en el caso de un interfaz a 8448 kbit/s

Nota — La necesidad de incluir en la presente Recomendación un requisito en materia de inmunidad a la interferencia se halla en estudio.

7.4 Puesta a tierra del conductor exterior o del blindaje

El conductor exterior del par coaxial deberá conectarse a tierra en el acceso de salida y también deberá preverse la conexión a tierra de este conductor en el acceso de entrada, si es necesario.

8 Interfaz a 34 368 kbit/s

8.1 Características generales

Velocidad binaria: 34 368 kbit/s $\pm$ 20 ppm

Código: HDB3 (en el anexo A figura una descripción de este código)

8.2 Especificación en los accesos de salida (indicada en el cuadro 8/G.703)

8.3 Especificaciones en los accesos de entrada

La señal digital presentada en los accesos de entrada deberá corresponder a la definición precedente, con las modificaciones que introduzcan las características del cable de interconexión. Deberá asegurarse que la atenuación de este cable siga una ley $\sqrt{f}$ y que la atenuación a la frecuencia de 17 184 kHz esté comprendida entre 0 y 12 dB.

En lo relativo a la fluctuación de fase que ha de tolerarse en los accesos de entrada, véase el § 3 de la Recomendación G.823.

CUADRO 8/G.703

Forma del impulso (forma nominal : rectangular)	Todas las marcas de una señal válida deberán ajustarse a la plantilla (figura 17/G.703), independientemente del signo
Par(es) en cada sentido de transmisión	Un par coaxial (véase el § 8.4)
Impedancia de carga de prueba	75 ohmios, resistiva
Tensión nominal de cresta de una marca (impulso)	1,0 V
Tensión de cresta de un espacio (ausencia de impulso)	0 $\pm$ 0,1 V
Anchura nominal del impulso	14,55 ns
Relación entre la amplitud de los impulsos positivos y la de los negativos en el punto medio del intervalo de un impulso	De 0,95 a 1,05
Relación entre la anchura de los impulsos positivos y la de los negativos, en los puntos de semi-amplitud nominal	De 0,95 a 1,05
Fluctuación de fase máxima cresta a cresta en un acceso de salida	Véase el § 2 de la Recomendación G.823

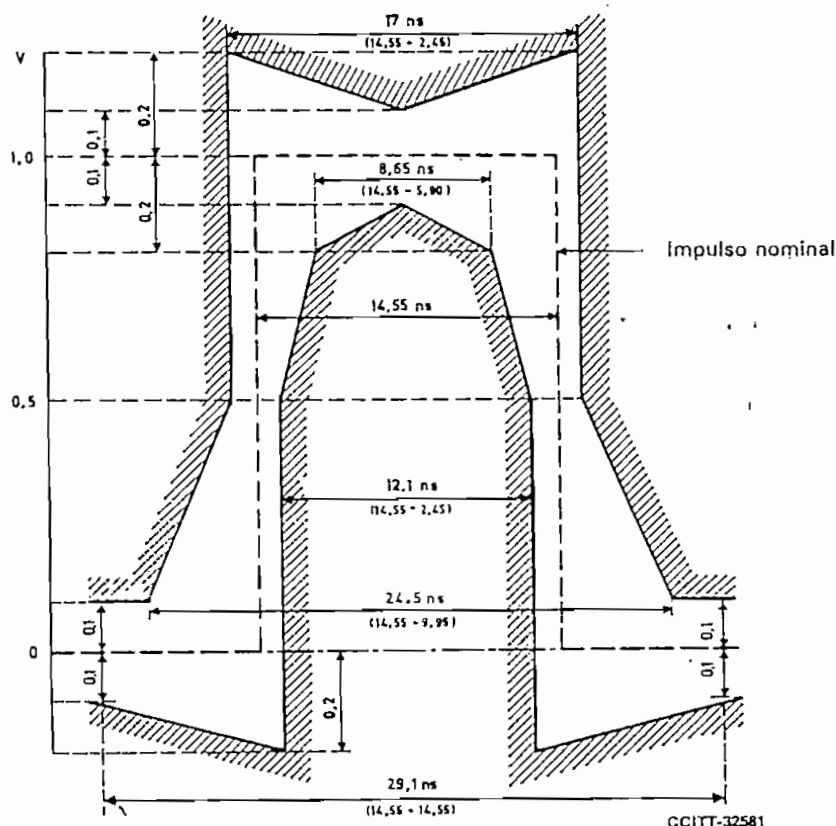


FIGURA 17/G.703

Plantilla para el impulso en el caso de un interfaz a 34368 kbit/s

La pérdida de retorno en los accesos de entrada deberá tener los siguientes valores mínimos provisionales:

Frecuencias correspondientes al porcentaje de la velocidad binaria nominal	Pérdida de retorno
2,5 a 5%	12 dB
5 a 100%	18 dB
100 a 150%	14 dB

Nota — La necesidad de incluir en la presente Recomendación un requisito en materia de inmunidad a la interferencia se halla en estudio.

Puesta a tierra del conductor exterior o del blindaje

Observación — El conductor exterior del par coaxial deberá conectarse a tierra en el acceso de salida: en el acceso de entrada, si es necesario.

Interfaz a 139 264 kbit/s

Características generales

Velocidad binaria: 139 264 kbit/s $\pm$ 15 ppm
Código: CMI (Coded Mark Inversion)

El código CMI es un código de 2 niveles sin retorno a cero en el cual el 0 binario se codifica de manera que los dos niveles de amplitud, A_1 y A_2 , se obtienen consecutivamente, cada uno durante un periodo igual a la mitad de un intervalo unitario ($T/2$).

El 1 binario se codifica de modo que los niveles de amplitud, A_1 y A_2 , se obtienen alternativamente cada uno durante un periodo igual a un intervalo unitario completo (T).

En la figura 18/G.703 se da un ejemplo.

Observación 1 — Para el 0 binario, existe siempre una transición positiva en el punto medio del intervalo de tiempo unitario binario.

Observación 2 — Para el 1 binario:

- a) existe una transición positiva al comienzo del intervalo de tiempo unitario binario si el nivel precedente era A_1 ;
- b) existe una transición negativa al comienzo del intervalo de tiempo unitario binario si el último 1 binario estaba codificado en el nivel A_2 .

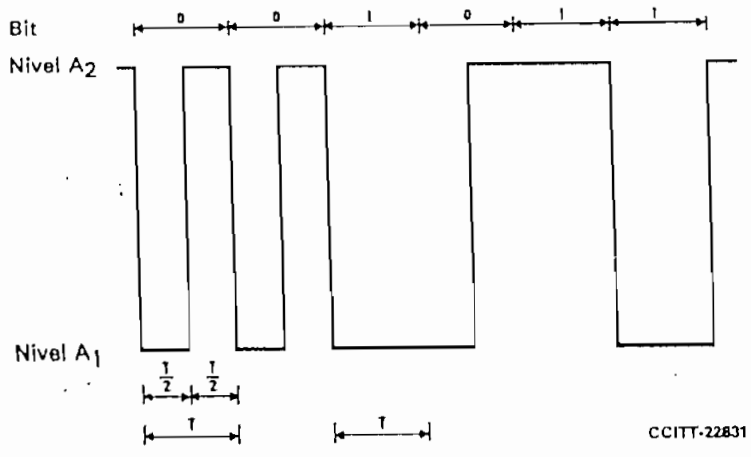


FIGURA 18/G.703
Ejemplos de señal binaria codificada en CMI

9.2 Especificaciones en los accesos de salida (indicadas en el cuadro 9/G.703)

Observación 1 — Se considera que un método basado en la medición de los niveles de la componente fundamental y del segundo (y posiblemente del tercer) armónico de una señal correspondiente a todos 0 binarios y todos 1 binarios es adecuado para verificar el cumplimiento de los requisitos indicados en el cuadro 9/G.703.

Los valores pertinentes están en estudio.

Observación 2 — Las plantillas de las figuras 19/G.703 y 20/G.703 se dan sólo como indicación, y no deben utilizarse necesariamente para mediciones.

9.3 Especificaciones en los accesos de entrada

La señal digital presentada en el acceso de entrada debe ser conforme a las indicaciones del cuadro 9/G.703, teniendo en cuenta las modificaciones producidas por las características del par coaxial de interconexión.

Debe suponerse que la atenuación del par coaxial sigue aproximadamente una ley $\sqrt{f}$ y que la pérdida de inserción máxima es de 12 dB a 70 MHz.

En lo relativo a la fluctuación de fase que ha de tolerarse en los accesos de entrada, véase el § 3 de la Recomendación G.823.

La característica de pérdida de retorno debe ser la misma que la especificada para el acceso de salida.

CUADRO 9/G.703

Forma nominal de los impulsos	Rectangular
Par(es) en cada sentido de transmisión	Un par coaxial
Impedancia de carga de prueba	75 ohmios, resistiva
Tensión cresta a cresta	$1 \pm 0,1$ voltios
Sobreoscilación	$< 5\%$ de la tensión medida de cresta a cresta
Tiempo de subida entre el 10% y el 90% de la amplitud medida	< 2 ns
Tolerancia para la temporización de las transiciones (referida al valor medio de los puntos de semiamplitud de transiciones negativas)	Transiciones negativas: $\pm 0,1$ ns Transiciones positivas en los extremos del intervalo unitario: $\pm 0,5$ ns Transiciones positivas en el punto medio del intervalo unitario: $+ 0,35$ ns
Pérdida de retorno	> 15 dB en la gama de frecuencias de 7 MHz a 210 MHz
Fluctuación de fase cresta a cresta máxima en un acceso de salida	Véase el § 2 de la Recomendación G.823

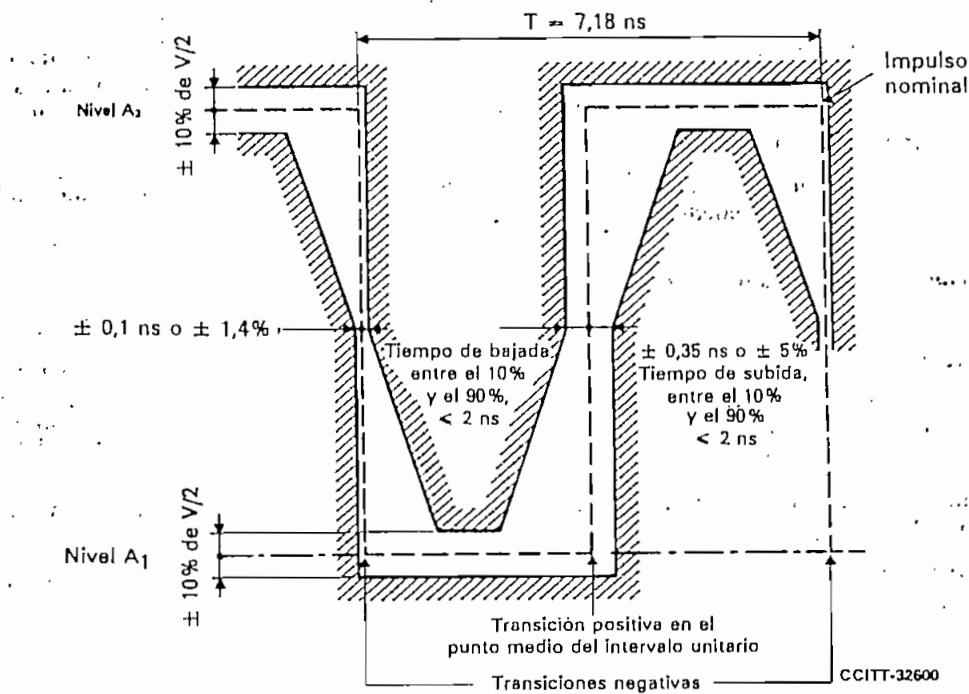
9.4 Puesta a tierra del conductor exterior o del blindaje

El conductor exterior del par coaxial debe estar conectado a tierra en el acceso de salida y debe preverse la puesta a tierra de este conductor, si es necesario, en el acceso de entrada.

10 Interfaz de sincronización a 2048 kHz

10.1 Características generales

Se recomienda la utilización de este interfaz en todas aquellas aplicaciones donde se necesite sincronizar un equipo digital mediante una señal de sincronización externa de 2048 kHz.

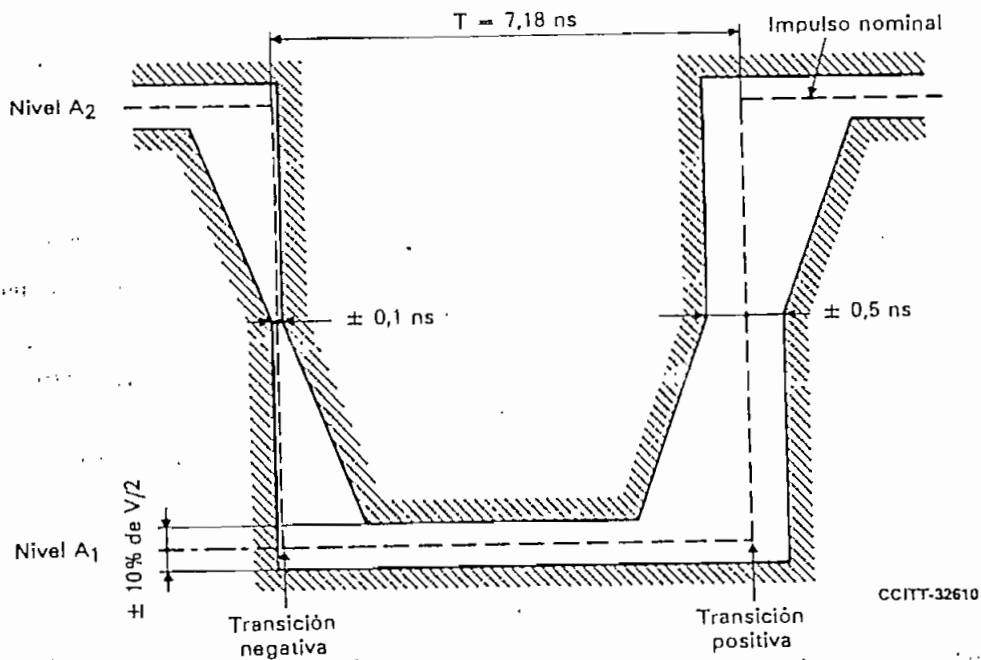


Observación 1 — V es la amplitud nominal cresta a cresta.

Observación 2 — La plantilla no incluye la tolerancia para la sobreoscilación; véase el cuadro 9/G.703.

FIGURA 19/G.703

Plantilla para un impulso que corresponde a un 0 binario



Observación 1 — El impulso inverso tendrá las mismas características.

Observación 2 — V es la amplitud nominal cresta a cresta.

Observación 3 — La plantilla no incluye la tolerancia para la sobreoscilación; véase el cuadro 9/G.703.

FIGURA 20/G.703

Plantilla para un impulso que corresponde a un 1 binario

CUADRO 10/G.703

Frecuencia	2048 kHz $\pm$ 50 ppm	
Forma de los impulsos	La señal debe ajustarse a la plantilla (figura 21/G.703) El valor V corresponde al valor de cresta máximo El valor V_1 corresponde al valor de cresta mínimo	
Tipo de par	Par coaxial (véase la observación en el § 10.3)	Par simétrico (véase la observación en el § 10.3)
Impedancia de carga de prueba	75 ohmios, resistiva	120 ohmios, resistiva
Tensión de cresta máxima (V_{op})	1,5	1,9
Tensión de cresta mínima (V_{op})	0,75	1,0
Fluctuación de fase máxima en el acceso de entrada	En estudio	

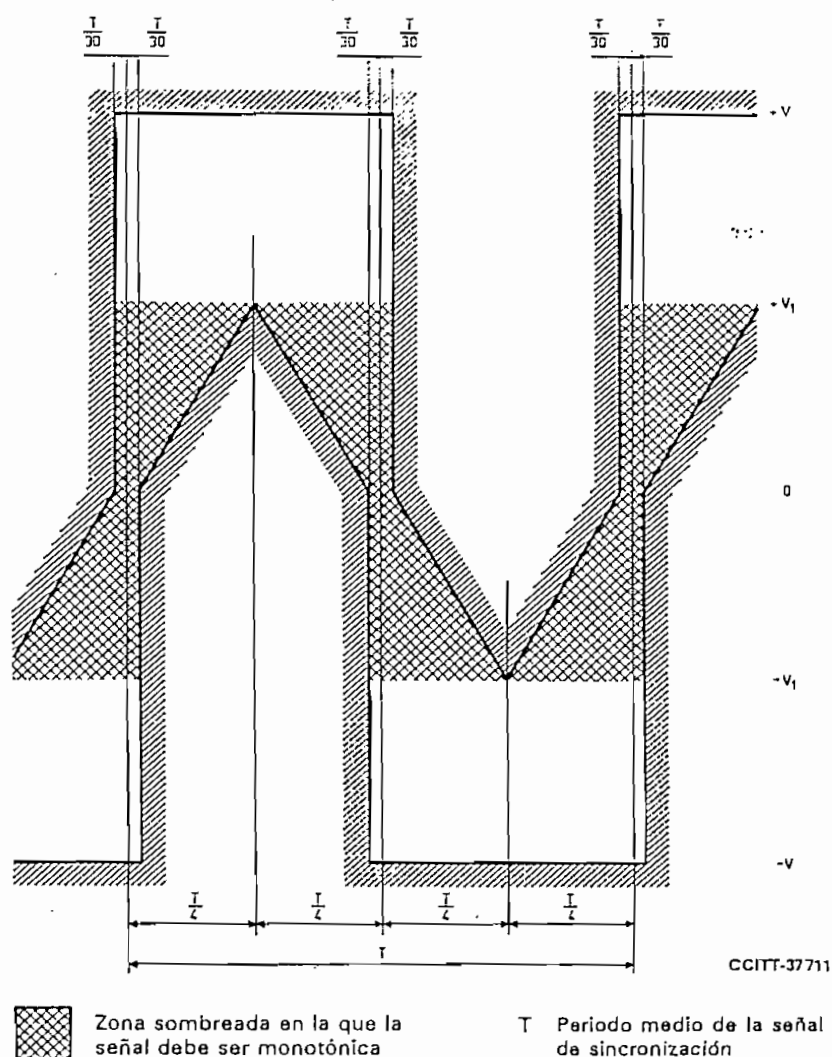


FIGURA 21/G.703

Forma de la onda en un acceso de salida

La señal presentada en los accesos de entrada deberá corresponder a la definición precedente, con las modificaciones que introduzcan las características del par de interconexión.

Se supone que la atenuación de este par obedece a la ley $\sqrt{f}$, y la atenuación a la frecuencia de 2048 kHz deberá estar comprendida entre 0 y 6 dB (valor mínimo). Esta atenuación deberá tomar en cuenta cualquier pérdida provocada por la presencia de un repartidor digital entre los equipos.

El acceso de entrada deberá ser capaz de tolerar una señal digital con estas características eléctricas, pero modulada por una fluctuación de fase. Los valores de la fluctuación de fase se hallan en estudio.

La atenuación de retorno a 2048 kHz debe ser ≥ 15 dB.

Observación — El conductor exterior del par coaxial o el blindaje del par simétrico deberán conectarse a tierra en el acceso de salida; también deberá preverse la conexión a tierra de estos elementos en el acceso de entrada, si es necesario.

11 Interfaz a 97 728 kbit/s

11.1 La interconexión de señales a 97 728 kbit/s a los fines de la transmisión se hace en un repartidor digital.

11.2 La velocidad binaria de la señal debe ser de 97 728 kbit/s ± 10 partes por millón (ppm).

11.3 Se utilizará un par coaxial para cada sentido de transmisión. El jack del repartidor conectado a un par por el que llegan las señales al repartidor se denomina jack de entrada. El jack del repartidor conectado a un par por el que salen las señales del repartidor se denomina jack de salida.

11.4 La impedancia de carga de prueba será de 75 ohmios $\pm 5\%$, resistiva.

11.5 Se utilizará un código AMI¹⁾ aleatorizado.

11.6 La forma de la señal a 97 728 kbit/s en el acceso de salida estará comprendida dentro de los límites de la plantilla de la figura 22/G.703. La forma de la señal en el jack de entrada estará modificada por las características del cable de interconexión.

11.7 Los conectores y los pares en cable en el repartidor tendrán una resistencia de 75 ohmios $\pm 5\%$.

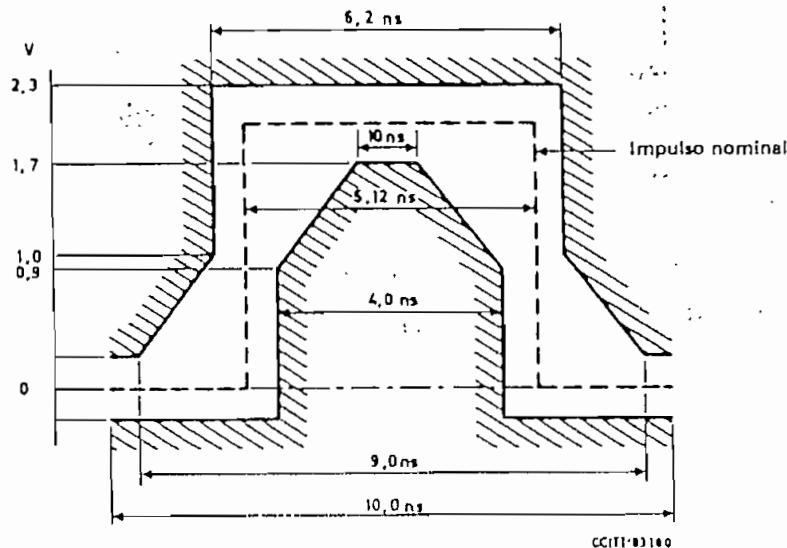


FIGURA 22/G.703

Plantilla del impulso en el acceso de salida a 97 728 kbit/s

¹⁾ Un código AMI se aleatoriza mediante un aleatorizador de cinco pasos, con reiniciación y con el polinomio generador $x^5 + x^3 + 1$.

ANEXO A

(a la Recomendación G.703)

Definición del código HDB3 (high density bipolar 3)

Para convertir una señal binaria en una señal HDB3 se aplican las siguientes reglas de codificación:

- 1) La señal HDB3 es pseudoternaria: sus tres estados se designan por B_+ , B_- y 0.
- 2) Los 0 de la señal binaria se codifican como 0 en la señal HDB3, pero en el caso de secuencias de cuatro 0 se aplican reglas particulares (véase el § 4).
- 3) Los 1 de la señal binaria se codifican alternadamente como B_+ y B_- en la señal HDB3 (inversión de marcas alternada AMI). Cuando se codifican secuencias de cuatro 0, se introducen violaciones de la regla de inversión de marcas alternada (véase el § 4).
- 4) Las secuencias de cuatro 0 de la señal binaria se codifican de acuerdo a lo siguiente:
 - a) El primer 0 de la secuencia se codifica como 0 si el 1 precedente de la señal HDB3 tiene una polaridad opuesta a la de la violación precedente y no constituye una violación; se codifica como un 1, que no constituye una violación (es decir, B_+ o B_-), si el 1 precedente de la señal HDB3 tiene la misma polaridad que la violación precedente o constituye en sí mismo una violación.
Esta regla asegura que las violaciones consecutivas sean de polaridad alternada, lo cual impide la introducción de una componente continua.
 - b) El segundo y tercer 0 de la secuencia se codifican siempre como 0.
 - c) El último 0 de la secuencia de cuatro se codifica como un 1 de polaridad tal que viole la regla de inversión de marcas alternada. Estas violaciones se designan V_+ o V_- , según su polaridad.

Recomendación G.704

CARACTERÍSTICAS FUNCIONALES DE LOS INTERFACES ASOCIADOS CON NODOS DE LA RED

(Málaga-Torremolinos, 1984)

1 Consideraciones generales

En esta Recomendación se dan las características funcionales de los interfaces asociados con:

- nodos de la red, en especial, equipos multiplex digitales sincrónicos y centrales digitales en redes digitales integradas (RDI) para telefonía y redes digitales de servicios integrados (RDSI) y,
- equipo de multiplexación MIC.

En la Recomendación G.703 se especifican las características eléctricas de estos interfaces.

Cabe señalar que esta Recomendación no se aplica necesariamente a aquellos casos en los que las señales que atraviesan los interfaces se destinan a conexiones no conmutadas, tales como el transporte de señales de banda ancha codificadas (por ejemplo señales de radiodifusión, TV o señales radiofónicas multiplexadas que no precisan un encaminamiento individual en la RDSI), véase también el anexo A a la Recomendación G.702.

Observación 1 — Las estructuras de trama recomendadas en esta Recomendación no se aplican a ciertas señales de mantenimiento, tales como la señal «todos unos» transmitida durante las condiciones de avería, u otras señales transmitidas durante las condiciones de fuera de servicio.

Observación 2 — Las Recomendaciones pertinentes para cada equipo correspondiente tratan las estructuras de trama asociadas con los equipos de multiplexación digital que utilizan justificación.

LISTADO DEL PROGRAMA "DEP" (Turbo Pascal 6.0)

```

Program DEP;

Uses
  Crt, Graph;
Type
  Vec = Array [1..4] Of String[26];
  Vec1= Array [1..10] Of String[26];
  VecB= Array [1..10] Of Boolean;
  VecI= Array [1..8] Of Integer;
  VecC= Array [1..8] Of Integer;
  VecPr=Array [1..3] Of Real;
Var
  B: vecB;
  C: vecC;
  D: vecI;
  Pr:vecPr;
  Gd, Gm          : Integer;
  Color           : Word;
  Dz,Mx,My,MxN,MyN,Pas: Integer;
  P,F             : Real;
  Is              : Integer;
  Isa,Isb,IsC     : Integer;
  Pro             : String[3];
  Ta              : Integer;
  Pa              : Real;
  ExitSave : Pointer;

Procedure MyExit; far;
label 1;
Begin
  ExitProc:= nil;
  {ExitCode:=0;}
  {ErrorAddr:=nil;}

  Window(1,1,80,25);
  NormVideo;
  ClrScr;
  {WriteLn('Formato incorrecto, P es un número real entre 0 y 1');}
  {WriteLn('Vuelva a correr el programa DEP.EXE');}
end;

{
  MOSTRAR EJES
}
Procedure Ejes(Mx,My: Integer; Color: Word);
Var
  I,J: Integer;

```


Begin

```
For I := 20 to My-20 Do
  PutPixel (22,I,Color);
For J := 22 to Mx-19 Do
  PutPixel (J,My-20,Color);
MoveTo(21,21);
LineTo(23,21);
MoveTo(20,22);
LineTo(24,22);
```

End;

```
{
  MOSTRAR ESCALAS
}
```

Procedure Escalas;

Var

I : Integer;

Begin

```
SetLineStyle(SolidLn,0,NormWidth);
MoveTo((22+(Mx-40) div 4),My-20);
LineTo((22+(Mx-40) div 4),My-17);
MoveTo((22+(Mx-40) div 2),My-20);
LineTo((22+(Mx-40) div 2),My-17);
MoveTo((22+3*(Mx-40) div 4),My-20);
LineTo((22+3*(Mx-40) div 4),My-17);
MoveTo((22+(Mx-40)),My-20);
LineTo((22+(Mx-40)),My-17);
SetTextJustify(LeftText,TopText);
SetTextStyle(SmallFont,HorizDir,4);
OutTextXY(12,GetMaxY-15,'0');
OutTextXY((15+(Mx-40) div 4),My-12,'0.5');
OutTextXY((15+(Mx-40) div 2),My-12,'1.0');
OutTextXY((15+ 3*(Mx-40) div 4),My-12,'1.5');
OutTextXY(Mx-25,My-12,'2.0');
OutTextXY(Mx-10,My-30,'ft');
OutTextXY(0,0,'S(f)');
MoveTo(1,15);
LineTo(17,15);
OutTextXY(0,15,'A²T');
```

```
{
  COLOCAR ESCALA VERTICAL SEGUN EL FACTOR DE ESCALA
}
```

If F = 0.2 then

begin

```
MoveTo(22,(My-20-((My-40) div 4)));
LineTo(25,(My-20-((My-40) div 4)));
OutTextXY(0,(My-25-((My-40) div 4)),'.05');
MoveTo(22,(My-20-((My-40) div 2)));
LineTo(25,(My-20-((My-40) div 2)));
OutTextXY(0,(My-25-((My-40) div 2)),'.10');
```

```

    MoveTo(22,(My-20-3*((My-40) div 4)));
    LineTo(25,(My-20-3*((My-40) div 4)));
    OutTextXY(0,(My-25-3*((My-40) div 4)),'.15');
end;
If F=0.7 then
begin
    Pas := (My-40) div 7;
    MoveTo(22,My-20-Pas);
    LineTo(25,My-20-Pas);
    OutTextXY(0,My-28-Pas,'0.1');
    MoveTo(22,My-20-2*Pas);
    LineTo(25,My-20-2*Pas);
    OutTextXY(0,My-28-2*Pas,'0.2');
    MoveTo(22,My-20-3*Pas);
    LineTo(25,My-20-3*Pas);
    OutTextXY(0,My-28-3*Pas,'0.3');
    MoveTo(22,My-20-4*Pas);
    LineTo(25,My-20-4*Pas);
    OutTextXY(0,My-28-4*Pas,'0.4');
    MoveTo(22,My-20-5*Pas);
    LineTo(25,My-20-5*Pas);
    OutTextXY(0,My-28-5*Pas,'0.5');
    MoveTo(22,My-20-6*Pas);
    LineTo(25,My-20-6*Pas);
    OutTextXY(0,My-28-6*Pas,'0.6');
end;
If F= 1.0 then
begin
    Pas := (My-40) div 10;
    MoveTo(22,My-20-Pas);
    LineTo(25,My-20-Pas);
    OutTextXY(0,My-28-Pas,'0.1');
    MoveTo(22,My-20-2*Pas);
    LineTo(25,My-20-2*Pas);
    OutTextXY(0,My-28-2*Pas,'0.2');
    MoveTo(22,My-20-3*Pas);
    LineTo(25,My-20-3*Pas);
    OutTextXY(0,My-28-3*Pas,'0.3');
    MoveTo(22,My-20-4*Pas);
    LineTo(25,My-20-4*Pas);
    OutTextXY(0,My-28-4*Pas,'0.4');
    MoveTo(22,My-20-5*Pas);
    LineTo(25,My-20-5*Pas);
    OutTextXY(0,My-28-5*Pas,'0.5');
    MoveTo(22,My-20-6*Pas);
    LineTo(25,My-20-6*Pas);
    OutTextXY(0,My-28-6*Pas,'0.6');
    MoveTo(22,My-20-7*Pas);
    LineTo(25,My-20-7*Pas);
    OutTextXY(0,My-28-7*Pas,'0.7');
    MoveTo(22,My-20-8*Pas);
    LineTo(25,My-20-8*Pas);

```

```

        OutTextXY(0,My-28-8*Pas,'0.8');
        MoveTo(22,My-20-9*Pas);
        LineTo(25,My-20-9*Pas);
        OutTextXY(0,My-28-9*Pas,'0.9');
    end;
    ' If F= 1.2 then
        begin
            Pas := (My-40) div 6;
            MoveTo(22,My-20-Pas);
            LineTo(25,My-20-Pas);
            OutTextXY(0,My-28-Pas,'0.2');
            MoveTo(22,My-20-2*Pas);
            LineTo(25,My-20-2*Pas);
            OutTextXY(0,My-28-2*Pas,'0.4');
            MoveTo(22,My-20-3*Pas);
            LineTo(25,My-20-3*Pas);
            OutTextXY(0,My-28-3*Pas,'0.6');
            MoveTo(22,My-20-4*Pas);
            LineTo(25,My-20-4*Pas);
            OutTextXY(0,My-28-4*Pas,'0.8');
            MoveTo(22,My-20-5*Pas);
            LineTo(25,My-20-5*Pas);
            OutTextXY(0,My-28-5*Pas,'1.0');
        end;

```

```

If F= 2.6 then
    begin
        Pas := (My-40) div 13;
        MoveTo(22,My-20-Pas);
        LineTo(25,My-20-Pas);
        OutTextXY(0,My-28-Pas,'0.2');
        MoveTo(22,My-20-2*Pas);
        LineTo(25,My-20-2*Pas);
        OutTextXY(0,My-28-2*Pas,'0.4');
        MoveTo(22,My-20-3*Pas);
        LineTo(25,My-20-3*Pas);
        OutTextXY(0,My-28-3*Pas,'0.6');
        MoveTo(22,My-20-4*Pas);
        LineTo(25,My-20-4*Pas);
        OutTextXY(0,My-28-4*Pas,'0.8');
        MoveTo(22,My-23-5*Pas);
        LineTo(25,My-23-5*Pas);
        OutTextXY(0,My-28-5*Pas,'1.0');
        MoveTo(22,My-20-6*Pas);
        LineTo(25,My-20-6*Pas);
        OutTextXY(0,My-28-6*Pas,'1.2');
        MoveTo(22,My-20-7*Pas);
        LineTo(25,My-20-7*Pas);
        OutTextXY(0,My-28-7*Pas,'1.4');
        MoveTo(22,My-20-8*Pas);
        LineTo(25,My-20-8*Pas);
    end;

```

```

    OutTextXY(0,My-28-8*Pas,'1.6');
    MoveTo(22,My-20-9*Pas);
    LineTo(25,My-20-9*Pas);
    OutTextXY(0,My-28-9*Pas,'1.8');
    MoveTo(22,My-20-10*Pas);
    LineTo(25,My-20-10*Pas);
    OutTextXY(0,My-28-10*Pas,'2.0');
    MoveTo(22,My-20-11*Pas);
    LineTo(25,My-20-11*Pas);
    OutTextXY(0,My-28-11*Pas,'2.2');
    MoveTo(22,My-20-12*Pas);
    LineTo(25,My-20-12*Pas);
    OutTextXY(0,My-28-12*Pas,'2.4');
end;

If F=3.5 then
begin
    Pas := (My-40) div 7;
    MoveTo(22,My-20-Pas);
    LineTo(25,My-20-Pas);
    OutTextXY(0,My-28-Pas,'0.5');
    MoveTo(22,My-20-2*Pas);
    LineTo(25,My-20-2*Pas);
    OutTextXY(0,My-28-2*Pas,'1.0');
    MoveTo(22,My-20-3*Pas);
    LineTo(25,My-20-3*Pas);
    OutTextXY(0,My-28-3*Pas,'1.5');
    MoveTo(22,My-20-4*Pas);
    LineTo(25,My-20-4*Pas);
    OutTextXY(0,My-28-4*Pas,'2.0');
    MoveTo(22,My-20-5*Pas);
    LineTo(25,My-20-5*Pas);
    OutTextXY(0,My-28-5*Pas,'2.5');
    MoveTo(22,My-20-6*Pas);
    LineTo(25,My-20-6*Pas);
    OutTextXY(0,My-28-6*Pas,'3.0');
end;

If F=5 then
begin
    Pas := (My-40) div 10;
    MoveTo(22,My-20-Pas);
    LineTo(25,My-20-Pas);
    OutTextXY(0,My-28-Pas,'0.5');
    MoveTo(22,My-20-2*Pas);
    LineTo(25,My-20-2*Pas);
    OutTextXY(0,My-28-2*Pas,'1.0');
    MoveTo(22,My-20-3*Pas);
    LineTo(25,My-20-3*Pas);
    OutTextXY(0,My-28-3*Pas,'1.5');
    MoveTo(22,My-20-4*Pas);
    LineTo(25,My-20-4*Pas);
    OutTextXY(0,My-28-4*Pas,'2.0');

```

```

    MoveTo(22,My-20-5*Pas);
    LineTo(25,My-20-5*Pas);
    OutTextXY(0,My-28-5*Pas,'2.5');
    MoveTo(22,My-20-6*Pas);
    LineTo(25,My-20-6*Pas);
    OutTextXY(0,My-28-6*Pas,'3.0');
    MoveTo(22,My-20-7*Pas);
    LineTo(25,My-20-7*Pas);
    OutTextXY(0,My-28-7*Pas,'3.5');
    MoveTo(22,My-20-8*Pas);
    LineTo(25,My-20-8*Pas);
    OutTextXY(0,My-28-8*Pas,'4.0');
    MoveTo(22,My-20-9*Pas);
    LineTo(25,My-20-9*Pas);
    OutTextXY(0,My-28-9*Pas,'4.5');
end;

{
Setear Colores
}
If ((Gd=3) or (Gd=4) or (Gd=9)) then
begin
    C[1]:=2;
    C[2]:=5;
    C[3]:=4;
    C[4]:=9;
    C[5]:=7;
    C[6]:=8;
    C[7]:=6;
    C[8]:=15;
end
else
    For I:=1 to 8 Do
        C[I] := 15;
End;

{Densidad espectral de potencia para el código NRZ unipolar o no polar}
Procedure Nrzu(P,F:real; Mx,My:Integer; Color:Word);
Var
    I,J      : Integer;
    X,A,B,S: Real;
Begin
    MxN := Mx;
    MyN := My;
    For I := 1 to Mx-40 Do
        Begin
            X := (2*I)/(Mx-40);
            A := Pi*X;
            B := Sin(A)/A;
            S := Abs(P*(1-P) * B * B);
            J := (My-Trunc(S*(My-40)/F)-20);
            PutPixel(I+22,J,Color);
        End
    End
End;

```

```

    If (I=41+(Mx-40)div 32) Then
        Begin
            MyN := J;
            MxN := I+22;
        End;
    End;
    I:= Trunc(P*(1-P)*(My-40)/F);
    If I>10 Then
        Begin
            For J := 1 to I-10 Do
                Begin
                    PutPixel(22,My-20-J,Color);
                    PutPixel(23,My-20-J,Color);
                    PutPixel(21,My-20-J,Color);
                    PutPixel(24,My-20-J,Color);
                End;
                PutPixel(19,My-11-I,Color);
                PutPixel(26,My-11-I,Color);
                PutPixel(19,My-12-I,Color);
                PutPixel(26,My-12-I,Color);
                PutPixel(20,My-11-I,Color);
                PutPixel(21,My-11-I,Color);
                PutPixel(22,My-11-I,Color);
                PutPixel(23,My-11-I,Color);
                PutPixel(24,My-11-I,Color);
                PutPixel(25,My-11-I,Color);
                PutPixel(20,My-12-I,Color);
                PutPixel(21,My-12-I,Color);
                PutPixel(22,My-12-I,Color);
                PutPixel(23,My-12-I,Color);
                PutPixel(24,My-12-I,Color);
                PutPixel(25,My-12-I,Color);
                PutPixel(21,My-13-I,Color);
                PutPixel(22,My-13-I,Color);
                PutPixel(23,My-13-I,Color);
                PutPixel(24,My-13-I,Color);
                PutPixel(22,My-14-I,Color);
                PutPixel(23,My-14-I,Color);
                PutPixel(22,My-15-I,Color);
                PutPixel(23,My-15-I,Color);
            End;
        End;
    End;

{Densidad espectral de potencia para el código NRZ polar}
Procedure Nrzp(P,F:real; Mx,My:Integer; Color:word);
Var
    I,J      : Integer;
    X,A,B,S: Real;
Begin
    MxN:=Mx;
    MyN:=My;
    For I:=1 to Mx-40 Do

```

```

Begin
  X := (2*I)/(Mx-40);
  A := Pi*X;
  B := sin(A)/A;
  S := Abs(4*P*(1-P)*B*B);
  J := My-Trunc(S*(My-40)/F)-20;
  PutPixel(I+22,J,Color);
  if (J<MyN) then
    begin
      MyN := J;
      MxN := I+22;
    end;
End;
I:= Trunc((2*P-1)*(2*P-1)*(My-40)/F);
If I>1 Then
Begin
For J := 1 to I Do
  Begin
    PutPixel(22,My-20-J,Color);
    {PutPixel(23,My-20-J,Color);}
  End;
  PutPixel(20,My-21-I,Color);
  PutPixel(21,My-21-I,Color);
  PutPixel(22,My-21-I,Color);
  PutPixel(23,My-21-I,Color);
  PutPixel(24,My-21-I,Color);
  PutPixel(25,My-21-I,Color);
  PutPixel(20,My-22-I,Color);
  PutPixel(21,My-22-I,Color);
  PutPixel(22,My-22-I,Color);
  PutPixel(23,My-22-I,Color);
  PutPixel(24,My-22-I,Color);
  PutPixel(25,My-22-I,Color);
  PutPixel(21,My-23-I,Color);
  PutPixel(22,My-23-I,Color);
  PutPixel(23,My-23-I,Color);
  PutPixel(24,My-23-I,Color);
  PutPixel(22,My-24-I,Color);
  PutPixel(23,My-24-I,Color);
  PutPixel(22,My-25-I,Color);
  PutPixel(23,My-25-I,Color);
End;
End;

{Densidad espectral de potencia para el código AMI}
Procedure Ami(P,F:real; Mx,My:Integer; Color:word);
Var
  I,J      : Integer;
  X,A,B,S: Real;
Begin
  MxN:=Mx;
  MyN:=My;

```

```

For I:=1 to Mx-40 Do
Begin
  X := (2*I)/(Mx-40);
  A := Pi*X;
  B := sin(A)*sin(A)/A;
  S := Abs(4*P*(1-P)*B*B/(1+(2*p-1)*(2*p-1)+2*(2*p-1)*cos(2*A)));
  J := My-Trunc(S*(My-40)/F)-20;
  PutPixel(I+22,J,Color);
  if (I=(-37+(Mx-40) div 4)) then
  begin
    MyN := J;
    MxN := I+22;
  end;
End;
End;

{Densidad espectral de potencia del código diferencial M polar}
Procedure Difm(P,F:real; Mx,My:Integer; Color:word);
Var
  I,J      : Integer;
  X,A,B,C,D,S: Real;
Begin
  MxN:=Mx;
  MyN:=My;
  For I:= 1 to Mx-40 Do
  Begin
    X := (2*I)/(Mx-40);
    A := Pi*X;
    B := sin(A)/A;
    C := 1*B*B*(1+(1-2*P)*((3-2*P)-4*(1-P)*cos(2*A)));
    D := (1+(1-2*P)*(1-2*P)-2*(1-2*P)*cos(2*A));
    S :=Abs(C/D);
    J := My-Trunc(S*(My-40)/F)-20;
    PutPixel(I+22,J,Color);
    If (I = 90+(Mx-40) div 32) then
    begin
      MyN := J-2;
      MxN := I+22;
    end;
  End;
End;

{Densidad espectral de potencia del código RZ-POLAR}
Procedure Rz(P,F:real; Mx,My:Integer; Color:word);
Var
  I,J      : Integer;
  X,A,B,S: Real;
Begin
  MxN:=Mx;
  MyN:=My;
  For I:= 1 to Mx-40 Do
  Begin

```



```

X := (2*I)/(Mx-40);
A := Pi*X;
B := sin(A/2)/(A/2);
S := Abs(P*(1-P)*B*B);
J := My -Trunc(S*(My-40)/F)-20;
PutPixel(I+22,J,Color);
if (I=(Mx-40) div 8+15) then
begin
  MyN := J+3;
  MxN := I+22;
end;
End;
I:= Trunc((2*P-1)*(2*P-1)*(1/4)*(My-40)/F);
If I>10 Then
Begin
For J := 1 to I-10 Do
  Begin
    PutPixel(22,My-20-J,Color);
    PutPixel(23,My-20-J,Color);
    {PutPixel(24,My-20-J,Color);}
  End;
  PutPixel(19,My-11-I,Color);
  PutPixel(26,My-11-I,Color);
  PutPixel(19,My-12-I,Color);
  PutPixel(26,My-12-I,Color);
  PutPixel(20,My-11-I,Color);
  PutPixel(21,My-11-I,Color);
  PutPixel(22,My-11-I,Color);
  PutPixel(23,My-11-I,Color);
  PutPixel(24,My-11-I,Color);
  PutPixel(25,My-11-I,Color);
  PutPixel(20,My-12-I,Color);
  PutPixel(21,My-12-I,Color);
  PutPixel(22,My-12-I,Color);
  PutPixel(23,My-12-I,Color);
  PutPixel(24,My-12-I,Color);
  PutPixel(25,My-12-I,Color);
  PutPixel(21,My-13-I,Color);
  PutPixel(22,My-13-I,Color);
  PutPixel(23,My-13-I,Color);
  PutPixel(24,My-13-I,Color);
  PutPixel(22,My-14-I,Color);
  PutPixel(23,My-14-I,Color);
  PutPixel(22,My-15-I,Color);
  PutPixel(23,My-15-I,Color);
End;

I:= Trunc((2/PI)*(2/PI)*(2*P-1)*(2*P-1)*(1/4)*(My-40)/F);
If I>10 Then
Begin
For J := 1 to I-10 Do
  Begin

```

```

        PutPixel(21+(Mx-40) div 2,My-20-J,Color);
        PutPixel(22+(Mx-40) div 2,My-20-J,Color);
    End;
    PutPixel(18+(Mx-40) div 2,My-11-I,Color);
    PutPixel(25+(Mx-40) div 2,My-11-I,Color);
    PutPixel(18+(Mx-40) div 2,My-12-I,Color);
    PutPixel(25+(Mx-40) div 2,My-12-I,Color);
    PutPixel(19+(Mx-40) div 2,My-11-I,Color);
    PutPixel(20+(Mx-40) div 2,My-11-I,Color);
    PutPixel(21+(Mx-40) div 2,My-11-I,Color);
    PutPixel(22+(Mx-40) div 2,My-11-I,Color);
    PutPixel(23+(Mx-40) div 2,My-11-I,Color);
    PutPixel(24+(Mx-40) div 2,My-11-I,Color);
    PutPixel(19+(Mx-40) div 2,My-12-I,Color);
    PutPixel(20+(Mx-40) div 2,My-12-I,Color);
    PutPixel(21+(Mx-40) div 2,My-12-I,Color);
    PutPixel(22+(Mx-40) div 2,My-12-I,Color);
    PutPixel(23+(Mx-40) div 2,My-12-I,Color);
    PutPixel(24+(Mx-40) div 2,My-12-I,Color);
    PutPixel(20+(Mx-40) div 2,My-13-I,Color);
    PutPixel(21+(Mx-40) div 2,My-13-I,Color);
    PutPixel(22+(Mx-40) div 2,My-13-I,Color);
    PutPixel(23+(Mx-40) div 2,My-13-I,Color);
    PutPixel(21+(Mx-40) div 2,My-14-I,Color);
    PutPixel(22+(Mx-40) div 2,My-14-I,Color);
    PutPixel(21+(Mx-40) div 2,My-15-I,Color);
    PutPixel(22+(Mx-40) div 2,My-15-I,Color);
End;

End;

{Densidad espectral de potencia del código de Miller}
Procedure Miller(F:real; Mx,My:Integer; Color:Word);
Var
    X,A,B,S: Real;
    I,J      : Integer;
Begin
    MxN:=Mx;
    MyN:=My;
    For I := 1 to Mx-40 Do
        Begin
            X := (2*I)/(Mx-40);
            A := Pi*X;
            B := A*A;
            S := 23-2*cos(A)-22*cos(2*A)-12*cos(3*A)+5*cos(4*A)+12*cos(5*A);
            S := S + 2*cos(6*A)-8*cos(7*A)+2*cos(8*A);
            S := S/((17+8*cos(A))*B);
            J := My-Trunc(S*(MY-40)/F)-20;
            PutPixel(I+22,J,Color);
            if (J<MyN) then
                begin

```

```

        MyN := J;
        MxN := I+22;
    end;
End;
End;

{Densidad espectral de potencia del código de Manchester}
Procedure Manchester(P,F:Real; Mx,My:Integer; Color:Word);
Var
    X,A,B,S: Real;
    I,J      : Integer;
Begin
    MxN:=Mx;
    MyN:=My;
    For I := 1 to Mx-40 Do
        Begin
            X := (2*I)/(Mx-40);
            A := Pi*X;
            B := sin(A/2)*sin(A/2)/(A/2);
            S := Abs(P*(1-P)*B*B/(1+(2*P-1)*(2*P-1)+2*(2*P-1)*cos(A)));
            J := My - Trunc(S*(My-40)/F)-20;
            PutPixel(I+22,J,Color);
            If P<0.9 then
                begin
                    if (J<MyN) then
                        begin
                            MyN := J;
                            MxN := I+22;
                        end;
                    end
                else
                    begin
                        if (I=(Mx-(Mx-40) div 2+10)) then
                            begin
                                MyN := J;
                                MxN := I+22;
                            end;
                        end;
                    end;
            End;
        End;
    End;
End;

```

```

{Densidad espectral de potencia para el código HDB3}
Procedure Hdb3(F:real; Mx,My:Integer; Color:Word);
Var
    X,A,B,C,D,E,S: Real;
    I,J          : Integer;
Begin
    MxN:=Mx;
    MyN:=My;
    For I := 1 to Mx-40 Do
        Begin
            X := (2*I)/(Mx-40);

```

```

A := Pi*X;
B := sin(A)/A;
C := (40-32*cos(2*A))/(465*(1025-64*cos(10*A)));
D := 7258.5-1929*cos(2*A)-1424*cos(4*A)-160*cos(6*A)+32*cos(8*A);
E := (131288.5-41399*cos(2*A)-86112*cos(4*A))/(85-44*cos(2*A)-24*cos(4*A)-16*cos(6*A));
S := B*B*C*(D-E);
J := My - Trunc(S*(My-40)/F)-20;
PutPixel(I+22,J,Color);
if (J<MyN) then
begin
MyN := J;
MxN := I+22;
end;
End;
End;

Procedure Nombre;
Begin
Dz:=1;
If B[1] then
begin
SetColor(C[1]);
OutTextXY(Mx-160,15*Dz,`( ) NRZ unipolar`);
Str(Dz,Pro);
OutTextXY(Mx-155,15*Dz,Pro);
SetColor(White);
Dz := Dz+1;
D[1] := 15*Dz;
end;
If B[2] then
begin
SetColor(C[2]);
OutTextXY(Mx-160,15*Dz,`( ) NRZ polar`);
Str(Dz,Pro);
OutTextXY(Mx-155,15*Dz,Pro);
SetColor(White);
Dz := Dz+1;
D[2] := 15*Dz;
end;
If B[3] then
begin
SetColor(C[3]);
OutTextXY(Mx-160,15*Dz,`( ) AMI`);
Str(Dz,Pro);
OutTextXY(Mx-155,15*Dz,Pro);
SetColor(White);
Dz := Dz+1;
D[3] := 15*Dz;
end;
If B[4] then

```

```

begin
  SetColor(C[4]);
  OutTextXY(Mx-160,15*Dz,`( ) Diferencial NRZ-M polar`);
  Str(Dz,Pro);
  OutTextXY(Mx-155,15*Dz,Pro);
  SetColor(White);
  Dz := Dz+1;
  D[4] := 15*Dz;
end;
If B[5] then
begin
  SetColor(C[5]);
  OutTextXY(Mx-160,15*Dz,`( ) RZ polar`);
  Str(Dz,Pro);
  OutTextXY(Mx-155,15*Dz,Pro);
  SetColor(White);
  Dz := Dz+1;
  D[5] := 15*Dz;
end;
If B[6] then
begin
  SetColor(C[6]);
  OutTextXY(Mx-160,15*Dz,`( ) Miller (P=0.5)`);
  Str(Dz,Pro);
  OutTextXY(Mx-155,15*Dz,Pro);
  SetColor(White);
  Dz := Dz+1;
  D[6] := 15*Dz;
end;
If B[7] then
begin
  SetColor(C[7]);
  OutTextXY(Mx-160,15*Dz,`( ) Manchester`);
  Str(Dz,Pro);
  OutTextXY(Mx-155,15*Dz,Pro);
  SetColor(White);
  Dz := Dz+1;
  D[7] := 15*Dz;
end;
If B[8] then
begin
  SetColor(C[8]);
  OutTextXY(Mx-160,15*Dz,`( ) HDB3 (P=0.5)`);
  Str(Dz,Pro);
  OutTextXY(Mx-155,15*Dz,Pro);
  SetColor(White);
  Dz := Dz+1;
  D[8] := 15*Dz;
end;
End;
{

```

```

    Pantalla de presentación
}
Procedure Pres_1;
Var
    I,J : Integer;

Begin
    For I:=2 to 79 Do
        Begin
            GotoXY(I,1);
            Write('=');
            GotoXY(I,24);
            Write('=');
        End;
    For I:=2 to 23 Do
        Begin
            GotoXY(1,I);
            Write('||');
            GotoXY(80,I);
            Write('||');
        End;
        GotoXY(1,1);
        Write('┌');
        GotoXY(1,24);
        Write('└');
        GotoXY(80,1);
        Write('┐');
        GotoXY(80,24);
        Write('┘');
    End;

Procedure Pres_2;
Var
    I,J : Integer;
Begin
    GotoXY(25,2);
    Write('ESCUELA POLITECNICA NACIONAL');
    GotoXY(23,3);
    Write('FACULTAD DE INGENIERIA ELECTRICA');
    GotoXY(16,4);
    Write('DEPARTAMENTO DE ELECTRONICA Y TELECOMUNICACIONES');
    GotoXY(1,5);
    Write('||');
    GotoXY(80,5);
    Write('||');
    For I:=2 to 79 Do
        Begin
            GotoXY(I,5);
            Write('=');
        End;
    GotoXY(11,23);
    Write('N. Avila');

```

```

GotoXY(65,23);
Write('1994');
GotoXY(1,22);
Write('||');
GotoXY(80,22);
Write('||');
For I:=2 to 79 Do
  Begin
    GotoXY(I,22);
    Write('-');
  End;
End;

Procedure Pres_3;
Var
  I,J : Integer;
Begin
  GotoXY(5,7);
  Write('ANALISIS DE LA DENSIDAD ESPECTRAL DE POTENCIA DE LOS CODIGOS DE LINEA');
  GotoXY(5,7);
  For I:=5 to 73 Do
    begin
      GotoXY(I,8);
      Write('-');
    end;
  For I:=28 to 52 Do
    begin
      GotoXY(I,12);
      Write('=');
      GotoXY(I,18);
      Write('=');
    end;
  For I := 13 To 17 Do
    begin
      GotoXY(27,I);
      Write('||');
      GotoXY(53,I);
      Write('||');
    end;
  GotoXY(27,12);
  Write('||');
  GotoXY(53,12);
  Write('||');
  GotoXY(27,18);
  Write('||');
  GotoXY(53,18);
  Write('||');
End;

Procedure Pres_4;
Var

```

```

    I : Integer;
Begin
    GotoXY(6,2);
    Write('ANALISIS DE LA DENSIDAD ESPECTRAL DE POTENCIA DE LOS CODIGOS DE
LINEA');
    GotoXY(1,4);
    Write('||');
    GotoXY(80,4);
    Write('||');
    For I := 2 To 79 Do
        Begin
            GoToXY(I,4);
            Write('-');
        End;
    End;

Procedure Pres_4a;
Var
    I,J : Integer;
Begin
    GotoXY(31,3);
    Write('SEGUN LOS CODIGOS');
End;

Procedure Pres_4b;
Var
    I,J : Integer;
Begin
    GotoXY(28,3);
    Write('SEGUN LAS PROBABILIDADES');
End;

Procedure Pres_5;
var
    I : integer;
Begin
    GotoXY(7,22);
    Write('Comparación de la d.e.p. de varios códigos de línea para una
misma');
    GotoXY(8,23);
    Write('probabilidad de ocurrencia de los bits de entrada al
codificador');
    GotoXY(1,21);
    Write('||');
    GotoXY(80,21);
    Write('||');
    For I := 2 To 79 Do
        Begin
            GoToXY(I,21);
            Write('-');
        End;
    End;

```



```

End;

Procedure Pres_6;
Var
  I : Integer;
Begin
  GotoXY(12,22);
  Write('Variación de la d.e.p. con la probabilidad de ocurrencia');
  GotoXY(22,23);
  Write('de los bits de entrada al codificador');
  GotoXY(1,21);
  Write('||');
  GotoXY(80,21);
  Write('||');
  For I := 2 To 79 Do
    Begin
      GoToXY(I,21);
      Write('-');
    End;
  End;

Procedure Pres_7;
var
  I : Integer;
Begin
  For I:=28 to 54 Do
    begin
      GotoXY(I,8);
      Write('-');
      GotoXY(I,14);
      Write('-');
    end;
  For I := 9 To 13 Do
    begin
      GotoXY(27,I);
      Write('|');
      GotoXY(55,I);
      Write('|');
    end;
  GotoXY(27,8);
  Write('f');
  GotoXY(55,8);
  Write('g');
  GotoXY(27,14);
  Write('L');
  GotoXY(55,14);
  Write('J');
End;

Procedure Pres_8;
Var
  I : Integer;

```

```

Begin
  For I:=26 to 55 Do
    begin
      GotoXY(I,6);
      Write('-');
      GotoXY(I,17);
      Write('-');
    end;
  For I := 7 To 16 Do
    begin
      GotoXY(25,I);
      Write('|');
      GotoXY(56,I);
      Write('|');
    end;
  GotoXY(25,6);
  Write('┌');
  GotoXY(56,6);
  Write('┐');
  GotoXY(25,17);
  Write('└');
  GotoXY(56,17);
  Write('┘');
End;

Procedure Pres_9;
Var
  I : Integer;
Begin
  GotoXY(4,22);
  Write('Elegir primero los códigos de línea cuya d.e.p. se desea comparar
y luego');
  GotoXY(10,23);
  Write('elegir la probabilidad "P" (posicionarse y presionar ENTER)');
  GotoXY(1,21);
  Write('||');
  GotoXY(80,21);
  Write('||');
  For I := 2 To 79 Do
    Begin
      GoToXY(I,21);
      Write('-');
    End;
End;

Procedure Pres_10;
var
  I : Integer;
Begin
  For I:=28 to 54 Do
    begin
      GotoXY(I,8);

```

```

        Write(' ');
        GotoXY(I,16);
        Write(' ');
    end;
    For I := 9 To 15 Do
    begin
        GotoXY(27,I);
        Write('| ');
        GotoXY(55,I);
        Write('| ');
    end;
    GotoXY(27,8);
    Write(' ');
    GotoXY(55,8);
    Write(' ');
    GotoXY(27,16);
    Write('L ');
    GotoXY(55,16);
    Write('J ');
End;

```

```

Procedure Pres_11;
Var
    I : Integer;
Begin
    For I:=29 to 52 Do
    begin
        GotoXY(I,7);
        Write(' ');
        GotoXY(I,14);
        Write(' ');
    end;
    For I := 8 To 13 Do
    begin
        GotoXY(28,I);
        Write('| ');
        GotoXY(53,I);
        Write('| ');
    end;
    GotoXY(28,7);
    Write(' ');
    GotoXY(53,7);
    Write(' ');
    GotoXY(28,14);
    Write('L ');
    GotoXY(53,14);
    Write('J ');
End;

```

```

Procedure Pres_12;
Var
    I : Integer;

```

```

Begin
  GotoXY(27, 19);
  If B[1] Then
    Write('      Código NRZ unipolar      ');
  If B[2] Then
    Write('      Código NRZ polar      ');
  If B[3] Then
    Write('      Código AMI      ');
  If B[4] Then
    Write('Código diferencial NRZ-M polar');
  If B[5] Then
    Write('      Código RZ polar      ');
  If B[6] Then
    Write('Código Manchester (bifase-L) ');
End;

Procedure Pres_13;
Var
  I : Integer;
Begin
  GotoXY(10,22);
  Write('Elegir el código de línea cuya d.e.p. variará en función de');
  GotoXY(13,23);
  Write('la probabilidad "P" (presionar ENTER para seleccionar)');
  GotoXY(1,21);
  Write('||');
  GotoXY(80,21);
  Write('||');
  For I := 2 To 79 Do
    Begin
      GoToXY(I,21);
      Write('-');
    End;
End;

Procedure Pres_14;
Var
  I : Integer;
Begin
  GotoXY(14,22);
  Write('Seleccionar dos valores de "P" para evaluar la d.e.p. ');
  GotoXY(22,23);
  Write('(se graficará también para P = 0.5)');
  GotoXY(1,21);
  Write('||');
  GotoXY(80,21);
  Write('||');
  For I := 2 To 79 Do
    Begin
      GoToXY(I,21);
      Write('-');
    End;
End;

```

```

End;

Procedure Mq;
Begin
  TextColor(White);
  GotoXY (44,15);
  Write('P = 0. ');
  Ta := Round(10000*P);
  Str(Ta,Pro);
  Write(Pro);
End;

{
  Seleccionar
}
Procedure Selec_1;
Var
  V : Vec1;
  I : Integer;
  Ch: Char;
Begin
  ExitSave := ExitProc;
  ExitProc := @MyExit;
  V[1] := 'NRZ UNIPOLAR';
  V[2] := 'NRZ POLAR';
  V[3] := 'AMI';
  V[4] := 'DIFERENCIAL NRZ-M POLAR';
  V[5] := 'RZ POLAR';
  V[6] := 'MILLER (DM), P = 0.5';
  V[7] := 'MANCHESTER (BIFASE L)';
  V[8] := 'HDB3, P = 0.5';
  V[9] := 'Probabilidad';
  V[10] := 'Regresar al menú anterior';

  Textcolor(White);
  TextBackground(Black);
  Clrscr;
  Pres_1;
  Pres_4;
  Pres_4a;
  Pres_8;
  Pres_9;
  For I := 7 to 16 Do
  Begin
    Mq;
    Gotoxy(26,I);
    If B[i-6] And ((i-6<>9)and(i-6<>10)) Then
      Write('[X] ')
    Else
      If ((i-6<>9)and(i-6<>10)) then
        Write('[ ] ')
      Else

```

```

        Write(' ^P ');
Write(V[i-6]);
End;

I := 1;
Repeat
    Gotoxy(26,i+6);
    TextBackground(White);
    Textcolor(Black);
    If B[i] And ((i<>9)and(i<>10)) Then
        Write('[X] ')
    Else
        if ((i<> 9)and(i<>10)) then
            Write('[ ] ')
        else
            Write(' ^P ');
        Write(V[i]);
    Repeat
        Ch := readkey;
        Until (Ch = #0) Or (Ch = Chr(13));
        If Ch <> Chr(13) Then
            CH := Readkey;
        If Ch = Chr(13) Then
            Begin
                B[i] := Not B[i];
                If i = 9 then
                    Begin
                        If (B[1] or B[2] or B[3] or B[4] or B[5] or B[7]) then
                            Begin
                                Repeat
                                    Window(26,18,56,20);
                                    TextBackground(Blue);
                                    TextColor(White);
                                    ClrScr;
                                    GotoXY(2,2);
                                    Write('Ingrese el valor de P (0<P<1)');
                                    Window(40,20,50,20);
                                    ClrScr;
                                    GotoXY(1,1);
                                    ReadLn(P);
                                    Until ( (P>0) and (P<1));
                                    Window(26,18,56,20);
                                    TextBackground(Black);
                                    ClrScr;
                                    Window(1,1,80,25);
                                End;
                            End;
                        End;
                    End;
                If Ch = 'P' Then
                    Begin
                        I := I + 1;
                        Textcolor(White);

```

```

    TextBackground(Black);
    Gotoxy(26,i+5);
    If B[i-1] And ((i-1<>9)and(i-1<>10))Then
        Write('[X] ')
    Else
        If ((i-1 <> 9)and(i-1 <> 10))then
            Write('[ ] ')
        Else
            Write(' ^P ');
        Write(V[i-1]);
    End;
    Normvideo;
    If Ch = 'H' Then
    Begin
        I := I-1;
        Textcolor(White);
        TextBackground(Black);
        Gotoxy(26,I+7);
        If B[i+1] And ((i+1<>9)and(i+1<>10)) Then
            Write('[X] ')
        Else
            If ((i+1 <> 9)and(i+1 <> 10))then
                Write('[ ] ')
            Else
                Write(' ^P ');
            Write(V[i+1]);
        End;
        Normvideo;
        If I > 10 Then
            I := 1;
        If I < 1 Then
            I := 10;
        If ((B[6] or B[8]) And Not (B[1] Or B[2] Or B[3] Or B[4] Or B[5] Or
B[7])) Then
            P := 0.5;
        Mq;
        Until (CH = Chr(13)) And (i=10);
    End;

Procedure Selec_2;
Var
    V : Vec1;
    I : Integer;
    Ch: Char;
Begin
    For I := 1 to 9 Do
        B[I] := False;
    V[1] := 'NRZ UNIPOLAR';
    V[2] := 'NRZ POLAR';
    V[3] := 'AMI';
    V[4] := 'DIFFERENCIAL NRZ-M POLAR';
    V[5] := 'RZ POLAR';

```

```

V[6] := 'MANCHESTER (BIFASE L)';

Textcolor(White);
TextBackground(Black);
Clrscr;
Pres_1;
Pres_4;
Pres_4b;
Pres_11;
Pres_13;
For I := 8 to 13 Do
Begin
    Gotoxy(29,I);
    Write(V[I-7]);
End;

I := 1;
Repeat
    Gotoxy(29,I+7);
    TextBackground(White);
    Textcolor(Black);
    Write(V[i]);
    Repeat
        Ch := readkey;
        Until (Ch = #0) Or (Ch = Chr(13));
        If Ch <> Chr(13) Then
            CH := Readkey;
        If Ch = Chr(13) Then
            B[i] := Not B[i];
        If Ch = 'P' Then
            Begin
                I := I + 1;
                Textcolor(White);
                TextBackground(Black);
                Gotoxy(29,i+6);
                Write(V[i-1]);
            End;
        Normvideo;
        If Ch = 'H' Then
            Begin
                I := I-1;
                Textcolor(White);
                TextBackground(Black);
                Gotoxy(29,I+8);
                Write(V[i+1]);
            End;
        Normvideo;
        If I > 6 Then
            I := 1;
        If I < 1 Then
            I := 6;
    Until (CH = Chr(13));

```



```

    Clrscr;
End;

Procedure Mp;
Begin
GotoXY(43,9);
    Write('P = 0. ');
    Ta := Round(10000*Pr[2]);
    Str(Ta,Pro);
    Write(Pro) ;
    GotoXY(43,11);
    Write('P = 0. ');
    Ta := Round(10000*Pr[3]);
    Str(Ta,Pro);
    Write(Pro) ;
End;

Procedure Selec_3;
Var
    I,J : integer;
    V    : vec;
    O    : byte;
    Ch   : String;
Begin
    Repeat
        TextColor(White);
        TextBackground(Black);
        ClrScr;
        Pres_1;
        Pres_4;
        Pres_4b;
        Pres_7;
        Pres_14;
        Mp;
        V[1] := 'Primera Opción';
        V[2] := 'Segunda Opción';
        V[3] := 'Regresar al menú anterior';

        Textcolor(White);
        TextBackground(Black);
        J := 9;
        For Isc := 1 to 3 Do
            Begin
                Gotoxy(28,J);
                Write(V[Isc]);
                J := J + 2;
            End;

        J := 9;
        Isc := 1;
        Repeat
            Gotoxy(28,j);

```

```

TextBackground(White);
Textcolor(Black);
Write(V[Isc]);
Repeat
  Ch := readkey;
Until (Ch = #0) Or (Ch = Chr(13));
If Ch <> Chr(13) Then
  CH := Readkey;
If Ch = 'P' Then
Begin
  J := j+2;
  Isc := Isc + 1;
  Textcolor(White);
  TextBackground(Black);
  Gotoxy(28,J-2);
  Write(V[Isc-1]);
End;
If Ch = 'H' Then
Begin
  J := J-2;
  Isc := Isc-1;
  Textcolor(White);
  TextBackground(Black);
  Gotoxy(28,J+2);
  Write(V[Isc+1]);
End;
If Isc > 3 Then
Begin
  Isc := 1;
  J := 9;
End;
If Isc < 1 Then
Begin
  Isc := 3;
  J := 13;
End;
Until Ch = Chr(13);
If Isc = 1 Then
Begin
  Repeat
    Window(26,16,56,18);
    TextColor(White);
    TextBackground(Blue);
    ClrScr;
    GotoXY(2,2);
    Write('Ingrese el valor de P (0<P<1)');
    Window(40,18,50,18);
    ClrScr;
    GotoXY(1,1);
    ReadLn(P);
  Until ( (P>0) and (P<1));
  Pr[2] := P;

```

```

        ClrScr;
        Window(1,1,80,25);
    End;
    If Isc = 2 Then
        Begin
            Repeat
                Window(26,16,56,18);
                TextColor(White);
                TextBackground(Blue);
                ClrScr;
                GotoXY(2,2);
                Write('Ingresa el valor de P ( $0 < P < 1$ )');
                Window(40,18,50,18);
                ClrScr;
                GotoXY(1,1);
                ReadLn(P);
            Until ( (P>0) and (P<1));
            Pr[3] := P;
            ClrScr;
            Window(1,1,80,25);
        End;
    Until Isc = 3;
    Ch := 'a';
    Normvideo;
End;

```

```

Procedure ver;
var
    Ban: Boolean;
    I : Integer;
    Ch : Char;

Begin
    {Clrscr;}
    Normvideo;
    Ban := True;
    For I := 1 to 8 Do
        If B[i] Then
            Ban := False;
    If Ban Then
        Begin
            Textcolor(Cyan+Blink);
            Gotoxy(26,15);
            Write('Seleccione primero los parámetros');
            Ch := Readkey;
        End
    Else
        Begin
            Gd := Detect;
            InitGraph(Gd, Gm, '');

```

```

if GraphResult <> grOk then Halt(1);
Color := GetMaxColor;
Mx := GetMaxX;
My := GetMaxY;

{MOSTRAR EJES}
Ejes(Mx,My,Color);

{ESCOGER EL FACTOR DE ESCALA EN EL EJE Y SEGUN EL CODIGO Y EL VALOR DE
P}
If (B[4] or B[6]) then
  begin
    If (B[6] and (Not B[4])) then
      F := 2.6;
    If (B[6] and B[4]) then
      begin
        If P<0.29 then
          F := 5
        else
          F := 2.6
        end;
      If ((Not B[6]) and B[4]) then
        begin
          If P<0.29 then
            F := 3.5
          else
            If P<0.465 then
              F := 2.6
            else
              F := 1.2
          end;
        end
      end
    else
      begin
        If B[2] then
          If P<0.2 then
            F:=0.7
          else
            F:=1.2
          end
        else
          If((not B[1]) and (not B[3]) and (not B[5]) and B[7] and (not
B[8])) then
            begin
              If ((P<0.4) or (P=0.4)) then
                F := 0.2;
              If ((P>0.4) and (P<0.7)) then
                F := 1.0;
              If (((P>0.7) or (P=0.7)) and (P<0.85)) then
                F:= 1.2;
              If ((P>0.85) or (P=0.85)) then
                F:=2.6;
            end
          end
        end
      end
    end
  end

```

```

        end
    else
        begin
            If ((B[1] or B[5] or B [7]) and (Not(B[3]))) then
                If B[7] then
                    begin
                        If ((P>0.7) and (P<0.85)) then
                            F:=1.2;
                        If ((P>0.85) or (P=0.85)) then
                            F:=2.6;
                        If ((P<0.8) or (P=0.8)) then
                            F:=1.0;
                    end
                else
                    F := 0.7
                else
                    F:=1.2;
                end
            end;
        {
            Mostrar Escalas
        }
        Escalas;

        {
        Mostrar el nombre de los códigos
        }
        Nombre;

        {
        Mostrar la probabilidad en curso
        }
        SetLineStyle(SolidLn,0,NormWidth);
        SetTextStyle(DefaultFont,HorizDir,1);
        If P<1 then
            begin
                OutTextXY(200 , 5 , 'P=0. ');
                Ta := Trunc(10000*P);
                Str(Ta,Pro);
                OutTextXY(230 , 5 , Pro);
            end
        else
            OutTextXY(200,5,'P=1.00');
            SetTextStyle(SmallFont,HorizDir,4);
            Rectangle(197,2,255,14);

        {
            Graficar la densidad espectral de potencia de los códigos escogidos
        }
        Dz := 1;
        If B[1] Then

```

```

Begin
  Nrzu(P, F, Mx, My, C[1]);
  SetColor(White);
  Str(Dz, Pro);
  OutTextXY(MxN-5, MyN-5, Pro);
  SetColor(White);
  Dz := Dz+1;
End;
If B[2] Then
  Begin
    Nrzp(P, F, Mx, My, C[2]);
    SetColor(White);
    Str(Dz, Pro);
    OutTextXY(MxN+5, MyN-5, Pro);
    SetColor(White);
    Dz := Dz+1;
  End;
If B[3] Then
  Begin
    Ami(P, F, Mx, My, C[3]);
    SetColor(White);
    Str(Dz, Pro);
    OutTextXY(MxN, MyN-5, Pro);
    SetColor(White);
    Dz := Dz+1;
  End;
If B[4] Then
  Begin
    Difm(P, F, Mx, My, C[4]);
    SetColor(White);
    Str(Dz, Pro);
    OutTextXY(MxN, MyN-5, Pro);
    SetColor(White);
    Dz := Dz+1;
  End;
If B[5] Then
  Begin
    Rz(P, F, Mx, My, C[5]);
    SetColor(White);
    Str(Dz, Pro);
    OutTextXY(MxN, MyN-8, Pro);
    SetColor(White);
    Dz := Dz+1;
  End;
If B[6] Then
  begin
    Miller(F, Mx, My, C[6]);
    SetColor(White);
    Str(Dz, Pro);
    OutTextXY(MxN+3, MyN-12, Pro);
    SetColor(White);
    Dz := Dz+1;
  end;

```

```

    End;
  If B[7] Then
    begin
      Manchester(P,F,Mx,My,C[7]);
      SetColor(White);
      Str(Dz,Pro);
      OutTextXY(MxN+22,MyN-4,Pro);
      SetColor(White);
      Dz := Dz+1;
    end;
  If B[8] Then
    begin
      Hdb3(F,Mx,My,C[8]);
      SetColor(White);
      Str(Dz,Pro);
      OutTextXY(MxN+5,MyN-12,Pro);
      SetColor(White);
      Dz := Dz+1;
    end;
  Ch := Readkey;
  CloseGraph;
End;

{
  Subrutina que muestra los gráficos de la densidad espectral de potencia
  para tres varios valores de probabilidad
}
Procedure Ver_P;
Var
  I,J : Integer;
  Ch : Char;
  Ban : Boolean;

Begin
  Normvideo;
  Ban := True;
  For I := 1 to 8 Do
    If B[i] Then
      Ban := False;
  If Ban Then
    Begin
      Textcolor(Cyan+Blink);
      Gotoxy(26,17);
      Write('Seleccione primero el código');
      Ch := Readkey;
      Normvideo;
    End
  Else
    Begin
      Gd := Detect;
      InitGraph(Gd, Gm, 'e:\tp\bgi');

```

```

If GraphResult <> grOk Then Halt(1);
Color := GetMaxColor;
Mx := GetMaxX;
My := GetMaxy;

{
  MOSTRAR EJES
}
Ejes(Mx,My,Color);

Pr[1] := 0.5;

If B[1] Then
  Begin
    Dz := 1;
    F:=0.7;
    Escalas;
    SetTextStyle(SmallFont,HorizDir,5);
    OutTextXY(200,0,'Codificación NRZ unipolar');
    OutTextXY(MX-125,0,'Probabilidad:');
    For I := 1 To 3 Do
      If (Pr[I] <> 0) then
        Begin
          P := Pr[I];
          Nrzu(P,F,Mx,My,C[I]);
          If ((Gd = 3) or (Gd = 4) or (Gd = 9)) then
            SetColor(C[I])
          else
            SetColor(yellow);
            If (P<1) Then
              Begin
                OutTextXY(Mx-100,Dz*20,'0. ');
                Ta := Round(10000*P);
                Str(Ta,Pro);
                OutTextXY(Mx-87,Dz*20,Pro);
              End
            Else
              OutTextXY(Mx-100, Dz*20,'1.000 ');
              SetColor(White);
              Str(Dz,Pro);
              SetTextStyle(SmallFont,HorizDir,4);
              OutTextXY(MxN-5+5*(Dz-1),MyN-11,Pro);
              {Circle(MxN-5,MyN-5,10);}
              SetTextStyle(SmallFont,HorizDir,5);
              OutTextXY(Mx-125,Dz*20,'( ) ');
              OutTextXY(Mx-120,Dz*20,Pro);
              Inc (Dz);
            End;
          End;
        End;
      End;
    End;
  End;
  If B[2] Then
    Begin

```



```

Dz := 1;
F := 1.2;
Escalas;
SetTextStyle(SmallFont, HorizDir, 5);
OutTextXY(200, 0, 'Codificación NRZ polar ');
OutTextXY(MX-125, 0, 'Probabilidad:');
For I := 1 To 3 Do
  If (Pr[I] <> 0) then
    Begin
      P := Pr[I];
      Nrzp(P, F, Mx, My, C[I]);
      SetColor(C[I]);
      If (P < 1) Then
        Begin
          OutTextXY(Mx-100, Dz*20, '0. ');
          Ta := Round(10000*P);
          Str(Ta, Pro);
          OutTextXY(Mx-87, Dz*20, Pro);
        End
      Else
        OutTextXY(Mx-100, Dz*20, '1.000 ');
        SetColor(White);
        Str(Dz, Pro);
        SetTextStyle(SmallFont, HorizDir, 4);
        OutTextXY(MxN+10+5*(Dz-1), MyN-9, Pro);
        SetTextStyle(SmallFont, HorizDir, 5);
        OutTextXY(Mx-125, Dz*20, '( ) ');
        OutTextXY(Mx-120, Dz*20, Pro);
        Inc (Dz);
      End;
    End;
  End;
If B[3] Then
  Begin
    Dz := 1;
    F := 1.2;
    For I:=1 To 3 Do
      If Pr[I]>0.71 Then F:=2.6;
    Escalas;
    SetTextStyle(SmallFont, HorizDir, 5);
    OutTextXY(200, 0, 'Codificación AMI ');
    OutTextXY(MX-125, 0, 'Probabilidad:');
    For I := 1 To 3 Do
      If (Pr[I] <> 0) then
        Begin
          P := Pr[I];
          Ami(P, F, Mx, My, C[I]);
          SetColor(C[I]);
          If (P < 1) Then
            Begin
              OutTextXY(Mx-100, Dz*20, '0. ');
              Ta := Round(10000*P);
              Str(Ta, Pro);
            End
          End;
        End;
      End;
    End;
  End;

```

```

        OutTextXY(Mx-87,Dz*20,Pro);
    End
    Else
        OutTextXY(Mx-100, Dz*20,`1.000`);
        SetColor(White);
        Str(Dz,Pro);
        SetTextStyle(SmallFont,HorizDir,4);
        OutTextXY(MxN-5+5*(Dz-1),MyN-11,Pro);
        SetTextStyle(SmallFont,HorizDir,5);
        OutTextXY(Mx-125,Dz*20,`( )`);
        OutTextXY(Mx-120,Dz*20,Pro);
        Inc (Dz);
    End;
End;
If B[4] Then
    Begin
        Dz := 1;
        F := 1.2;
        For I:=1 To 3 Do
            If ((Pr[I]>0.71) or (Pr[I]<0.469)) Then F:=3.5;
        Escalas;
        SetTextStyle(SmallFont,HorizDir,5);
        OutTextXY(180,0,`Codificación Diferencial NRZ-M Polar`);
        OutTextXY(MX-125,0,`Probabilidad:`);
        For I := 1 To 3 Do
            If (Pr[I] <> 0) then
                Begin
                    P := Pr[I];
                    Difm(P,F,Mx,My,C[I]);
                    SetColor(C[I]);
                    If (P<1) Then
                        Begin
                            OutTextXY(Mx-100,Dz*20,`0.`);
                            Ta := Round(10000*P);
                            Str(Ta,Pro);
                            OutTextXY(Mx-87,Dz*20,Pro);
                        End
                    Else
                        OutTextXY(Mx-100, Dz*20,`1.000`);
                        SetColor(White);
                        Str(Dz,Pro);
                        SetTextStyle(SmallFont,HorizDir,4);
                        OutTextXY(MxN-5+3*(Dz-1),MyN-10,Pro);
                        SetTextStyle(SmallFont,HorizDir,5);
                        OutTextXY(Mx-125,Dz*20,`( )`);
                        OutTextXY(Mx-120,Dz*20,Pro);
                        Inc (Dz);
                    End;
                End;
        End;
    End;
End;
If B[5] Then
    Begin
        Dz := 1;

```

```

F := 0.7;
{For I:=1 To 3 Do
  If ((Pr[I]>0.71) or (Pr[I]<0.469)) Then F:=3.5;}
Escalas;
SetTextStyle(SmallFont,HorizDir,5);
OutTextXY(200,0,'Codificación RZ Polar');
OutTextXY(MX-125,0,'Probabilidad:');
For I := 1 To 3 Do
  If (Pr[I] <> 0) then
    Begin
      P := Pr[I];
      Rz(P,F,Mx,My,C[I]);
      SetColor(C[I]);
      If (P<1) Then
        Begin
          OutTextXY(Mx-100,Dz*20,'0. ');
          Ta := Round(10000*P);
          Str(Ta,Pro);
          OutTextXY(Mx-87,Dz*20,Pro);
        End
      Else
        OutTextXY(Mx-100, Dz*20,'1.000 ');
        SetColor(White);
        Str(Dz,Pro);
        SetTextStyle(SmallFont,HorizDir,4);
        OutTextXY(MxN-5+5*(Dz-1),MyN-14,Pro);
        SetTextStyle(SmallFont,HorizDir,5);
        OutTextXY(Mx-125,Dz*20,'( ) ');
        OutTextXY(Mx-120,Dz*20,Pro);
        Inc (Dz);
      End;
    End;
  End;
If B[6] Then
  Begin
    Dz := 1;
    F := 0.7;
    For I:=1 To 3 Do
      If ((Pr[I]>0.71) ) Then F:=1.0;
    Escalas;
    SetTextStyle(SmallFont,HorizDir,5);
    OutTextXY(200,0,'Codificación Manchester');
    OutTextXY(MX-125,0,'Probabilidad:');
    For I := 1 To 3 Do
      If (Pr[I] <> 0) then
        Begin
          P := Pr[I];
          Manchester(P,F,Mx,My,C[I]);
          SetColor(C[I]);
          If (P<1) Then
            Begin
              OutTextXY(Mx-100,Dz*20,'0. ');
              Ta := Round(10000*P);

```

```

        Str(Ta,Pro);
        OutTextXY(Mx-87,Dz*20,Pro);
    End
    Else
        OutTextXY(Mx-100, Dz*20,`1.000`);
        SetColor(White);
        Str(Dz,Pro);
        SetTextStyle(SmallFont,HorizDir,4);
        OutTextXY(MxN,MyN,Pro);
        SetTextStyle(SmallFont,HorizDir,5);
        OutTextXY(Mx-125,Dz*20,`( )`);
        OutTextXY(Mx-120,Dz*20,Pro);
        Inc (Dz);
    End;
End;
Ch := Readkey;
CloseGraph;
End;
End;

{
    Menú que escoge los códigos dada una probabilidad
}
Procedure Menu_P;

Var
    V: vec;
    J: Integer;
    O: byte;
    Ch: String;
Begin
    V[1] := `Seleccionar los parámetros`;
    V[2] := `Ver espectros de potencia`;
    V[3] := `Regresar al menú principal`;

    Textcolor(White);
    TextBackground(Black);
    Clrscr;
    Pres_1;
    Pres_4;
    Pres_4a;
    Pres_5;
    Pres_7;
    J := 9;
    For Isa := 1 to 3 Do
        Begin
            Gotoxy(28,J);
            Write(V[Isa]);
            J := J + 2;
        End;
    End;

```

```

J := 9;
Isa := 1;
Repeat
  Gotoxy(28,j);
  TextBackground(White);
  Textcolor(Black);
  Write(V[Isa]);
  Repeat
    Ch := readkey;
  Until (Ch = #0) Or (Ch = Chr(13));
  If Ch <> Chr(13) Then
    CH := Readkey;
  If Ch = 'P' Then
  Begin
    J := j+2;
    Isa := Isa + 1;
    Textcolor(White);
    TextBackground(Black);
    Gotoxy(28,J-2);
    Write(V[Isa-1]);
  End;
  If Ch = 'H' Then
  Begin
    J := J-2;
    Isa := Isa-1;
    Textcolor(White);
    TextBackground(Black);
    Gotoxy(28,J+2);
    Write(V[Isa+1]);
  End;
  If Isa > 3 Then
  Begin
    Isa := 1;
    J := 9;
  End;
  If Isa < 1 Then
  Begin
    Isa := 3;
    J := 13;
  End;
Until Ch = Chr(13);
If Isa = 1 Then Selec_1;
If Isa = 2 Then Ver;
Ch := 'a';
Normvideo;
End;
{
  Menú para varias probabilidades
}
Procedure Menu_Q;

```

Var

```

V: vec;
J: Integer;
O: byte;
Ch: String;
Begin
  V[1] := 'Seleccionar código';
  V[2] := 'Seleccionar probabilidades';
  V[3] := 'Ver espectros de potencia';
  V[4] := 'Regresar al menú principal';

  Textcolor(White);
  TextBackground(Black);
  Clrscr;
  Pres_1;
  Pres_4;
  Pres_4b;
  Pres_6;
  Pres_10;
  Pres_12;
  J := 9;
  For Isb := 1 to 4 Do
    Begin
      Gotoxy(28,j);
      Write(V[Isb]);
      J := J + 2;
    End;

  J := 9;
  Isb := 1;
  Repeat
    Gotoxy(28,j);
    TextBackground(White);
    Textcolor(Black);
    Write(V[Isb]);
    Repeat
      Ch := readkey;
    Until (Ch = #0) Or (Ch = Chr(13));
    If Ch <> Chr(13) Then
      CH := Readkey;
    If Ch = 'P' Then
      Begin
        J := J+2;
        Isb := Isb + 1;
        Textcolor(White);
        TextBackground(Black);
        Gotoxy(28,J-2);
        Write(V[Isb-1]);
      End;
    If Ch = 'H' Then
      Begin
        J := J-2;
        Isb := Isb-1;

```

```

    Textcolor(White);
    TextBackground(Black);
    Gotoxy(28,J+2);
    Write(V[Isb+1]);
End;
If Isb > 4 Then
Begin
    Isb := 1;
    J := 9;
End;
If Isb < 1 Then
Begin
    Isb := 4;
    J := 15;
End;
Until Ch = Chr(13);
If Isb = 1 Then Selec_2;
If Isb = 2 Then Selec_3;
If Isb = 3 Then Ver_p;
Ch := 'a';
Normvideo;
End;

```

```

Procedure Menu_1;
Var
    V: vec;
    J: Integer;
    O: byte;
    Ch: String;
Begin
    P := 0.5;
    V[1] := 'Según los Códigos';
    V[2] := 'Según las Probabilidades';
    V[3] := 'Salir al D. O. S.';

    Textcolor(White);
    TextBackground(Blue);
    Clrscr;
    Pres_1;
    Pres_2;
    Pres_3;
    J := 13;
    For Is := 1 to 3 Do
    Begin
        Gotoxy(28,j);
        Write(V[Is]);
        J := J + 2;
    End;

    J := 13;
    Is := 1;

```

```

Repeat
  Gotoxy(28,j);
  TextBackground(White);
  Textcolor(Black);
  Write(V[is]);
  Repeat
    Ch := readkey;
  Until (Ch = #0) Or (Ch = Chr(13));
  If Ch <> Chr(13) Then
    CH := Readkey;
  If Ch = 'P' Then
  Begin
    J := j+2;
    Is := Is + 1;
    Textcolor(White);
    TextBackground(Blue);
    Gotoxy(28,J-2);
    Write(V[is-1]);
  End;
  If Ch = 'H' Then
  Begin
    J := J-2;
    Is := Is-1;
    Textcolor(White);
    TextBackground(Blue);
    Gotoxy(28,J+2);
    Write(V[is+1]);
  End;
  If Is > 3 Then
  Begin
    Is := 1;
    J := 13;
  End;
  If Is < 1 Then
  Begin
    Is := 3;
    J := 17;
  End;
Until Ch = Chr(13);
For J := 1 to 9 Do
  B[J] := False;
If Is = 1 Then
  Begin
    Repeat
      Menu_P;
    Until Isa=3;
  End;
If Is = 2 Then
  Begin
    Repeat
      Menu_Q;
    Until Isb=4;

```



```
        End;  
        Ch := 'a';  
End;  
  
{$F+}  
Begin  
    For Is := 1 to 9 Do  
        B[is] := False;  
        Pr[1] := 0.5;  
        Pr[2] := 0;  
        Pr[3] := 0;  
        Repeat  
            Menu_1;  
        Until is = 3;  
        TextBackground(Black);  
        NormVideo;  
        clrscr;  
End.
```

LISTADO DEL PROGRAMA DE CONTROL DEL EQUIPO
(ASSEMBLER DE LA FAMILIA INTEL MCS-51)

```

;-----
;ETIQUETAS
;-----
;
CODIGO      EQU      30H      ;ALMACENA EL TIPO DE CODIGO
C_TH1       EQU      31H      ;VALOR A CARGAR EN TH1
C_TL1       EQU      32H      ;VALOR A CARGAR EN TL1
C_TH0       EQU      33H      ;VALOR A CARGAR EN TH0
C_TL0       EQU      34H      ;VALOR A CARGAR EN TL0
AD_TH1      EQU      35H      ;LOCALIDADES ADICIONALES
AD_TL1      EQU      36H      ;PARA CARGA EN TEMPORIZADOR 1
AD_TH0      EQU      37H      ;LOCALIDADES ADICIONALES
AD_TL0      EQU      38H      ;PARA CARGA EN TEMPORIZADOR 0
DDRAM       EQU      39H      ;DISPLAY DATA RAM
ASCII       EQU      3AH      ;ASCII A MOSTRAR EN EL DISPLAY
NUM_CAR     EQU      3BH      ;NUMERO DE CARACTERES A MOSTRAR
ON_LINE     EQU      3CH      ;IGUAL A 00H CUANDO SE INICIA TX
C_SELEC     EQU      3DH      ;CONTADOR DE SELECCION
RITMO_TX    EQU      3EH      ;CONTIENE EL RITMO DE TX
MAX_RITMO   EQU      3FH      ;CONTIENE EL MAXIMO RITMO DE TX
RESPALDO    EQU      40H      ;BYTE QUE RESPALDA EL PORTICO 1
DEM_SINC    EQU      41H      ;RETARDO QUE DURA EL SINCRONISMO (BIFASE)
BR_SINCRO   EQU      42H      ;CODIGO DE LA VELOCIDAD
;
BIT_SIG     EQU      7FH      ;ALMACENA UN BIT DE SIGNO
SIG_DA      EQU      7EH      ;SIGNO DE DISPARIDAD ACUMULADA (4B3T, MS43)
TEMP        EQU      7DH      ;ALMACENAMIENTO TEMPORAL DE UN BIT
TEMPO       EQU      7CH      ;ALMACENAMIENTO TEMPORAL EXTRA
ATEND       EQU      7BH      ;ATENDER O NO A INT. EX1
OCURRIO     EQU      7AH      ;SI OCURRIO O NO UNA IT_SERIAL
ACEPTAR     EQU      79H      ;ACEPTAR EL ITEM EN PANTALLA
ARRIBA      EQU      78H      ;DESPLAZAR CURSOR HACIA ARRIBA
ABAJO       EQU      77H      ;DESPLAZAR CURSOR HACIA ABAJO
ENTRADA     EQU      76H      ;TECNOLOGIA DE ENTRADA (TTL-RS232)
;
DISP        EQU      20H      ;CONTIENE DISPARIDAD ACUMULADA (4B3T, MS43)
BUF4        EQU      21H      ;BUFFER DE 4 BITS (TX)
BUF3        EQU      22H      ;BUFFER DE 3 BITS (TX)
BUFRX4      EQU      23H      ;BUFFER DE RX CON 4 BITS DECODIFICADOS
BUFRX3      EQU      24H      ;BUFFER DE RX CON 3 PULSOS RECIBIDOS
LATCH_C1    EQU      25H      ;LATCH DE CONFIGURACION 1
LATCH_C2    EQU      26H      ;LATCH DE CONFIGURACION 2
ENLACE      EQU      27H      ;TX HALF O FULL DUPLEX
TIPO_HF     EQU      28H      ;TX HALF O FULL DUPLEX
TIPO_TR     EQU      29H      ;TRANSMISOR O RECEPTOR?
;
;LATCH_C1 EN BITS

```

DB 03H ;9600 BPS (03)
DB 05H ;4800 BPS (05)
DB 0DH ;2400 BPS (0E)
DB 24H ;1200 BPS (24)
DB 38H ;600 BPS (38)
DB 7FH ;300 BPS (7F)
DB 0FFH ;150 BPS (FF)

END